

Modelación y Simulación Computacional

Libro de apoyo de la asignatura

Maestría en Ingeniería
Facultad de Ingeniería

Daniel D. Otero Meza, Ph.D.

Universidad de Sucre
Sincelejo, Colombia

2026

Prefacio

Un ingeniero decide todos los días sobre sistemas que no puede ensayar. Nadie construye tres versiones de un canal para ver cuál conduce mejor la creciente, nadie fermenta cien lotes para averiguar a qué temperatura conviene operar, y nadie deja caer una estructura para saber cuánto resiste. La decisión se toma antes, sobre una representación del sistema que se comporta como el sistema, y esa representación es un modelo. La simulación es el acto de hacerla funcionar bajo condiciones que todavía no se han observado. La [Figura 0.1](#) resume ese recorrido, que va del sistema real al modelo y de la simulación a la decisión, con el lazo de verificación, calibración y validación que sostiene cada uno de sus pasos.

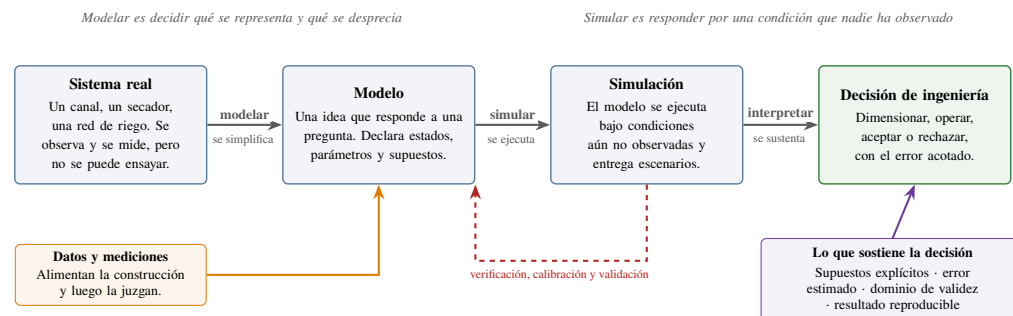


Figura 0.1. Del sistema real al modelo y de la simulación a la decisión de ingeniería.

Este libro es el material de apoyo de la asignatura *Modelación y Simulación Computacional*, de la Maestría en Ingeniería de la Universidad de Sucre. Su propósito no es enseñar un programa sino formar un criterio, el que permite decidir qué se representa y qué se desprecia, con qué método se resuelve, cuánto se puede creer del número que sale y cómo se comunica. La herramienta de trabajo es el ecosistema científico de Python, elegido porque es libre, multiplataforma y publica las implementaciones de referencia de casi todos los métodos que un ingeniero necesita. Lo aprendido aquí no caduca con una licencia.

Organización de la obra

El libro se organiza en cuatro capítulos que corresponden exactamente a las cuatro unidades del plan de asignatura, y cada uno se dicta en una semana de trabajo intensivo. El [Capítulo 1](#) construye el marco conceptual, con la tipología de los modelos, las etapas del proceso de modelación, el vocabulario de estados, parámetros y condiciones de frontera, y la discusión honesta de los alcances y las limitaciones. El [Capítulo 2](#) instala el entorno de trabajo, enseña a ingresar y verificar datos, a representar un modelo en código, a visualizar sus resultados y a sostener buenas prácticas. El [Capítulo 3](#) simula sistemas estacionarios, dinámicos y distribuidos, compara escenarios y mide la sensibilidad de la respuesta frente a lo que se supuso. El [Capítulo 4](#) cierra con la verificación, la calibración, la validación frente a datos independientes, el tratamiento de la incertidumbre y la comunicación de los resultados. La [Figura 0.2](#) muestra ese hilo conductor junto con la pregunta que responde cada unidad y la entrega del proyecto integrador que produce, pues el problema de investigación de cada estudiante entra en la primera semana y no se abandona hasta la sustentación final.

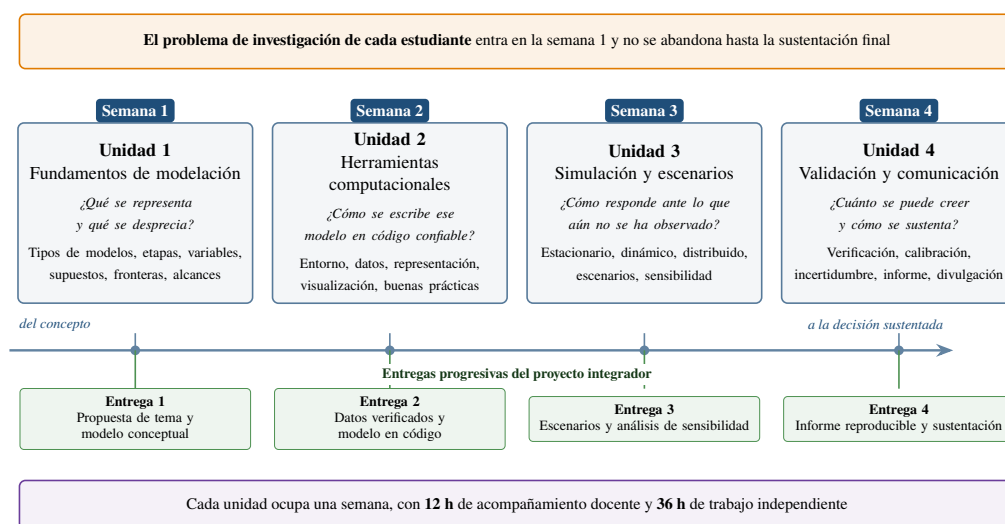


Figura 0.2. Hilo conductor de las cuatro unidades, con la pregunta que responde cada una y la entrega del proyecto que produce.

Cada capítulo sigue una misma arquitectura. Se abre con una motivación que sitúa el problema en un contexto de ingeniería reconocible. A continuación se enuncian las *definiciones* que fijan el vocabulario y los *teoremas* que sostienen los resultados en los que se apoya la práctica, ambos destacados tipográficamente para que el estudiante pueda volver a ellos con facilidad. Cada método se acompaña de su fundamento, de su expresión algorítmica en pseudocódigo, de un listado breve de Python que muestra la implementación real y de al menos un *ejemplo* completamente resuelto, en el cual se

separan de manera explícita las consideraciones adoptadas, el análisis, la formulación y la solución. Los capítulos terminan con una colección de problemas propuestos, organizados por tema en complejidad creciente y numerados de forma continua dentro del capítulo.

Sobre el código, las figuras y los cálculos

Los listados que aparecen en el texto son fragmentos, no programas completos. Se eligieron para mostrar la idea esencial de cada método en el menor número de líneas legibles, y su desarrollo íntegro vive en los cuadernos ejecutables que acompañan a cada unidad. Esos cuadernos son el segundo componente del material, se abren tanto en JupyterLab como en un entorno de ejecución en la nube, incluyen los datos que utilizan y están pensados para ser modificados. El libro explica por qué, el cuaderno muestra cómo.

Las figuras conceptuales, los esquemas y los diagramas de flujo se construyeron con el paquete *TikZ* y se compilan de forma independiente, lo cual garantiza que el trazo, la tipografía y la notación matemática de cada ilustración sean idénticos a los del texto que la rodea. Las figuras que representan datos, resultados de una simulación o de un ajuste se generaron con Python y se exportaron en formato vectorial con la misma tipografía del libro, de modo que la transición entre unas y otras resulta imperceptible. No hay imágenes tomadas de otras fuentes.

Los resultados numéricos de los ejemplos y de las tablas se calcularon con la precisión que se declara en cada caso y se verificaron computacionalmente. El estudiante debe reproducirlos. Conviene advertir desde ahora que pequeñas diferencias en la última cifra decimal son normales, pues dependen de la versión de la biblioteca, del orden de las operaciones en punto flotante y de la semilla del generador aleatorio, y documentar esas tres cosas es parte del oficio que la asignatura enseña.

Sobre el curso y su evaluación

El libro acompaña un módulo intensivo de cuatro semanas, con doce horas de acompañamiento docente y treinta y seis de trabajo independiente por semana, en el que cada unidad ocupa una semana y cierra con sus propios entregables. La evaluación combina cuatro instrumentos, que son los talleres guiados de cada unidad, las dos presentaciones orales del proyecto, una prueba práctica individual sobre computador y el proyecto integrador en cuatro entregas progresivas. La [Figura 0.3](#) reparte esos instrumentos sobre las cuatro semanas, y el esquema de evaluación que se entrega el primer día fija sus pesos, sus formatos y sus rúbricas.

Instrumento	Semana 1	Semana 2	Semana 3	Semana 4	Peso
Unidad que se dicta	Unidad 1 Fundamentos de modelación	Unidad 2 Herramientas computacionales	Unidad 3 Simulación y escenarios	Unidad 4 Validación y comunicación	12 h + 36 h
Talleres y laboratorios	Taller 1 cierre de la semana 1 7.5 %	Taller 2 cierre de la semana 2 7.5 %	Taller 3 cierre de la semana 3 7.5 %	Taller 4 cierre de la semana 4 7.5 %	30 %
Presentaciones orales	sin sustentación	Sustentación 1 avance del proyecto 5 %	sin sustentación	Sustentación 2 sustentación final del proyecto 5 %	10 %
Exámenes parciales	sin prueba	sin prueba	sin prueba	Prueba práctica unidades 1 a 4, en computador, 3 h, versiones A y B, 20 %	20 %
Proyecto integrador	Entrega 1 propuesta de tema 4 %	Entrega 2 modelo formulado y verificado 8 %	Entrega 3 escenarios, sensibilidad y calibración 10 %	Entrega 4 informe reproducible y repositorio 18 %	40 %

Todo entregable se sube al repositorio con el archivo DECLARACION.md. Los cuatro pesos suman el 100 % del curso. Las fechas exactas se fijan en el calendario del periodo 2026-2

Figura 0.3. Calendario de la evaluación, con los instrumentos repartidos en las cuatro semanas del módulo.

El texto guía de la asignatura es el libro de Velten, Schmidt y Kahlen sobre modelación y simulación matemática, del cual este material toma la definición de modelo matemático como terna de sistema, pregunta y enunciados, la distinción entre modelos fenomenológicos y mecanicistas y el esquema de modelación y simulación que organiza el ciclo de trabajo, y al cual se remite al lector en cada capítulo para la deducción completa de lo que aquí se resume. Lo que sostiene una decisión de ingeniería defendible al terminar el módulo se recoge en la [Figura 0.4](#), con el lugar de cada unidad en esa construcción.

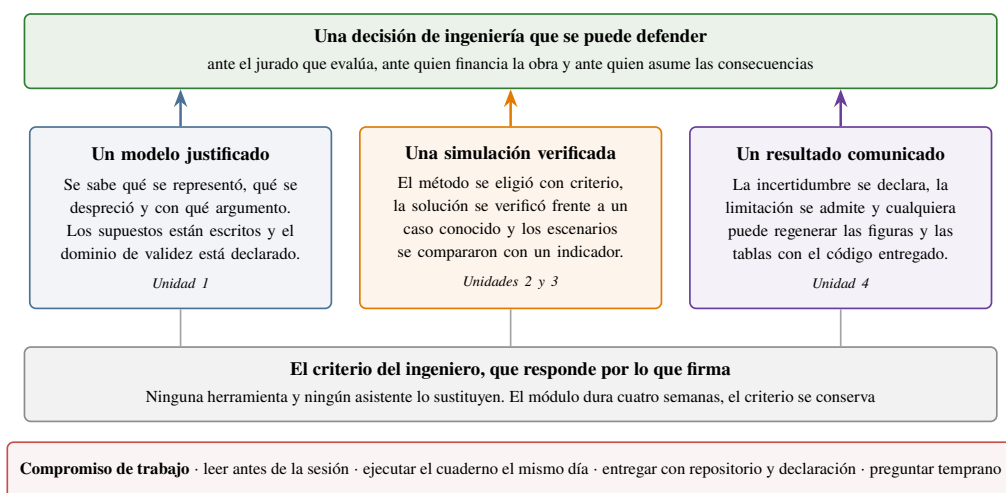


Figura 0.4. Lo que sostiene una decisión de ingeniería defendible y el lugar de cada unidad en esa construcción.

Sobre los asistentes de programación

El libro se escribe en un momento en el que un modelo de lenguaje redacta en segundos el código que un estudiante tardaría una hora en escribir. Negar esa realidad sería inútil, y prohibirla, ingenuo. La posición que este texto adopta es distinta, pues el ingeniero responde por el resultado que firma con independencia de quién o qué haya tecleado las líneas. De ahí que el libro insista una y otra vez en la verificación, en el orden de magnitud esperado, en la coherencia dimensional, en el caso límite de comprobación y en el balance que debe cerrar. Un asistente produce código plausible con la misma facilidad con la que produce código correcto, y solo el criterio del ingeniero distingue una cosa de la otra.

Cómo estudiar con este libro

La recomendación es sencilla y exigente a la vez. Lea el fundamento de cada método con lápiz y papel, reproduciendo los pasos que el texto resume. Ejecute después el cuaderno de la unidad y verifique que sus resultados coinciden con los del ejemplo resuelto. Rompa entonces el modelo a propósito, cambiando un parámetro, relajando una tolerancia o alterando una condición de frontera, y observe qué se degrada y qué no. Solo entonces pase a los problemas propuestos. La competencia para modelar no se adquiere leyendo modelos ajenos, se adquiere construyéndolos, viéndolos fallar y entendiendo por qué fallaron.

Daniel D. Otero Meza, Ph.D.
Sincelejo, Sucre, Colombia

Índice general

Prefacio	III
1 Fundamentos de Modelación en Ingeniería	1
1.1 Modelación y simulación en la decisión de ingeniería	2
1.2 Tipos de modelos y criterios de clasificación	6
1.3 Etapas del proceso de modelación	14
1.4 Variables, parámetros, supuestos y condiciones de frontera	19
1.5 Análisis de escalas y simplificación justificada	27
1.6 Alcances, limitaciones y responsabilidad del modelador	37
1.7 Problemas	43
1.7.1 Apropiación conceptual	43
1.7.2 Clasificación de modelos y alcance	43
1.7.3 Variables, grados de libertad y condiciones de frontera	44
1.7.4 Análisis de escalas y grupos adimensionales	44
1.7.5 Problemas de programación	45
1.7.6 Problemas de diseño y ensayo	45
2 Herramientas Computacionales para Modelación y Simulación	47
2.1 El ecosistema científico libre y el entorno reproducible	49
2.2 Ingreso, verificación y preparación de datos	54
2.3 Representación computacional de sistemas y procesos	62
2.4 Cálculo simbólico y paso del símbolo al código	72
2.5 Visualización e interpretación preliminar de resultados	76
2.6 Buenas prácticas, verificación del código y asistentes	81
2.7 Problemas	87
2.7.1 Apropiación conceptual	88
2.7.2 Preparación y verificación de datos	88
2.7.3 Representación numérica y condicionamiento	88
2.7.4 Cálculo simbólico y linealización	89
2.7.5 Problemas de programación	89
2.7.6 Problemas de diseño y ensayo	90

3	Simulación de Sistemas y Análisis de Escenarios	91
3.1	Simulación en estado estacionario	92
3.1.1	Métodos para una sola incógnita	94
3.1.2	Sistemas acoplados y estructura del jacobiano	98
3.2	Simulación dinámica y comportamiento bajo diferentes condiciones	101
3.2.1	Orden, error y control automático del paso	102
3.2.2	Estabilidad absoluta y rigidez	105
3.3	Sistemas distribuidos y método de líneas	110
3.3.1	Discretización espacial y condiciones de frontera	111
3.3.2	Estabilidad del esquema explícito y condición de Courant	113
3.4	Análisis de escenarios y comparación de alternativas	117
3.4.1	Indicadores de desempeño y comparación multicriterio	119
3.5	Simulación estocástica y propagación de la variabilidad	122
3.6	Análisis de sensibilidad	127
3.6.1	Sensibilidad local y elasticidad	127
3.6.2	Tamizado y sensibilidad global	130
3.7	Discusión de resultados en función del problema de estudio	134
3.8	Problemas	136
3.8.1	Apropiación conceptual	137
3.8.2	Simulación estacionaria	137
3.8.3	Simulación dinámica y rigidez	137
3.8.4	Sistemas distribuidos	138
3.8.5	Escenarios, Monte Carlo y sensibilidad	138
3.8.6	Problemas de programación	139
3.8.7	Problemas de diseño y ensayo	139
4	Validación, Interpretación y Comunicación de Resultados	141
4.1	Verificación del código y de la solución numérica	142
4.1.1	Método de las soluciones fabricadas	144
4.1.2	Estudio de convergencia y orden observado	145
4.1.3	Extrapolación de Richardson e índice de convergencia de malla	148
4.2	Calibración y estimación de parámetros	151
4.2.1	Precisión de los parámetros estimados	152
4.2.2	Identificabilidad, equifinalidad y parsimonia	155
4.3	Validación frente a datos independientes	158
4.3.1	Métricas de desempeño y lo que cada una esconde	159
4.3.2	Diagrama de paridad y análisis de residuales	161
4.4	Incertidumbre, limitaciones y consistencia	165
4.4.1	Propagación de la incertidumbre	166
4.4.2	Consistencia, cifras significativas y limitaciones	170
4.5	Tablas, gráficas e informes técnicos	171

4.5.1	El informe reproducible	173
4.6	Comunicación científica y aplicaciones en investigación	176
4.6.1	Métodos reproducibles y publicación de datos y de código	177
4.6.2	La simulación en el diseño de una investigación	178
4.7	Problemas	180
4.7.1	Apropiación conceptual	180
4.7.2	Verificación y convergencia	181
4.7.3	Calibración e identificabilidad	181
4.7.4	Validación y métricas de desempeño	181
4.7.5	Incertidumbre y consistencia	182
4.7.6	Problemas de programación	182
4.7.7	Problemas de diseño y ensayo	183
Referencias		185

Capítulo 1

Fundamentos de Modelación en Ingeniería

Un canal de riego que todavía no existe no se puede aforar, una planta de tratamiento que aún se diseña no se puede muestrear y un banco de módulos fotovoltaicos que se instalará el año entrante no entrega todavía ningún dato. Sin embargo, el ingeniero debe decidir hoy la sección del canal, el volumen del reactor y la potencia del arreglo, y debe hacerlo con un margen que él mismo pueda defender ante quien firma el contrato. Esa asimetría entre el momento de la decisión y el momento de la evidencia es la que da origen a la modelación. Un modelo es el instrumento que permite razonar cuantitativamente sobre un sistema antes de tenerlo enfrente.

La formación de pregrado suele entregar al ingeniero un repertorio de fórmulas y de procedimientos ya empaquetados, en los que alguien más decidió qué se desprecia, sobre qué rango vale la correlación y con qué incertidumbre se reporta el resultado. Esas decisiones son invisibles mientras el problema se parezca al del libro, y se vuelven críticas en cuanto deja de parecerse. La maestría exige el paso contrario, que consiste en construir la representación desde el balance, declarar las hipótesis que la sostienen, resolverla con un método cuyo error se controla y luego someterla a la evidencia disponible.

Este capítulo construye el marco conceptual sobre el que se apoya el resto del libro. Se fija el vocabulario que distingue modelar de simular, se ordena la variedad de modelos con criterios que tienen consecuencias prácticas sobre el método numérico, se recorre el ciclo completo del proceso de modelación con sus lazos de realimentación y se establece la disciplina de separar estados, entradas, parámetros y perturbaciones. Se estudia después cómo el análisis de escalas convierte la pregunta de qué se desprecia en un cálculo verificable, y se cierra discutiendo con franqueza hasta dónde llega un modelo y dónde empieza la responsabilidad de quien lo firma.

Al terminar el capítulo, el lector debe ser capaz de traducir un enunciado de ingeniería en una pregunta que un modelo pueda responder, de clasificar el modelo resultante según seis criterios y de anticipar qué implica esa clasificación, de contar los grados de libertad de una formulación antes de programarla, de decidir con un número adimensional si una simplificación es admisible y de redactar el registro de supuestos que acompaña a todo entregable profesional. Las herramientas computacionales que hacen operativo ese marco se instalan en el [Capítulo 2](#), la ejecución de las simulaciones se desarrolla en el [Capítulo 3](#) y el tratamiento formal de la validación y de la incertidumbre ocupa el [Capítulo 4](#).

1.1. Modelación y simulación en la decisión de ingeniería

La práctica profesional de la ingeniería consiste en comprometer recursos sobre sistemas que no se pueden ensayar de forma exhaustiva. Nadie construye tres versiones de un canal para averiguar cuál conduce mejor la creciente decamilenaria, nadie fermenta cien lotes de pulpa para saber a qué temperatura conviene operar y nadie deja fallar una subestación para medir el costo de la interrupción. El ensayo directo es imposible por costo, por riesgo, por escala temporal o porque el sistema no existe todavía. Lo que sí se puede ensayar cuantas veces se quiera es una representación que se comporte como el sistema en aquello que interesa.

Conviene insistir en la última cláusula, porque es la que separa un modelo útil de una maqueta inútil. Un modelo no busca parecerse al sistema en todo, sino reproducir con fidelidad suficiente el comportamiento del que depende la decisión. Un modelo hidráulico de una red de distribución que predice presiones nodales con error de 0.5 m de columna de agua es excelente para dimensionar un tanque elevado y es completamente inservible para estudiar un transitorio por cierre rápido de válvula, aunque ambas preguntas se refieran a la misma red física. La pregunta de ingeniería precede al modelo y determina qué se representa y qué se ignora.

Definición 1.1. Modelo. Un *modelo* es una representación simplificada de un sistema real, formulada con un propósito declarado, que reproduce las relaciones entre las magnitudes que ese propósito exige y omite deliberadamente las restantes. Su validez se juzga siempre frente al propósito y nunca en abstracto.

La definición encierra una particularidad decisiva. Como el modelo se juzga frente a un propósito, la misma representación matemática puede ser correcta para una pregunta y errónea para otra, sin que ninguna línea de su formulación haya cambiado. Esa naturaleza teleológica del modelo, que ya estaba en la formulación clásica de Minsky según la cual un objeto es modelo de otro en la medida en que sirve para responder las preguntas que interesan sobre el segundo, es el punto de partida del texto guía de la asignatura [1].

De ahí que la primera obligación del modelador no sea escribir ecuaciones sino escribir la pregunta, con su variable de interés, su horizonte temporal, su resolución espacial y su tolerancia admisible. Un enunciado como «simular el comportamiento del embalse» no es una pregunta de ingeniería. «Estimar la probabilidad de que el nivel supere la cota de vertimiento durante los próximos treinta días, con una tolerancia de 0.10 m» sí lo es, y de ella se deduce casi todo el resto. La Figura 1.1 sitúa esa reformulación sobre la línea de tiempo que separa el momento de decidir del momento en que llega la evidencia.

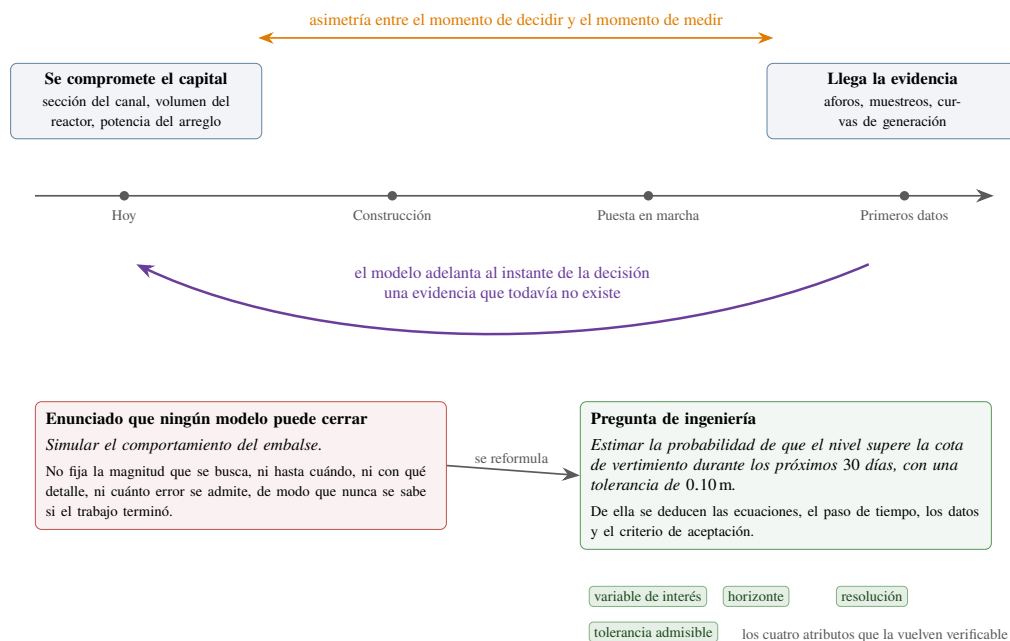


Figura 1.1. Asimetría entre el momento de decidir y el momento de medir, y reformulación de un enunciado difuso como pregunta de ingeniería.

Construir la representación y ejecutarla son actos distintos, y el vocabulario técnico los separa. Modelar es el trabajo de abstracción que produce un conjunto de relaciones entre magnitudes, junto con las hipótesis que las sostienen. Simular es el trabajo de operar esas relaciones bajo condiciones concretas, incluidas condiciones que jamás se han observado, para obtener trayectorias, escenarios o distribuciones de la respuesta. Un modelo mal construido se simula sin dificultad y produce números perfectamente presentables, de modo que la facilidad de la simulación no es evidencia de la calidad del modelo.

Definición 1.2. Simulación. Una *simulación* es la ejecución de un modelo bajo un conjunto específico de entradas, parámetros y condiciones auxiliares, con el fin de obtener la respuesta del sistema representado en un escenario que no necesariamente ha sido observado. La simulación produce resultados condicionados al modelo, nunca evidencia independiente sobre el sistema.

El aporte propio de la simulación se entiende mejor por contraste con el registro histórico. Una serie de veinte años de caudales dice lo que ocurrió con la infraestructura que existía, con la cobertura vegetal que había y con la operación que se aplicó. No dice nada sobre lo que habría ocurrido con un embalse de regulación aguas arriba, porque esa condición no está en los datos. La simulación es el único procedimiento disponible para explorar la condición no observada, y es también el único que permite recorrer sistemáticamente un espacio de alternativas de diseño antes de comprometer capital. Por eso el modelo se sitúa entre el sistema real y la decisión, tal como resume la [Figura 1.2](#).

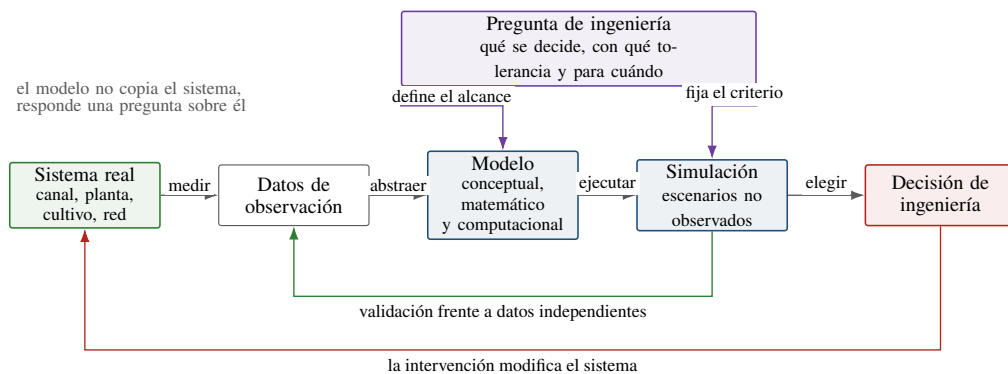


Figura 1.2. Posición del modelo entre el sistema real y la decisión de ingeniería, con el lazo de validación que compara la predicción contra datos independientes y el lazo por el cual la intervención modifica el sistema representado.

El esquema muestra dos lazos que conviene no confundir. El de validación compara lo que el modelo predice contra observaciones que no se usaron para construirlo, y es el que permite afirmar algo sobre su credibilidad [2]. El segundo recuerda que la decisión modifica el sistema y con ello invalida parte de la evidencia sobre la que el modelo se calibró, pues una red de alcantarillado ampliada ya no es la red cuyos caudales se midieron.

La postura que este libro adopta se resume en la advertencia de Box, quien escribió que, puesto que todos los modelos están equivocados, el científico debe permanecer atento a aquello que está equivocado de manera importante [3]. La frase no invita al relativismo sino a jerarquizar los errores, esto es, a distinguir la simplificación que desplaza el resultado en la tercera cifra decimal de la que lo desplaza en un factor de dos. Buena parte de la competencia profesional consiste en saber cuál es cuál, y el análisis de escalas de la [Sección 1.5](#) convierte esa intuición en un cálculo.

Queda una objeción práctica frecuente, según la cual modelar cuesta tiempo que el proyecto no tiene. La objeción se resuelve comparando el costo del modelo con el costo de la decisión que sostiene, y conviene ponerle cifras sobre una decisión hidráulica elemental en la que la única variable de diseño es el diámetro de una tubería de impulsión.

Considérese una estación de bombeo que eleva 45 L/s de agua potable por 1200 m de tubería de PVC hasta un tanque situado 28 m por encima del pozo de succión, y supóngase que el proyectista duda entre un diámetro de 200 mm y uno de 250 mm. El modelo que responde la pregunta consta de las dos relaciones de la Ecuación (1.1), que son la altura de bombeo y la energía anual que exige,

$$H = H_g + f \frac{L}{D} \frac{V^2}{2g}, \quad V = \frac{4Q}{\pi D^2}, \quad E = \frac{\rho g Q H}{\eta} t_{\text{op}}, \quad (1.1)$$

donde H es la altura total de bombeo (m); H_g es la altura estática (m); f es el factor de fricción de Darcy, obtenido de la ecuación implícita de Colebrook–White (adimensional); L es la longitud de la conducción (m); D es el diámetro interno (m); V es la velocidad media (m/s); Q es el caudal (m³/s); g es la aceleración de la gravedad (m/s²); ρ es la densidad del agua (kg/m³); η es el rendimiento del conjunto motobomba (adimensional); E es la energía anual (Wh); y t_{op} es el tiempo anual de operación (h).

Con agua a 20 °C, rugosidad absoluta de 0.0015 mm, rendimiento de 0.72 y operación de 18 h/d durante todo el año, el diámetro de 200 mm conduce a una velocidad de 1.432 m/s, una pérdida por fricción de 9.21 m y un consumo de 149 619 kWh anuales, mientras que el de 250 mm baja la velocidad a 0.917 m/s, la pérdida a 3.14 m y el consumo a 125 231 kWh. La diferencia es de 24 388 kWh al año, que con una tarifa de 0.20 USD/kWh equivalen a 4878 USD/a. El resultado se verifica advirtiendo que la pérdida escala como D^{-5} , de modo que aumentar el diámetro en un factor 1.25 debe reducirla cerca de 3.05 veces, y la relación calculada es 2.93.

Si el sobrecosto de la tubería mayor asciende a 21 000 USD, se recupera en 4.3 a y produce un beneficio neto de 31 068 USD descontado a veinticinco años con una tasa del 8 %. El cálculo cabe en media hora de trabajo y sostiene una decisión de esa magnitud. Ninguna serie histórica de la estación habría podido responder la pregunta, porque la alternativa de 250 mm no existe en ningún registro, y ese es precisamente el aporte de la simulación.

El cálculo anterior es deliberadamente modesto, pues su modelo consta de dos relaciones algebraicas resueltas con la rutina de búsqueda de raíz de SciPy [4]. Con todo, exhibe la estructura completa del razonamiento, que consiste en una pregunta, una representación con hipótesis declaradas, una ejecución sobre escenarios alternativos y una lectura ingenieril del resultado. Lo que cambia en los problemas grandes no es la estructura sino la complejidad de la representación, y esa complejidad se ordena con la tipología que se establece a continuación [5].

1.2. Tipos de modelos y criterios de clasificación

La variedad de objetos que reciben el nombre de modelo es tan amplia que la palabra, por sí sola, no informa. Un diagrama de flujo, un sistema de ecuaciones diferenciales, una regresión ajustada a datos de planta y un programa que integra ese sistema en una malla de cien mil celdas son todos modelos, y pertenecen a niveles distintos de una misma construcción. Conviene ordenarlos en cuatro estratos sucesivos, que son el conceptual, el matemático, el estadístico y el computacional, y la palabra clave es sucesivos, porque cada estrato presupone el anterior y lo hace operativo.

Definición 1.3. Modelo conceptual. Un *modelo conceptual* es la descripción cualitativa del sistema que fija su frontera, enumera los componentes que se retienen, identifica los flujos y los procesos que los vinculan y declara las hipótesis simplificadoras adoptadas, todo ello antes de escribir ninguna ecuación.

El modelo conceptual es el estrato que con más frecuencia se omite y el que más errores irreparables produce. Definir la frontera equivale a decidir qué es interior, qué es entrada y qué es irrelevante, y esa decisión no se enmienda después con un método numérico mejor. Si el modelo conceptual de una laguna de estabilización no incluye la estratificación térmica, ningún refinamiento de la malla hará aparecer su efecto en los resultados. Por eso el entregable de esta etapa es un diagrama con la frontera dibujada y una lista escrita de hipótesis, ambos revisables por un tercero.

El estrato matemático traduce esa descripción en balances de masa, de energía, de cantidad de movimiento y de carga, junto con las ecuaciones constitutivas que cierran el sistema. Conviene fijar con precisión qué se entiende por modelo matemático, porque la definición corriente, que lo reduce a un conjunto de ecuaciones, deja fuera lo que lo convierte en modelo. Una ecuación diferencial escrita en una pizarra no modela nada mientras no se declare de qué sistema habla y qué pregunta responde, y esa observación conduce a la definición en tres componentes que adopta el texto guía [1].

Definición 1.4. Modelo matemático. Un *modelo matemático* es una terna formada por un sistema, una pregunta relativa a ese sistema y un conjunto de enunciados matemáticos, que son ecuaciones algebraicas, diferenciales, en diferencias o integrales junto con sus condiciones auxiliares, que pueden usarse para responder la pregunta. Los tres componentes son indispensables, pues sin el sistema y la pregunta el conjunto de ecuaciones es un ejercicio de matemáticas y no un modelo.

La definición explica por qué dos ingenieros que parten del mismo sistema físico pueden construir modelos matemáticos distintos y ambos correctos, ya que responden preguntas distintas, y por qué el mismo conjunto de ecuaciones cambia de estatus cuando cambia la pregunta. La ecuación de Manning es un modelo matemático del canal cuando la pregunta es el tirante normal, y deja de serlo cuando la pregunta es la propagación de una onda de creciente, aunque no haya cambiado una sola letra.

Los cuatro estratos y su encadenamiento se representan en la [Figura 1.3](#), que añade una segunda dimensión, la del origen de las relaciones, sobre la cual se sitúan los modelos mecanicistas, los empíricos y los híbridos. Leerla de arriba abajo describe la construcción de un modelo, pues del sistema y de la pregunta sale el modelo conceptual, de él las ecuaciones, de estas la inferencia de lo que los datos deben aportar y, al final, el programa que lo resuelve. Leerla de abajo arriba describe la depuración, porque un resultado sospechoso se persigue estrato por estrato hasta encontrar en cuál se cometió el error.

El estrato estadístico se ocupa de la parte del comportamiento que no se deduce de un principio de conservación sino que se infiere de datos, y es el que conecta el modelo matemático con el mundo real, porque toda comparación entre una predicción y una medición pasa por un enunciado sobre el error [1]. Ese estrato no se limita a ajustar curvas, ya que decide con qué distribución se describe la discrepancia, cómo se estiman los parámetros y con qué precisión quedan determinados, cuestiones que ocupan el [Capítulo 4](#).

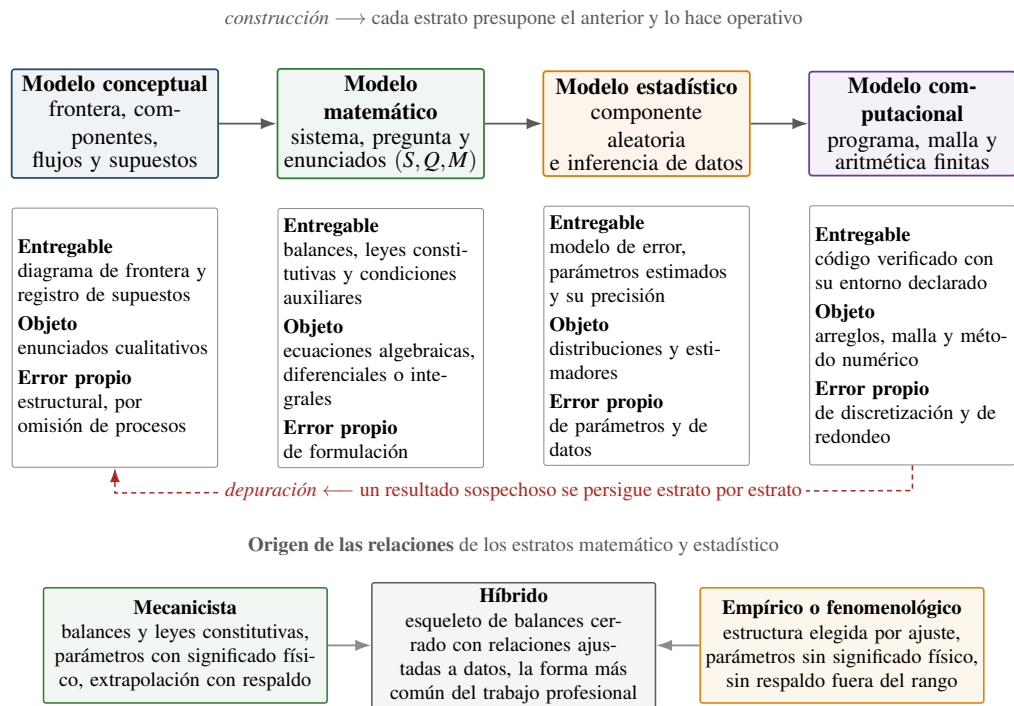


Figura 1.3. Los cuatro estratos sucesivos de un modelo, con el entregable, el objeto matemático y el error propio de cada uno, y la dimensión transversal del origen de las relaciones que separa los modelos mecanicistas de los empíricos.

Definición 1.5. Modelo estadístico. Un *modelo estadístico* es un modelo en el cual la relación entre las entradas y la salida incorpora de manera explícita una componente aleatoria, descrita por una distribución de probabilidad, de modo que su respuesta es una distribución y no un valor, y sus parámetros, incluidos los que describen la dispersión, se estiman a partir de observaciones mediante un procedimiento de inferencia.

El modelo estadístico puede envolver a un modelo mecanicista, al que añade el término de error de medición que hace posible calibrarlo, o sostenerse solo, como ocurre en una regresión o en una red neuronal que aprende la relación directamente de los datos [6, 7]. En el segundo caso su estructura se elige por conveniencia de ajuste y no por un principio físico, y comparte con el modelo empírico la limitación de fondo que se cuantifica en la [Sección 1.6](#), pues fuera del rango observado carece de respaldo por más flexible que sea [8]. El estrato computacional, por último, convierte todo lo anterior en un programa que lo resuelve sobre una malla y con una aritmética finitas, y recibe su definición formal en la [Sección 1.3](#).

Los dos estratos intermedios se distinguen mejor por su origen que por su forma, pues lo determinante es si la relación proviene de un principio de conservación o de un ajuste a observaciones, distinción que tiene consecuencias directas sobre la capacidad de extrapolar. El texto guía la fórmula como la oposición entre modelos fenomenológicos, contruidos solo con datos y sin información previa sobre el sistema, y modelos mecanicistas, en los que al menos parte de los enunciados proviene de conocimiento previo sobre su funcionamiento interno [1].

Definición 1.6. Modelo mecanicista. Un *modelo mecanicista* es aquel cuyas ecuaciones se derivan de principios de conservación y de leyes constitutivas conocidas, de modo que sus parámetros tienen significado físico propio y pueden medirse o estimarse con independencia de los datos de respuesta del sistema.

Un modelo mecanicista admite en principio la extrapolación fuera del rango observado, porque los principios de conservación siguen valiendo allí. La licencia no es incondicional, pues las leyes constitutivas que cierran el sistema sí tienen rango de validez, y una correlación de transferencia de calor obtenida entre 10^4 y 10^5 en número de Reynolds no se vuelve confiable por estar incrustada en un balance impecable.

Definición 1.7. Modelo empírico. Un *modelo empírico* es aquel cuya estructura se elige por conveniencia de ajuste y cuyos parámetros se estiman exclusivamente a partir de datos de entrada y salida del sistema, sin que dichos parámetros correspondan necesariamente a magnitudes físicas identificables.

Un modelo empírico bien ajustado puede ser superior a uno mecanicista dentro del rango en que se calibró, y suele ser más barato de construir y de evaluar, pero fuera de ese rango carece por completo de respaldo, tal como se cuantifica en la [Sección 1.6](#). Entre ambos extremos se sitúan los modelos híbridos, en los cuales un esqueleto de balances se cierra con relaciones ajustadas a datos, que son la forma más común del trabajo profesional real.

Sobre estos estratos operan seis criterios transversales de clasificación, que no compiten entre sí sino que se aplican de forma simultánea, y cada uno corresponde a una decisión con consecuencias precisas sobre el método numérico y sobre lo que el modelo puede responder. La [Figura 1.4](#) los representa como seis ejes independientes y sitúa sobre ellos dos modelos muy distintos, para dejar claro que un modelo no pertenece a una categoría sino que ocupa un punto en un espacio de decisiones, mientras que la [Tabla 1.1](#) recoge la consecuencia práctica de cada elección.

Tras el origen, ya discutido, viene el tratamiento del tiempo, que distingue el modelo estacionario, reducido a un sistema algebraico, del dinámico, que exige integración y una condición inicial. Sigue la aleatoriedad, que separa el modelo determinístico, de salida fijada por sus entradas, del estocástico, cuya salida es una distribución y cuya evaluación

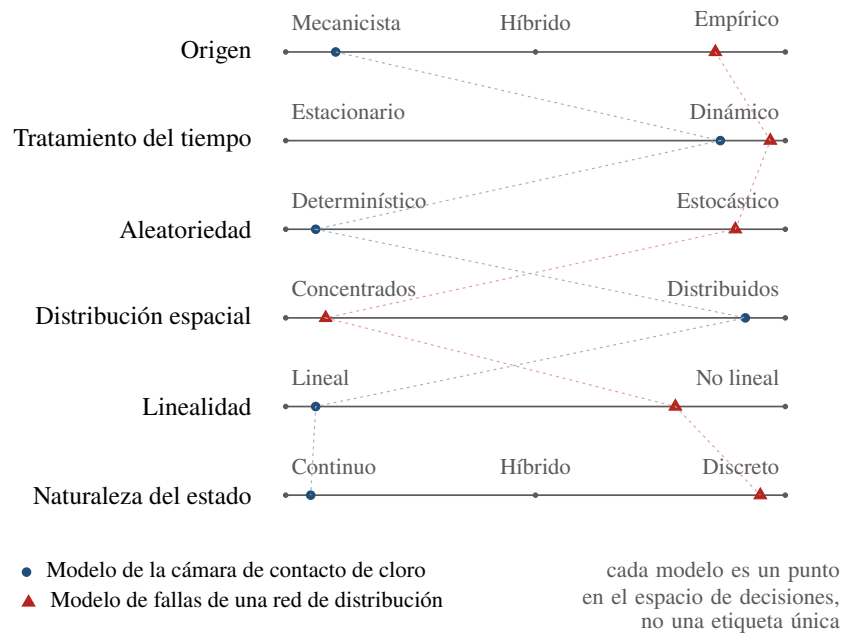


Figura 1.4. Los seis criterios transversales de clasificación como ejes independientes, con la posición de dos modelos de dominios distintos sobre cada uno de ellos.

requiere repetición. La distribución espacial opone los parámetros concentrados, que conducen a ecuaciones diferenciales ordinarias, a los distribuidos, que resuelven los gradientes y conducen a ecuaciones en derivadas parciales. La linealidad decide si vale la superposición y si hay solución analítica de referencia. La naturaleza del estado, por último, separa lo continuo de lo discreto y da lugar a los modelos de eventos y a los híbridos con conmutación.

Esa tabla conviene leerla como la lista de preguntas que el modelador se hace antes de programar. Responderlas obliga a explicitar decisiones que de otro modo se toman por inercia, como suponer estado estacionario porque el problema del libro lo suponía. El efecto conjunto de estas decisiones se aprecia mejor cuando se aplican tres niveles de complejidad a un mismo sistema físico. El [Ejemplo 1.1](#) lo hace con una cámara de contacto de cloro, un elemento presente en cualquier planta de potabilización, y muestra que la elección entre estratos no es un asunto de elegancia sino de respuestas numéricamente distintas.

Tabla 1.1. Criterios transversales de clasificación, consecuencia práctica de cada elección y ejemplos por dominio de la Facultad.

Criterio	Polos	Consecuencia práctica	Ejemplo
Origen	Mecanicista, empírico	Define si la extrapolación tiene respaldo	Secador con balance de energía frente a curva ajustada
Tiempo	Estacionario, dinámico	Sistema algebraico frente a integración	Tirante normal frente a tránsito de creciente
Aleatoriedad	Determinístico, estocástico	Una corrida frente a un conjunto de realizaciones	Demanda media frente a curva de duración de carga
Espacio	Concentrados, distribuidos	Ecuaciones ordinarias frente a derivadas parciales	Tanque agitado frente a columna de percolación
Linealidad	Lineal, no lineal	Superposición y solución analítica de referencia	Pozos en acuífero confinado frente a acuífero libre
Estado	Continuo, discreto	Integrador numérico frente a planificador de eventos	Cuarto frío frente a fallas de una red de distribución

^a Los seis criterios se aplican de forma simultánea, de modo que un mismo modelo recibe una posición en cada fila.

Ejemplo 1.1. Tres modelos de complejidad creciente para una cámara de contacto de cloro. Una cámara de contacto rectangular tiene 30 m de longitud y una sección transversal de 4 m², de modo que su volumen útil es de 120 m³. Recibe un caudal permanente de 40 m³/h y el desinfectante residual decae con cinética de primer orden y constante $k = 0.35/\text{h}$. Se pide estimar la concentración relativa a la salida en régimen permanente y el tiempo que tarda la cámara en alcanzarlo tras un cambio escalón en la concentración de entrada.

Consideraciones. 1. Caudal, volumen y temperatura constantes; 2. decaimiento de primer orden con constante conocida; 3. coeficiente de dispersión longitudinal $D_L = 25 \text{ m}^2/\text{h}$, estimado a partir de la geometría de la cámara; 4. la cámara arranca limpia, con concentración nula en todo su interior; 5. no hay cortocircuitos ni zonas muertas adicionales a las que el término dispersivo representa.

Formulación. El primer modelo supone mezcla completa y estado estacionario, con lo cual el balance se reduce a una relación algebraica. El segundo mantiene la mezcla completa pero admite acumulación, y produce una ecuación diferencial ordinaria. El tercero resuelve el gradiente longitudinal y produce la ecuación en derivadas parciales de la Ecuación (1.2),

$$\frac{\partial C}{\partial t} = -u \frac{\partial C}{\partial x} + D_L \frac{\partial^2 C}{\partial x^2} - k C, \quad (1.2)$$

donde C es la concentración del desinfectante (mg/L); t es el tiempo (h); x es la coordenada longitudinal (m); $u = Q/A$ es la velocidad media del flujo (m/h); D_L es el coeficiente de dispersión longitudinal (m²/h); y k es la constante de decaimiento de primer orden (1/h).

Los tres modelos comparten dos grupos adimensionales, que son el número de Péclet $Pe = uL/D_L$ y el número de Damköhler $Da = k\tau$, con $\tau = V/Q$ el tiempo medio de residencia. Con los datos del enunciado resultan $\tau = 3$ h, $Pe = 12$ y $Da = 1.05$.

Solución. El modelo de mezcla completa estacionario entrega $C/C_0 = 1/(1 + Da) = 0.488$. El modelo dinámico de mezcla completa entrega la misma concentración final y añade la constante de tiempo $\tau_e = (1/\tau + k)^{-1} = 1.463$ h. El modelo distribuido se integra por el método de líneas y converge al valor estacionario $C/C_0 = 0.376$, que coincide con la solución analítica del reactor de flujo disperso. La Figura 1.5 muestra las tres respuestas.

Verificación. La solución analítica del reactor disperso vale 0.376441 y con 600 celdas el esquema numérico entrega 0.376659, con error relativo de 5.8×10^{-4} . Al duplicar el número de celdas el error se reduce a la mitad, lo que confirma el orden uno del esquema advectivo empleado.

Comentario. Los dos modelos de mezcla completa sobrestiman la concentración residual en un 29.6 % respecto del distribuido, porque suponen dilución instantánea en todo el volumen. Para verificar un residual mínimo esa sobrestimación es conservadora y aceptable, y para dimensionar la dosis de cloro es antieconómica. La elección del estrato no es estética, se paga en reactivo.

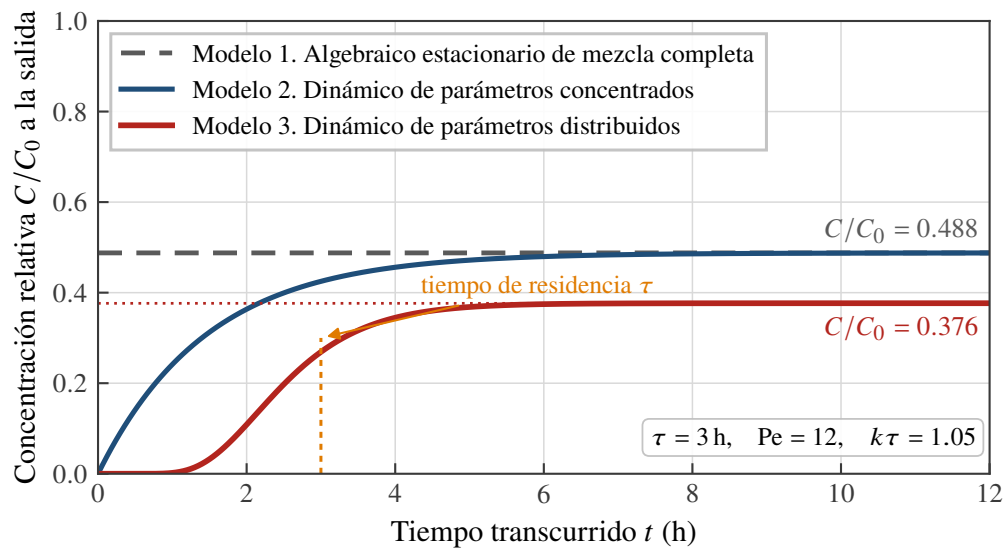


Figura 1.5. Respuesta de la cámara de contacto a un escalón de concentración a la entrada, según tres modelos de complejidad creciente construidos sobre el mismo sistema físico.

El salto entre el segundo y el tercer modelo no consiste solo en resolver una ecuación más difícil, pues el modelo distribuido exige condiciones de frontera que el concentrado no necesita, y elegir las es una decisión de modelación por derecho propio. La condición de entrada de Danckwerts, que iguala el flujo advectivo entrante al flujo total en la sección de ingreso, es la que corresponde a un reactor cerrado y produce un salto de concentración que sorprende a quien esperaba continuidad [9]. El [Listado 1.1](#) recoge las tres formulaciones en forma comparable.

Listado 1.1. Los tres modelos de la cámara de contacto expresados como funciones comparables.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 Q, A, L, k, D_L = 40.0, 4.0, 30.0, 0.35, 25.0
5 u, tau = Q / A, A * L / Q
6
7 def estacionario_mezclado():
8     return 1.0 / (1.0 + k * tau)
9
10 def dinamico_mezclado(t):
11     return estacionario_mezclado() * (1 - np.exp(-(1 / tau + k) *
12         t))

```

```

13 def distribuido(t_final=12.0, nodos=600): # parámetros
    distribuidos
14     dx = L / nodos
15     def derivada(_, c):
16         flujo = np.concatenate(([u], u * c)) #
            advección aguas arriba
17         flujo[1:nodos] -= D_L * (c[1:] - c[:-1]) / dx #
            dispersión interna
18         flujo[nodos] = u * c[-1] # salida
            sin gradiente
19     return -(flujo[1:] - flujo[:-1]) / dx - k * c
20     sol = solve_ivp(derivada, (0.0, t_final), np.zeros(nodos),
21                     method="BDF", rtol=1e-8, atol=1e-10)
22     return sol.t, sol.y[-1, :]

```

Las tres funciones del listado reciben la misma información física y devuelven respuestas distintas, sin que ninguna sea incorrecta. La diferencia está en el modelo conceptual que cada una encarna. Elegir entre ellas requiere un procedimiento explícito, y ese procedimiento es el ciclo de modelación que se describe enseguida.

1.3. Etapas del proceso de modelación

Un modelo no se escribe de una sentada. Se construye recorriendo una secuencia de etapas, cada una con un entregable propio y con un criterio que decide si se puede pasar a la siguiente, y ese recorrido casi nunca se completa en un solo paso. La literatura de modelación ambiental sistematizó la secuencia en diez pasos iterativos, que van desde la definición del propósito hasta la evaluación del modelo frente a los objetivos declarados [10], y el texto guía la condensa en un esquema de cinco bloques, que son las definiciones del problema y del sistema, el análisis del sistema, la modelación, la simulación y la validación, con la advertencia de que ningún proyecto real los recorre en línea recta [1]. La versión que se adopta aquí desarrolla esos bloques en ocho etapas y se representa en la [Figura 1.6](#).

Las cuatro primeras etapas construyen el modelo. La primera formula el problema y fija el alcance, la segunda conceptualiza el sistema y declara las hipótesis, la tercera traduce esa descripción en ecuaciones y condiciones auxiliares, y la cuarta las implementa en un programa. Ninguna de las cuatro produce evidencia alguna sobre la calidad de lo construido, y confundir el hecho de haber terminado un programa con el de tener un modelo utilizable es el atajo que más retrabajo genera.

Las etapas quinta, sexta y séptima no construyen nada nuevo, sino que deciden si lo construido puede usarse. La verificación pregunta si el programa resuelve correctamente las ecuaciones que se pretendía resolver, asunto puramente matemático que se dirime con soluciones manufacturadas, casos límite analíticos y estudios de refinamiento. La

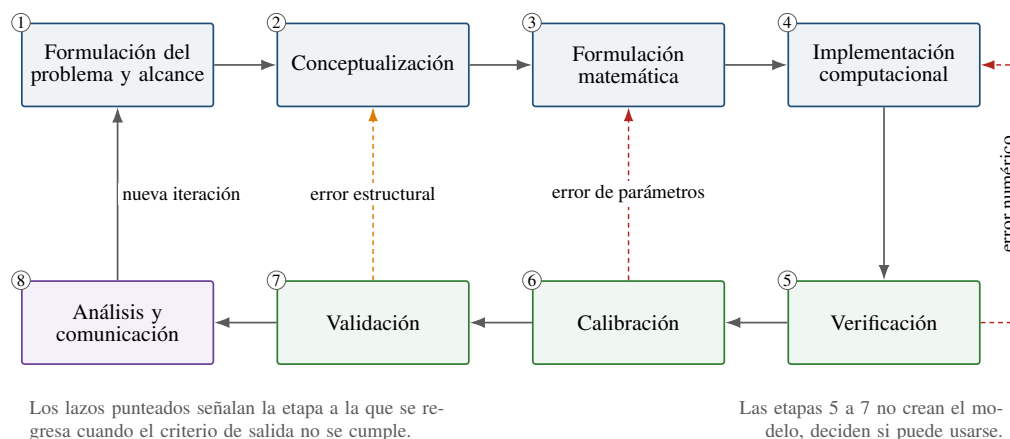


Figura 1.6. Ciclo de modelación con sus ocho etapas y los tres lazos de realimentación que se activan cuando el criterio de salida de una etapa de evaluación no se cumple.

validación pregunta si esas ecuaciones representan el sistema real dentro de la tolerancia declarada, asunto empírico que exige datos independientes [2]. Confundirlas es el error metodológico más costoso del oficio, porque un código impecable puede resolver con exactitud un modelo equivocado.

Definición 1.8. Modelo computacional. Un *modelo computacional* es la implementación de un modelo matemático en un programa que lo resuelve mediante un método numérico, sobre una discretización finita y con aritmética de precisión finita. Al modelo matemático original le añade, por tanto, un error de discretización y un error de redondeo que le son propios y que deben acotarse por separado.

Entre la verificación y la validación se sitúa la calibración, que ajusta los parámetros que no pudieron medirse de forma independiente para que la respuesta reproduzca un conjunto de observaciones. Calibrar no es validar, y usar los mismos datos para ambas cosas invalida la conclusión, porque cualquier modelo con suficientes parámetros libres reproduce cualquier conjunto de datos. La regla mínima consiste en reservar observaciones que no participaron del ajuste, y el tratamiento riguroso de esta separación ocupa el Capítulo 4 [2]. La Figura 1.7 muestra las tres actividades sobre conjuntos de datos disjuntos y recoge la regla de diagnóstico que indica, cuando una de ellas falla, a qué etapa se regresa.

La octava etapa analiza y comunica. Su entregable no es un archivo de resultados sino un documento que responde la pregunta inicial, declara las hipótesis, reporta la incertidumbre y delimita el dominio de validez, y un modelo que no se comunica en esos términos no ha terminado el ciclo. La Tabla 1.2 reúne las ocho etapas con su entregable, su criterio de salida y el error que cada una controla.

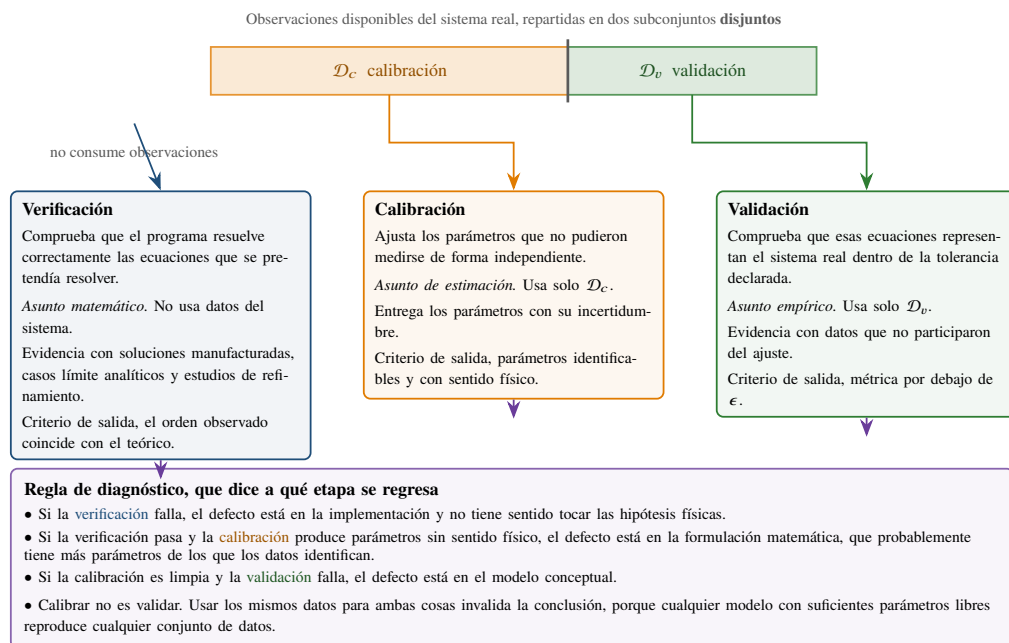


Figura 1.7. Verificación, calibración y validación sobre conjuntos de datos disjuntos, con la regla de diagnóstico que indica a qué etapa se regresa.

Los lazos de realimentación codifican una regla de diagnóstico que ahorra semanas de trabajo. Si la verificación falla, el problema está en la implementación y no tiene sentido tocar las hipótesis físicas. Si la verificación pasa y la calibración produce parámetros sin sentido físico, el problema está en la formulación matemática, que probablemente tiene más parámetros de los que los datos identifican. Si la calibración es limpia y la validación falla, el problema está en el modelo conceptual. El [Algoritmo 1.1](#) escribe ese protocolo en forma ejecutable.

La estructura de repetición no es un adorno, pues el ciclo se recorre varias veces antes de que el modelo sea utilizable y cada vuelta suele empezar por descubrir que la pregunta inicial estaba mal planteada. Conviene por eso mantener desde el principio el registro de versiones del código y de los datos, ya que la trazabilidad entre una versión del modelo y los resultados que produjo es la única defensa disponible cuando alguien pregunta, dos años después, de dónde salió una cifra [11].

Conviene recorrer el ciclo sobre un problema de programación de riego, elegido porque su modelo es lo bastante simple para exhibir todas las etapas. Un lote de maíz de 1 ha en el departamento de Sucre se riega por aspersión sobre un suelo franco arenoso, con capacidad de campo 0.24, punto de marchitez permanente 0.11 y profundidad radicular efectiva de 0.60 m. La evapotranspiración de referencia es de 5.2 mm/d, el coeficiente de cultivo vale 1.05 y la fracción de agotamiento admisible es 0.55. El modelo conceptual

Tabla 1.2. Etapas del ciclo de modelación, con el entregable de cada una, el criterio que autoriza el paso a la siguiente y el error que se controla.

Etapas	Entregable	Criterio de salida	Error controlado
Formulación	Pregunta escrita con tolerancia y horizonte	Un tercero deduce qué se decide	De alcance
Conceptualización	Diagrama de frontera y registro de supuestos	Cada supuesto tiene prueba prevista	Estructural
Formulación matemática	Ecuaciones, condiciones auxiliares y nomenclatura	Grados de libertad nulos y coherencia dimensional	De formulación
Implementación	Programa con datos de prueba y control de versiones	Reproduce casos límite de solución conocida	De programación
Verificación	Estudio de refinamiento y orden observado	El orden observado coincide con el teórico	Numérico
Calibración	Parámetros estimados con su incertidumbre	Parámetros identificables y admisibles	De parámetros
Validación	Comparación contra datos independientes	Métrica dentro de la tolerancia declarada	Estructural residual
Comunicación	Informe con incertidumbre y dominio de validez	La pregunta inicial queda respondida	De interpretación

^a Incumplir un criterio de salida obliga a regresar a la etapa que la [Figura 1.6](#) indica.

es un depósito con una entrada, una salida y un rebose, y su formulación matemática es el balance diario de agotamiento de la zona radicular que recoge la Ecuación (1.3),

$$D_{r,i} = D_{r,i-1} + ET_{c,i} - P_i - I_i + DP_i, \quad (1.3)$$

Algoritmo 1.1 Protocolo de construcción de un modelo con sus criterios de salida.

Entradas: Pregunta de ingeniería $\mathcal{P}$, tolerancia admisible ϵ , datos disponibles $\mathcal{D}$

Salidas: Modelo utilizable $\mathcal{M}$ con su dominio de validez, o el diagnóstico de por qué no lo hay

- 1: Particionar $\mathcal{D}$ en un conjunto de calibración $\mathcal{D}_c$ y uno de validación $\mathcal{D}_v$ disjuntos
 - 2: **repetir**
 - 3: Redactar el alcance, construir el modelo conceptual y registrar cada supuesto
 - 4: Escribir las ecuaciones y verificar coherencia dimensional y grados de libertad nulos
 - 5: Implementar el modelo y estimar el orden observado por refinamiento
 - 6: **si** el orden observado no coincide con el teórico **entonces**
 - 7: Corregir la implementación y repetir la verificación
 - 8: **fin si**
 - 9: Calibrar los parámetros no medibles usando solo $\mathcal{D}_c$
 - 10: **si** algún parámetro carece de sentido físico o no es identificable **entonces**
 - 11: Reducir la complejidad de la formulación y volver al paso 3
 - 12: **fin si**
 - 13: Evaluar la métrica de desempeño sobre $\mathcal{D}_v$
 - 14: **hasta que** la métrica sobre $\mathcal{D}_v$ es menor que ϵ **o** se agota el presupuesto
 - 15: **devolver** $\mathcal{M}$, sus supuestos, su incertidumbre y su dominio de validez
-

donde $D_{r,i}$ es el agotamiento de la zona radicular al final del día i (mm); $ET_{c,i}$ es la evapotranspiración del cultivo (mm); P_i es la lluvia efectiva (mm); I_i es la lámina neta de riego aplicada (mm); y DP_i es la percolación profunda (mm), que se activa solo cuando el balance arrojaría un agotamiento negativo. El agua total disponible es $TAW = 1000 (\theta_{CC} - \theta_{PMP})$ $Z_r = 78.0$ mm y el umbral de riego $RAW = p TAW = 42.9$ mm, con θ_{CC} y θ_{PMP} los contenidos volumétricos a capacidad de campo y a punto de marchitez, Z_r la profundidad radicular en m y p la fracción admisible.

La implementación recorre el balance día a día, según muestra el [Listado 1.2](#).

Listado 1.2. Balance hídrico diario de la zona radicular y programación del riego.

```
1 def programar_riego(agotamiento_inicial, etc_diaria, lluvia,
2   umbral,
3   dias=30):
4     agotamiento, percolacion = agotamiento_inicial, 0.0
5     for dia in range(1, dias + 1):
6         balance = agotamiento + etc_diaria - lluvia.get(dia, 0.0)
7         percolacion += max(0.0, -balance)
8         agotamiento = max(0.0, balance)
9         if agotamiento > umbral:
10             return dia, agotamiento, percolacion
```

```

10     return None, agotamiento, percolacion
11
12     dia, neto, percolacion = programar_riego(
13         12.0, 1.05 * 5.2, {3: 18.0}, 0.55 * 1000 * (0.24 - 0.11) *
14         0.60)
15     bruto = neto / 0.75

```

Partiendo de un agotamiento inicial de 12 mm y con una lluvia efectiva de 18 mm el tercer día, el umbral se supera el noveno día con un agotamiento de 43.14 mm. La lámina bruta, con una eficiencia de aplicación de 0.75, asciende a 57.52 mm, esto es 575.2 m³ por hectárea. Sin esa lluvia el riego habría correspondido al sexto día. La verificación es inmediata, pues el balance debe cerrar de forma exacta, y en efecto $9 \times 5.46 \text{ mm} - 18 \text{ mm} = 31.14 \text{ mm}$ coincide con la diferencia entre el agotamiento final y el inicial. Un cierre distinto de cero delataría un error de programación antes que un fenómeno físico.

El modelo resultante es de parámetros concentrados, determinístico, dinámico, de estado continuo con conmutación en el rebose y de origen híbrido, pues el balance es mecanicista y el coeficiente de cultivo es empírico [12]. Su validación exigiría contenidos de humedad medidos con sonda y su calibración se reduce a ajustar ese coeficiente. Adviértase que la lluvia entró como dato observado y no como predicción, lo cual limita el uso del resultado a la programación de muy corto plazo.

El recorrido deja al descubierto un asunto que hasta ahora se ha usado sin nombrar. El agotamiento es un estado, la evapotranspiración es una entrada, la fracción admisible es un parámetro y la lluvia es una perturbación, y el modelo solo funciona porque cada magnitud ocupa uno de esos papeles y no otro. Ese reparto es el objeto de la sección siguiente.

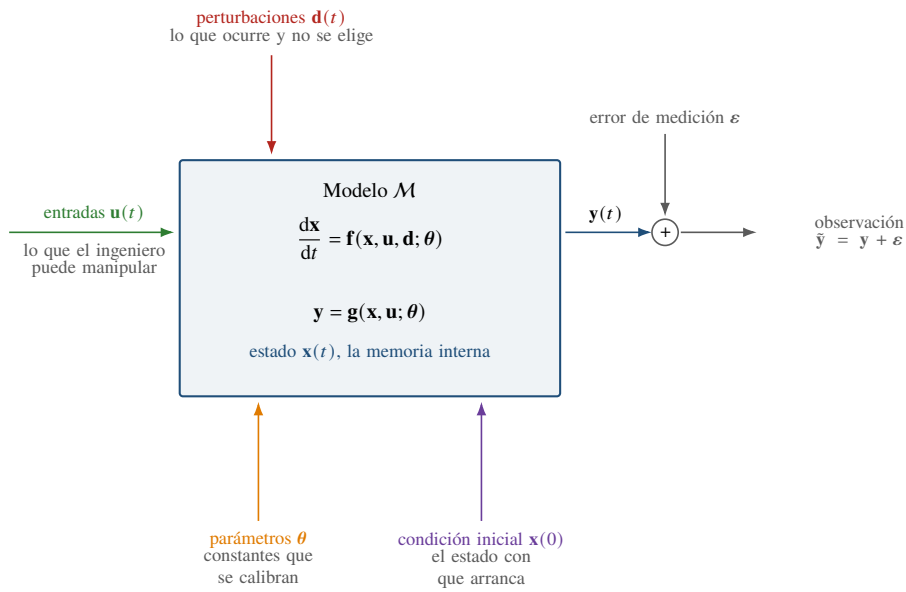
1.4. Variables, parámetros, supuestos y condiciones de frontera

Casi todos los modelos dinámicos de la ingeniería, con independencia del dominio, admiten una misma forma canónica. Escribirla explícitamente obliga a clasificar cada magnitud del problema y evita la confusión más común del principiante, que consiste en tratar como parámetro algo que evoluciona o como entrada algo que el sistema mismo determina. La Ecuación (1.4) la expresa en su forma general,

$$\frac{dx}{dt} = f(x, u, d; \theta), \quad y = g(x, u; \theta), \quad x(t_0) = x_0, \quad (1.4)$$

donde $\mathbf{x}$ es el vector de estado, que reúne las variables cuya evolución el modelo resuelve; $\mathbf{u}$ es el vector de entradas manipulables por el ingeniero; $\mathbf{d}$ es el vector de perturbaciones, que actúan sobre el sistema sin que puedan elegirse; $\boldsymbol{\theta}$ es el vector de parámetros, constantes durante la simulación; $\mathbf{y}$ es el vector de salidas de interés; $\mathbf{f}$ y $\mathbf{g}$ son las funciones que definen la dinámica y la observación; y $\mathbf{x}_0$ es la condición inicial.

La Figura 1.8 despliega la Ecuación (1.4) como diagrama de bloques y añade un elemento que la ecuación no contiene, que es el error de medición. La distinción entre la salida del modelo y la observación disponible es indispensable para entender la calibración, porque lo que se compara nunca es el estado verdadero sino una medición contaminada.



El estatus de una variable depende del alcance elegido. Lo que en un modelo es un parámetro puede ser un estado en otro de mayor resolución.

Figura 1.8. Clasificación de las variables de un modelo sobre su diagrama de bloques, con la distinción entre la salida del modelo y la observación efectivamente disponible.

Definición 1.9. Variable de estado. Una *variable de estado* es una magnitud cuyo valor en un instante, junto con las entradas y las perturbaciones posteriores, determina de forma única la evolución futura del sistema. El conjunto mínimo de tales magnitudes constituye el vector de estado y su cardinalidad es el orden del modelo.

La condición de minimalidad es esencial y con frecuencia se pasa por alto. Si al vector de estado se le agregan magnitudes que son combinación algebraica de las demás, el sistema deja de tener solución única y el integrador falla o converge a resultados dependientes de la tolerancia, como ocurre al incluir a la vez la masa de soluto, el volumen y la concentración de un tanque. La forma canónica se escribe en código con la misma separación de papeles, según muestra el [Listado 1.3](#).

Listado 1.3. Forma canónica de un modelo dinámico, con separación explícita de estados, entradas, perturbaciones y parámetros.

```

1 from scipy.integrate import solve_ivp
2
3 def dinamica(t, x, entradas, perturbaciones, parametros):
4     concentracion, = x                                # vector de estado
5     caudal = entradas(t)                               # manipulable
6     entrada = perturbaciones(t)                       # no se elige
7     volumen, constante = parametros
8     return [caudal / volumen * (entrada - concentracion)
9             - constante * concentracion]
10
11 def simular(x0, entradas, perturbaciones, parametros, horizonte):
12     return solve_ivp(dinamica, (0.0, horizonte), x0,
13                      method="LSODA",
14                      args=(entradas, perturbaciones, parametros),
15                      dense_output=True, rtol=1e-8, atol=1e-10)

```

El estatus de una magnitud no es una propiedad del mundo sino del alcance elegido. En un modelo de operación diaria de una planta de tratamiento la concentración de biomasa es un parámetro estable, en un modelo del arranque de esa misma planta es un estado que crece según una cinética de saturación, y en un modelo de diseño es una variable de decisión. Ninguna de las tres lecturas es incorrecta, y confundirlas produce modelos que responden preguntas distintas de las que se les hizo.

Todo modelo descansa además sobre supuestos, y la disciplina profesional exige que estén escritos, pues un supuesto no declarado no desaparece sino que se vuelve inauditable. El registro de supuestos es un entregable de cuatro columnas, que son el enunciado, la razón por la cual se adopta, el efecto esperado si resultara falso y la prueba con la que podría contrastarse. La [Tabla 1.3](#) lo muestra para el modelo del canal que se resuelve enseguida.

Antes de programar conviene comprobar que la formulación está bien planteada, y el instrumento es el conteo de grados de libertad, una comprobación de treinta segundos que evita horas de depuración. Un sistema con grados de libertad positivos no tiene solución única y uno con grados negativos está sobredeterminado y no tiene solución alguna.

Tabla 1.3. Registro de supuestos del modelo de flujo uniforme en un canal trapezoidal.

Supuesto	Razón	Efecto si es falso	Prueba
Flujo uniforme y permanente	El tramo es prismático y largo frente al tirante	El tirante real varía a lo largo del tramo	Perfil de flujo gradualmente variado
Rugosidad constante en el perímetro	Revestimiento homogéneo de concreto	Subestimación del tirante en la zona vegetada	Aforo con medición de tirante
Velocidad uniforme en la sección	Canal ancho y poco profundo	Error en la energía específica y en el resalto	Perfil con molinete
Sección estable en el tiempo	Canal revestido sin arrastre	Cambio de la relación entre tirante y caudal	Batimetría periódica de la sección

^a La última columna convierte cada supuesto en una hipótesis contrastable, que es lo que distingue un supuesto de una excusa.

Definición 1.10. Grados de libertad. Los *grados de libertad* de un modelo estacionario son la diferencia entre el número de incógnitas independientes y el número de ecuaciones independientes que las vinculan. El modelo queda bien planteado cuando esa diferencia se anula tras especificar las variables de diseño y las condiciones de operación.

El conteo tiene dos trampas frecuentes, que son contar como independientes ecuaciones que no lo son, como ocurre al escribir los balances de todos los componentes de una mezcla junto con el balance global que es su suma, y olvidar que las relaciones geométricas y constitutivas también son ecuaciones. El [Algoritmo 1.2](#) fija el procedimiento y el [Listado 1.4](#) lo implementa por medio del rango de la matriz jacobiana, calculado con las rutinas de álgebra lineal de NumPy, que detecta las dependencias lineales locales [13].

Listado 1.4. Conteo de grados de libertad mediante el rango del jacobiano de las ecuaciones.

```

1 import numpy as np
2
3 def grados_libertad(residuos, punto, n_especificadas, paso=1e-6):
4     punto = np.asarray(punto, dtype=float)
5     base = np.asarray(residuos(punto), dtype=float)
6     jacobiano = np.empty((base.size, punto.size))
7     for j in range(punto.size):

```

Algoritmo 1.2 Análisis de grados de libertad de un modelo estacionario.

Entradas: Lista de incógnitas $\mathcal{V}$, lista de ecuaciones $\mathcal{E}$, lista de valores especificados $\mathcal{S}$

Salidas: Grados de libertad GL y diagnóstico de la formulación

```
1:  $n \leftarrow |\mathcal{V}|$ 
2:  $m \leftarrow$  rango del jacobiano de  $\mathcal{E}$  respecto de  $\mathcal{V}$  en un punto admisible
3: si  $m < |\mathcal{E}|$  entonces
4:   Reportar que hay  $|\mathcal{E}| - m$  ecuaciones redundantes y eliminarlas
5: fin si
6:  $GL \leftarrow n - m - |\mathcal{S}|$ 
7: si  $GL > 0$  entonces
8:   Reportar que faltan GL especificaciones
9: en otro caso si  $GL < 0$  entonces
10:   Reportar que el sistema está sobredeterminado en  $|GL|$  condiciones
11: en otro caso
12:   Reportar formulación bien planteada y proceder a resolver
13: fin si
14: devolver GL
```

```
8     desplazado = punto.copy()
9     desplazado[j] += paso
10    jacobiano[:, j] = (np.asarray(residuos(desplazado)) -
11                       base) / paso
12    rango = np.linalg.matrix_rank(jacobiano, tol=1e-8)
13    redundantes = base.size - rango
14    return punto.size - rango - n_especificadas, redundantes
```

El [Ejemplo 1.2](#) aplica el conteo a un canal trapezoidal y resuelve después el tirante normal, con lo cual cierra el circuito entre el análisis de la formulación y su solución numérica.

Ejemplo 1.2. Grados de libertad y tirante normal de un canal trapezoidal. Un canal de riego de sección trapezoidal, revestido en concreto, tiene ancho de fondo 2.50 m, taludes con relación horizontal a vertical de 1.5, pendiente longitudinal 0.0008 y coeficiente de rugosidad de Manning 0.025. Debe conducir un caudal de diseño de 12 m³/s. Se pide verificar que la formulación está bien planteada, calcular el tirante normal y clasificar el régimen del flujo.

Consideraciones. 1. Flujo uniforme y permanente, con pendiente de la línea de energía igual a la del fondo; 2. sección prismática y rugosidad uniforme en todo el perímetro mojado; 3. pendiente suficientemente pequeña para que el tirante medido vertical y el medido normal al fondo coincidan; 4. distribución de velocidad uniforme en la sección, con coeficiente de energía unitario; 5. agua limpia sin arrastre de sedimentos que modifique la sección.

Análisis. El modelo consta de la ecuación de resistencia y de las tres relaciones geométricas de la Ecuación (1.5),

$$Q = \frac{1}{n} A R^{2/3} S_0^{1/2}, \quad A = (b + z y) y, \quad P = b + 2 y \sqrt{1 + z^2}, \quad R = \frac{A}{P}, \quad (1.5)$$

donde Q es el caudal (m^3/s); n es el coeficiente de rugosidad de Manning, de unidades $\text{s} \cdot \text{m}^{-1/3}$; A es el área mojada (m^2); R es el radio hidráulico (m); S_0 es la pendiente longitudinal del fondo (adimensional); b es el ancho de fondo (m); z es la relación horizontal a vertical del talud (adimensional); y es el tirante (m); P es el perímetro mojado (m).

Las incógnitas son nueve, esto es, Q , y , b , z , n , S_0 , A , P y R , y las ecuaciones independientes son cuatro. Los grados de libertad valen por tanto cinco, de modo que el problema queda determinado al especificar cinco magnitudes. El enunciado especifica b , z , n , S_0 y Q , y deja y como única incógnita efectiva.

Solución. Sustituyendo las relaciones geométricas en la ecuación de resistencia se obtiene una ecuación no lineal en y , que el Listado 1.5 resuelve por bisección con la garantía de convergencia que ofrece el cambio de signo.

Listado 1.5. Tirante normal de un canal trapezoidal por búsqueda de raíz.

```
1 import numpy as np
2 from scipy.optimize import brentq
3
4 def tirante_normal(caudal, ancho, talud, rugosidad,
5 pendiente):
6     area = lambda y: (ancho + talud * y) * y
7     perimetro = lambda y: ancho + 2 * y * np.sqrt(1 +
8 talud**2)
9     def residuo(y):
10         radio = area(y) / perimetro(y)
11         return area(y) * radio**(2 / 3) * np.sqrt(pendiente)
12         / rugosidad \
13         - caudal
14     return brentq(residuo, 1e-4, 10.0, xtol=1e-12)
15
16 def froude(caudal, tirante, ancho, talud):
```

```

14 area = (ancho + talud * tirante) * tirante
15 espejo = ancho + 2 * talud * tirante
16 return caudal / area / np.sqrt(9.81 * area / espejo)

```

El tirante normal resulta $y_n = 1.888$ m, con área mojada 10.066 m^2 , perímetro mojado 9.307 m, radio hidráulico 1.082 m y velocidad media 1.192 m/s. El número de Froude vale 0.343 y el tirante crítico 1.067 m, de modo que el flujo es subcrítico.

Verificación. Un canal con $y_n > y_c$ es de pendiente suave y admite control aguas abajo, lo que concuerda con el número de Froude calculado. Al variar el coeficiente de rugosidad entre 0.022 y 0.030 , banda habitual para concreto en servicio, el tirante recorre el intervalo de 1.774 m a 2.061 m, esto es que una variación del 32% en n produce una del 15% en el tirante.

Comentario. La sensibilidad calculada tiene consecuencias de diseño inmediatas, pues el borde libre habitual de 0.30 m apenas cubre la banda de incertidumbre del coeficiente de rugosidad. Reportar el tirante como 1.888 m sin acompañarlo de esa banda transmite una precisión que el modelo no posee.

La [Figura 1.9](#) resume el conteo del ejemplo, los tres diagnósticos posibles según el signo del resultado y las dos trampas que lo invalidan, y conviene tenerla a la vista cada vez que se formula un modelo estacionario, porque el conteo es la comprobación más barata de todo el ciclo y la que con más frecuencia se omite.

Conteo sobre el canal trapezoidal del ejemplo

Incógnitas independientes	9
Ecuaciones independientes	4
Valores especificados	5

$GL = n - m - s = 9 - 4 - 5 = 0$, formulación bien planteada.

Antes de especificar nada la formulación tiene $9 - 4 = 5$ grados de libertad, esto es, cinco magnitudes que hay que fijar. Tras fijarlas, el tirante y queda como única incógnita efectiva y el problema se cierra con una búsqueda de raíz.

$GL > 0$ faltan especificaciones

El sistema no tiene solución única y el resolutor entrega la que dependa del punto de partida.

$GL = 0$ bien planteado

Tantas ecuaciones independientes como incógnitas libres. Se procede a resolver.

$GL < 0$ sobredeterminado

Hay $|GL|$ condiciones de más y no existe solución alguna que las satisfaga todas.

Las dos trampas que invalidan el conteo

- Contar como independientes ecuaciones que no lo son, como al escribir los balances de todos los componentes de una mezcla junto con el balance global, que es su suma. El rango del jacobiano delata la redundancia.
- Olvidar que las relaciones geométricas y las constitutivas también son ecuaciones. En el canal, el área, el perímetro y el radio hidráulico aportan tres de las cuatro.

Figura 1.9. Conteo de grados de libertad de un modelo estacionario, los tres diagnósticos posibles y las dos trampas que invalidan el conteo.

Los valores tabulados del coeficiente de rugosidad provienen de campañas de aforo y llevan asociada esa dispersión [14]. Cuando el modelo pasa a resolver gradientes espaciales la condición inicial ya no basta, y aparecen las condiciones de frontera, que son la parte de la formulación donde se concentran los errores más difíciles de diagnosticar, porque un problema mal planteado en la frontera produce resultados que parecen razonables y no lo son.

Definición 1.11. Condición de frontera. Una *condición de frontera* es una relación que la solución de una ecuación en derivadas parciales debe satisfacer sobre el contorno del dominio de cálculo. Se denomina de Dirichlet cuando prescribe el valor de la variable, de Neumann cuando prescribe su derivada normal y de Robin cuando impone una combinación lineal de ambas.

El significado físico de los tres tipos es directo. La de Dirichlet corresponde a un nivel impuesto por un elemento externo de capacidad muy grande, como un embalse que fija el potencial en el borde de un acuífero o un baño termostático. La de Neumann corresponde a un flujo impuesto, y su caso homogéneo describe una pared impermeable, un aislamiento perfecto o un plano de simetría. La de Robin describe el intercambio con un medio cuya resistencia no es despreciable, como la convección sobre una superficie o la infiltración a través de una capa semiconfinante. La Figura 1.10 las sitúa sobre un dominio genérico.

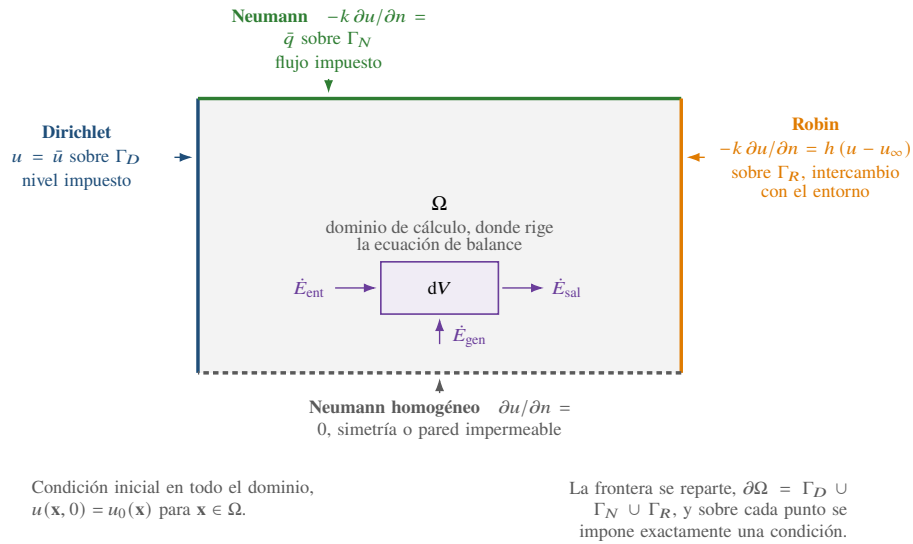


Figura 1.10. Volumen de control genérico dentro del dominio de cálculo y los tres tipos de condición de frontera con su interpretación física.

El error más frecuente consiste en imponer condiciones incompatibles. Un problema de difusión con todas las fronteras aisladas y sin término fuente no tiene solución única, porque cualquier constante aditiva satisface las mismas ecuaciones, y el síntoma en el computador es una matriz singular. El error simétrico consiste en imponer Dirichlet donde el sistema exige un flujo, con lo cual el modelo entrega el flujo que haga falta para sostener el valor prescrito, aunque sea físicamente imposible. Queda entonces por preguntar si el problema de valor inicial tiene solución y si esa solución es única, cuestión que no es académica, pues un modelo sin unicidad produce resultados que dependen del integrador y del paso, comportamiento que se confunde con un error de programación.

Teorema 1.1. Existencia y unicidad de Picard–Lindelöf. *Sea $\mathbf{f}: [t_0, t_0 + a] \times \overline{B}(\mathbf{x}_0, b) \rightarrow \mathbb{R}^n$ continua y lipschitziana respecto de su segundo argumento, es decir, tal que existe $L > 0$ con $\|\mathbf{f}(t, \mathbf{x}_1) - \mathbf{f}(t, \mathbf{x}_2)\| \leq L\|\mathbf{x}_1 - \mathbf{x}_2\|$ para todo t del intervalo y todo par de puntos de la bola. Si $M = \max\|\mathbf{f}\|$ sobre ese conjunto, entonces el problema de valor inicial $\mathbf{x}' = \mathbf{f}(t, \mathbf{x})$, $\mathbf{x}(t_0) = \mathbf{x}_0$ posee una solución y solo una en el intervalo $[t_0, t_0 + \alpha]$, con $\alpha = \min\{a, b/M\}$.*

La hipótesis que falla en la práctica es la de Lipschitz, y falla de maneras concretas. Una raíz cuadrada del estado cerca del origen, como la de la descarga por un orificio cuando el tanque se vacía, tiene pendiente infinita en cero y admite infinitas soluciones. Un término de la forma x^2 es lipschitziano solo localmente y la solución escapa en tiempo finito, de modo que el teorema garantiza existencia en un intervalo que puede ser mucho menor que el horizonte de la simulación. Una conmutación discontinua, como la de un termostato ideal, viola la continuidad y produce castaño. En los tres casos el remedio es de modelación y no de método numérico, pues consiste en regularizar la expresión o en tratar la conmutación como un evento explícito [15].

1.5. Análisis de escalas y simplificación justificada

Decidir qué se desprecia es la operación más delicada de la modelación y también la que con más frecuencia se resuelve por costumbre. La alternativa disciplinada consiste en escribir las ecuaciones en variables adimensionales, porque así cada término queda multiplicado por un grupo cuyo valor numérico indica su importancia relativa, y despreciar un término deja de ser una licencia para convertirse en una desigualdad verificable.

Definición 1.12. Adimensionalización. La *adimensionalización* es el procedimiento que consiste en referir cada variable de un modelo a una escala característica del propio problema, de modo que las variables resultantes son de orden unitario y los coeficientes de la ecuación se agrupan en un número reducido de parámetros adimensionales que gobiernan la solución.

El procedimiento se aprecia sobre la conducción transitoria en una placa de semiespesor L enfriada por convección. Partiendo de $\partial T/\partial t = \alpha \partial^2 T/\partial x^2$, con $-k \partial T/\partial x = h(T - T_\infty)$ en la superficie y simetría en el plano central, se definen $\theta = (T - T_\infty)/(T_i - T_\infty)$, $\xi = x/L$ y $Fo = \alpha t/L^2$, con lo cual el problema se transforma en la Ecuación (1.6),

$$\frac{\partial \theta}{\partial Fo} = \frac{\partial^2 \theta}{\partial \xi^2}, \quad - \left. \frac{\partial \theta}{\partial \xi} \right|_{\xi=1} = Bi \theta(1, Fo), \quad \theta(\xi, 0) = 1, \quad (1.6)$$

donde θ es la temperatura reducida (adimensional); ξ es la coordenada reducida (adimensional); $Fo = \alpha t/L^2$ es el número de Fourier (adimensional); $Bi = hL/k$ es el número de Biot (adimensional); α es la difusividad térmica (m^2/s); h es el coeficiente convectivo ($W/(m^2 K)$); y k es la conductividad térmica del sólido ($W/(m K)$).

El resultado tiene consecuencias que ninguna cantidad de simulaciones dimensionales habría revelado, pues toda la familia de problemas de enfriamiento de placas, con cualquier material, espesor y coeficiente convectivo, se reduce a una única familia de curvas gobernada por un solo número. Cuando Bi es pequeño la condición de frontera indica que el gradiente interno también lo es y el sólido puede tratarse como isotérmico [16]. El [Algoritmo 1.3](#) generaliza el procedimiento.

El último paso no es opcional, pues un grupo pequeño indica que un término es pequeño en la ecuación, no que su efecto acumulado sobre la respuesta lo sea, y solo la comparación entre el modelo reducido y el completo cuantifica esa diferencia. Cuando la ecuación gobernante no se conoce, el análisis dimensional sigue siendo posible por medio del resultado clásico de Buckingham [17].

Teorema 1.2. Teorema Π de Buckingham. Sea $F(q_1, q_2, \dots, q_n) = 0$ una relación físicamente significativa entre n magnitudes dimensionales, y sea k el rango de la matriz dimensional cuyas columnas son los exponentes de las dimensiones fundamentales en cada magnitud. Entonces la relación puede reescribirse de manera equivalente como $\Phi(\Pi_1, \dots, \Pi_{n-k}) = 0$, donde los Π_i son $n - k$ productos de potencias adimensionales e independientes de las magnitudes originales.

Algoritmo 1.3 Adimensionalización de un modelo y jerarquización de sus términos.

Entradas: Ecuación gobernante con sus condiciones auxiliares, rango esperado de cada magnitud

Salidas: Ecuación adimensional y lista de términos despreciables con su cota de error

- 1: Elegir una escala característica para cada variable independiente y dependiente
 - 2: Sustituir cada variable por el producto de su escala y una variable reducida
 - 3: Dividir la ecuación por el coeficiente del término que se sabe dominante
 - 4: Identificar los grupos adimensionales que multiplican a los demás términos
 - 5: **para** cada grupo Π_i **hacer**
 - 6: Evaluar Π_i con el rango esperado de las magnitudes
 - 7: **si** $\Pi_i \ll 1$ en todo el rango **entonces**
 - 8: Marcar el término como candidato a despreciarse
 - 9: **fin si**
 - 10: **fin para**
 - 11: Resolver el modelo reducido y el completo en un caso de prueba
 - 12: **devolver** la diferencia entre ambos como cota del error de la simplificación
-

La lectura práctica del teorema es doble. Reduce el número de variables de un programa experimental, pues una campaña que exigiría explorar n magnitudes se reduce a explorar $n - k$ grupos, y advierte que k es el rango de la matriz dimensional y no el número de dimensiones fundamentales presentes, distinción que importa cuando alguna dimensión aparece siempre en la misma combinación. El [Listado 1.6](#) calcula el número de grupos y sus exponentes con álgebra simbólica [18].

Listado 1.6. Número y forma de los grupos adimensionales a partir de la matriz dimensional.

```
1 import sympy as sp
2
3 # Filas M, L, T; columnas gH, Q, N, D, rho, mu
4 dimensional = sp.Matrix([[0, 0, 0, 0, 1, 1],
5                          [2, 3, 0, 1, -3, -1],
6                          [-2, -1, -1, 0, 0, -1]])
7 nombres = ["gH", "Q", "N", "D", "rho", "mu"]
8 repetidas, restantes = [4, 2, 3], [0, 1, 5] # rho, N, D como
   base
9
10 rango = dimensional.rank()
11 n_grupos = dimensional.shape[1] - rango
12 base = dimensional[:, repetidas]
13 for j in restantes:
14     exponentes = base.solve(-dimensional[:, j])
15     print(nombres[j], [sp.nsimplify(e) for e in exponentes])
```

El listado resuelve, para cada magnitud no repetida, el sistema lineal que anula las dimensiones del producto correspondiente. La elección de las repetidas es libre mientras su submatriz sea de rango completo, y determina la forma de los grupos sin alterar su número. El [Ejemplo 1.3](#) lo aplica a una bomba centrífuga para predecir su comportamiento a una velocidad no ensayada.

Ejemplo 1.3. Grupos adimensionales y semejanza de una bomba centrífuga.

Una bomba centrífuga de rodete 200 mm se ensaya con agua a 20 °C girando a 1750 rpm, y en su punto de mejor rendimiento entrega 22 L/s contra una altura de 18 m. Se pide predecir el caudal, la altura y la potencia hidráulica del mismo rodete girando a 2400 rpm, y justificar bajo qué condición la predicción es válida.

Consideraciones. 1. Las magnitudes relevantes son la energía específica gH , el caudal Q , la velocidad de giro N , el diámetro del rodete D , la densidad ρ y la viscosidad dinámica μ ; 2. la geometría es idéntica en ambos puntos, de modo que la semejanza geométrica es exacta; 3. no hay cavitación y el rendimiento hidráulico se mantiene constante entre ambas velocidades.

Análisis. Con seis magnitudes y una matriz dimensional de rango tres, el [Teorema 1.2](#) anticipa tres grupos independientes. Tomando ρ , N y D como repetidas, el [Listado 1.6](#) entrega los grupos de la Ecuación (1.7),

$$\Pi_1 = \frac{Q}{ND^3}, \quad \Pi_2 = \frac{gH}{N^2D^2}, \quad \Pi_3 = \frac{\rho ND^2}{\mu}, \quad (1.7)$$

donde Π_1 es el coeficiente de caudal (adimensional); Π_2 es el coeficiente de altura (adimensional); Π_3 es el número de Reynolds del rodete (adimensional); N es la velocidad de giro (1/s); D es el diámetro del rodete (m); μ es la viscosidad dinámica (Pa s); y g es la aceleración de la gravedad (m/s^2).

Solución. La semejanza exige que Π_1 y Π_2 se conserven. De la primera condición se sigue $Q_2 = Q_1 (N_2/N_1) = 30.17 \text{ L/s}$ y de la segunda $H_2 = H_1 (N_2/N_1)^2 = 33.85 \text{ m}$. La potencia hidráulica ρgQH escala entonces con el cubo de la velocidad, de modo que pasa de 3.88 kW a 10.00 kW, con una razón de 2.579 que coincide con $(N_2/N_1)^3$.

Verificación. El tercer grupo no se conserva con el mismo fluido y el mismo rodete, de manera que la predicción solo vale si el número de Reynolds es lo bastante alto para que la respuesta no dependa de él. Se obtiene $\Pi_3 = 1.16 \times 10^6$ a 1750 rpm y 1.59×10^6 a 2400 rpm, ambos muy por encima del umbral de 10^5 , mientras que $\Pi_1 = 0.0943$ y $\Pi_2 = 5.189$ coinciden en los dos puntos.

Comentario. Las leyes de semejanza de las bombas no son una regla empírica sino una consecuencia del teorema Π más la hipótesis de independencia respecto del número de Reynolds, y enunciarlas así deja explícita la condición bajo la cual dejan de valer, que es la de las bombas pequeñas o los fluidos viscosos.

Un puñado de grupos adimensionales resuelve la mayor parte de las decisiones de simplificación de los dominios de la Facultad, y la [Tabla 1.4](#) los reúne con su definición, su lectura y su umbral de decisión.

Tabla 1.4. Grupos adimensionales de uso frecuente, con la razón de magnitudes que expresan y el umbral de decisión asociado.

Grupo	Definición	Razón que expresa	Umbral de decisión
Reynolds	$\rho u L / \mu$	Inercia frente a viscosidad	Transición cerca de 2300 y turbulencia plena sobre 4000
Froude	$u / \sqrt{g D_h}$	Inercia frente a gravedad	Subcrítico si es menor que 1, supercrítico si es mayor
Biot	$h L_c / k$	Resistencia interna frente a externa	Sólido isotérmico si es menor que 0.1
Fourier	$\alpha t / L_c^2$	Difusión frente a almacenamiento	Serie de un término si es mayor que 0.2
Péclet	$u L / D_L$	Advección frente a dispersión	Flujo pistón si supera 500, mezcla completa si es menor que 5
Damköhler	$k \tau$	Reacción frente a residencia	La reacción gobierna si es mayor que 1
Courant	$u \Delta t / \Delta x$	Avance por paso frente a celda	Estabilidad de esquemas explícitos si no supera 1

^a Los umbrales son convenciones y no fronteras exactas, de modo que cerca de ellos conviene resolver el modelo completo.

El umbral del número de Biot merece un examen detenido, porque es el que más se invoca y el que más se aplica sin comprobar. La afirmación de que un sólido con $Bi < 0.1$ puede tratarse como isotérmico proviene de comparar la resistencia interna con la externa, y su costo en exactitud es acotado pero no nulo [19], según cuantifica el [Ejemplo 1.4](#) sobre un proceso de enfriamiento de alimentos.

Ejemplo 1.4. Costo de la hipótesis de resistencia interna despreciable en el enfriamiento de un producto. Una lámina de pulpa de fruta concentrada de 12 mm de espesor se enfría por sus dos caras en una cámara con aire a 20 °C, desde una temperatura inicial uniforme de 80 °C. El producto tiene conductividad térmica 0.52 W/(m K), densidad 1080 kg/m³ y calor específico 3600 J/(kg K), y el coeficiente convectivo es de 8.0 W/(m² K). Se pide el tiempo necesario para que la temperatura descienda hasta 32 °C y el error que introduce suponer resistencia interna despreciable.

Consideraciones. 1. Propiedades térmicas constantes en el intervalo considerado; 2. la lámina es lo bastante extensa para tratarla como placa infinita, con conducción unidimensional; 3. coeficiente convectivo uniforme sobre ambas caras y temperatura del aire constante; 4. sin cambio de fase ni migración de humedad.

Análisis. La longitud característica es el semiespesor, $L_c = 6$ mm, de suerte que $Bi = hL_c/k = 0.0923$ y la difusividad térmica vale $\alpha = k/(\rho c_p) = 1.337 \times 10^{-7}$ m²/s. El modelo de resistencia interna despreciable conduce a un decaimiento exponencial con constante de tiempo $\tau = \rho c_p L_c/h$, mientras que la solución exacta es la serie de la Ecuación (1.8), obtenida por separación de variables sobre la Ecuación (1.6),

$$\theta_0(Fo) = \sum_{m=1}^{\infty} C_m e^{-\zeta_m^2 Fo}, \quad \zeta_m \tan \zeta_m = Bi, \quad C_m = \frac{4 \sin \zeta_m}{2\zeta_m + \sin 2\zeta_m}, \quad (1.8)$$

donde θ_0 es la temperatura reducida del plano central (adimensional); ζ_m son las raíces positivas de la ecuación característica (adimensionales); y C_m son los coeficientes de la serie (adimensionales).

Solución. La temperatura reducida objetivo es $\theta = (32 - 20)/(80 - 20) = 0.20$. Con $\tau = 2916$ s, el modelo isotérmico predice $t = -\tau \ln \theta = 4693$ s, esto es 78.2 min. La solución exacta, con $\zeta_1 = 0.2992$ y $C_1 = 1.0149$, exige $Fo = 18.14$ y por tanto 4883 s, es decir 81.4 min. El Listado 1.7 realiza ambos cálculos.

Listado 1.7. Evaluación del número de Biot y comparación del modelo isotérmico con la serie exacta.

```
1 import numpy as np
2 from scipy.optimize import brentq
3
4 def theta_centro(biot, fourier, terminos=40):
5     z = np.array([brentq(lambda x: x * np.tan(x) - biot,
6                         m * np.pi + 1e-13, (m + 0.5) *
7                         np.pi - 1e-13)
```

```

7         for m in range(terminos)]]
8     c = 4 * np.sin(z) / (2 * z + np.sin(2 * z))
9     return float(np.sum(c * np.exp(-z**2 * fourier)))
10
11 def tiempos(h, k, rho, cp, semiespesor, objetivo):
12     biot, alfa = h * semiespesor / k, k / (rho * cp)
13     isotermico = -(rho * cp * semiespesor / h) *
14         np.log(objetivo)
15     fourier = brentq(lambda f: theta_centro(biot, f) -
16         objetivo, 1e-6, 500)
17     return biot, isotermico, fourier * semiespesor**2 / alfa

```

Verificación. La hipótesis subestima el tiempo en 190 s, esto es un 3.9 %, de suerte que al cabo de los 78.2 min que predice el modelo isotérmico el plano central está todavía a 32.8 °C. La [Figura 1.11](#) extiende el cálculo y muestra que el error se mantiene bajo el 5 % mientras $Bi < 0.1$ y crece hasta el 18 % cuando Bi alcanza 0.5.

Comentario. Ese error es despreciable frente a la variabilidad del espesor de lámina en una línea real, de modo que la simplificación está justificada. La conclusión cambiaría por completo si el producto se enfriara por inmersión en agua, pues el coeficiente convectivo subiría dos órdenes de magnitud y con él el número de Biot.

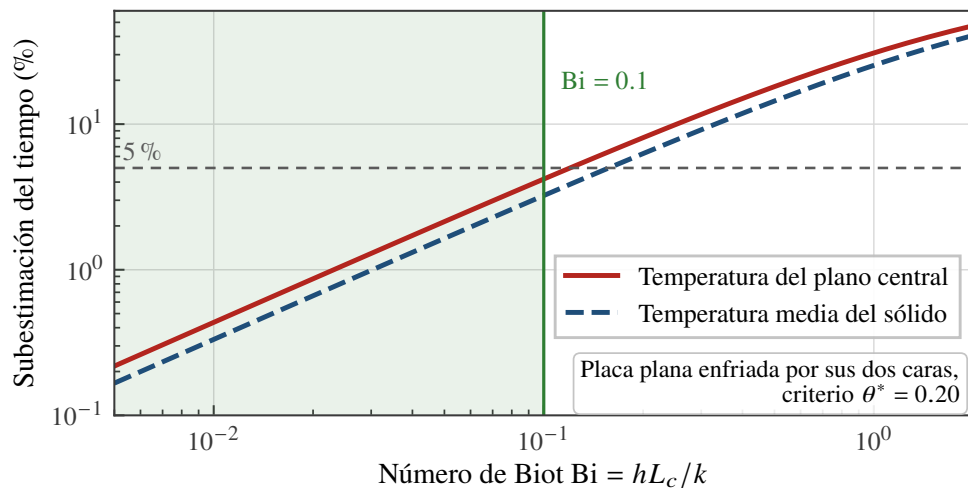


Figura 1.11. Error del modelo de resistencia interna despreciable en el tiempo de enfriamiento de una placa, en función del número de Biot, para un descenso hasta el veinte por ciento del salto térmico inicial.

La otra gran fuente de simplificación legítima es la linealidad, que permite construir soluciones complejas sumando soluciones elementales y cubre desde la superposición de descensos en un campo de pozos hasta la respuesta de un circuito a varias fuentes.

Teorema 1.3. Principio de superposición. Sean $\mathcal{L}$ un operador lineal y $\mathcal{B}$ un operador de frontera también lineal. Si u_i satisface $\mathcal{L}u_i = f_i$ en el dominio y $\mathcal{B}u_i = g_i$ en su contorno para $i = 1, \dots, m$, entonces, para cualesquiera constantes α_i , la combinación $u = \sum_i \alpha_i u_i$ satisface $\mathcal{L}u = \sum_i \alpha_i f_i$ con $\mathcal{B}u = \sum_i \alpha_i g_i$.

El valor práctico del principio es doble, pues permite resolver problemas con varias fuentes evaluando una sola solución elemental y proporciona soluciones analíticas de referencia con las cuales verificar un código numérico. Su límite es igualmente claro, ya que basta un término no lineal para invalidarlo. En un acuífero confinado la transmisividad no depende del descenso y la superposición es exacta, mientras que en uno libre el espesor saturado disminuye al descender el nivel, la ecuación se vuelve no lineal y la suma de descensos individuales subestima el conjunto. La solución elemental que se superpone en el caso confinado es la de Theis [20], cuya aproximación logarítmica para argumentos pequeños se debe a Cooper y Jacob [21].

Ejemplo 1.5. Superposición de descensos en un campo de dos pozos de bombeo. Un acuífero confinado de transmisividad $250 \text{ m}^2/\text{d}$ y coeficiente de almacenamiento 1.5×10^{-4} se explota con dos pozos que bombean de forma continua $900 \text{ m}^3/\text{d}$ y $1200 \text{ m}^3/\text{d}$. Se requiere estimar el descenso al cabo de 30 d en un punto de observación situado a 144.2 m del primer pozo y a 197.0 m del segundo.

Consideraciones. 1. Acuífero confinado, homogéneo, isótropo y de extensión infinita; 2. pozos totalmente penetrantes y de diámetro despreciable; 3. caudales constantes desde el instante inicial; 4. transmisividad independiente del descenso, lo cual hace lineal la ecuación de flujo.

Formulación. Para un pozo aislado, la solución de Theis se aproxima por la expresión logarítmica de Cooper y Jacob de la Ecuación (1.9), válida cuando el argumento $u = r^2 S / (4Tt)$ es pequeño,

$$s(r, t) = \frac{2.303 Q}{4\pi T} \log_{10} \left(\frac{2.25 T t}{r^2 S} \right), \quad u = \frac{r^2 S}{4Tt} < 0.01, \quad (1.9)$$

donde s es el descenso del nivel piezométrico (m); Q es el caudal de bombeo (m^3/d); T es la transmisividad (m^2/d); S es el coeficiente de almacenamiento (adimensional); r es la distancia al pozo (m); y t es el tiempo transcurrido desde el inicio del bombeo (d).

Solución. El [Teorema 1.3](#) autoriza sumar los descensos individuales, ya que la ecuación de flujo en acuífero confinado es lineal en el descenso, y el [Listado 1.8](#) realiza la suma.

Listado 1.8. Densco por superposición en un campo de pozos y contraste con la solución de Theis.

```
1 import numpy as np
2 from scipy.special import exp1
3
4 def cooper_jacob(caudal, radio, transmisividad,
5 almacenamiento, tiempo):
6     u = radio**2 * almacenamiento / (4 * transmisividad *
7         tiempo)
8     return 2.303 * caudal / (4 * np.pi * transmisividad) *
9         np.log10(
10             2.25 * transmisividad * tiempo / (radio**2 *
11                 almacenamiento)), u
12
13 def theis(caudal, radio, transmisividad, almacenamiento,
14 tiempo):
15     u = radio**2 * almacenamiento / (4 * transmisividad *
16         tiempo)
17     return caudal / (4 * np.pi * transmisividad) * exp1(u)
18
19 def descenso_total(pozos, *acuifero):
20     return sum(theis(q, r, *acuifero) for q, r in pozos)
```

El primer pozo aporta 2.463 m y el segundo 3.046 m, de modo que el descenso total es de 5.509 m.

Verificación. Los argumentos valen $u_1 = 1.04 \times 10^{-4}$ y $u_2 = 1.94 \times 10^{-4}$, ambos muy inferiores a 0.01, de manera que la aproximación logarítmica es aplicable. La solución exacta de Theis entrega 2.462 m y 3.045 m, con una diferencia relativa de 0.04 % respecto de la aproximación.

Comentario. Si el acuífero fuese libre y el descenso representara una fracción apreciable del espesor saturado, la suma anterior carecería de fundamento y habría que corregir los descensos o resolver directamente el problema no lineal. Comprobar la linealidad antes de superponer es tan importante como saber superponer.

La aproximación logarítmica empleada procede del análisis de Cooper y Jacob sobre la solución de Theis [21]. Una última comprobación no cuesta nada. Cada término de la ecuación gobernante debe tener las mismas dimensiones, y esa verificación se automatiza representando cada magnitud por su vector de exponentes en las dimensiones fundamentales, como hace el [Listado 1.9](#) sobre la ecuación de la energía.

Listado 1.9. Verificación automática de la coherencia dimensional de los términos de un balance.

```

1 import numpy as np
2 BASE = ("M", "L", "T", "K")
3
4 def dimension(**exponentes):
5     return np.array([exponentes.get(s, 0) for s in BASE],
6                     dtype=int)
7
8 def coherente(terminos):
9     vectores = [sum(factoros) for factores in terminos.values()]
10    return all(np.array_equal(v, vectores[0]) for v in vectores)
11
12 terminos = {
13     "rho*cp*dT/dt": [dimension(M=1, L=-3), dimension(L=2, T=-2,
14     K=-1),
15                     dimension(K=1, T=-1)],
16     "k*d2T/dx2": [dimension(M=1, L=1, T=-3, K=-1),
17     dimension(K=1, L=-2)],
18     "h*(T-Tinf)/L": [dimension(M=1, T=-3, K=-1), dimension(K=1),
19     dimension(L=-1)],
20 }

```

Los tres términos devuelven el mismo vector de exponentes, correspondiente a una potencia por unidad de volumen, de suerte que el balance es dimensionalmente coherente. Una comprobación así intercepta el error más común al programar una ecuación tomada de la literatura, que consiste en mezclar formulaciones por unidad de masa con formulaciones por unidad de volumen. Con el modelo simplificado, verificado y dimensionalmente sano, resta preguntarse hasta dónde puede llevarse su respuesta.

1.6. Alcances, limitaciones y responsabilidad del modelador

Todo modelo tiene una región del espacio de condiciones dentro de la cual su respuesta merece crédito. Delimitarla es tan parte del entregable como el resultado mismo, y omitirla es la forma más frecuente de mala práctica profesional, porque un número presentado sin su dominio de validez invita a usarlo donde no vale.

Definición 1.13. Dominio de validez. El *dominio de validez* de un modelo es el conjunto de valores de sus entradas, de sus parámetros y de sus condiciones de operación para los cuales se ha comprobado, contra evidencia independiente, que el error de su respuesta no supera la tolerancia declarada. Fuera de ese conjunto el modelo sigue produciendo números, pero ya no produce predicciones respaldadas.

La distinción importa sobre todo en los modelos empíricos, cuyo respaldo proviene únicamente del rango observado. Una correlación con un coeficiente de determinación excelente puede comportarse de forma absurda pocos pasos fuera de la ventana de calibración, sin que ninguna estadística calculada dentro de ella lo anuncie, según exhibe el [Ejemplo 1.6](#).

Ejemplo 1.6. Costo de extrapolar una correlación empírica de un módulo fotovoltaico. Una campaña de medición sobre un módulo fotovoltaico registra la potencia entregada frente a la irradiancia en el plano del módulo, en diez puntos repartidos entre 700 W/m^2 y 1000 W/m^2 , que son las condiciones del mediodía despejado en que se hizo la campaña. Se ajusta una recta y se pretende usarla para estimar la energía diaria del módulo. Se pide evaluar el error que introduce ese uso.

Consideraciones. 1. Los datos se generan de forma sintética con la semilla 20 262 del curso, a partir de un comportamiento de referencia que incorpora la caída de rendimiento a baja irradiancia y el efecto de la temperatura de celda; 2. la temperatura de celda sigue el modelo de temperatura nominal de operación, con 45°C de valor nominal, aire a 30°C y coeficiente de potencia de $-0.38\%/K$; 3. el perfil diario de irradiancia es un semiseno entre las seis y las dieciocho horas.

Análisis. El ajuste por mínimos cuadrados sobre la ventana medida entrega la recta $P = 0.2644 G + 20.32 \text{ W}$, con coeficiente de determinación 0.9979 y error relativo inferior al 1 % dentro de esa ventana. El término independiente positivo revela el problema de fondo, pues afirma que el módulo entrega 20.3 W con irradiancia nula y que su potencia se anula en -76.9 W/m^2 .

Solución. El [Listado 1.10](#) ajusta la recta y evalúa el error dentro y fuera de la ventana de calibración.

Listado 1.10. Ajuste de una correlación empírica y evaluación de su error al extrapolar.

```
1 import numpy as np
2
3 def ajustar_recta(irradiancia, potencia):
4     pendiente, corte = np.polyfit(irradiancia, potencia, 1)
5     residuo = potencia - (pendiente * irradiancia + corte)
6     varianza = ((potencia - potencia.mean())**2).sum()
7     return pendiente, corte, 1 - (residuo @ residuo) /
8         varianza
9
10 def energia_diaria(potencia, irradiancia_maxima=950.0,
11     puntos=20001):
12     horas = np.linspace(6.0, 18.0, puntos)
13     irradiancia = irradiancia_maxima * np.sin(np.pi * (horas
14         - 6) / 12)
15     return np.trapezoid(np.maximum(potencia(irradiancia),
16         0.0), horas)
```

A 100 W/m^2 la recta sobrestima la potencia en un 63.1 %, y el error se mantiene por encima del 5 % para toda irradiancia inferior a 447 W/m^2 . Al integrar el día completo, la energía estimada supera la de referencia en un 2.9 % en jornada despejada y en un 21.0 % en jornada nublada. La [Figura 1.12](#) muestra ambas cosas.

Comentario. El resultado desmonta la creencia de que un coeficiente de determinación alto certifica un modelo, pues solo informa sobre el ajuste dentro de la ventana medida. Un dimensionamiento de baterías basado en la energía de días nublados, que es el caso crítico, quedaría con un déficit cercano al veinte por ciento por una extrapolación que en el informe pasaría inadvertida.

El error de una predicción no proviene de una sola causa, y por eso perfeccionar un solo ingrediente rara vez mejora el resultado. Conviene descomponerlo en cuatro contribuciones, que son el error estructural, por los procesos que el modelo omite o representa mal, el de parámetros, por la incertidumbre de sus valores, el de datos de entrada, por el error de medición de las entradas y de las condiciones auxiliares, y el numérico, por la discretización y el redondeo [22]. La [Tabla 1.5](#) indica cómo se detecta y cómo se reduce cada uno.

De las cuatro componentes, la de parámetros admite un tratamiento analítico sencillo cuando la incertidumbre es moderada y la respuesta es suave, y es la primera herramienta que conviene aplicar antes de montar una campaña de Monte Carlo.

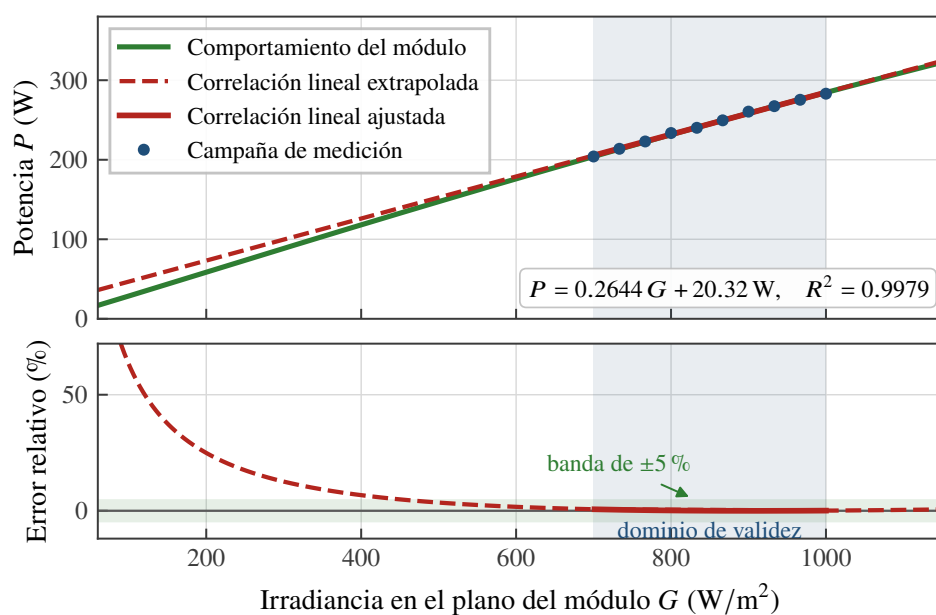


Figura 1.12. Dominio de validez de una correlación empírica lineal ajustada en una ventana estrecha de irradiancia y error relativo que se comete al extrapolarla fuera de esa ventana.

Tabla 1.5. Componentes del error de una predicción, señal que delata a cada uno y acción que efectivamente lo reduce.

Componente	Origen	Señal característica	Acción que lo reduce
Estructural	Procesos omitidos o mal representados	Residuos con patrón sistemático	Revisar el modelo conceptual
De parámetros	Estimación con datos limitados	Intervalos de confianza amplios o correlacionados	Más datos informativos o menos parámetros
De datos de entrada	Instrumentación y series de forzamiento	Dispersión que no cambia con el modelo	Mejor medición y control de calidad
Numérico	Discretización y aritmética finita	El resultado cambia al refinar	Refinar hasta que el cambio sea despreciable

^a Reducir una sola componente por debajo de las demás no mejora la predicción, solo desplaza el costo.

Teorema 1.4. Propagación de la varianza en primer orden. Sea $y = f(\mathbf{x})$ con f de clase C^1 en un entorno de $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$, y sea Σ la matriz de covarianza de $\mathbf{x}$. Entonces, en la aproximación de primer orden del desarrollo de Taylor alrededor de $\boldsymbol{\mu}$, se cumple $\mathbb{E}[y] \approx f(\boldsymbol{\mu})$ y $\text{Var}[y] \approx \nabla f(\boldsymbol{\mu})^\top \Sigma \nabla f(\boldsymbol{\mu})$, es decir que la varianza de la salida es la forma cuadrática de la covarianza de las entradas evaluada en el gradiente.

La aproximación es exacta si f es lineal y se degrada al crecer la curvatura o la dispersión de las entradas, de modo que debe contrastarse con Monte Carlo cuando la incertidumbre es grande [23]. Aplicada al canal del [Ejemplo 1.2](#), con rugosidad de valor esperado 0.025 y desviación estándar 0.0025, la derivada del tirante respecto de la rugosidad vale 36.6 m por unidad de n y arroja una desviación estándar de 0.091 m. Una simulación con cinco mil realizaciones y la semilla del curso entrega 0.092 m y un intervalo central del noventa y cinco por ciento entre 1.700 m y 2.056 m, según el [Listado 1.11](#).

Listado 1.11. Propagación de incertidumbre de primer orden contrastada con Monte Carlo.

```
1 import numpy as np
2
3 def propagacion_primer_orden(modelo, valor_medio, desviacion,
4     paso=1e-6):
5     derivada = (modelo(valor_medio + paso)
6                 - modelo(valor_medio - paso)) / (2 * paso)
7     return modelo(valor_medio), abs(derivada) * desviacion
8
9 def monte_carlo(modelo, valor_medio, desviacion, muestras=5000,
10     semilla=20262):
11     generador = np.random.default_rng(semilla)
12     respuesta = np.array([modelo(v) for v in
13         generador.normal(valor_medio,
14             desviacion, muestras)]]
15     return (respuesta.mean(), respuesta.std(ddof=1),
16         np.percentile(respuesta, [2.5, 97.5]))
```

La coincidencia entre ambos procedimientos no siempre se da, y su discrepancia es en sí misma una señal diagnóstica, pues un intervalo de Monte Carlo marcadamente asimétrico respecto de la media indica que la no linealidad es apreciable y que la fórmula de primer orden subestima el riesgo del lado desfavorable. En el canal ocurre eso en grado leve, ya que el tirante crece más rápido que linealmente con la rugosidad.

La segunda gran limitación estructural es el compromiso entre complejidad y datos disponibles. Añadir procesos reduce el error estructural y aumenta el número de parámetros por estimar, de suerte que a partir de cierto punto el error de estimación crece más rápido de lo que el estructural disminuye. El resultado es una curva de error total en forma de valle cuyo mínimo se desplaza hacia mayor complejidad a medida que se dispone de más datos, tal como muestra la [Figura 1.13](#).

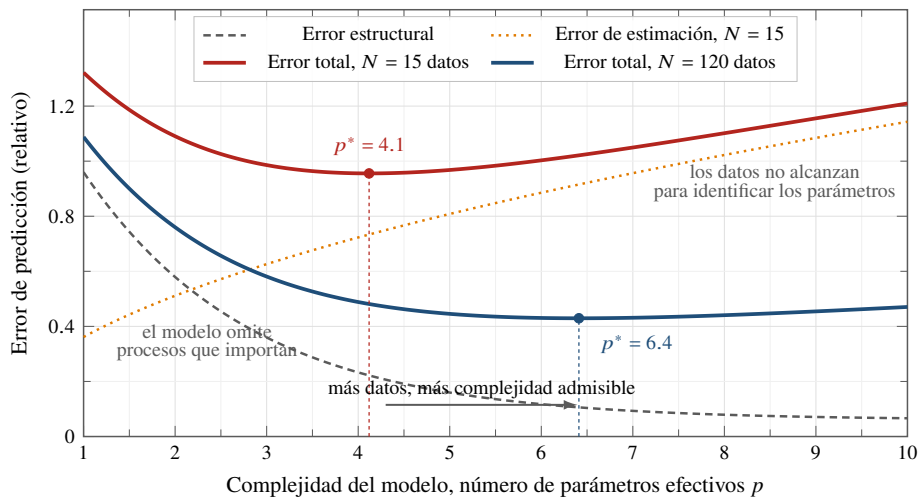


Figura 1.13. Compromiso entre la complejidad de un modelo, los datos disponibles para estimar sus parámetros y el error total de predicción, con el desplazamiento del óptimo al aumentar el tamaño de la muestra.

El principio de parsimonia no es una preferencia estética por lo sencillo, sino la constatación de que un modelo cuyos parámetros no pueden estimarse con los datos disponibles predice peor que uno más pobre pero bien determinado. Los criterios de información penalizan el número de parámetros y ofrecen un procedimiento objetivo para elegir entre estructuras ajustadas a los mismos datos [24].

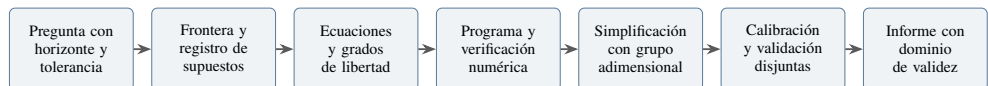
Con esa penalización aparece un fenómeno desconcertante. En modelos con varios parámetros correlacionados, conjuntos de valores muy distintos reproducen las observaciones con calidad casi idéntica, de modo que el ajuste no identifica un óptimo sino una región difusa. Esa multiplicidad recibe el nombre de equifinalidad y no es un defecto del algoritmo sino una propiedad del problema, ligada a que los datos no contienen información suficiente para separar los efectos de los parámetros [25]. Su consecuencia es severa, pues un juego calibrado puede reproducir el pasado y fallar en el futuro sin que ninguna métrica de ajuste lo anuncie.

De ahí se desprende una obligación profesional concreta, que es entregar el modelo con la declaración de lo que supone y de lo que no puede hacer, junto con el registro de supuestos, el dominio de validez con sus límites numéricos, la incertidumbre de la respuesta y la trazabilidad entre las versiones del código, de los datos y del resultado [11].

Los asistentes de programación basados en modelos de lenguaje añaden un componente nuevo a esa declaración, pues el código generado de forma automática es plausible con la misma facilidad con la que es correcto y el ingeniero responde por el resultado que firma con independencia de quién teclee las líneas. La práctica que este libro promueve consiste en declarar su uso, someter el código generado a las mismas pruebas que el escrito a mano y verificar siempre el orden de magnitud esperado, la coherencia dimensional, el caso límite y el balance que debe cerrar. El capítulo cierra con el ejercicio inverso, que consiste en leer críticamente el modelo de otro con un protocolo fijo.

Ese protocolo procede por preguntas. Cuál es la pregunta que el modelo dice responder y si coincide con la decisión en curso, cuáles supuestos carecen de prueba prevista, si las ecuaciones son dimensionalmente coherentes, si los datos de calibración y los de validación son disjuntos, si existe estudio de refinamiento, si el rango de entradas cae dentro del dominio de validez y si un resultado puede reproducirse con el código entregado. Léida al revés, esa lista describe la estructura del informe que el lector deberá producir, tal como sintetiza la [Figura 1.14](#), y construir modelos que la resistan es el objeto de los tres capítulos siguientes.

La unidad produce una cadena de entregables, y cada eslabón tiene criterio de salida propio



Protocolo para leer el modelo de otro

1. Cuál es la pregunta que el modelo dice responder y si coincide con la decisión en curso.
2. Cuáles supuestos carecen de prueba prevista.
3. Si las ecuaciones son dimensionalmente coherentes.
4. Si los datos de calibración y los de validación son disjuntos.
5. Si existe estudio de refinamiento con orden observado.
6. Si el rango de entradas cae dentro del dominio de validez.
7. Si un resultado puede reproducirse con el código entregado.

Contenido mínimo del entregable propio

- La pregunta respondida, con su variable de interés y su tolerancia.
- El registro de supuestos con su prueba prevista.
- La evidencia de verificación y el orden observado.
- Los datos usados en el ajuste y los reservados para la validación.
- El dominio de validez con sus límites numéricos.
- La incertidumbre de la respuesta.
- La trazabilidad entre versiones del código, de los datos y del resultado, con el uso declarado de asistentes de programación.

la misma lista, léida al revés

Un modelo se juzga siempre frente al propósito declarado y nunca en abstracto. La simulación produce resultados condicionados al modelo y jamás evidencia independiente sobre el sistema, de modo que la firma del ingeniero respalda el resultado con independencia de quién teclee las líneas de código.

Figura 1.14. Cadena de entregables de la unidad y protocolo de lectura crítica, que leído en sentido inverso describe el informe que el estudiante debe producir.

El protocolo tiene un antecedente breve y memorable en los cinco mandamientos negativos que Golomb formuló para la modelación matemática y que el texto guía recoge, a saber, no creer que el modelo es la realidad, no extrapolar fuera de la región del ajuste, no distorsionar la realidad para que encaje en el modelo, no conservar un modelo desacreditado y no enamorarse del propio modelo [1]. Los dos primeros son la advertencia sobre el dominio de validez que este capítulo acaba de cuantificar, y los tres restantes exigen abandonar el modelo que no supera la validación, que es la disciplina que el [Capítulo 4](#) formaliza.

1.7. Problemas

Los problemas siguientes están ordenados en complejidad creciente, desde la apropiación del vocabulario hasta las situaciones abiertas de diseño. Se admite trabajar con las bibliotecas del curso y se exige reportar unidades y cifras significativas coherentes con los datos.

1.7.1. Apropiación conceptual

- 1-1 Explique la diferencia entre modelar y simular con un ejemplo de su área en el cual un modelo bien construido se simule de manera inadecuada.
- 1-2 Un colega afirma que su modelo es válido porque reproduce con error inferior al 2 % los datos con los que lo calibró. Objete esa afirmación.
- 1-3 Reformule «estudiar el comportamiento térmico de un cuarto frío» como pregunta de ingeniería, con variable de interés, horizonte, resolución y tolerancia.
- 1-4 Argumente por qué la creciente máxima de un registro de veinte años no dice cuál será el nivel máximo tras construir un dique de encauzamiento aguas arriba.
- 1-5 Discuta por qué la advertencia de Box sobre los modelos equivocados es compatible con la exigencia de reportar la incertidumbre.

1.7.2. Clasificación de modelos y alcance

- 1-6 Clasifique según los seis criterios el modelo de agotamiento de la zona radicular de la [Sección 1.3](#) y justifique cada posición.
- 1-7 Un fabricante entrega la eficiencia de un colector solar plano frente a la temperatura reducida, medida entre 700 W/m^2 y 1000 W/m^2 . Indique qué tipo de modelo es y cuándo carece de respaldo.

- 1–8 Para una laguna de estabilización proponga un modelo estacionario de parámetros concentrados y uno dinámico de parámetros distribuidos, e indique qué pregunta responde cada uno y qué datos exige el segundo.
- 1–9 Explique por qué un modelo mecanicista admite extrapolación y por qué esa licencia cesa cuando incorpora una correlación empírica de cierre.
- 1–10 Una empresa modela las fallas de sus circuitos rurales con una tasa media anual constante. Clasifique el modelo, señale qué fenómeno no representa y proponga una alternativa estocástica.

1.7.3. Variables, grados de libertad y condiciones de frontera

- 1–11 Escriba en la forma de la Ecuación (1.4) el modelo dinámico de mezcla completa del [Ejemplo 1.1](#), con su estado, entrada, perturbación, parámetro y salida.
- 1–12 Un intercambiador de doble tubo se describe con las temperaturas de entrada y salida, los caudales másicos, los calores específicos, el coeficiente global y el área. Cuente incógnitas, ecuaciones y grados de libertad.
- 1–13 Para un canal trapezoidal con $b = 3.0$ m, $z = 2.0$, $n = 0.030$ y $S_0 = 0.0005$, determine el tirante normal para $15 \text{ m}^3/\text{s}$.
- 1–14 Clasifique como de Dirichlet, de Neumann o de Robin un embalse que fija el nivel piezométrico en el borde de un acuífero, una pared de horno aislada, la superficie de un producto enfriado por aire forzado y la base de un lisímetro.
- 1–15 El vaciado de un tanque por un orificio da una ecuación cuyo miembro derecho es proporcional a la raíz del nivel. Discuta qué hipótesis del [Teorema 1.1](#) falla al vaciarse el tanque y qué consecuencia tiene sobre la integración.

1.7.4. Análisis de escalas y grupos adimensionales

- 1–16 Adimensionalice el balance de un reactor de mezcla completa con reacción de primer orden y muestre que la respuesta depende solo del número de Damköhler.
- 1–17 Un cubo de 40 mm de lado y conductividad $0.48 \text{ W}/(\text{m K})$ se enfría en agua agitada con $450 \text{ W}/(\text{m}^2 \text{ K})$. Evalúe el número de Biot y dictamine si vale la hipótesis de sólido isotérmico.
- 1–18 Obtenga los grupos adimensionales que gobiernan la potencia de un agitador a partir de la potencia, el diámetro del impulsor, la velocidad de giro, la densidad y la viscosidad.

- 1–19 Una cámara de 18 m tiene residencia media de 45 min y dispersión longitudinal de $0.9 \text{ m}^2/\text{min}$. Calcule el número de Péclet y decida si se aproxima al flujo pistón o a la mezcla completa.
- 1–20 Un esquema explícito para la advección en un río de velocidad 0.4 m/s usa celdas de 50 m. Determine el paso de tiempo máximo para un número de Courant unitario y qué ocurre si se excede.
- 1–21 Demuestre que la superposición no aplica al descenso en un acuífero libre, partiendo de la ecuación de flujo en la que el espesor saturado depende del propio descenso.

1.7.5. Problemas de programación

- 1–22 Implemente una función que reciba los residuos de un modelo estacionario y devuelva sus grados de libertad, con la estrategia del [Listado 1.4](#), y pruébela sobre el canal trapezoidal.
- 1–23 Programe la verificación del [Listado 1.9](#) y aplíquela a un balance de masa que incluya un término escrito por error en base másica.
- 1–24 Implemente el modelo distribuido de la cámara de contacto con 150, 300, 600 y 1200 celdas y estime el orden observado.
- 1–25 Escriba una rutina que devuelva la desviación estándar de la salida de un modelo de una entrada incierta, por primer orden y por Monte Carlo.
- 1–26 Amplíe el modelo del [Listado 1.2](#) para que acepte una serie diaria de lluvia y devuelva el calendario de riegos de una temporada.

1.7.6. Problemas de diseño y ensayo

- 1–27 Diseñe el modelo conceptual de un sistema de aprovechamiento de agua lluvia para una vivienda rural de Sucre, con su frontera, sus estados, entradas, parámetros y dominio de validez.
- 1–28 Repita el análisis de la tubería de impulsión de la [Sección 1.1](#) con un tercer diámetro y una tarifa incierta en un 20 %, y decida si la elección cambia.
- 1–29 Elija un artículo reciente de su área que reporte simulaciones y aplíquelo el protocolo de lectura crítica de la [Sección 1.6](#).
- 1–30 Ensayo de dos páginas. Discuta si los asistentes de programación modifican la responsabilidad de quien firma un informe de simulación, con apoyo en la [Tabla 1.5](#).

Capítulo 2

Herramientas Computacionales para Modelación y Simulación

Una estación de bombeo del acueducto municipal registra cada cinco minutos el caudal impulsado y la presión en la descarga. Al cabo de un año el archivo acumula más de cien mil filas, con marcas de tiempo repetidas por un reinicio del registrador, con lecturas negativas producidas por un sensor destapado durante el mantenimiento y con un intervalo de tres días en blanco. Sobre ese archivo alguien debe decidir si la bomba está operando fuera de su curva. La decisión no se toma leyendo el archivo, se toma construyendo con él un modelo, y entre el archivo y el modelo hay un trabajo de ingeniería que ningún programa realiza solo.

El [Capítulo 1](#) estableció qué es un modelo, cómo se clasifica, qué etapas recorre su construcción y hasta dónde alcanza su validez. Ese marco es necesario pero no suficiente, porque un modelo formulado con precisión sobre el papel todavía tiene que existir como código que otra persona pueda ejecutar, verificar y corregir. Ahí aparecen preguntas que la teoría de la modelación no responde. Cómo se ingresa un dato sin destruirlo, cómo se representa un sistema distribuido en la memoria de una máquina, por qué dos expresiones algebraicamente idénticas devuelven números distintos, y cómo se distingue un resultado correcto de uno meramente plausible.

Este capítulo responde esas preguntas con las herramientas del ecosistema científico libre de Python. No enseña el lenguaje, que se supone conocido a nivel básico, sino el oficio de usarlo para representar modelos. Se recorre el ecosistema y el proyecto reproducible como unidad de trabajo, el ingreso y la verificación de datos de instrumentación, la representación numérica de sistemas con arreglos y álgebra lineal, el cálculo simbólico como puente entre la formulación y el código, la visualización como instrumento de diagnóstico y no de adorno, y las prácticas de verificación que un ingeniero aplica a todo fragmento de código, lo haya escrito él o se lo haya sugerido un asistente automático.

Al terminar, el lector debe ser capaz de montar un proyecto reproducible desde cero, cargar una serie de instrumentación y demostrar que la cargó bien, expresar un balance sobre una malla sin escribir un solo bucle, estimar cuánta confianza merece la última cifra de un resultado y producir una figura que permita detectar un artefacto numérico antes de que se convierta en una conclusión. La [Figura 2.1](#) anticipa ese recorrido sobre el registro de la estación de bombeo, con las seis capas de trabajo que separan el archivo crudo de la decisión y la responsabilidad única de cada una. Con eso queda instalada la infraestructura sobre la cual el [Capítulo 3](#) ejecuta las simulaciones y el [Capítulo 4](#) las somete a validación.

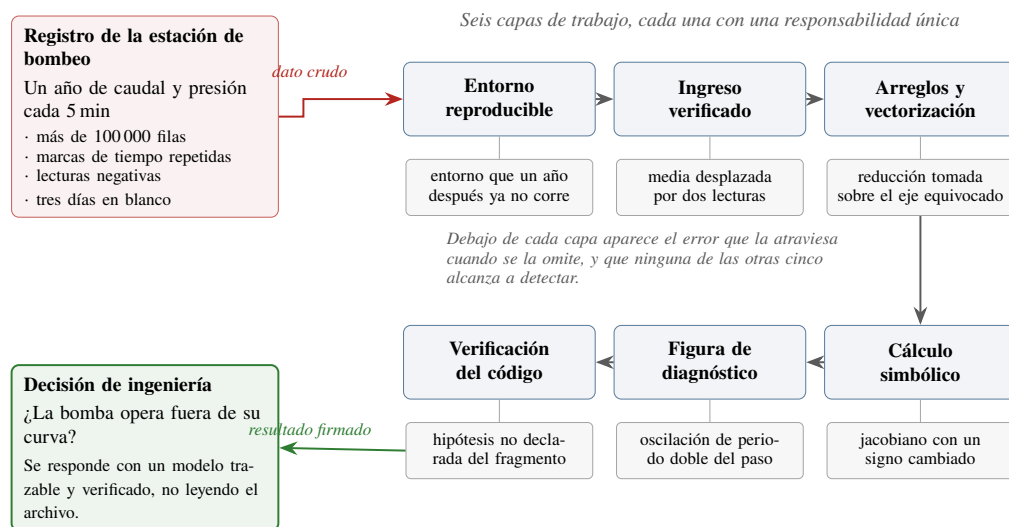


Figura 2.1. Recorrido que va del registro crudo de la estación de bombeo hasta la decisión de ingeniería, con las seis capas de trabajo de la unidad.

2.1. El ecosistema científico libre y el entorno reproducible

La elección de una herramienta de modelación suele decidirse por la comodidad inmediata de quien la va a usar, y ese criterio es el peor de todos. Un modelo de ingeniería tiene una vida útil larga, se hereda entre profesionales, se audita cuando algo falla y se reutiliza en proyectos que nadie previó. Cuatro criterios sostienen esa vida útil, a saber, la licencia, que determina si el resultado puede reproducirse sin pagar; la trazabilidad, que permite saber qué operación produjo cada número; la interoperabilidad, que decide si el modelo puede acoplarse a otro; y la permanencia, que dice si dentro de diez años el archivo seguirá abriéndose. Una interfaz agradable no compensa la pérdida de ninguno.

El ecosistema científico de Python satisface los cuatro porque está construido por estratos independientes, cada uno con una responsabilidad única. La [Figura 2.2](#) los muestra junto con aquello que no debe pedírsele a cada uno, advertencia que importa tanto como la capacidad, pues la mayoría de los tropiezos provienen de exigirle a un estrato una función que corresponde a otro, como esperar que una tabla de pandas garantice la coherencia de unidades o que un cuaderno de Jupyter garantice la reproducibilidad.

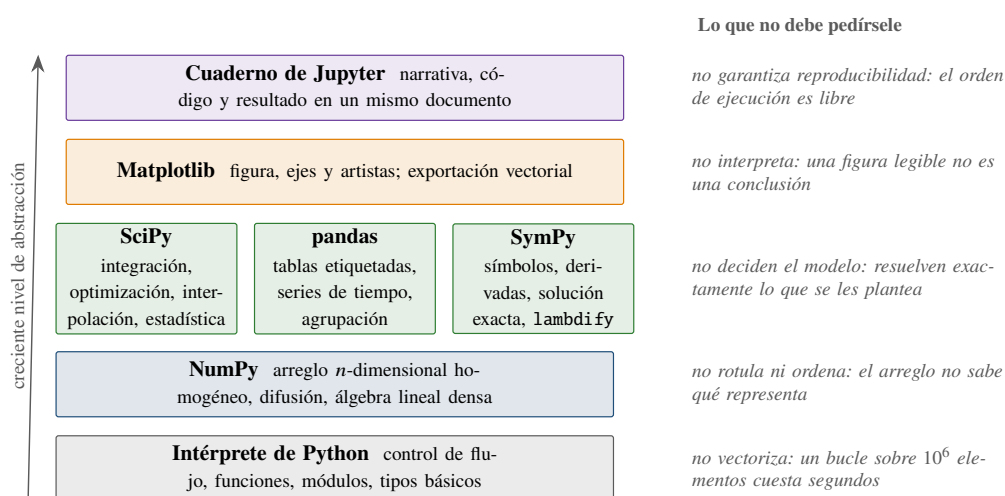


Figura 2.2. Estratos del ecosistema científico de Python, con lo que aporta cada uno y lo que no debe pedírsele.

Conviene precisar qué resuelve cada estrato antes de escribir la primera línea. NumPy aporta el arreglo homogéneo de tamaño fijo sobre el cual descansa todo lo demás, con la difusión y el álgebra lineal densa [13]; SciPy, las implementaciones de referencia de integración, optimización, interpolación y estadística [4]; pandas, la tabla etiquetada con índice temporal, que es la forma natural de un registro de instrumentación [26]; SymPy,

la manipulación exacta de expresiones [18]; Matplotlib, la representación gráfica [27]; y el cuaderno de Jupyter, la mezcla de narrativa, código y resultado [28]. Los tratamientos de conjunto de ese ecosistema orientado al análisis de datos y al cómputo científico son abundantes [29, 30]. La [Tabla 2.1](#) resume la correspondencia entre tarea y estrato responsable.

Tabla 2.1. Correspondencia entre las tareas de un proyecto de modelación y el estrato del ecosistema que las resuelve.

Tarea del proyecto	Estrato	Objeto central	Riesgo de mal uso
Ingresar y depurar datos	pandas	DataFrame	cálculo fila a fila
Representar campos	NumPy	ndarray	bucles explícitos
Resolver el modelo	SciPy	rutina especializada	tolerancia por omisión
Derivar y linealizar	SymPy	expresión exacta	explosión simbólica
Diagnosticar resultados	Matplotlib	Figure, Axes	figura sin unidades
Documentar el análisis	Jupyter	cuaderno	ejecución desordenada
Versionar el trabajo	Git	repositorio	versionar salidas

El ecosistema de Python no es la única opción y conviene situarlo dentro del panorama del software especializado con el que un maestrante se encontrará en su vida profesional. Existen entornos de propósito general que compiten con él, como R para el análisis estadístico, MATLAB y su equivalente libre Octave para el cálculo matricial, Julia para el cómputo de alto desempeño y los sistemas de álgebra computacional como Maxima. El texto guía de la asignatura desarrolla toda su exposición sobre R, Maxima y OpenFOAM, precisamente porque son libres, y muestra que la elección del entorno importa menos que la disciplina con que se usa [1]. Los tratamientos de la programación científica desde Python cubren el terreno común de los cuatro entornos [31, 32].

Junto a los entornos generales existen los simuladores de dominio, que resuelven de manera cerrada una familia de modelos con una interfaz propia, como los simuladores de redes hidráulicas y de drenaje urbano, los de tránsito de crecientes en cauces, los de procesos químicos, los de dinámica de fluidos computacional, los de sistemas fotovoltaicos y las plataformas de teledetección en la nube que procesan imágenes satelitales a escala regional [33]. La [Tabla 2.2](#) los ordena por categoría, con lo que resuelven y con el riesgo que su comodidad esconde. El criterio que gobierna la elección es el mismo de los cuatro requisitos enunciados al comienzo de la sección, y un simulador de dominio se justifica cuando el modelo que encierra coincide con el que la pregunta exige, mientras que un entorno general se justifica cuando el modelo debe construirse, modificarse o acoplarse.

Tabla 2.2. Panorama del software especializado para modelación y simulación, con la categoría, ejemplos representativos, lo que resuelve y el riesgo que conviene vigilar.

Categoría	Ejemplos	Qué resuelve	Riesgo
Entornos generales de cómputo científico	Python, R, MATLAB, Octave, Julia	Cualquier modelo que el usuario programe	Reinventar lo que una biblioteca ya verificó
Álgebra computacional	SymPy, Maxima, Mathematica	Derivación, linealización y soluciones cerradas	Expresiones que crecen sin control
Simuladores hidráulicos e hidrológicos	Redes de tuberías, drenaje urbano, tránsito en cauces	Balances y ecuaciones de flujo ya implementados	Usar el modelo interno fuera de sus hipótesis
Simuladores de procesos y de CFD	Diagramas de flujo de proceso, OpenFOAM, elementos finitos	Equilibrios, balances de planta y campos distribuidos	Aceptar una malla o una tolerancia por omisión
Modelado de sistemas dinámicos	OpenModelica, entornos de bloques	Sistemas con conmutación y control	Ocultar la rigidez tras la interfaz
Aprendizaje estadístico y teledetección	scikit-learn, marcos de redes neuronales, Google Earth Engine	Modelos fenomenológicos y series de imágenes	Extrapolar un ajuste sin respaldo físico

^a La asignatura trabaja en el ecosistema de Python, y el resto de la tabla se presenta como mapa para reconocer qué herramienta corresponde a cada pregunta y qué hipótesis carga cada una.

Las plataformas de aprendizaje estadístico y de teledetección merecen una mención aparte porque su uso crece con rapidez en las ingenierías agrícola y ambiental, donde alimentan modelos de balance hídrico y de monitoreo de cultivos con datos de sensores remotos y de drones [34, 35, 36]. Son modelos fenomenológicos en el sentido de la [Definición 1.5](#), y por eso el criterio de dominio de validez del [Capítulo 1](#) les aplica con todo su rigor, cualquiera que sea la sofisticación del algoritmo que los ajusta.

La disponibilidad de buenas bibliotecas no basta. Un resultado de simulación solo tiene valor profesional si alguien más puede obtenerlo de nuevo, y esa propiedad se diseña desde el primer día del proyecto [37]. Conviene fijarla formalmente.

Definición 2.1. Reproducibilidad computacional. Un análisis es *computacionalmente reproducible* cuando un tercero, disponiendo únicamente de los datos originales, del código y de la descripción del entorno de ejecución, obtiene los mismos resultados numéricos y las mismas figuras sin necesidad de consultar al autor.

La definición encierra una particularidad decisiva. No exige que el modelo sea correcto ni que sus predicciones se ajusten a la realidad, sino solamente que el camino desde el dato hasta la conclusión esté completo y sea recorrible. Un análisis reproducible puede estar equivocado, y de hecho la reproducibilidad es la condición que permite descubrir el error, porque un resultado que nadie puede rehacer tampoco puede refutarse.

Tres elementos hacen reproducible un proyecto, a saber, una estructura de carpetas fija con rutas relativas y separación estricta entre el dato crudo, que jamás se modifica, y todo lo que se deriva de él; la declaración explícita del entorno de ejecución; y el control de versiones, que registra qué cambió, cuándo y por qué [38]. La Figura 2.3 muestra esa estructura junto al flujo de los datos e indica qué se versiona y qué se regenera con una orden.

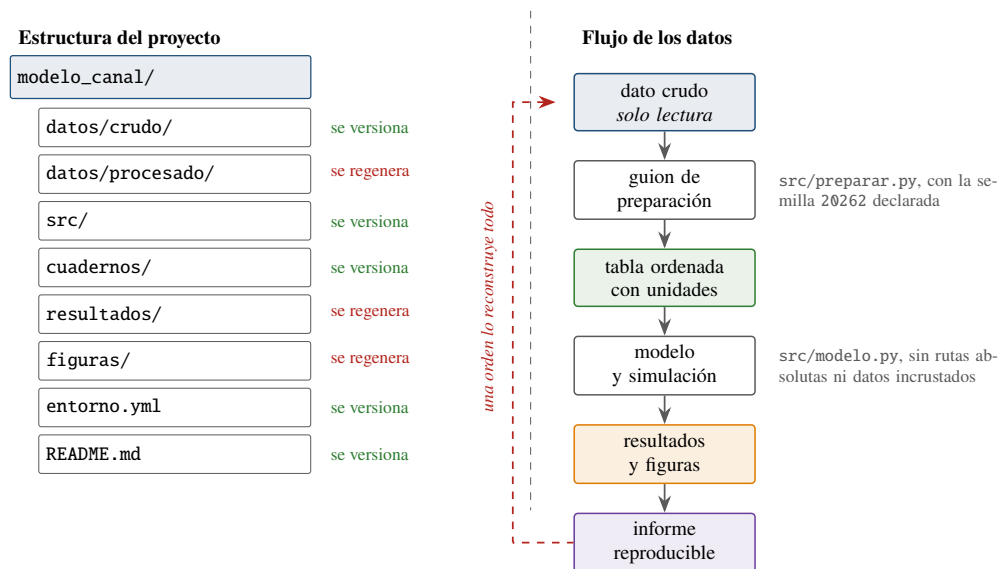


Figura 2.3. Estructura del proyecto reproducible y flujo de los datos, con la distinción entre lo que se versiona y lo que se regenera.

La regla que ordena el diagrama es sencilla de enunciar y difícil de sostener. Se versiona todo lo que un ser humano escribió y no puede reconstruirse, esto es, el dato crudo, el código, los cuadernos, el entorno y la documentación, y no se versiona nada que una orden pueda regenerar. Quien versiona figuras termina con un repositorio pesado, con conflictos irresolubles y con la duda permanente de si la figura del informe corresponde a la última versión del código [11].

Definición 2.2. Entorno declarativo. Un *entorno declarativo* es un archivo de texto versionado que enumera el intérprete, las bibliotecas y sus versiones exactas, de modo que la orden de instalación reconstruya en otra máquina un entorno funcionalmente idéntico al del autor.

El entorno declarativo resuelve el problema más frecuente de la reproducibilidad, que no es el código perdido sino el código que ya no corre, porque una función cambió su valor por omisión entre dos versiones menores y el resultado se desplazó sin que nadie lo notara. Junto con el entorno debe declararse la semilla del generador aleatorio, que en esta asignatura es 20262. El [Listado 2.1](#) deja constancia de ambos dentro de los propios resultados.

Listado 2.1. Registro del entorno de ejecución y de la semilla junto con los resultados.

```

1 import json, platform, sys
2 import matplotlib, numpy as np, pandas as pd, scipy
3
4 SEMILLA = 20262
5 rng = np.random.default_rng(SEMILLA)
6
7 entorno = {"python": sys.version.split()[0],
8           "plataforma": platform.platform(),
9           "numpy": np.__version__, "scipy": scipy.__version__,
10          "pandas": pd.__version__, "matplotlib":
11          matplotlib.__version__,
12          "semilla": SEMILLA}
13 with open("resultados/entorno.json", "w", encoding="utf-8") as f:
14     json.dump(entorno, f, indent=2, ensure_ascii=False)

```

El archivo que produce ese fragmento pesa unos pocos cientos de bytes y responde por adelantado la pregunta de cualquiera que intente repetir el trabajo. La [Tabla 2.3](#) completa el cuadro con el contenido de cada carpeta y el criterio que decide si entra o no al repositorio.

Tabla 2.3. Contenido y tratamiento de cada carpeta del proyecto reproducible.

Carpeta o archivo	Contenido	Versionado	Modificable
datos/crudo/	registros tal como llegaron	sí	nunca
datos/procesado/	tablas depuradas	no	por el guion
src/	funciones del modelo	sí	sí
cuadernos/	narrativa del análisis	sí	sí
resultados/	tablas y valores calculados	no	por el guion
figuras/	gráficas exportadas	no	por el guion
entorno.yml	versiones del entorno	sí	sí
README.md	propósito y orden de ejecución	sí	sí

El cuaderno de Jupyter merece una advertencia aparte porque su comodidad esconde una trampa. Las celdas pueden ejecutarse en cualquier orden y el estado de la sesión sobrevive a la edición del código, de modo que un cuaderno puede mostrar resultados que ninguna ejecución ordenada reproduce. La disciplina mínima consiste en reiniciar el núcleo y ejecutar todo de arriba abajo antes de dar por buena cualquier salida, y en mantener en el cuaderno la narrativa mientras las funciones viven en módulos importables [39].

Cuando el trabajo se entrega, los datos y el código deben además quedar localizables, accesibles, interoperables y reutilizables, que es lo que resumen los principios FAIR [40]. En la asignatura esto se traduce en tres exigencias, a saber, que cada conjunto de datos lleve su procedencia y sus unidades declaradas, que cada figura del informe pueda regenerarse con una orden, y que el entorno y la semilla estén registrados. Con esa base ya es posible ocuparse del primer paso real del trabajo, que es meter los datos en la máquina sin estropearlos.

2.2. Ingreso, verificación y preparación de datos

Los datos de un modelo rara vez llegan en la forma que el modelo necesita. Llegan como archivos de texto exportados por un registrador, con separador de punto y coma porque el sistema operativo estaba configurado en español, con coma decimal, con encabezados que mezclan la variable y su unidad, y con marcas de tiempo en un formato que depende del fabricante. La tabla etiquetada de pandas es el objeto adecuado para recibirlos, porque conserva el nombre de cada columna, admite un índice temporal y distingue el dato ausente del cero [26]. El [Listado 2.2](#) muestra una lectura declarada, en la que nada se deja al azar.

Listado 2.2. Lectura tipada de un archivo de instrumentación, con unidades declaradas.

```
1 from pathlib import Path
2 import pandas as pd
3
4 RUTA = Path("datos") / "caudal_bombeo.csv"
5 tipos = {"caudal_l_s": "float64", "presion_bar": "float64"}
6 crudo = pd.read_csv(RUTA, sep=";", decimal=",", encoding="utf-8",
7                     dtype=tipos, parse_dates=["fecha_hora"])
8 crudo = crudo.rename(columns={"caudal_l_s": "Q", "presion_bar":
9                               "p"})
9 UNIDADES = {"Q": "L/s", "p": "bar"}
10 crudo["Q"] = crudo["Q"] / 1000.0
11 UNIDADES["Q"] = "m3/s"
```

Tres decisiones del fragmento merecen comentario. La primera es declarar el tipo de cada columna en lugar de dejar que la biblioteca lo infiera, porque una sola celda con texto convierte toda una columna numérica en texto y el error se propaga sin aviso. La segunda es declarar el separador decimal, ya que interpretar «41,8» como dos campos produce una tabla con más columnas de las esperadas. La tercera es mantener un diccionario de unidades que se actualiza al convertir la magnitud, pues el arreglo no recuerda en qué unidad está y el ingeniero tampoco lo recuerda tres semanas después. Leída la tabla, resta darle una forma canónica, que en este libro es la de los datos ordenados [41].

Definición 2.3. Datos ordenados. Un conjunto de *datos ordenados* es una tabla en la que cada variable ocupa una columna, cada observación ocupa una fila y cada unidad de observación se guarda en una tabla distinta.

La forma ordenada no es una preferencia estética. Una tabla en la que cada columna corresponde a un mes, o en la que una misma columna guarda a veces caudal y a veces presión, obliga a escribir código específico para ese archivo, y ese código muere con él. El costo de convertir se paga una vez; el de no hacerlo se paga en cada análisis posterior. Ahora bien, una tabla bien formada puede contener datos falsos. La verificación de ingreso es el conjunto de comprobaciones que se ejecutan sobre la tabla recién leída y antes de cualquier cálculo, y su propósito no es limpiar sino detectar. La [Figura 2.4](#) sitúa esas comprobaciones dentro del recorrido que va del archivo crudo a la tabla ordenada, y muestra el lazo que devuelve al origen cuando alguna falla.

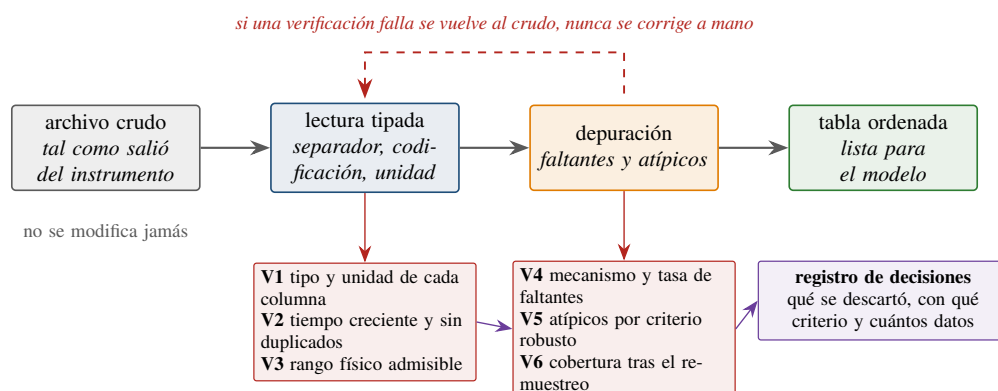


Figura 2.4. Recorrido del dato crudo hasta la tabla ordenada, con los seis puntos de verificación y el registro de decisiones.

El principio que gobierna el diagrama es que el archivo crudo no se corrige nunca. Si una lectura es imposible se marca, se documenta y se descarta en el guion de preparación, de modo que el descarte quede escrito en el código y no en la memoria de quien lo hizo. Corregir a mano el archivo original destruye la única evidencia de lo que el instrumento registró. La [Tabla 2.4](#) detalla las seis comprobaciones con el síntoma que delata cada falla.

Tabla 2.4. Verificaciones de ingreso aplicables a cualquier serie de instrumentación.

Clave	Qué comprueba	Síntoma de falla	Acción
V1	tipo y unidad	columna numérica leída como texto	redeclarar tipos
V2	orden temporal	marcas repetidas o fuera de secuencia	ordenar y eliminar duplicados
V3	rango físico	valores negativos o imposibles	marcar como ausentes
V4	datos faltantes	huecos agrupados en el tiempo	documentar el mecanismo
V5	valores atípicos	saltos que exceden el criterio robusto	marcar sin borrar
V6	cobertura	pocas muestras por intervalo agregado	rechazar el agregado

^a Ninguna acción modifica el archivo original. Todas se ejecutan en el guion de preparación y quedan registradas en el repositorio.

Las tres primeras comprobaciones se implementan en pocas líneas y conviene que devuelvan un informe cuantitativo y no un simple aviso, porque el número de anomalías es parte del resultado. El [Listado 2.3](#) construye ese informe y el [Algoritmo 2.1](#) describe el procedimiento del que forma parte.

Listado 2.3. Informe cuantitativo de las verificaciones de ingreso de una tabla.

```

1 import pandas as pd
2
3
4 def verificar(df, tiempo, limites):
5     """Devuelve el conteo de anomalías de ingreso por tipo."""
6     d = {"filas": len(df),
7         "marcas duplicadas": int(df[tiempo].duplicated().sum()),
8         "tiempo no monótono": int(not
9             df[tiempo].is_monotonic_increasing)}
10    for col, (lo, hi) in limites.items():
11        v = df[col]
```

```

11     d[f"{col} faltante"] = int(v.isna().sum())
12     d[f"{col} fuera de rango"] = int((~v.between(lo, hi)
13                                         & v.notna()).sum())
14     return pd.Series(d)
15
16
17 limites = {"Q": (0.0, 0.120), "p": (0.5, 6.0)}
18 informe = verificar(crudo, "fecha_hora", limites)

```

Algoritmo 2.1 Preparación de una serie de instrumentación para su uso en un modelo.

Entradas: Archivo crudo A , límites físicos $[\ell_j, u_j]$ por variable, paso nominal Δt

Salidas: Tabla ordenada T y registro de decisiones R

- 1: Leer A con separador, codificación y tipos declarados
 - 2: Renombrar columnas y registrar las unidades de cada una
 - 3: $R \leftarrow$ informe de verificaciones V1 a V3
 - 4: Ordenar por marca de tiempo y eliminar filas duplicadas
 - 5: **para** cada variable j **hacer**
 - 6: Marcar como ausente todo valor fuera de $[\ell_j, u_j]$
 - 7: Marcar los atípicos con el criterio robusto y anotarlos en R
 - 8: **fin para**
 - 9: Reindexar sobre la rejilla regular de paso Δt y agregar
 - 10: **si** la cobertura de un intervalo es menor que el umbral **entonces**
 - 11: Descartar el agregado y registrarlo en R
 - 12: **fin si**
 - 13: **devolver** T , R
-

El procedimiento se aprecia mejor sobre un caso concreto. El siguiente ejemplo aplica las tres primeras verificaciones a un extracto de doce registros de la estación de bombeo descrita en la apertura del capítulo.

Ejemplo 2.1. Verificación de ingreso de un registro de caudal. Un registrador de una estación de bombeo entrega un archivo con separador de punto y coma y coma decimal, que contiene doce filas correspondientes al periodo comprendido entre las 8:00 a.m. y las 8:50 a.m. del 4 de marzo. El caudal nominal de la bomba es 45 L/s y el diámetro de la impulsión admite como máximo 120 L/s. Determínese cuántas anomalías de ingreso contiene el extracto y qué efecto tiene ignorarlas.

Consideraciones. 1. El instrumento registra cada 5 min; 2. el caudal admisible está en 0 a 120 L/s; 3. ninguna lectura se corrige, solo se marca.

Análisis. Se aplican las verificaciones V1 a V3 de la [Tabla 2.4](#) con el procedimiento del [Listado 2.3](#). El efecto de ignorarlas se mide comparando la media aritmética de la columna cruda con la media de las lecturas que sobreviven al filtro físico, es decir, aquellas cuyo caudal cae dentro del intervalo admisible.

Solución. El informe de verificación arroja doce filas, una marca de tiempo duplicada, una secuencia temporal no monótona, una lectura ausente y dos lecturas fuera del intervalo físico, que son -9.9 L/s y 186.0 L/s . La media de las nueve lecturas válidas vale 43.478 L/s , mientras que la media de la columna cruda, calculada sobre once valores no ausentes, asciende a 51.582 L/s .

Comentario. Ignorar dos lecturas de doce desplaza la media un 18.6% hacia arriba y sugiere que la bomba opera por encima de su caudal nominal, que es la conclusión contraria a la verdadera. La marca duplicada, además, duplicaría el peso de ese instante en cualquier agregación posterior, y ninguna de las tres anomalías levanta un error al leer el archivo.

Las lecturas ausentes exigen un tratamiento propio porque su origen determina qué se puede hacer con ellas. Un hueco producido por una falla de energía no guarda relación con el valor que se habría medido, mientras que uno producido por la saturación del sensor ocurre precisamente cuando la variable era alta, de modo que descartarlo sesga el resultado hacia abajo. Solo los mecanismos completamente aleatorio y aleatorio condicionado admiten imputación sin sesgo [42], de manera que la regla de la asignatura es no imputar antes de caracterizar el mecanismo, y nunca imputar con la media, que reduce artificialmente la varianza.

Los valores atípicos plantean un problema más sutil, porque no son necesariamente errores. Un pico de caudal puede ser una falla del sensor o el arranque real de una segunda bomba, y solo el conocimiento del proceso distingue una cosa de la otra. Lo que sí puede automatizarse es la detección.

Definición 2.4. Valor atípico. Un *valor atípico* es una observación cuya distancia al centro de la distribución, medida en unidades de una escala estimada a partir de los propios datos, excede un umbral fijado de antemano.

La definición traslada toda la dificultad a dos decisiones, que son la elección del centro y la elección de la escala. El criterio clásico usa la media y la desviación estándar, y falla precisamente cuando más se le necesita, porque tanto la media como la desviación estándar se contaminan con los mismos valores que se pretende detectar. Basta un dato aberrante para inflar la desviación estándar lo suficiente como para que ese dato quede dentro de la banda de tres sigmas. El criterio robusto sustituye la media por la mediana y la desviación estándar por la desviación absoluta mediana, que se define como $\text{MAD} = \text{med}_i |x_i - \text{med}_j x_j|$ [43, 44].

Como la desviación absoluta mediana de una muestra normal converge a 0.6745σ , la escala robusta se normaliza multiplicando por 1.4826 para hacerla comparable con la desviación estándar en ausencia de contaminación. El [Listado 2.4](#) implementa el puntaje y el umbral habitual, y el [Algoritmo 2.2](#) describe su lógica sobre una serie con tendencia.

Listado 2.4. Criterio robusto de detección de valores atípicos basado en la desviación absoluta mediana.

```

1 import numpy as np
2
3
4 def atipicos_mad(x, umbral=3.5):
5     """Marca como atípico todo dato cuyo puntaje robusto excede
6         el umbral."""
7     x = np.asarray(x, dtype=float)
8     mediana = np.median(x)
9     mad = np.median(np.abs(x - mediana))
10    if mad == 0.0: # más de la mitad son
11        idénticos
12        return np.zeros(x.shape, bool), np.zeros(x.shape), 0.0
13    z = 0.6745 * (x - mediana) / mad # escala robusta =
14        1.4826 * mad
15    return np.abs(z) > umbral, z, 1.4826 * mad

```

Algoritmo 2.2 Detección robusta de valores atípicos sobre una serie con tendencia.

Entradas: Serie $x_1, \dots, x_n$, ancho de ventana w , umbral k

Salidas: Máscara de atípicos m y escala robusta $\hat{\sigma}$

```

1:  $b_i \leftarrow$  mediana móvil de ancho  $w$  centrada en  $i$ 
2:  $r_i \leftarrow x_i - b_i$                                 ▶ residuo sin la tendencia
3:  $\mu \leftarrow \text{med}(r)$ 
4:  $\text{MAD} \leftarrow \text{med}|r - \mu|$ 
5: si  $\text{MAD} = 0$  entonces
6:     devolver máscara vacía                            ▶ más de la mitad de los datos son idénticos
7: fin si
8:  $\hat{\sigma} \leftarrow 1.4826 \text{ MAD}$ 
9: para  $i = 1$  hasta  $n$  hacer
10:     $z_i \leftarrow 0.6745 (r_i - \mu) / \text{MAD}$ 
11:     $m_i \leftarrow (|z_i| > k)$ 
12: fin para
13: devolver  $m, \hat{\sigma}$ 

```

La superioridad del criterio robusto no es una cuestión de gusto sino una propiedad demostrable, que se formaliza mediante el punto de ruptura del estimador.

Definición 2.5. Punto de ruptura. El *punto de ruptura* de un estimador es la fracción más pequeña de observaciones que, al ser reemplazadas por valores arbitrarios, basta para llevar el valor del estimador más allá de cualquier cota prefijada.

El punto de ruptura mide cuánta contaminación soporta un estimador antes de volverse inútil, y separa de manera tajante a los dos criterios en discusión.

Teorema 2.1. Punto de ruptura de la mediana y de la media. Sea una muestra de n observaciones reales. El punto de ruptura de la media aritmética y el de la desviación estándar valen $1/n$, de modo que tienden a cero cuando n crece. El punto de ruptura de la mediana y el de la desviación absoluta mediana valen $\lfloor (n+1)/2 \rfloor / n$, que tiende a $1/2$, y ningún estimador equivariante ante traslaciones puede superar ese valor.

El enunciado se comprueba con un razonamiento directo. Si se reemplaza una sola observación por un valor M arbitrariamente grande, la media se desplaza en M/n y crece sin cota, mientras que la mediana no cambia si la mayoría de los datos permanece intacta. En sentido contrario, al sustituir más de la mitad de la muestra por M la mediana pasa a valer M , lo cual muestra que el valor $1/2$ no puede excederse. En la práctica, la banda de tres desviaciones estándar deja de detectar cualquier cosa en cuanto la contaminación supera unos pocos puntos porcentuales, tal como ilustra la [Figura 2.5](#).

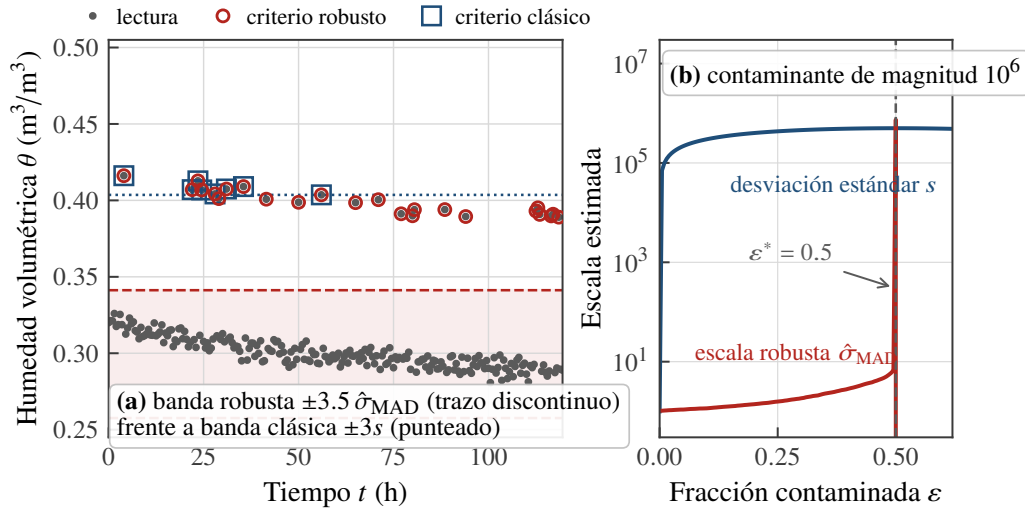


Figura 2.5. Detección de valores atípicos en una serie de humedad de suelo y comportamiento de las dos escalas frente a la fracción contaminada.

El panel derecho resume el teorema en una imagen, pues con un contaminante de magnitud 10^6 la desviación estándar se desploma como estimador apenas la fracción contaminada abandona el cero, mientras que la escala robusta permanece del orden de la unidad hasta la mitad de la muestra. El siguiente ejemplo cuantifica el efecto sobre un caso de campo.

Ejemplo 2.2. Atípicos en una sonda de humedad de suelo. Una sonda de capacitancia instalada a 30 cm de profundidad en un lote de riego registra la humedad volumétrica cada 30 min durante cinco días, para un total de 240 lecturas. Un contacto defectuoso en el conector produce 24 lecturas espurias, todas ellas elevadas en $0.10 \text{ m}^3/\text{m}^3$ sobre el valor verdadero. Compárese la capacidad de detección del criterio clásico de tres desviaciones estándar con la del criterio robusto de la Ecuación (2.1).

Consideraciones. 1. La serie presenta un secado lento de $0.320 \text{ m}^3/\text{m}^3$ hacia $0.285 \text{ m}^3/\text{m}^3$; 2. el ruido del instrumento tiene desviación estándar $0.004 \text{ m}^3/\text{m}^3$; 3. las lecturas espurias se distribuyen al azar.

Análisis. El puntaje robusto de cada lectura se calcula como

$$z_i = \frac{0.6745 (x_i - \text{med}(x))}{\text{MAD}(x)}, \quad \text{MAD}(x) = \text{med}_i |x_i - \text{med}(x)|, \quad (2.1)$$

donde x_i es la lectura de humedad volumétrica del registro i (m^3/m^3); $\text{med}(x)$ es la mediana de la serie completa; y $\text{MAD}(x)$ es la desviación absoluta mediana, expresada en la misma unidad que la lectura. Se marca como atípica toda lectura con $|z_i| > 3.5$, y como referencia se aplica el criterio clásico $|x_i - \bar{x}| > 3s$.

Solución. La mediana de la serie contaminada vale $0.29943 \text{ m}^3/\text{m}^3$ y la escala robusta normalizada asciende a $0.01194 \text{ m}^3/\text{m}^3$, mientras que la desviación estándar muestral llega a $0.03119 \text{ m}^3/\text{m}^3$. El criterio robusto marca las 24 lecturas espurias y ninguna válida. El criterio clásico marca solamente 8 de las 24, porque su propia banda, de semiancho $0.0936 \text{ m}^3/\text{m}^3$, alcanza a cubrir la mayoría de los picos.

Verificación. Aplicando el [Algoritmo 2.2](#) sobre el residuo de una mediana móvil de ancho 11, la escala robusta baja a $0.00511 \text{ m}^3/\text{m}^3$, mucho más cercana al ruido real del instrumento, lo cual confirma que el valor de $0.01194 \text{ m}^3/\text{m}^3$ estaba inflado por la tendencia de secado y no por las lecturas espurias.

Comentario. El criterio clásico no falla por poco, falla por diseño. Una contaminación del 10 % basta para que dos tercios de los datos falsos queden dentro de su propia banda, y con un 15 % deja de detectar cualquier cosa.

Depurada la serie queda la agregación, que puede introducir un error mayor que todos los anteriores si se ejecuta sin cuidado. La razón es la cobertura, es decir, la fracción de muestras válidas dentro del intervalo agregado. Un promedio diario calculado sobre cuatro lecturas de las cuarenta y ocho esperadas no es un promedio diario, es un número que se le parece. El [Listado 2.5](#) agrega la serie horaria y calcula la cobertura en la misma operación.

Listado 2.5. Agregación temporal con cálculo simultáneo de la cobertura del intervalo.

```
1 serie =
    crudo.drop_duplicates("fecha_hora").sort_values("fecha_hora")
2 valida = serie.set_index("fecha_hora")["Q"].where(lambda q:
3                                                    q.between(0.0,
4                                                    0.120))
4 horaria = valida.resample("1h").agg(media="mean", n="count")
5 horaria["cobertura"] = horaria["n"] / valida.resample("1h").size()
6 horaria = horaria.where(horaria["cobertura"] >= 0.80)
```

La última línea invalida los agregados con cobertura insuficiente en lugar de calcularlos de todos modos, y esa es la diferencia entre una serie preparada y una serie maquillada. Con los datos ya en condiciones de usarse, corresponde ocuparse de cómo se representa el sistema que va a consumirlos.

2.3. Representación computacional de sistemas y procesos

Un modelo de ingeniería se apoya casi siempre en colecciones de números del mismo tipo y de tamaño conocido, como el vector de estados de un sistema dinámico, la malla de nodos de un dominio o la matriz de coeficientes de un sistema de balances. La estructura que las representa con eficiencia es el arreglo homogéneo, cimiento de todo el ecosistema [\[13\]](#).

Definición 2.6. Arreglo n -dimensional. Un *arreglo n -dimensional* es un bloque contiguo de memoria acompañado de tres descriptores, a saber, la forma que fija el número de elementos en cada eje, el tipo de dato común a todos los elementos y el vector de zancadas que indica cuántos bytes hay que avanzar para moverse una posición a lo largo de cada eje.

Los tres descriptores explican por qué el arreglo es rápido y por qué desconcierta a quien viene de las listas. Como el tipo es común a todos los elementos, el intérprete no necesita examinar cada uno antes de operar; como la memoria es contigua, el procesador trae varios elementos de una vez; y como el recorrido se describe con zancadas, cambiar la forma o tomar una porción no exige copiar nada, basta con reinterpretar el mismo bloque. La [Figura 2.6](#) muestra esa correspondencia y la distinción entre una vista, que comparte el búfer, y una copia, que no.

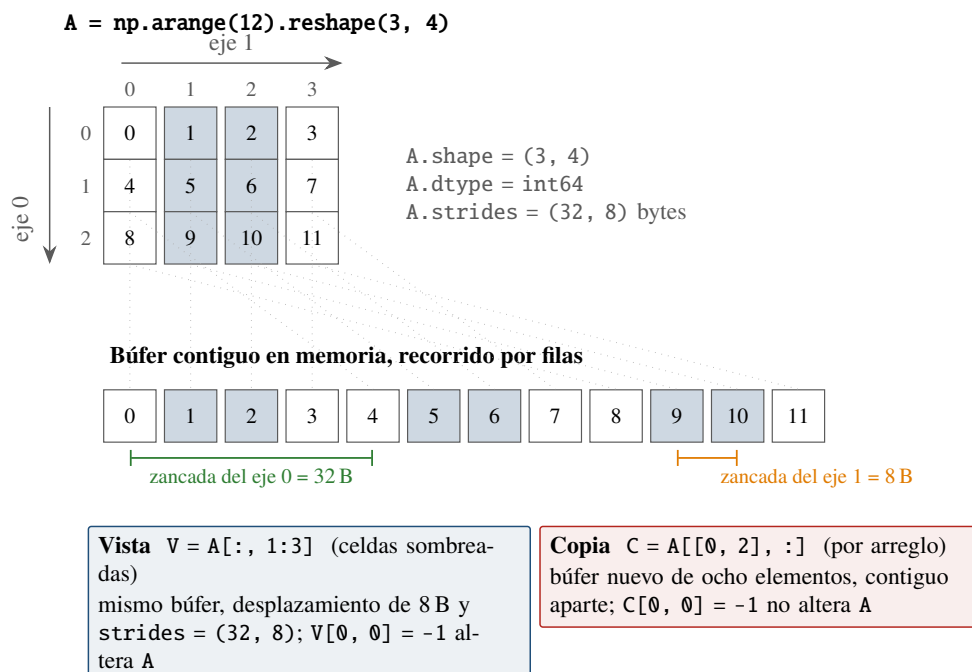


Figura 2.6. Correspondencia entre un arreglo de dos ejes y su búfer en memoria, con las zancadas y la diferencia entre vista y copia.

Esa distinción es la fuente de un error silencioso frecuente. Modificar una porción obtenida por rebanado altera el arreglo original, mientras que modificar una obtenida por indexación con una lista de índices no lo altera, y quien supone lo contrario descubre el problema cuando el resultado depende del orden en que se llamaron las funciones. La regla operativa es que el rebanado produce vistas, la indexación por arreglo o por máscara produce copias, y ante la duda se llama explícitamente al método de copia. El segundo pilar de la representación es la vectorización.

Definición 2.7. Vectorización. La *vectorización* es la práctica de formular un cálculo como una operación única sobre arreglos completos, de manera que el bucle sobre los elementos se ejecute dentro de una rutina compilada en lugar de en el intérprete.

La ganancia proviene de eliminar tres costos por elemento, que son la interpretación de la instrucción, la creación del objeto numérico que envuelve cada valor y la comprobación de tipo, y que en un bucle sobre un millón de elementos se pagan un millón de veces. Antes de medirla conviene presentar el mecanismo que hace expresiva a la forma vectorizada, que es la difusión.

Definición 2.8. Difusión. La *difusión* es la regla que hace compatibles dos arreglos de formas distintas alineando sus ejes por la derecha, completando la forma más corta con ejes de tamaño uno y estirando sin copiar todo eje cuyo tamaño valga uno frente a un eje de tamaño mayor.

La regla se aplica eje por eje y falla explícitamente cuando dos tamaños distintos son ambos mayores que uno, lo cual es una virtud, pues una incompatibilidad de formas suele delatar un error de concepción del modelo. La [Figura 2.7](#) la ilustra sobre tres pares de formas.

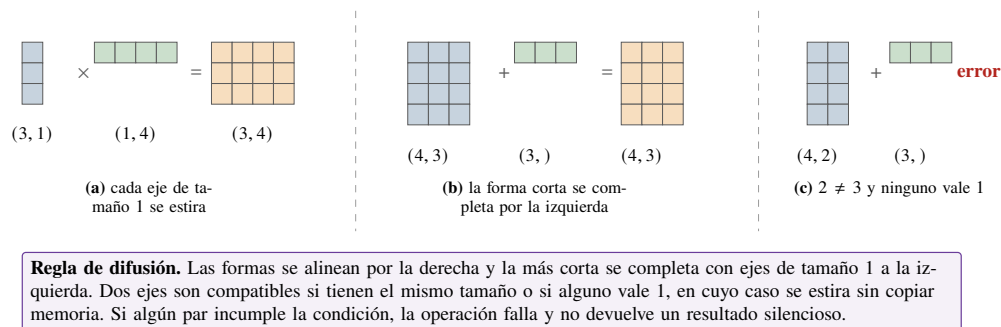


Figura 2.7. Aplicación de la regla de difusión a tres pares de formas, con el caso incompatible que produce un error.

Un uso característico de la difusión en modelación consiste en evaluar un modelo sobre el producto cartesiano de una coordenada espacial y un conjunto de escenarios, sin construir explícitamente la malla ni recorrerla. La [Figura 2.8](#) anticipa lo que se gana al hacerlo así, sobre un balance de oxígeno disuelto evaluado en un número creciente de nodos y con los tiempos medidos mediante repeticiones calibradas.

La figura contiene una lección que suele pasar inadvertida. El bucle más lento no es el que recorre listas de Python sino el que recorre un arreglo de NumPy elemento a elemento, porque cada acceso indexado construye un objeto escalar y cada llamada a una función universal sobre un escalar paga el costo de despacho completo sin amortizarlo. Recorrer un arreglo con un bucle reúne lo peor de los dos mundos. En la máquina que

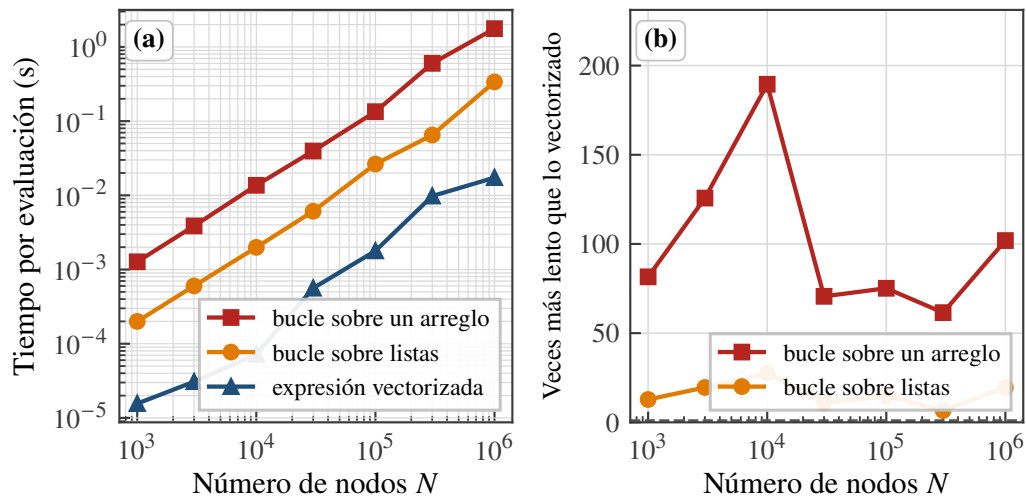


Figura 2.8. Costo de evaluar un mismo balance con un bucle y con una expresión vectorizada, en función del número de nodos.

produjo la figura la razón entre ambos tiempos supera las 150 veces para arreglos de decena de millar y desciende a unas 70 a 85 veces cuando el tamaño alcanza el millón, donde el costo ya está dominado por el tránsito de memoria. Las cifras dependen de la máquina, pero el orden de magnitud no.

El Listado 2.6 lleva la difusión al déficit de oxígeno disuelto a lo largo de un tramo de río, con doscientos escenarios de velocidad media.

Listado 2.6. Evaluación de un balance sobre una malla de posición y escenario mediante difusión.

```

1 import numpy as np
2
3 kd, ka, L0, D0 = 0.35, 0.62, 18.0, 1.2          # 1/d, 1/d, mg/L, mg/L
4 x = np.linspace(0.0, 80.0e3, 2000)           # m
5 u = np.linspace(0.15, 0.60, 200)             # m/s
6
7 t = x[None, :] / (u[:, None] * 86400.0)       # (200, 2000) días
8 D = (kd * L0 / (ka - kd)) * (np.exp(-kd * t) - np.exp(-ka * t)) \
9     + D0 * np.exp(-ka * t)
10 OD = 7.5 - D                                  # mg/L
11 critico = OD.min(axis=1)                      # un valor por
    escenario

```

Las dos últimas líneas ilustran el otro concepto que ordena el trabajo con arreglos, que es la reducción a lo largo de un eje. Una reducción colapsa un eje y devuelve un arreglo con un eje menos, de modo que el argumento del eje decide si el mínimo se toma por escenario o por posición. Confundirlo produce un número plausible y equivocado, razón por la cual conviene comprobar siempre la forma del resultado y no solo su valor.

Ejemplo 2.3. Vectorización de un balance de oxígeno sobre una malla. Un tramo de 80 km de un río recibe una descarga con demanda bioquímica de oxígeno inicial de 18 mg/L y un déficit inicial de 1.2 mg/L, con constantes de desoxigenación y de reaeración de 0.35/d y 0.62/d. Se requiere evaluar el déficit en 2000 posiciones para 200 escenarios de velocidad media entre 0.15 m/s y 0.60 m/s. Compárese el costo del bucle anidado con el de la expresión vectorizada.

Consideraciones. 1. Régimen permanente y flujo pistón sin dispersión longitudinal; 2. constantes uniformes en todo el tramo; 3. concentración de saturación de 7.5 mg/L.

Análisis. El déficit de oxígeno en función del tiempo de viaje responde al balance de desoxigenación y reaeración de la Ecuación (2.2),

$$D(t) = \frac{k_d L_0}{k_a - k_d} (e^{-k_d t} - e^{-k_a t}) + D_0 e^{-k_a t}, \quad t = \frac{x}{u}, \quad (2.2)$$

donde D es el déficit de oxígeno disuelto (mg/L); L_0 y D_0 son la demanda bioquímica de oxígeno y el déficit en el punto de descarga (mg/L); k_d y k_a son las constantes de desoxigenación y de reaeración (1/d); x es la distancia desde la descarga (m); y u es la velocidad media del flujo (m/s).

Solución. La forma vectorizada del Listado 2.6 produce el arreglo completo de 400 000 evaluaciones. Con repeticiones calibradas, el bucle anidado tarda 0.60 s y la expresión vectorizada 0.0083 s, unas 73 veces menos, y ambas coinciden dígito a dígito. El tiempo crítico de viaje resulta de anular la derivada de la Ecuación (2.2) y vale 1.9222 d, con un déficit máximo de 5.185 mg/L y una concentración mínima de 2.315 mg/L.

Verificación. Para 0.30 m/s el punto crítico cae a 49.8 km de la descarga. La condición $x_c \leq 80$ km exige $u \leq 0.48$ m/s, de modo que los escenarios más rápidos alcanzan el extremo del tramo antes de tocar el mínimo.

Comentario. Los tiempos dependen de la máquina y no deben citarse como propiedad del método, pero la razón entre ellos sí es estructural. Más importante que el ahorro es la legibilidad, pues la forma vectorizada expresa la Ecuación (2.2) casi con la notación del papel.

Muchos modelos estacionarios se reducen a un sistema lineal de balances, y la forma de resolverlo tiene consecuencias que van más allá de la velocidad. El [Listado 2.7](#) mezcla tres corrientes en una planta de jugos para alcanzar a la vez una masa, un contenido de sólidos solubles y una acidez objetivo.

Listado 2.7. Solución de un sistema de balances de masa y de componentes con álgebra lineal densa.

```

1 import numpy as np
2
3 # Mezcla de concentrado, jugo simple y agua para 1000 kg/h a 13.0
  Bx
4 # y 0.95 % de acidez.
5 A = np.array([[1.00, 1.00, 1.00],          # masa total
6               [65.0, 11.0, 0.00],         # sólidos solubles, Bx
7               [4.20, 0.85, 0.00]])        # acidez, %
8 b = np.array([1000.0, 13.0e3, 0.95e3])
9
10 m = np.linalg.solve(A, b)                  # kg/h de cada corriente
11 residuo = np.abs(A @ m - b).max()
12 kappa = np.linalg.cond(A)

```

La solución entrega 66.298 kg/h de concentrado, 790.055 kg/h de jugo simple y 143.646 kg/h de agua, con residuo nulo. Ahora bien, el residuo nulo no garantiza que la solución sea confiable, y la magnitud que cuantifica esa confianza es el número de condición, que aquí vale 624.8.

Definición 2.9. Número de condición. El *número de condición* de una matriz cuadrada no singular $\mathbf{A}$ respecto de una norma es $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$, y mide el factor por el cual una perturbación relativa de los datos puede amplificarse en la solución.

El número de condición no describe un defecto del algoritmo sino una propiedad del problema, pues una matriz mal condicionada lo sigue estando por más cuidadoso que sea el método, cuyo único mérito posible es no empeorarla. El resultado que hace operativo el concepto acota el error relativo de la solución.

Teorema 2.2. Cota del error relativo en función del número de condición.

Sea $\mathbf{Ax} = \mathbf{b}$ con $\mathbf{A}$ no singular y $\mathbf{b} \neq \mathbf{0}$, y sea $\mathbf{x} + \delta\mathbf{x}$ la solución exacta del sistema perturbado $\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}$. Entonces

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|},$$

y la cota se alcanza para alguna elección de $\mathbf{b}$ y de $\delta\mathbf{b}$.

La lectura práctica es una regla de dedo que conviene memorizar. Si $\kappa(\mathbf{A}) \approx 10^p$ y los datos se conocen con q cifras significativas, la solución merece confianza en aproximadamente $q - p$ cifras. Con el sistema de la mezcla, $\kappa \approx 6 \times 10^2$ y datos de laboratorio con cuatro cifras, quedan algo menos de dos cifras confiables en los caudales, lo cual sirve para operar una línea de producción pero no para certificar una etiqueta. La cota explica además por qué nunca debe formarse la matriz de las ecuaciones normales, cuyo número de condición es el cuadrado del original.

Todo lo anterior supone que la máquina calcula con números reales, lo cual es falso. Un computador representa un subconjunto finito de racionales y aproxima el resto, de modo que casi toda operación introduce un error [45, 46].

Definición 2.10. Épsilon de máquina. El *épsilon de máquina* ε de un formato de punto flotante es la distancia entre 1 y el siguiente número representable, y la unidad de redondeo es $u = \varepsilon/2$, que acota el error relativo cometido al redondear un real al representable más próximo.

En doble precisión, que es lo que usan por omisión los arreglos del ecosistema, $\varepsilon = 2.220446049250313 \times 10^{-16}$ y $u = 1.1102230246251565 \times 10^{-16}$, unas dieciséis cifras decimales. La consecuencia inmediata es que 0.1 no existe como tal en la máquina, sino como el racional $3602879701896397/2^{55}$, cuyo error relativo vale $5.551115123125783 \times 10^{-17}$, menor que u como debe ser. La suma $0.1 + 0.2$ difiere de 0.3 en esa misma cantidad, y por eso comparar en punto flotante con igualdad exacta es una fuente inagotable de fallas intermitentes.

Teorema 2.3. Error de representación en punto flotante. Sea x un número real no nulo dentro del rango normalizado de un formato de punto flotante con unidad de redondeo u , y sea $\text{fl}(x)$ su representable más próximo. Entonces existe δ con $|\delta| \leq u$ tal que $\text{fl}(x) = x(1 + \delta)$. Además, para cada operación aritmética básica $\circ$ entre representables se cumple $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$ con $|\delta| \leq u$.

El teorema afirma que cada operación individual es casi exacta, y de ahí se deduce el resultado incómodo. Como el error es relativo al valor de cada operando, restar dos cantidades muy próximas conserva el error absoluto heredado pero reduce el valor del resultado, con lo cual el error relativo se dispara. Ese fenómeno, llamado cancelación catastrófica, no es un defecto del computador sino del modo de escribir la fórmula. La raíz de menor magnitud de $x^2 + 10^8x + 1 = 0$ calculada con la fórmula habitual da $-7.450580596923828 \times 10^{-9}$, mientras que la forma equivalente $x = 2c/(-b - \sqrt{b^2 - 4ac})$ da -1×10^{-8} , con un error relativo del 25.5 % en la primera. La Figura 2.9 muestra, cifra por cifra, cómo la resta de dos entalpías de una tabla de vapor conserva solo las cifras que ambas no compartían, y cómo la misma raíz escrita de las dos maneras conserva o pierde la precisión disponible.

Resta de dos entalpías próximas de una tabla de vapor

$$\begin{array}{rcl}
 h_1 & = & \begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 7 & 5 & 0 & \cdot & 4 & 2 & ? \\ \hline \end{array} 2750.42 \text{ kJ kg}^{-1} \\
 h_2 & = & \begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 7 & 4 & 9 & \cdot & 6 & 2 & ? \\ \hline \end{array} 2749.62 \text{ kJ kg}^{-1} \\
 \Delta h & = & \begin{array}{|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & \cdot & 8 & ? & ? \\ \hline \end{array} 0.80 \text{ kJ kg}^{-1}
 \end{array}$$

Cada entalpía trae seis cifras significativas y una incertidumbre de $\pm 0.01 \text{ kJ kg}^{-1}$.

La diferencia hereda esa misma incertidumbre absoluta, $\pm 0.02 \text{ kJ kg}^{-1}$, pero conserva una cifra significativa.

error relativo
operandos 4×10^{-6}
resultado 2.5×10^{-2}
amplificación 6×10^3

La misma raíz escrita de dos maneras

Raíz de menor magnitud de $x^2 + 10^8 x + 1 = 0$ en doble precisión

Forma habitual resta dos números casi iguales

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = -7.4506 \times 10^{-9}$$

Error relativo de 25.5 %

Forma equivalente suma dos del mismo signo

$$x = \frac{2c}{-b - \sqrt{b^2 - 4ac}} = -1.0000 \times 10^{-8}$$

Exacta hasta la última cifra almacenable

El álgebra no distingue las dos expresiones. La aritmética de la máquina sí, porque el error de cada operación es relativo al valor de sus propios operandos.

Figura 2.9. Cifras significativas que sobreviven a la resta de dos cantidades próximas y efecto de reescribir la misma fórmula.

El mismo fenómeno aparece al ajustar un polinomio en una variable que no ha sido centrada, situación cotidiana en las curvas de calibración de instrumentos, y el ejemplo siguiente lo cuantifica sobre un refractómetro de laboratorio.

Ejemplo 2.4. Condicionamiento de una curva de calibración polinómica. La calibración de un refractómetro de laboratorio relaciona el índice de refracción medido a 20°C con la concentración de sólidos solubles de una solución de sacarosa. Se dispone de ocho puntos de calibración uniformemente espaciados entre 0°Bx y 70°Bx , con índices entre 1.33299 y 1.46554. Ajústese un polinomio de grado cinco y determínese cuántas cifras del resultado merecen confianza.

Consideraciones. 1. El instrumento resuelve el índice hasta la quinta cifra decimal; 2. los patrones de sacarosa tienen incertidumbre despreciable; 3. el ajuste se realiza por mínimos cuadrados.

Análisis. El ajuste plantea el sistema sobredeterminado $\mathbf{V}\mathbf{c} \approx \mathbf{y}$, con $\mathbf{V}$ la matriz de Vandermonde de la variable independiente. El condicionamiento depende de esa variable, de modo que se comparan la base cruda y la base normalizada de la Ecuación (2.3),

$$z_i = \frac{n_i - \bar{n}}{s_n}, \quad (2.3)$$

donde n_i es el índice de refracción del punto de calibración i (adimensional); $\bar{n}$ es la media de los índices; y s_n es su desviación estándar poblacional.

Solución. Con la base cruda el número de condición vale 5.451×10^9 , mientras que con la base normalizada de la Ecuación (2.3) desciende a 57.55. Al formar la matriz de las ecuaciones normales asciende a 3.120×10^{17} , por encima de $1/\varepsilon = 4.504 \times 10^{15}$. Ambos caminos entregan curvas que difieren a lo sumo en 0.0014°Bx , pero los coeficientes discrepan en un 93 % relativo y en la base cruda alcanzan magnitudes del orden de 3×10^5 .

Verificación. Al evaluar el polinomio de la base cruda en $n = 1.39986$, los términos individuales llegan a 5.96×10^5 y su suma vale 40.00, de modo que la evaluación pierde 4.2 cifras por cancelación. El residuo máximo del ajuste es de 0.0018°Bx con grado cinco, 0.0263°Bx con grado tres y 0.2307°Bx con grado dos.

Comentario. La lección es doble. La solución por descomposición en valores singulares es lo bastante estable como para que la curva sobreviva a un condicionamiento de 5×10^9 , pero los coeficientes por separado carecen de sentido físico y no deben reportarse. Y dado que el grado tres ya alcanza un residuo diez veces menor que la resolución del instrumento, el grado cinco solo agrega inestabilidad sin ganancia de exactitud.

El error de redondeo también se acumula al sumar muchos términos, y ahí existe un remedio elegante. La suma compensada de Kahan arrastra en una variable auxiliar la parte del sumando perdida en el último redondeo y la reincorpora en el paso siguiente [47].

Teorema 2.4. Error de la suma compensada. Sea $S_n = \sum_{i=1}^n x_i$ calculada con suma compensada en una aritmética de unidad de redondeo u , y sea $\hat{S}_n$ el valor obtenido. Entonces

$$|\hat{S}_n - S_n| \leq (2u + O(nu^2)) \sum_{i=1}^n |x_i|,$$

mientras que la suma secuencial ingenua solo admite la cota $(n-1)u \sum |x_i|$ en primer orden.

La diferencia entre ambas cotas es la dependencia del número de términos, que desaparece en la suma compensada [48]. El Listado 2.8 lo comprueba sumando un millón de copias de 0.1, cuyo valor exacto es 100 000.

Listado 2.8. Suma compensada de Kahan y comparación con la suma secuencial.

```
1 import numpy as np
```

```

2
3
4 def suma_kahan(v):
5     """Suma compensada: arrastra el error de redondeo de cada
6         término."""
7     s = 0.0
8     c = 0.0
9     for x in v:
10         y = x - c
11         t = s + y
12         c = (t - s) - y
13         s = t
14     return s
15
16 v = np.full(1_000_000, 0.1)
17 s_ingenua = 0.0
18 for x in v:
19     s_ingenua += x
20 error = (abs(s_ingenua - 1e5), abs(suma_kahan(v) - 1e5),
21         abs(float(v.sum()) - 1e5))

```

Los errores absolutos resultantes valen 1.333×10^{-6} para la suma secuencial, exactamente cero para la suma compensada y 2.910×10^{-11} para la reducción de NumPy, que emplea suma por pares. El resultado enseña dos cosas. La reducción de la biblioteca ya es cinco órdenes de magnitud mejor que el bucle ingenuo, de modo que reimplementar sumas a mano rara vez conviene, pero cuando se acumulan millones de contribuciones pequeñas, como en un balance de energía a lo largo de un año de operación, la elección del algoritmo deja de ser un detalle.

Queda por resolver cómo se organiza todo esto dentro del código, esto es, cómo se configura el modelo para que sus parámetros, sus unidades y su función de evaluación queden declarados en un solo lugar y no repartidos por el cuaderno. La estructura que mejor resiste el paso del tiempo separa los datos, el modelo y el análisis. El modelo se expresa como una función pura que recibe el estado y los parámetros y devuelve la derivada o el residuo, sin leer archivos ni imprimir nada, y los parámetros se agrupan en una estructura inmutable con sus unidades documentadas. El [Listado 2.9](#) muestra ese patrón sobre un quimiostato.

Listado 2.9. Separación entre los parámetros y la función que evalúa la derivada del modelo.

```

1 from dataclasses import dataclass
2 import numpy as np
3
4

```

```

5 @dataclass(frozen=True)
6 class Quimiostato:
7     mu_max: float = 0.40          # 1/d
8     K_s: float = 25.0             # mg/L
9     Y: float = 0.42               # mg biomasa por mg sustrato
10    D: float = 0.15               # 1/d
11    S_in: float = 800.0           # mg/L
12
13
14 def derivada(t, y, p):
15     """Devuelve dy/dt del modelo, con y = (X, S)."""
16     X, S = y
17     mu = p.mu_max * S / (p.K_s + S)
18     return np.array([(mu - p.D) * X, p.D * (p.S_in - S) - mu * X
19                     / p.Y])

```

El decorador que congela la estructura impide modificar un parámetro por descuido a mitad de una simulación, que es una de las formas más difíciles de rastrear del error silencioso. Con el modelo así escrito, la misma función sirve para integrarlo, para buscar sus estados estacionarios y para derivarlo simbólicamente, que es lo que ocupa la sección siguiente.

2.4. Cálculo simbólico y paso del símbolo al código

El cálculo simbólico manipula expresiones en lugar de números, de manera que una derivada se obtiene exacta y no aproximada, y una sustitución conserva la forma algebraica en vez de colapsarla a un valor [18]. En modelación su utilidad no está en resolver el modelo, que casi nunca tiene solución cerrada, sino en tres tareas auxiliares que ahorran una cantidad desproporcionada de trabajo, a saber, obtener el jacobiano de un sistema, linealizar alrededor de un punto de operación y construir soluciones analíticas de casos límite que después sirven de referencia para verificar el código numérico. El jacobiano, en particular, aparece en casi todos los procedimientos posteriores del libro, desde los integradores implícitos hasta el análisis de estabilidad.

Definición 2.11. Matriz jacobiana. Dada una función vectorial $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^n$ diferenciable, su *matriz jacobiana* en el punto $\mathbf{y}$ es la matriz $\mathbf{J}(\mathbf{y})$ de elementos $J_{ij} = \partial f_i / \partial y_j$, evaluados en ese punto.

Derivar a mano un jacobiano de tres o cuatro estados es tedioso y propenso a errores de signo, y derivarlo numéricamente introduce un error de truncamiento que se confunde con el del método. Obtenerlo de forma simbólica y convertirlo después en una función evaluable elimina ambos problemas. El [Listado 2.10](#) realiza el recorrido completo para el quimiostato del [Listado 2.9](#).

Listado 2.10. Jacobiano simbólico de un modelo de dos estados y su conversión a función evaluable.

```

1 import numpy as np
2 import sympy as sp
3
4 X, S = sp.symbols("X S", nonnegative=True)
5 mu_max, K_s, Y, D, S_in = sp.symbols("mu_max K_s Y D S_in",
6                                         positive=True)
7
8 mu = mu_max * S / (K_s + S)
9 f = sp.Matrix([(mu - D) * X, D * (S_in - S) - mu * X / Y])
10 J = sp.simplify(f.jacobian([X, S]))
11
12 par = {mu_max: 0.40, K_s: 25.0, Y: 0.42, D: 0.15, S_in: 800.0}
13 J_num = sp.lambdify((X, S), J.subs(par), "numpy")
14 autoval = np.linalg.eigvals(np.array(J_num(329.7, 15.0),
15                                         dtype=float))

```

La llamada a `lambdify` es el puente entre los dos mundos, pues toma una expresión simbólica y genera una función de Python que la evalúa con las operaciones de NumPy, de modo que admite arreglos como argumento y se ejecuta a velocidad numérica. Sin ese paso, sustituir valores uno a uno resulta cientos de veces más lento y la ventaja del cálculo exacto se pierde en la ejecución. Los autovalores que devuelve el fragmento valen $-0.15/d$ y $-4.906/d$, ambos negativos, de modo que el estado estacionario con sustrato residual $S^* = 15.0$ mg/L y biomasa $X^* = 329.7$ mg/L es asintóticamente estable. El autovalor de menor magnitud fija la constante de tiempo dominante en 6.67 d, dato que después determina cuánto debe durar una simulación para alcanzar el régimen permanente. El cociente entre ambos, cercano a 33, anticipa además una rigidez moderada.

El jacobiano es también la herramienta que formaliza la aproximación lineal de un modelo alrededor de una condición de operación.

Definición 2.12. Linealización. La *linealización* de un modelo $\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y})$ alrededor de un punto $\mathbf{y}^*$ es el sistema lineal $\dot{\boldsymbol{\eta}} = \mathbf{J}(\mathbf{y}^*) \boldsymbol{\eta}$ que se obtiene al truncar en primer orden la serie de Taylor de $\mathbf{f}$, donde $\boldsymbol{\eta} = \mathbf{y} - \mathbf{y}^*$ es la desviación respecto del punto.

La linealización es el fundamento del diseño de instrumentación y de control, pero también la fuente de un abuso frecuente, que consiste en aplicarla lejos del punto de operación sin cuantificar el error que se comete. El siguiente ejemplo lo evalúa sobre un sensor de temperatura de uso común.

Ejemplo 2.5. Linealización simbólica de un termistor en un divisor de tensión.

Un termistor de coeficiente negativo con resistencia nominal $10\text{ k}\Omega$ a 25°C y constante $B = 3950\text{ K}$ se conecta en un divisor de tensión con una resistencia fija de $10\text{ k}\Omega$ y una alimentación de 3.30 V . La tensión de salida se toma sobre la resistencia fija. Obténgase la sensibilidad del sensor en el punto de operación y determínese el intervalo de temperatura en el que el error de linealidad no excede el equivalente a 0.5 K .

Consideraciones. 1. El autocalentamiento del termistor es despreciable; 2. la constante B es independiente de la temperatura en el intervalo de interés; 3. la etapa de medición no carga el divisor.

Formulación. La resistencia del termistor y la tensión de salida obedecen las relaciones de la Ecuación (2.4),

$$R_T(T) = R_0 e^{B\left(\frac{1}{T} - \frac{1}{T_0}\right)}, \quad V_o(T) = V_{cc} \frac{R_s}{R_s + R_T(T)}, \quad (2.4)$$

donde R_T es la resistencia del termistor (Ω); R_0 es su resistencia nominal a la temperatura de referencia T_0 (Ω); B es la constante característica del material (K); T es la temperatura absoluta (K); R_s es la resistencia fija (Ω); V_{cc} es la tensión de alimentación (V); y V_o es la tensión de salida (V).

Análisis. Derivando simbólicamente la Ecuación (2.4) y evaluando en $T = T_0$, donde $R_T = R_0$, la sensibilidad se reduce a la expresión cerrada de la Ecuación (2.5),

$$\left. \frac{dV_o}{dT} \right|_{T_0} = \frac{B R_0 R_s V_{cc}}{T_0^2 (R_0 + R_s)^2}, \quad (2.5)$$

en la que se aprecia de inmediato que la sensibilidad se maximiza cuando $R_s = R_0$.

Solución. En el punto de operación $T_0 = 298.15\text{ K}$ resulta $V_o = 1.650\text{ V}$ y la Ecuación (2.5) entrega 36.659 mV/K . La segunda derivada vale $-2.459 \times 10^{-4}\text{ V/K}^2$, de modo que el término cuadrático de Taylor predice un error de 12.3 mV a 10 K del punto. El error real de la aproximación lineal es de 0.54 mV a 2 K , de 3.74 mV a 5 K , de 17.26 mV a 10 K y de 83.23 mV a 20 K .

Verificación. Como error equivalente de temperatura, esos valores corresponden a 0.01 K , 0.10 K , 0.47 K y 2.27 K . El intervalo que cumple el requisito de 0.5 K va desde 24.2 K por debajo del punto de operación hasta 10.3 K por encima, es decir, de 1°C a 35°C .

Comentario. La asimetría del intervalo es la información más valiosa del ejercicio. La curva de respuesta es cóncava, de modo que la aproximación lineal se degrada mucho más rápido hacia arriba que hacia abajo, y un diseñador que fijara el intervalo útil de forma simétrica alrededor de 25 °C subestimaría el error en el extremo caliente por un factor cercano a cinco.

La segunda tarea en la que rinde el cálculo simbólico es la construcción de soluciones de referencia. Casi ningún modelo de ingeniería tiene solución cerrada, pero casi todos tienen un caso límite que sí la tiene, obtenido al anular un término, al suponer un parámetro constante o al llevar una condición de frontera al infinito. Esa solución cerrada permite verificar que el código numérico resuelve la ecuación que dice resolver [2]. La tercera tarea es la comprobación de coherencia dimensional, pues un modelo puede ser dimensionalmente incorrecto y aun así producir números. El [Listado 2.11](#) comprueba que una constante de tiempo formada con una capacidad calorífica y un coeficiente convectivo tiene efectivamente dimensión de tiempo.

Listado 2.11. Comprobación de la coherencia dimensional de una expresión del modelo.

```
1 from sympy.physics.units import (Dimension, convert_to, joule,
   kelvin,
2                                     kilogram, meter, second, watt)
3 from sympy.physics.units.systems.si import SI
4
5 h = 25 * watt / (meter**2 * kelvin)          # coeficiente
   convectivo
6 A = 0.48 * meter**2                          # área de intercambio
7 m, cp = 1.2 * kilogram, 3900 * joule / (kilogram * kelvin)
8
9 tau = m * cp / (h * A)                      # constante de tiempo
10 dim = SI.get_dimension_system().get_dimensional_dependencies(
11     Dimension(SI.get_dimensional_expr(tau)))
12 assert dim == {Dimension("time"): 1}, f"dimensión inesperada:
   {dim}"
13 tau_s = convert_to(tau, second)
```

El fragmento devuelve 390 s, y lo importante no es el valor sino que la aserción se ejecute cada vez que el módulo se importa. Si alguien invierte por descuido el cociente y escribe el producto $mc_p h A$, la comprobación falla de inmediato con un diagnóstico que nombra la dimensión encontrada, en lugar de dejar circular un número con unidades absurdas hasta el informe final.

Conviene por último delimitar el alcance del cálculo simbólico, porque su promesa es seductora y sus límites llegan pronto. Ayuda cuando la expresión es corta y la exactitud importa, como en un jacobiano, en una linealización o en la integración de una ley cinética sencilla. Estorba cuando el número de términos crece de manera combinatoria, situación

que se presenta al derivar sistemas de más de cinco o seis estados con parámetros literales, al invertir matrices simbólicas de tamaño mediano o al resolver de forma exacta una ecuación diferencial no lineal. En esos casos la expresión ocupa páginas, se evalúa con lentitud y sufre de cancelación al sustituir valores. La [Figura 2.10](#) resume el paso del modelo simbólico a la función evaluable y separa las tareas en las que el cálculo simbólico rinde de aquellas en las que estorba.

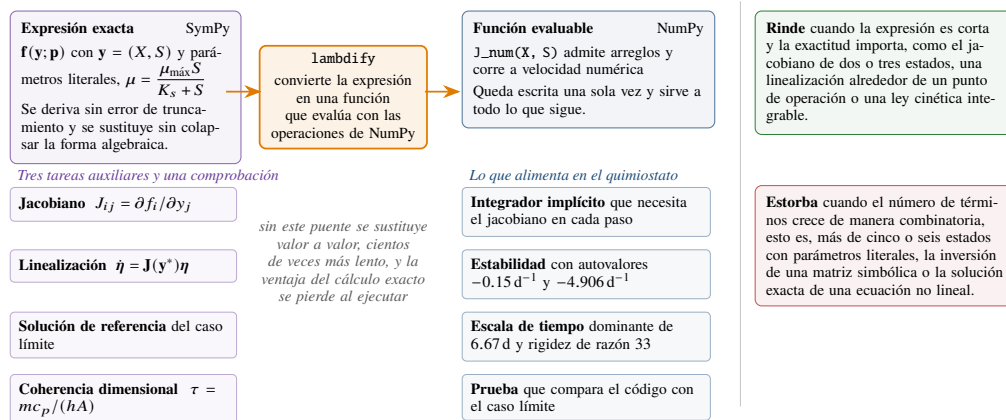


Figura 2.10. Paso del modelo simbólico a la función evaluable, con las tareas en que el cálculo simbólico rinde y aquellas en que estorba.

2.5. Visualización e interpretación preliminar de resultados

La primera figura de una simulación no se hace para el informe, se hace para uno mismo. Su propósito es detectar lo que las tablas esconden, es decir, la oscilación que delata un paso de tiempo excesivo, la deriva que denuncia una fuga de masa o el tramo plano que indica que una variable se saturó contra un límite del código. Para eso conviene entender la gramática de Matplotlib, que organiza toda figura en tres niveles jerárquicos identificados en la [Figura 2.11](#) [27].

El lienzo contiene uno o varios sistemas de ejes, y cada uno contiene los artistas, que son los objetos dibujables como líneas, marcadores, textos y leyendas. La consecuencia práctica es que conviene programar sobre el objeto de ejes y no sobre la interfaz de estado global, pues el objeto puede pasarse como argumento, reutilizarse en una subgráfica y probarse. Una función que recibe un sistema de ejes y dibuja en él es componible; una que dibuja en la figura activa no lo es. El [Listado 2.12](#) sigue ese estilo.

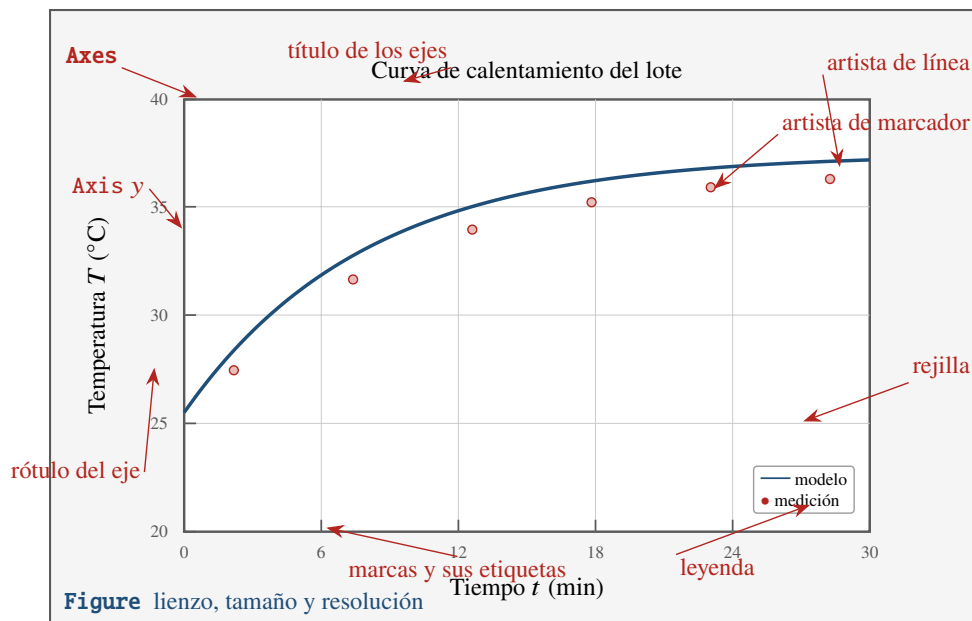


Figura 2.11. Anatomía de una figura de Matplotlib, con el lienzo, el sistema de ejes y los artistas que lo pueblan.

Listado 2.12. Figura de diagnóstico con ejes rotulados, unidades y referencia física explícita.

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 t = np.arange(0.0, 2401.0, 11.0)
5 T = 121.0 - 96.0 * np.exp(-t / 1450.0)
6
7 fig, ax = plt.subplots(figsize=(13.5 / 2.54, 7.0 / 2.54),
8                           layout="constrained")
9 ax.plot(t / 60.0, T, color="#1F4E79", lw=1.4, label="paso
10         estable")
11 ax.axhline(121.0, color="#5A5A5A", lw=0.8, ls="--") # medio
12         calefactor
13 ax.set_xlabel("Tiempo desde el inicio del proceso (min)")
14 ax.set_ylabel("Temperatura en el centro de la lata (°C)")
15 ax.set_xlim(0.0, 40.0)
16 ax.set_ylim(20.0, 130.0)
17 ax.legend(loc="lower right")
18 fig.savefig("figuras/centro_lata.pdf")

```

Cuatro decisiones del fragmento son deliberadas. Los ejes llevan su magnitud y su unidad, la referencia física relevante aparece como una línea horizontal que permite juzgar el resultado sin calcular, los límites se fijan de manera explícita para que la figura no cambie de escala al cambiar los datos, y la exportación se hace en formato vectorial [49].

Esas decisiones tienen su contraparte. Ciertas elecciones gráficas, cometidas casi siempre sin intención de engañar, distorsionan la lectura de manera sistemática y sobreviven al escrutinio porque la figura se ve profesional [50]. El eje vertical truncado magnifica diferencias pequeñas de una magnitud acotada, el doble eje ajustado sugiere una relación entre series que solo comparten estacionalidad, la escala transformada sin declarar convierte un decaimiento exponencial en una recta aparente y el promedio sin dispersión presenta como tendencia una diferencia menor que el ruido. La Figura 2.12 muestra las cuatro junto a su corrección, que consiste, respectivamente, en anclar el eje en cero, en graficar la relación directa entre las series, en rotular el eje como logarítmico y en añadir la dispersión con los datos.

El segundo par de paneles merece comentario aparte porque su desenlace es el menos intuitivo. Las dos series comparten un ciclo anual y, superpuestas en ejes ajustados, exhiben una correlación de 0.923 que invita a proponer un mecanismo causal. Al retirar de ambas la componente estacional, la correlación de los residuos cae a 0.212, valor que con treinta y seis observaciones no sostiene ninguna afirmación.

Una vez producida la figura hay que leerla, y esa lectura sigue un orden. Se comprueba primero el orden de magnitud frente al valor esperado del proceso, después el signo y el sentido de la tendencia, y luego los extremos, es decir, el valor inicial y el asintótico. La cuarta comprobación rechaza el resultado si alguna variable excede una cota física del sistema, y la quinta lo rechaza si aparecen oscilaciones de periodo igual al doble del paso de integración. Solo si las cinco se superan tiene sentido repetir el cálculo con la malla o el paso refinados a la mitad, y aceptar el resultado únicamente cuando el cambio queda por debajo de la tolerancia declarada.

La cuarta y la quinta comprobación son las que más resultados salvan, porque atrapan el artefacto numérico antes de que se convierta en una afirmación sobre el proceso. La Tabla 2.5 recoge un caso de tratamiento térmico en el que ambos criterios se activan, y el ejemplo que le sigue lo desarrolla.

Ejemplo 2.6. Lectura preliminar de un perfil térmico simulado. El centro de una conserva de 40 mm de espesor, con difusividad térmica de $1.4 \times 10^{-7} \text{ m}^2/\text{s}$, se simula mediante un esquema explícito de diferencias finitas sobre once nodos, a partir de 25°C y con las superficies mantenidas a 121°C . Un analista escoge un paso de tiempo de 15 s por comodidad de registro. Interpretese el resultado antes de aceptarlo.

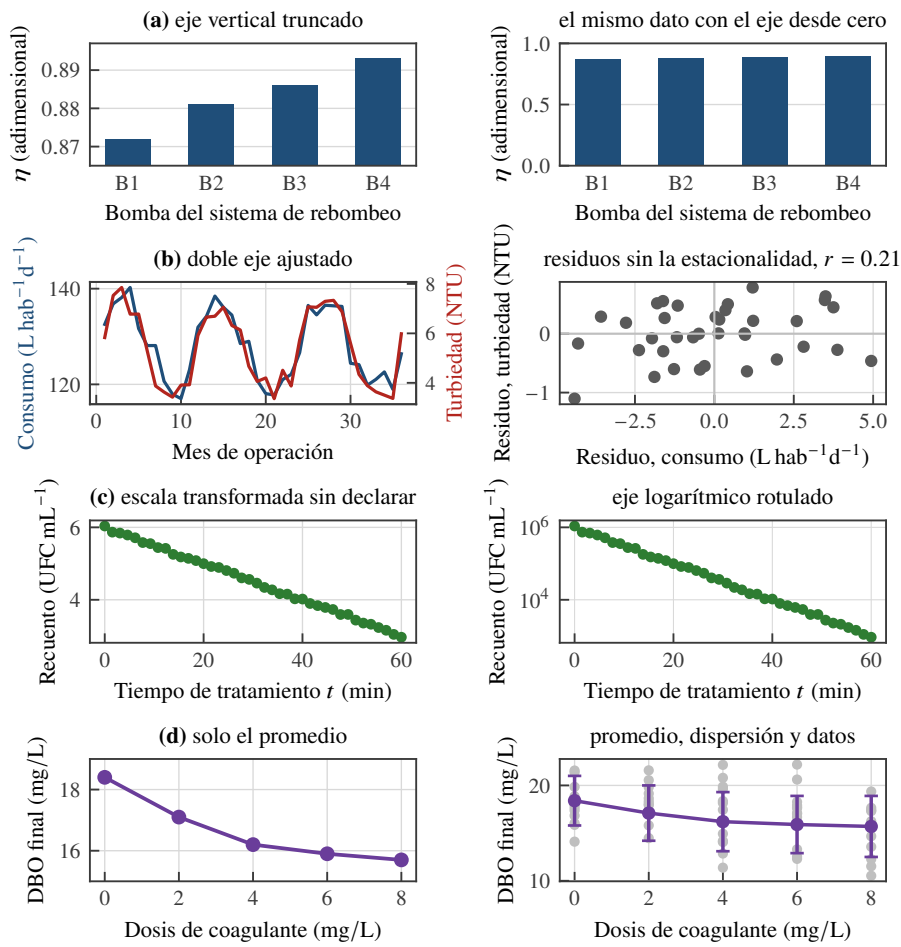


Figura 2.12. Cuatro decisiones gráficas que distorsionan la lectura, cada una junto a la representación que la corrige.

Consideraciones. 1. Conducción unidimensional con simetría en el plano medio; 2. propiedades térmicas constantes; 3. resistencia externa despreciable frente a la interna.

Análisis. El esquema explícito avanza mediante el número de Fourier de malla de la Ecuación (2.6),

$$Fo = \frac{\alpha \Delta t}{\Delta x^2}, \quad (2.6)$$

donde α es la difusividad térmica del alimento (m²/s); Δt es el paso de tiempo (s); y Δx es el espaciamiento de la malla (m). Con media placa de 20 mm y once nodos, $\Delta x = 2$ mm.

Tabla 2.5. Temperaturas calculadas con un paso estable y con un paso que excede el límite de estabilidad.

Tiempo (min)	$\Delta t = 11 \text{ s}$	$\Delta t = 15 \text{ s}$	
	centro ($^{\circ}\text{C}$)	centro ($^{\circ}\text{C}$)	6 mm ($^{\circ}\text{C}$)
5	30.41	32.80	34.50
10	49.08	68.85	37.68
15	65.17	177.37	-29.48
20	77.81	678.30	-452.85
25	87.60	3279.65	-2752.95

^a El paso de 15 s corresponde a un número de Fourier de malla de 0.525, superior al límite de 0.5.

Solución. El paso elegido da $Fo = 0.525$, mientras que el paso de 11 s usado como referencia da $Fo = 0.385$. La [Tabla 2.5](#) recoge la temperatura calculada en el centro y en el nodo situado a 6 mm del plano medio. Con el paso mayor la temperatura del centro sube a 68.85°C a los 10 min, cae a 177.37°C a los 15 min y alcanza 678.30°C a los 20 min, mientras el nodo intermedio se vuelve negativo.

Verificación. La cuarta comprobación rechaza el resultado en cuanto algún nodo excede la temperatura del medio, lo cual ocurre por primera vez a los 795 s, esto es, a los 13.25 minutos de proceso. La quinta lo habría rechazado antes, pues los nodos oscilan con periodo igual al doble del paso desde las primeras iteraciones.

Comentario. El paso elegido excede en apenas un 5 % el límite de 14.286 s que impone la condición $Fo \leq 1/2$, y ese exceso mínimo basta para destruir la solución. Lo peligroso es la primera parte de la tabla, donde los valores todavía parecen razonables, pues un analista que reportara únicamente el instante de 10 min habría publicado un número equivocado en 40 % sin ningún síntoma visible.

Las figuras destinadas al informe exigen un cuidado adicional que no es estético sino funcional. Deben usar una paleta distinguible por lectores con deficiencia en la percepción del rojo y del verde, y no deben depender únicamente del color para separar series, sino combinarlo con el tipo de trazo o el marcador. Conviene además que el pie describa lo que se ve y no lo que se concluye. Una figura que necesita ser explicada por su autor todavía no está terminada.

2.6. Buenas prácticas, verificación del código y asistentes

Un modelo escrito para durar se organiza en funciones pequeñas, y la propiedad que distingue a las funciones fáciles de probar es la pureza.

Definición 2.13. Función pura. Una *función pura* es aquella cuyo valor de retorno depende únicamente de sus argumentos y que no modifica ningún estado externo, de modo que dos llamadas con los mismos argumentos devuelven siempre el mismo resultado.

La pureza no es un dogma sino una conveniencia, pues una función pura puede probarse en aislamiento, ejecutarse en paralelo sin riesgo y sustituirse por su resultado sin alterar el programa. Las funciones con efecto lateral son necesarias en los extremos del proyecto, donde se leen archivos y se escriben figuras, pero el núcleo del modelo debe mantenerse puro. El incumplimiento más común de esta regla en Python es también el más silencioso, y proviene de usar un objeto mutable como valor por omisión de un argumento, tal como muestra el [Listado 2.13](#).

Listado 2.13. Argumento mutable por omisión y su reescritura como función pura.

```
1 def promedio_acumulado(q, historial=[]):      # incorrecto
2     """La lista por omisión se crea una sola vez y sobrevive."""
3     historial.append(q)
4     return sum(historial) / len(historial)
5
6
7 def promedio_puro(historial, q):
8     """Devuelve el nuevo historial y su media, sin alterar el
9         original."""
10    nuevo = (*historial, q)
11    return nuevo, sum(nuevo) / len(nuevo)
```

La primera función devuelve 41.8, 42.05 y 42.4 al procesar las tres primeras lecturas de la estación de bombeo, que son los valores correctos. Al comenzar una serie nueva con una única lectura de 10.0 devuelve 34.3 en lugar de 10.0, porque la lista por omisión se creó al definir la función y conserva las lecturas anteriores. El error no levanta excepción alguna y solo se manifiesta cuando alguien compara dos corridas. La segunda versión no puede fallar así porque no guarda estado.

Las anotaciones de tipo cumplen una función parecida a la de las unidades en una fórmula, pues documentan la intención de manera verificable. Escribir que una función recibe un arreglo de flotantes y devuelve un escalar no impide pasarle una lista, ya que Python no las hace cumplir en tiempo de ejecución, pero permite que una herramienta de análisis estático detecte la incompatibilidad. En modelación la anotación más valiosa no es la del tipo sino la de la unidad, y como el lenguaje no la ofrece, se suple con el nombre del argumento y con la línea de documentación.

Las excepciones y las aserciones son el mecanismo por el cual un modelo declara su propio dominio de validez. Una función que calcula el caudal por la ecuación de Manning no tiene sentido con rugosidad nula ni con pendiente negativa, y debe negarse a hacerlo en lugar de devolver un número. La excepción se lanza ante una entrada externa inválida y el usuario debe poder manejarla, mientras que la aserción comprueba una condición que el propio código garantiza y cuyo incumplimiento indica un defecto de programación. El [Listado 2.14](#) aplica los dos.

Listado 2.14. Aserciones de dominio físico como contrato explícito de una función del modelo.

```

1 import numpy as np
2
3
4 def caudal_manning(n, A, R_h, S_0):
5     """Caudal por la ecuación de Manning, en m3/s."""
6     if not (0.008 <= n <= 0.20):
7         raise ValueError(f"rugosidad fuera de rango: n = {n}")
8     if A <= 0.0 or R_h <= 0.0:
9         raise ValueError("el área y el radio hidráulico deben ser
10                             positivos")
11     if not (1e-6 <= S_0 <= 0.20):
12         raise ValueError(f"pendiente fuera de rango: S_0 = {S_0}")
13     Q = A * R_h ** (2.0 / 3.0) * np.sqrt(S_0) / n
14     assert np.isfinite(Q) and Q > 0.0, "caudal no físico"
15     return Q

```

Con una rugosidad de 0.014, un área de 2.40 m², un radio hidráulico de 0.62 m y una pendiente de 0.0015, la función devuelve 4.8275 m³/s, y con rugosidad nula lanza una excepción que nombra el valor ofensivo en lugar de devolver un infinito que contaminaría todo el cálculo aguas abajo.

Las pruebas automáticas de un modelo no se parecen a las de un programa administrativo, pues no comprueban que una función devuelva un valor previamente anotado sino que el modelo respete una ley física. Tres pruebas bastan para atrapar la mayoría de los errores estructurales, a saber, la comparación con un caso límite de solución analítica, el cierre de un balance de masa o de energía y la invariancia del resultado ante un cambio coherente de unidades. El [Listado 2.15](#) implementa la segunda sobre una cadena de dos tanques.

Listado 2.15. Prueba de conservación de masa aplicada a un modelo dinámico cerrado.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4
5 def tanques(t, y, k):
6     """Dos tanques en serie sin fuentes ni sumideros, en kg."""
7     return np.array([-k * y[0], k * y[0] - k * y[1], k * y[1]])
8
9
10 def test_conservacion_masa():
11     y0 = np.array([120.0, 0.0, 0.0]) # kg
12     sol = solve_ivp(tanques, (0.0, 40.0), y0, args=(0.18,),
13                     rtol=1e-9, atol=1e-12)
14     deriva = np.max(np.abs(sol.y.sum(axis=0) - y0.sum()))
15     assert deriva < 1e-8 * y0.sum(), f"fuga de masa de
16         {deriva:.3e} kg"
17     return deriva

```

La deriva de masa que arroja la prueba es de 4.263×10^{-14} kg sobre un total de 120 kg, del orden de la precisión de la máquina, lo cual confirma que el modelo conserva. Si alguien cambiara un signo en la segunda ecuación, la prueba fallaría de inmediato con un mensaje que cuantifica la fuga. La [Tabla 2.6](#) generaliza esas comprobaciones.

Todo lo anterior apunta a un mismo adversario, que es la clase de fallas que no se anuncian.

Definición 2.14. Error silencioso. Un *error silencioso* es aquel que no interrumpe la ejecución ni produce mensaje alguno, y cuyo resultado es un número del orden de magnitud esperado, de modo que solo puede detectarse mediante una comprobación independiente del propio cálculo.

Tabla 2.6. Catálogo de verificaciones mínimas aplicables a todo modelo antes de usar sus resultados.

Verificación	Qué se comprueba	Error que atrapa
Coherencia dimensional	unidades de cada término	factor de conversión omitido
Caso límite analítico	coincidencia con la solución exacta	signo cambiado o índice desplazado
Balance global	conservación de masa o energía	fuga por condición de frontera
Dominio de validez	rechazo de entradas imposibles	extrapolación silenciosa
Invariancia de unidades	mismo resultado en otro sistema	constante empírica mal aplicada
Refinamiento	independencia de la malla y del paso	error de discretización
Orden de magnitud	escala esperada del proceso	error de un factor de mil

^a Las tres primeras deben quedar escritas como pruebas ejecutables dentro del repositorio del proyecto.

El error silencioso sobrevive a la revisión precisamente porque nada llama la atención sobre él. Un signo cambiado en un término pequeño, un índice desplazado en un extremo de la malla, una constante empírica aplicada en unidades distintas de aquellas en que fue ajustada o un promedio calculado sobre un eje equivocado producen resultados plausibles. La única defensa es un protocolo aplicado siempre y no solo cuando algo parece raro, ya que por definición nada parecerá raro. La [Figura 2.13](#) recoge ese protocolo y el [Algoritmo 2.3](#) lo expresa como procedimiento.

El protocolo adquiere urgencia particular frente al código sugerido por un asistente automático. Un modelo de lenguaje produce código sintácticamente correcto, idiomático y bien comentado con la misma facilidad con la que produce código que resuelve un problema ligeramente distinto del planteado. El riesgo no está en el disparate evidente, que cualquiera detecta, sino en la solución plausible que usa una fórmula válida en condiciones que aquí no se cumplen. La posición de la asignatura es que el ingeniero responde por el resultado que firma con independencia de quién haya tecleado las líneas, de modo que todo fragmento de origen asistido se declara en el encabezado del módulo y se somete al protocolo completo.

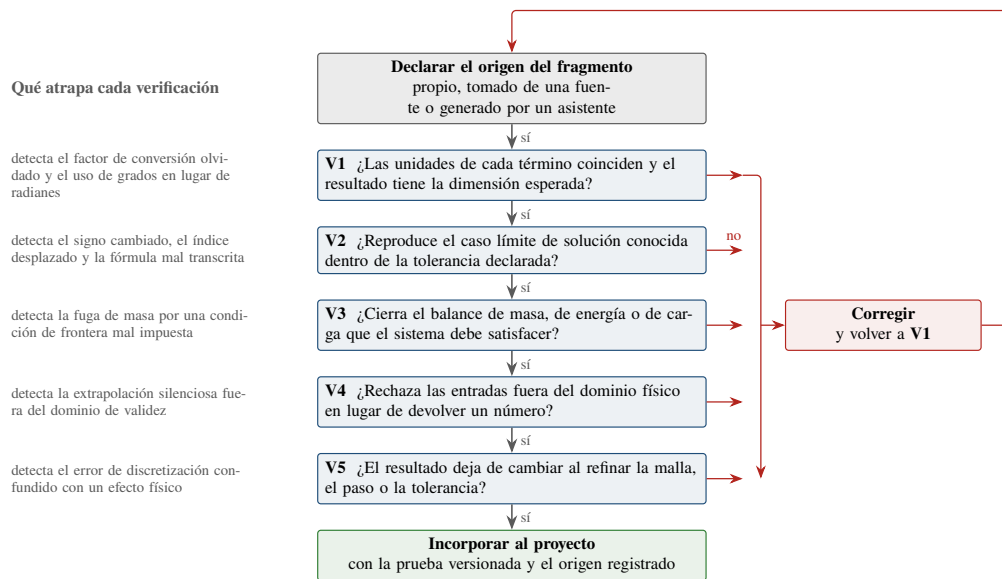


Figura 2.13. Protocolo de verificación de un fragmento de código propio o generado por un asistente, con lo que atrapa cada puerta.

Algoritmo 2.3 Verificación de un fragmento de código antes de incorporarlo al proyecto.

Entradas: Fragmento F , origen declarado, caso límite L con solución conocida u^*

Salidas: Decisión de incorporar o corregir, y registro de la prueba

- 1: Registrar el origen de F en el encabezado del módulo
 - 2: Comprobar la dimensión de cada término y del resultado
 - 3: $u \leftarrow$ evaluar F sobre L
 - 4: **si** $\|u - u^*\|$ excede la tolerancia o el balance no cierra **entonces**
 - 5: **devolver** corregir
 - 6: **fin si**
 - 7: **para** cada entrada fuera del dominio físico **hacer**
 - 8: Comprobar que F levanta una excepción
 - 9: **fin para**
 - 10: Repetir con la malla, el paso y la tolerancia refinados
 - 11: **si** el resultado cambia significativamente **entonces**
 - 12: **devolver** corregir
 - 13: **fin si**
 - 14: Escribir las comprobaciones como pruebas y versionarlas
 - 15: **devolver** incorporar
-

Ejemplo 2.7. Diagnóstico de un fragmento asistido con error silencioso. Un piranómetro registra la irradiancia global sobre el plano de un arreglo fotovoltaico cada 15 min entre las 6:00 a.m. y las 6:00 p.m., para un total de 49 muestras. Una falla del registrador cerca del mediodía elimina tres muestras consecutivas. Un asistente automático propone la función `energia_diaria` para estimar la energía generada, con área de módulos de 34 m^2 y rendimiento global del sistema de 0.156. Verifíquese el fragmento.

Consideraciones. 1. El arreglo tiene una potencia nominal de 6.80 kW; 2. el rendimiento global incluye el módulo y las pérdidas del sistema; 3. la irradiancia máxima registrada es de 962.3 W/m^2 .

Análisis. El fragmento propuesto calcula la integral con la regla del trapecio suponiendo espaciamiento uniforme, mediante `np.trapezoid(irradiancia, dx=900.0)`. La energía diaria responde a la Ecuación (2.7),

$$E = \frac{A \eta}{3.6 \times 10^6} \int_{t_0}^{t_f} G(t) dt, \quad (2.7)$$

donde E es la energía generada en el día (kW h); A es el área total de módulos (m^2); η es el rendimiento global del sistema (adimensional); G es la irradiancia sobre el plano del arreglo (W/m^2); y el divisor convierte julios en kilovatios hora.

Solución. Con la serie completa la Ecuación (2.7) entrega 35.374 kW h, que corresponde a una irradiación de 6.669 kW h/m^2 y a un rendimiento específico de 5.20 kW h/kW . El fragmento asistido, aplicado a las 46 muestras sobrevivientes, devuelve 31.597 kW h, un 10.7 % por debajo. Al pasar la marca de tiempo real como abscisa en lugar del espaciamiento supuesto, el resultado sube a 35.322 kW h, con una desviación de apenas 0.15 % respecto de la serie completa.

Verificación. El fragmento supone implícitamente una duración de $(46 - 1) \times 900 \text{ s} = 11.25 \text{ h}$, mientras que el registro abarca 12.00 h. Esa discrepancia es la comprobación que lo delata. El rendimiento específico que produce, de 4.65 kW h/kW , es plausible para la región y por sí solo no habría levantado sospecha.

Comentario. El fragmento no contiene ningún error de programación, pues la función existe, los argumentos son válidos y el resultado tiene las unidades correctas. El error está en una hipótesis no declarada, la del muestreo uniforme, que era cierta cuando el asistente escribió el código y dejó de serlo cuando el registrador falló. Un fragmento que reciba explícitamente la marca de tiempo no puede cometer ese error.

El protocolo de declaración que exige la asignatura consta de tres elementos y no admite excepciones. El módulo indica en su encabezado qué partes fueron sugeridas por un asistente y cuáles se escribieron a mano; cada fragmento de origen asistido lleva asociada al menos una prueba ejecutable que comprueba una propiedad física del resultado y no simplemente su valor; y el informe final declara el uso de la herramienta junto con los supuestos y las limitaciones del modelo. Esa declaración permite a quien audite el trabajo concentrar la revisión donde el riesgo es mayor.

Con esto queda montada la infraestructura completa. El proyecto es reproducible, los datos entran verificados, el modelo se representa con arreglos y se deriva con símbolos, los resultados se leen antes de creerse y todo fragmento de código pasa por un protocolo. La [Figura 2.14](#) enumera lo que queda escrito al terminar la unidad, lo que esa infraestructura permite afirmar y lo que todavía no autoriza a afirmar, porque un modelo bien instalado no es aún un modelo validado. El [Capítulo 3](#) usa esa infraestructura para simular sistemas estacionarios, dinámicos y distribuidos, y el [Capítulo 4](#) la somete al escrutinio de la validación y de la incertidumbre.

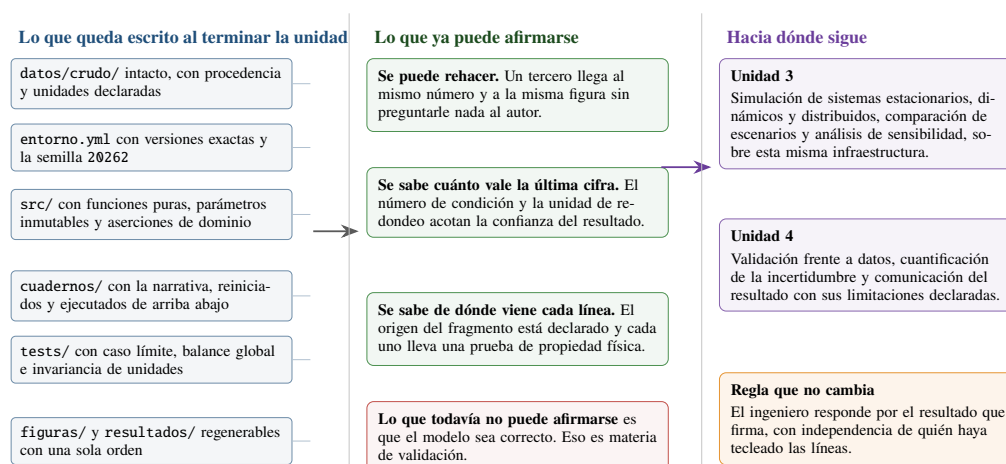


Figura 2.14. Infraestructura que queda instalada al terminar la unidad, con lo que permite afirmar y lo que todavía queda pendiente.

2.7. Problemas

Los problemas siguientes recorren el capítulo en complejidad creciente. Salvo indicación contraria se trabaja con la semilla 20262 y se declara el entorno de ejecución junto con la respuesta.

2.7.1. Apropiación conceptual

- 2-1 Explique por qué un análisis reproducible puede ser erróneo, y por qué la reproducibilidad sigue siendo condición necesaria del trabajo profesional.
- 2-2 Un colega dice que su proyecto es reproducible porque guardó las figuras en el repositorio. Dé tres razones por las cuales eso es falso.
- 2-3 Distinga vista de copia de un arreglo, y describa un caso de modelación en el que confundirlas dé un resultado equivocado sin levantar excepción.
- 2-4 Defina el punto de ruptura y explique por qué la desviación estándar es inservible para detectar atípicos con un diez por ciento de lecturas espurias.
- 2-5 Distinga entre lanzar una excepción y afirmar una aserción, e indique cuál corresponde a una pendiente negativa recibida y cuál a un caudal infinito.

2.7.2. Preparación y verificación de datos

- 2-6 Un archivo contiene la fila 2026-04-12 07;30;00;12,4;1,03. Explique qué produce leerla con separador de punto y coma y coma decimal.
- 2-7 Una serie de nivel tiene 18 marcas duplicadas en 8760 registros horarios. Estime el efecto sobre el volumen medio anual si se concentran en el estiaje.
- 2-8 Sobre 500 lecturas de conductividad se obtienen $\bar{x} = 1.42$ dS/m, $s = 0.31$ dS/m, mediana 1.38 dS/m y MAD = 0.06 dS/m. Calcule la escala robusta y explique la discrepancia.
- 2-9 Un promedio diario de evapotranspiración usa 6 de las 48 lecturas esperadas. Discuta si debe reportarse y proponga un umbral de cobertura.
- 2-10 Proponga las verificaciones V1 a V6 con sus límites numéricos para una temperatura de pasteurización registrada cada 2 s en un intercambiador de placas.

2.7.3. Representación numérica y condicionamiento

- 2-11 Determine la forma de sumar un arreglo (365, 1) con uno (24,) y qué representa si el primero trae temperaturas diarias y el segundo un perfil horario.
- 2-12 Un sistema de balances tiene $\kappa_2 = 4.7 \times 10^6$ y coeficientes con seis cifras. Estime las cifras confiables y si sirven para dimensionar una tubería.
- 2-13 Demuestre con el [Teorema 2.2](#) que formar las ecuaciones normales duplica las cifras perdidas frente a resolver los mínimos cuadrados directamente.

- 2–14** Calcule el error relativo de la fórmula habitual para la raíz menor de $x^2 + bx + 1 = 0$ con $b = 10^6$, $b = 10^8$ y $b = 10^{10}$, y explique la tendencia.
- 2–15** Un balance resta dos entalpías cercanas a 2750 kJ/kg que difieren en 0.8 kJ/kg. Estime las cifras que conserva y evite la cancelación.

2.7.4. Cálculo simbólico y linealización

- 2–16** Obtenga el jacobiano simbólico de dos tanques en serie con descarga por orificio y discuta qué ocurre cuando un nivel tiende a cero.
- 2–17** Linealice la ecuación de Arrhenius en 80 °C con energía de activación de 85 kJ/mol y halle el intervalo con error de linealidad menor al cinco por ciento.
- 2–18** Deduzca la sensibilidad de un divisor con termistor respecto de la resistencia fija y demuestre que se maximiza cuando ambas se igualan en el punto de operación.
- 2–19** Discuta en qué situaciones de la ingeniería de la Facultad el cálculo simbólico deja de ser útil, y proponga un criterio para decidir cuándo abandonarlo.

2.7.5. Problemas de programación

- 2–20** Amplíe verificar del [Listado 2.3](#) para que reporte el hueco más largo, y aplíquela a 2000 registros con diez por ciento de faltantes agrupados.
- 2–21** Aplique el criterio robusto sobre el residuo de una mediana móvil, con la ventana como argumento, y estudie las detecciones al variarla entre 5 y 51.
- 2–22** Vectorice un cálculo de evapotranspiración sobre 365 días por 50 estaciones y mida con repeticiones calibradas la razón frente a los bucles anidados.
- 2–23** Compare la suma compensada, la secuencial y la reducción de la biblioteca sobre un año de potencia fotovoltaica horaria, frente a una suma exacta en aritmética racional.
- 2–24** Escriba un módulo con los parámetros de un digestor anaerobio como estructura inmutable, la derivada y tres pruebas de caso límite, balance de masa e invariancia de unidades.
- 2–25** Reproduzca los cuatro pares de la [Figura 2.12](#) con datos de su área y redacte la frase que un lector desprevenido concluiría de cada versión engañosa.

2.7.6. Problemas de diseño y ensayo

- 2–26** Diseñe un proyecto reproducible para el monitoreo de calidad de agua de un río, con carpetas, entorno declarativo, descripción y política de versionado justificados.
- 2–27** Aplique el [Algoritmo 2.3](#) a un guion de análisis ajeno y redacte un informe con las anomalías, la corrección propuesta y las pruebas agregadas.
- 2–28** Un informe muestra la eficiencia de cuatro alternativas fotovoltaicas en barras con el eje entre 0.86 y 0.90. Critíquelo y discuta si la diferencia justifica la inversión.
- 2–29** Argumente en dos páginas cuándo un asistente automático mejora la calidad de un modelo de ingeniería y cuándo la degrada, con dos ejemplos de error silencioso.
- 2–30** Proponga un catálogo de verificaciones para su línea de investigación, con seis entradas que den la propiedad física comprobada, la prueba ejecutable y la tolerancia.

Capítulo 3

Simulación de Sistemas y Análisis de Escenarios

Un modelo escrito no sirve de nada mientras no se ejecute. El [Capítulo 1](#) enseñó a decidir qué se representa y qué se desprecia, y el [Capítulo 2](#) instaló el entorno, ordenó los datos y tradujo el modelo a código. Queda la parte que justifica todo el esfuerzo anterior, que consiste en hacer funcionar la representación bajo condiciones que nadie ha observado y extraer de allí una recomendación de ingeniería. Ese acto de ejecución dirigida es la simulación, y este capítulo se ocupa de ella en sus tres formas habituales, el régimen permanente, la evolución en el tiempo y la variación en el espacio.

El obstáculo no es conceptual sino operativo. Un balance de masa en régimen permanente conduce a un sistema de ecuaciones que casi nunca tiene solución cerrada, y el ingeniero debe elegir un método de solución, un punto de partida y un criterio de parada sin los cuales el programa devuelve un número sin significado. Una ecuación diferencial que describe el arranque de un fermentador exige elegir un integrador, unas tolerancias y, cuando el sistema mezcla escalas de tiempo muy distintas, un método capaz de sortear una restricción de estabilidad que multiplicaría por mil el costo de cómputo. Una pluma de contaminante que avanza por un río obliga a discretizar el espacio antes de poder integrar el tiempo.

El capítulo no deduce los métodos numéricos, tarea que corresponde a un curso especializado, sino que enseña el criterio para elegirlos, invocarlos correctamente y desconfiar de lo que devuelven. Cada método se presenta con la pregunta que responde, con la condición bajo la cual falla y con la verificación que permite detectar la falla antes de que llegue al informe, siempre contra una solución analítica, un caso límite o un balance que debe cerrar.

La segunda mitad cambia de escala, porque una corrida aislada rara vez responde a una pregunta de ingeniería, y de allí surgen la construcción sistemática de escenarios, la propagación de la variabilidad por muestreo aleatorio, el análisis de sensibilidad y la discusión de los resultados en función del problema de estudio. Lo que este capítulo produce son resultados, y decidir cuánto de ellos es creíble corresponde al [Capítulo 4](#). La [Figura 3.1](#) traza esa ruta, desde el régimen del modelo hasta la pregunta que una corrida no puede responder por sí sola.

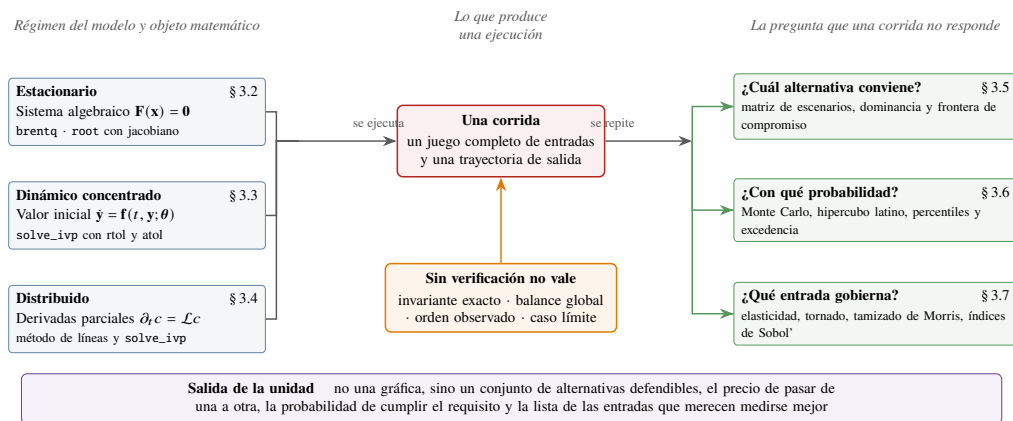


Figura 3.1. Ruta de la unidad, desde el régimen del modelo hasta la recomendación defendible, con la sección que atiende cada pregunta.

3.1. Simulación en estado estacionario

El régimen permanente es la condición en la cual las acumulaciones se anulan y el estado deja de depender del tiempo. Un balance escrito en esa condición pierde el término derivativo y se convierte en una relación algebraica entre las variables de estado, los parámetros y las condiciones de frontera, de modo que la simulación estacionaria resuelve un sistema de ecuaciones algebraicas casi siempre no lineales cuya solución es el punto de operación. Ese es el régimen más frecuente en el diseño, porque dimensionar una tubería o una laguna se hace sobre la condición sostenida y no sobre el arranque.

Conviene situar este régimen dentro del panorama completo antes de entrar en los métodos. La [Figura 3.2](#) organiza los cuatro casos que resultan de cruzar el tratamiento del tiempo con el del espacio, e indica para cada uno la forma matemática del modelo y la familia de algoritmos que le corresponde. La aleatoriedad de las entradas se superpone a cualquiera de los cuatro y no constituye una quinta categoría.

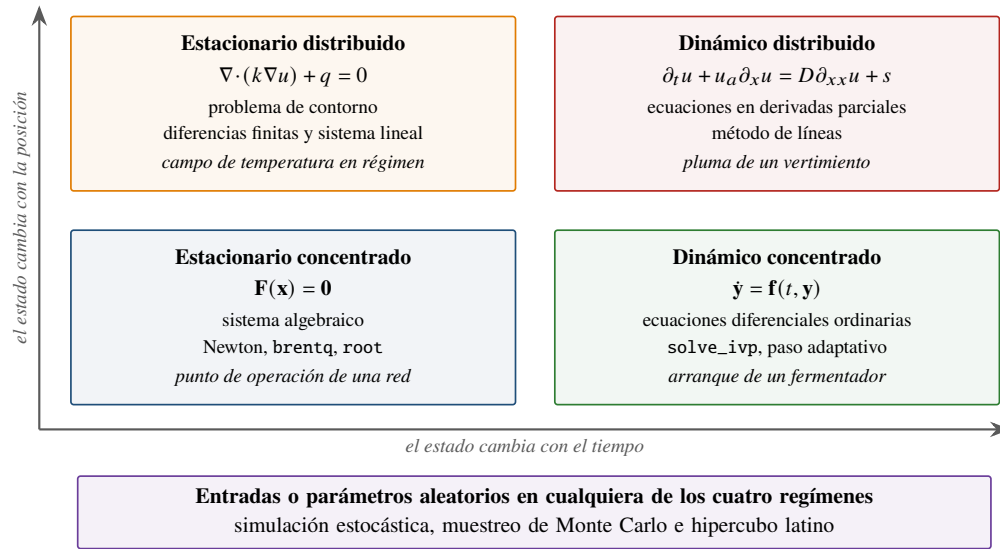


Figura 3.2. Mapa de los regímenes de simulación según el tratamiento del tiempo y del espacio, con la forma matemática del modelo y la familia de métodos numéricos que corresponde a cada uno.

Todo método de solución de ecuaciones no lineales opera sobre una misma reescritura del problema. En lugar de despejar la incógnita, cosa que en general resulta imposible, se agrupan todos los términos en un miembro y se busca el valor que anula la expresión resultante. Esa expresión recibe un nombre propio porque es el objeto que el algoritmo manipula miles de veces.

Definición 3.1. Residuo de un modelo estacionario. El *residuo* de un modelo estacionario es la función vectorial $\mathbf{F}$ que se obtiene al pasar todos los términos de las ecuaciones de balance a un mismo miembro, de modo que la solución del modelo es el vector $\mathbf{x}^*$ que satisface $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$. Cada componente del residuo tiene las unidades del balance del cual proviene.

La observación sobre las unidades no es un detalle. Un balance de masa produce residuos en kilogramos por segundo y uno de energía en vatios, de modo que un vector con componentes de magnitudes dispares no admite una norma única sin escalamiento previo. La práctica correcta divide cada componente por una magnitud característica de su balance, con lo cual el criterio de parada adquiere el significado de un error relativo. Un sistema bien planteado converge mal cuando ese escalamiento se omite.

El problema escalar es el caso más simple y el que mejor exhibe las diferencias entre familias de métodos. Su forma canónica es la de la Ecuación (3.1),

$$F(x) = 0, \quad F: [a, b] \subset \mathbb{R} \rightarrow \mathbb{R}, \quad (3.1)$$

donde x es la incógnita del modelo, expresada en las unidades de la variable física correspondiente; F es el residuo definido en la [Definición 3.1](#); y $[a, b]$ es un intervalo dentro del cual el sentido físico del problema garantiza que la solución existe.

3.1.1. Métodos para una sola incógnita

Los tres métodos de uso corriente se distinguen por la información que exigen y por la rapidez con que reducen el error. La *bisección* solo necesita un intervalo con cambio de signo del residuo y descarta en cada paso la mitad que no contiene la raíz, de modo que el error se divide por dos. La *iteración de punto fijo* reescribe el residuo como $x = g(x)$ y repite la sustitución, con lo cual el error se multiplica por un factor próximo a $|g'(x^*)|$. El *método de Newton–Raphson* sustituye la función por su recta tangente, lo que duplica de forma aproximada el número de cifras correctas en cada paso.

Esa diferencia se formaliza con el *orden de convergencia*, el exponente $p \geq 1$ tal que $|x_{k+1} - x^*| \leq C |x_k - x^*|^p$ para todo k suficientemente grande, con $p = 1$ para la convergencia lineal y $p = 2$ para la cuadrática. La noción permite comparar métodos sin recurrir a un caso particular y se puede estimar a partir de los errores de una corrida real.

El orden se estima con tres iteraciones consecutivas, mediante el cociente de logaritmos de errores sucesivos, y basta reemplazar el valor verdadero por la última iteración disponible. Si un método que debería ser cuadrático exhibe orden uno, la causa suele ser una derivada mal escrita.

El resultado que sostiene el uso de Newton no es una promesa de convergencia universal, sino un enunciado local cuyas hipótesis señalan, una por una, un modo de falla concreto.

Teorema 3.1. Convergencia local del método de Newton. Sea F dos veces continuamente diferenciable en un entorno de x^* , con $F(x^*) = 0$ y $F'(x^*) \neq 0$. Entonces existe $\delta > 0$ tal que, para todo punto inicial x_0 con $|x_0 - x^*| < \delta$, la iteración $x_{k+1} = x_k - F(x_k)/F'(x_k)$ está bien definida, converge a x^* y satisface

$$|x_{k+1} - x^*| \leq \frac{\max |F''|}{2 \min |F'|} |x_k - x^*|^2.$$

Si la derivada se anula en la raíz, el orden cae a uno y la convergencia se vuelve penosamente lenta. Si el punto inicial está lejos, la iteración puede divergir, oscilar o saltar a una raíz sin sentido físico, como un factor de fricción negativo. Si la derivada se evalúa por diferencias finitas con un paso mal elegido, la cancelación destruye la propiedad cuadrática. La [Tabla 3.1](#) resume estos rasgos y el costo por iteración.

Tabla 3.1. Comparación de los métodos de solución de una ecuación no lineal escalar.

Método	Orden	Evaluaciones ^a	Requisito	Falla característica
Bisección	1 ^b	1	cambio de signo	muy lenta
Punto fijo	1	1	$ g' < 1$ cerca de x^*	diverge si $ g' > 1$
Secante	1.618	1	dos puntos iniciales	pierde el encierro
Newton–Raphson	2	2	F' no nula	diverge si x_0 está lejos
Brent	1.618 ^c	1	cambio de signo	ninguna en el intervalo

^a Evaluaciones de función por iteración, contando la derivada como una evaluación adicional.

^b La bisección reduce el error a la mitad en cada paso, lo que equivale a una razón lineal de 0.5.

^c Orden superlineal típico, con retroceso garantizado a bisección cuando la interpolación no mejora.

El método de Brent combina la seguridad de la bisección con la rapidez de la interpolación cuadrática inversa, y es la elección por omisión cuando existe un intervalo con cambio de signo. La implementación de referencia vive en SciPy [4] y se invoca con `scipy.optimize.brentq`. El [Listado 3.1](#) la aplica a la ecuación de Colebrook–White, en la cual el factor de fricción aparece en ambos miembros y no admite despeje [51].

Listado 3.1. Residuo de la ecuación de Colebrook–White resuelto con el algoritmo de Brent.

```

1 import numpy as np
2 from scipy.optimize import brentq
3 def residuo(f, Re, rug):
4     """Forma residual de la ecuación de Colebrook-White."""
5     return 1/np.sqrt(f) + 2*np.log10(rug/3.7 +
6         2.51/(Re*np.sqrt(f)))
7 D, eps, Q, nu = 0.30, 0.26e-3, 0.12, 1.004e-6
8 V = Q/(np.pi*D**2/4)
9 Re, rug = V*D/nu, eps/D

```

```
9 f = brentq(residuo, 0.005, 0.10, args=(Re, rug), xtol=1e-14)
```

El comportamiento comparado aparece en la [Figura 3.3](#), con el error absoluto en escala logarítmica frente al número de iteración. La bisección describe una recta de pendiente suave, el punto fijo una recta más inclinada y Newton una curva que se dobla hacia abajo, firma inconfundible del orden cuadrático. El [Ejemplo 3.1](#) desarrolla el caso completo.

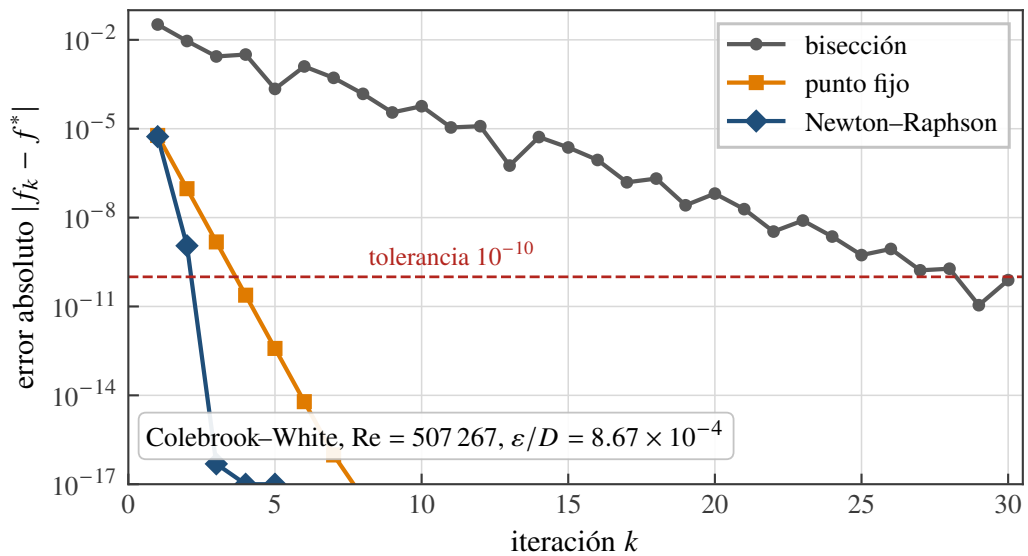


Figura 3.3. Error absoluto del factor de fricción frente al número de iteración para la bisección, la iteración de punto fijo y el método de Newton-Raphson aplicados a la ecuación de Colebrook-White.

Ejemplo 3.1. Factor de fricción de una conducción de acueducto. Una conducción de hierro dúctil de diámetro interno 0.30 m y rugosidad absoluta 0.26 mm transporta un caudal de $0.12 \text{ m}^3/\text{s}$ de agua a 20°C , con viscosidad cinemática de $1.004 \times 10^{-6} \text{ m}^2/\text{s}$. Se pide el factor de fricción de Darcy-Weisbach y la pérdida de carga en un tramo recto de 850 m.

Consideraciones. 1. Flujo turbulento plenamente desarrollado e incompresible; 2. conducto circular de sección constante, lleno y de rugosidad uniforme; 3. pérdidas locales excluidas del balance.

Análisis. De la continuidad y de la definición del número de Reynolds se obtiene $V = 1.6977 \text{ m/s}$ y $\text{Re} = 507\,267$, valor que confirma el régimen turbulento, con rugosidad relativa $\varepsilon/D = 8.667 \times 10^{-4}$. El factor de fricción satisface la Ecuación (3.2), implícita en f ,

$$\frac{1}{\sqrt{f}} = -2 \log_{10} \left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{\text{Re} \sqrt{f}} \right), \quad (3.2)$$

donde f es el factor de fricción de Darcy–Weisbach (adimensional); ε/D es la rugosidad relativa (adimensional); y Re es el número de Reynolds (adimensional). El residuo se construye pasando el miembro derecho al izquierdo, tal como exige la [Definición 3.1](#).

Solución. Con el punto inicial $f_0 = 0.02$, tomado del diagrama de Moody, y tolerancia de 1×10^{-10} sobre el paso, Newton converge en 4 iteraciones y 8 evaluaciones, contando la derivada. El punto fijo sobre $x = 1/\sqrt{f}$ necesita 5 iteraciones y 5 evaluaciones, y la bisección sobre $[0.005, 0.10]$ necesita 29 iteraciones y 31 evaluaciones. Los tres entregan $f = 0.0196228$ y la pérdida de carga vale

$$h_f = f \frac{L}{D} \frac{V^2}{2g} = 0.0196228 \cdot \frac{850}{0.30} \cdot \frac{1.6977^2}{2 \cdot 9.81} = 8.167 \text{ m}.$$

Verificación. El orden de convergencia estimado con los errores de las tres primeras iteraciones de Newton vale $\hat{p} = 2.00$, consistente con el [Teorema 3.1](#). La correlación explícita de Swamee y Jain entrega $f = 0.019742$, que se aparta un 0.61 % del valor implícito, diferencia compatible con la exactitud declarada de esa aproximación.

Comentario. El costo de las tres alternativas difiere en un factor cercano a cuatro y ninguna supera el milisegundo. La elección importa cuando la ecuación se resuelve dentro de un lazo, como al simular una red con centenares de tramos.

La correlación explícita usada en la verificación es la de Swamee y Jain, cuya deducción y rango de validez están en los textos de mecánica de fluidos [52]. Su papel es el de una comprobación independiente y no el de un sustituto, porque el error del 0.6 % se traslada íntegro a la pérdida de carga y en una red mallada se acumula tramo a tramo.

3.1.2. Sistemas acoplados y estructura del jacobiano

Un modelo estacionario realista rara vez tiene una sola incógnita. Una cascada de etapas, una red de tuberías o un tren de unidades genera tantos residuos como balances independientes, y todos comparten variables. La extensión de Newton reemplaza la derivada por la matriz de derivadas parciales y el cociente por la solución del sistema lineal de la Ecuación (3.3),

$$\mathbf{J}(\mathbf{x}_k) \Delta \mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \Delta \mathbf{x}_k, \quad (3.3)$$

donde $\mathbf{J}$ es la matriz jacobiana de componentes $J_{ij} = \partial F_i / \partial x_j$; $\Delta \mathbf{x}_k$ es la corrección de Newton; y $\lambda_k \in (0, 1]$ es un factor de amortiguamiento que vale la unidad en la iteración de Newton pura.

El factor de amortiguamiento convierte un método local en uno utilizable. La corrección de Newton señala una buena dirección de descenso para la norma del residuo, pero su longitud puede resultar excesiva lejos de la solución y llevar a una región sin sentido físico. El retroceso reduce a la mitad el paso hasta que la norma disminuya, con lo cual se preserva la rapidez cuadrática cerca de la solución y se gana robustez lejos de ella. El [Algoritmo 3.1](#) recoge el procedimiento.

Algoritmo 3.1 Newton amortiguado con retroceso para un sistema de residuos.

Entradas: Residuo $\mathbf{F}$, jacobiano $\mathbf{J}$, punto inicial $\mathbf{x}_0$, tolerancia τ , máximo de iteraciones K

Salidas: Solución aproximada $\mathbf{x}$ o aviso de fracaso

```
1:  $\mathbf{x} \leftarrow \mathbf{x}_0$ ;  $\mathbf{r} \leftarrow \mathbf{F}(\mathbf{x})$ ;  $\rho \leftarrow \|\mathbf{r}\|_\infty$ ;  $\rho_0 \leftarrow \rho$ 
2: para  $k = 1$  hasta  $K$  hacer
3:   resolver  $\mathbf{J}(\mathbf{x}) \Delta \mathbf{x} = -\mathbf{r}$ ;  $\lambda \leftarrow 1$ 
4:   repetir
5:      $\mathbf{x}^+ \leftarrow \mathbf{x} + \lambda \Delta \mathbf{x}$ ;  $\mathbf{r}^+ \leftarrow \mathbf{F}(\mathbf{x}^+)$ ;  $\lambda \leftarrow \lambda/2$ 
6:   hasta que  $\mathbf{x}^+$  respeta el dominio físico y  $\|\mathbf{r}^+\|_\infty < \rho$ , o  $\lambda < \lambda_{\min}$ 
7:   si  $\lambda < \lambda_{\min}$  entonces
8:     devolver fracaso por estancamiento
9:   fin si
10:   $\mathbf{x} \leftarrow \mathbf{x}^+$ ;  $\mathbf{r} \leftarrow \mathbf{r}^+$ ;  $\rho \leftarrow \|\mathbf{r}\|_\infty$ 
11:  si  $\rho \leq \tau \rho_0$  entonces
12:    devolver  $\mathbf{x}$ 
13:  fin si
14: fin para
15: devolver fracaso por número de iteraciones
```

La segunda decisión es cómo se obtiene el jacobiano. Las diferencias finitas cuestan n evaluaciones adicionales del residuo por iteración y arrastran un error de truncamiento que degrada el orden. La derivación analítica es exacta y barata, pero es una fuente frecuente de errores de signo. El camino intermedio deriva el residuo con SymPy y genera la función evaluable con `lambdify`, técnica descrita en el [Capítulo 2](#) [18].

La estructura del jacobiano importa tanto como su exactitud. En una cascada, cada balance involucra la etapa propia y sus dos vecinas, de modo que la matriz es tridiagonal y contiene $3n - 2$ elementos no nulos en lugar de n^2 . La [Figura 3.4](#) muestra esa correspondencia entre topología y dispersión. Aprovecharla reduce el costo de la factorización de $O(n^3)$ a $O(n)$.

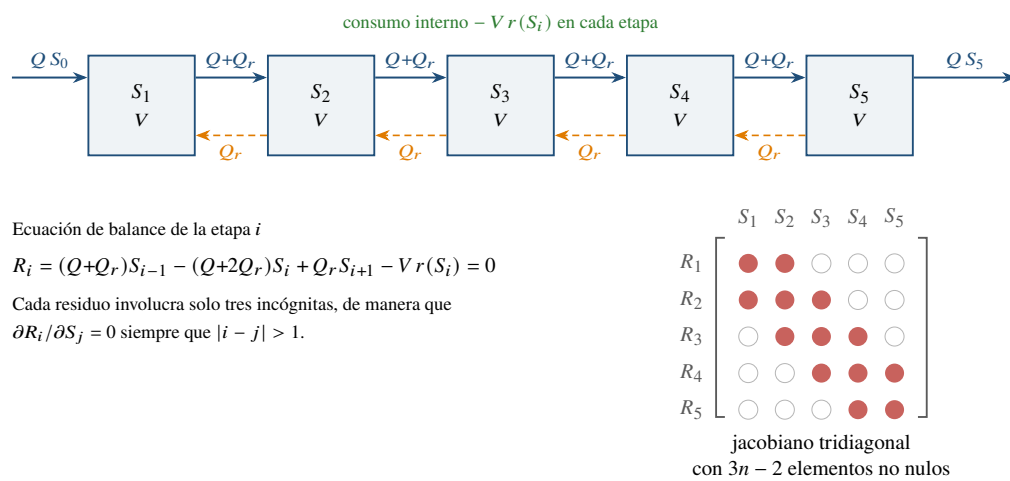


Figura 3.4. Cascada de balances acoplados con retromezcla entre etapas contiguas y patrón de dispersión tridiagonal del jacobiano que resulta de esa topología.

La función `scipy.optimize.root` reúne varias familias de algoritmos y acepta el jacobiano como argumento opcional. El [Listado 3.2](#) arma el residuo de la cascada de forma vectorizada y construye el jacobiano explícito, con las dos filas de frontera tratadas aparte porque su balance carece de un vecino.

Listado 3.2. Residuo y jacobiano de una cascada de reactores resueltos con `scipy.optimize.root`.

```
1 import numpy as np
2 from scipy.optimize import root
3 def residuos(S, par):
4     Q, Qr, V, S0 = par["Q"], par["Qr"], par["V"], par["S0"]
5     r = par["kmax"]*S/(par["Ks"] + S)
6     R = np.empty_like(S)
7     R[0] = Q*S0 - (Q + Qr)*S[0] + Qr*S[1] - V*r[0]
```

```

8   R[1:-1] = ((Q + Qr)*S[:-2] - (Q + 2*Qr)*S[1:-1]
9           + Qr*S[2:] - V*r[1:-1])
10  R[-1] = (Q + Qr)*S[-2] - (Q + Qr)*S[-1] - V*r[-1]
11  return R
12  def jacobiano(S, par):
13      Q, Qr, V = par["Q"], par["Qr"], par["V"]
14      dr = par["kmax"]*par["Ks"]/(par["Ks"] + S)**2
15      J = np.diag(-(Q + 2*Qr) - V*dr)
16      J += np.diag(np.full(S.size - 1, Q + Qr), -1)
17      J += np.diag(np.full(S.size - 1, Qr), 1)
18      J[0, 0] = -(Q + Qr) - V*dr[0]
19      J[-1, -1] = -(Q + Qr) - V*dr[-1]
20  return J
21  par = dict(Q=1200.0, Qr=600.0, V=2000.0, S0=250.0, kmax=60.0,
22           Ks=40.0)
23  sol = root(residuos, np.full(5, par["S0"]), jac=jacobiano,
24           args=(par,), tol=1e-12)

```

La aplicación de referencia es un sistema de lagunas de estabilización, con retro-mezcla entre unidades contiguas que representa la dispersión longitudinal y cinética de saturación en la forma propuesta por Monod [53]. La verificación se apoya en un balance global que debe cerrar con precisión de máquina [54].

El caso de referencia es un sistema de 5 lagunas iguales de 2000 m³ que recibe 1200 m³/d con una demanda bioquímica de oxígeno afluente de 250 mg/L, con un intercambio por dispersión de 600 m³/d entre lagunas contiguas y una cinética de saturación de parámetros $k_{\text{máx}} = 60 \text{ mg}/(\text{L d})$ y $K_s = 40 \text{ mg}/\text{L}$. El balance de la laguna i , con $1 < i < n$, iguala en la Ecuación (3.4) el flujo neto que entra al que sale más lo que se consume,

$$(Q + Q_r) S_{i-1} - (Q + 2Q_r) S_i + Q_r S_{i+1} - V \frac{k_{\text{máx}} S_i}{K_s + S_i} = 0, \quad (3.4)$$

donde Q es el caudal de paso (m³/d); Q_r es el caudal de retromezcla (m³/d); V es el volumen de cada laguna (m³); S_i es la concentración en la laguna i (mg/L); $k_{\text{máx}}$ es la velocidad máxima de remoción volumétrica (mg/(L d)); y K_s es la constante de semisaturación (mg/L). Las lagunas primera y última tienen un vecino menos y sus balances se escriben aparte.

Partiendo del perfil uniforme $S_i = 250 \text{ mg}/\text{L}$, el algoritmo híbrido converge con 19 evaluaciones del residuo y 2 del jacobiano y entrega el perfil 142.42, 83.40, 41.50, 17.65 y 7.33 en mg/L, con una remoción global del 97.07 % y un tiempo de retención total de 8.33 d. Sin el jacobiano analítico, el mismo algoritmo necesita 29 evaluaciones. El Newton amortiguado del Algoritmo 3.1 alcanza la misma solución en 6 iteraciones partiendo de 1 mg/L, punto muy alejado del correcto.

La verificación se hace sumando los cinco residuos, cuya suma se reduce al balance global $QS_0 - QS_n - V \sum_i r(S_i) = 0$. Con la solución obtenida, la carga afluente vale 300 000 mg/d, la efluente 8792.84 mg/d y el consumo 291 207.16 mg/d, con un cierre relativo de 1.9×10^{-16} . Ese cierre no prueba que el modelo sea adecuado, solo que el sistema algebraico se resolvió bien, distinción entre verificación del cálculo y validación del modelo que el [Capítulo 4](#) desarrolla.

Cuando ni el amortiguamiento ni un punto inicial razonable bastan, queda la *continuación*, que resuelve primero una versión fácil del problema y desplaza el parámetro difícil hasta su valor final, usando cada solución como punto inicial de la siguiente. En la cascada se puede partir de un caudal de retromezcla nulo, caso en el cual las etapas se desacoplan y se resuelven una tras otra. Es la herramienta de último recurso antes de reformular el modelo, y los textos clásicos de métodos numéricos para ingenieros la documentan con detalle [55, 56, 57].

3.2. Simulación dinámica y comportamiento bajo diferentes condiciones

El régimen permanente describe la condición sostenida, pero muchas preguntas se refieren al camino que conduce hasta ella. Cuánto tarda un fermentador en agotar el azúcar, si el enfriamiento alcanza la temperatura objetivo antes del amanecer o cómo responde un lazo de control ante un cambio brusco de carga son preguntas sobre el transitorio. Cuando la hipótesis de parámetros concentrados vale, el balance transitorio conserva la acumulación y se convierte en una ecuación diferencial ordinaria. Esta sección enseña a integrarla y, sobre todo, a examinar cómo cambia la respuesta cuando cambian las condiciones de operación, las tolerancias del integrador y las escalas de tiempo del propio sistema, que son las tres fuentes de comportamiento distinto que un mismo modelo puede exhibir.

La forma canónica reúne todas las ecuaciones en un sistema de primer orden. Un balance de segundo orden se reduce a ella introduciendo la derivada como estado adicional, y un modelo con varias especies agrega componentes al vector, tal como recoge la Ecuación (3.5),

$$\frac{dy}{dt} = \mathbf{f}(t, \mathbf{y}; \boldsymbol{\theta}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \quad t \in [t_0, t_f], \quad (3.5)$$

donde $\mathbf{y}$ es el vector de estado, con las unidades físicas de cada componente; $\mathbf{f}$ es el campo vectorial que resulta de los balances; $\boldsymbol{\theta}$ es el vector de parámetros; $\mathbf{y}_0$ es la condición inicial; y $[t_0, t_f]$ es el horizonte de simulación.

El conjunto formado por ese sistema, la condición inicial y el horizonte constituye un *problema de valor inicial*, cuya solución es la función $y(t)$ que satisface ambas cosas a la vez. Su existencia y su unicidad se garantizan bajo la condición de Lipschitz estudiada en el [Capítulo 1](#) y aquí se dan por descontadas. Queda por decidir cómo se aproxima la solución, elección que liga tres cosas, el método de integración, el tamaño del paso y la tolerancia con la cual se acepta cada paso. Confundirlas es la fuente más común de resultados dinámicos incorrectos, tal como insisten los textos que tratan la integración numérica desde Python [58, 31] y el texto guía, que ofrece una regla práctica para fijar las tolerancias a partir de la exactitud con que se conocen los datos [1].

3.2.1. Orden, error y control automático del paso

Todo método construye y_{n+1} en $t_{n+1} = t_n + h$ a partir de información previa. Los métodos de un paso, como la familia de Runge–Kutta, usan solo y_n y varias evaluaciones del campo dentro del intervalo. Los de paso múltiple, como las fórmulas de diferenciación regresiva, reutilizan valores anteriores y se abaratan cuando el campo es costoso. El *error local de truncamiento* se comete en un solo paso partiendo de un valor exacto y se comporta como $O(h^{p+1})$ para un método de orden p , mientras que el *error global* acumula esos errores y se comporta como $O(h^p)$ [15].

Los integradores modernos no trabajan con paso fijo. Estiman el error local comparando dos aproximaciones de órdenes distintos que comparten las mismas evaluaciones del campo, artificio llamado par encajado, y ajustan el paso siguiente para que ese estimado quede bajo una cota. El par de Dormand y Prince de órdenes cinco y cuatro es el más difundido y SciPy lo implementa como RK45 [59]. La cota se construye con dos tolerancias, una relativa y una absoluta, según la Ecuación (3.6),

$$|e_{n,i}| \leq \text{atol}_i + \text{rtol} \cdot |y_{n,i}|, \quad (3.6)$$

donde $e_{n,i}$ es el error local estimado de la componente i en el paso n ; rtol es la tolerancia relativa, común a todas las componentes; y atol_i es la tolerancia absoluta de la componente i , expresada en sus mismas unidades.

El papel de cada tolerancia se ve en los extremos. Cuando la componente es grande domina el término relativo y la cota equivale a exigir un número fijo de cifras significativas. Cuando se aproxima a cero, solo la tolerancia absoluta impide que el controlador exija una exactitud imposible. De ahí que atol se fije en el orden de magnitud por debajo del cual la componente carece de significado físico, por ejemplo 1×10^{-6} g/L para una concentración de decenas de gramos por litro.

Conviene insistir en un punto que se malinterpreta con frecuencia. Las tolerancias controlan el error local estimado de cada paso, no el error al final del horizonte. La propagación de los errores locales puede amplificarlos cuando el problema tiene direcciones inestables y amortiguarlos cuando el sistema es disipativo. La tolerancia no es el error, es una perilla que lo regula de forma indirecta, y la única manera honesta de conocerlo consiste en repetir la corrida con tolerancias más estrictas hasta que la respuesta deje de cambiar.

El [Listado 3.3](#) muestra la invocación de `scipy.integrate.solve_ivp` sobre un fermentador en lote, con el campo, las tolerancias explícitas, la solución densa y un evento que detecta el agotamiento del sustrato. Los eventos se declaran como funciones que se anulan en el instante buscado, y el integrador localiza el cruce por interpolación sobre el polinomio denso, de modo que el instante hereda la exactitud del método y no la del paso.

Listado 3.3. Integración de un fermentador en lote con tolerancias explícitas y detección de eventos.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 def fermentador(t, y, mu_max, Ks, Y):
4     X, S = y
5     mu = mu_max*S/(Ks + S)
6     return [mu*X, -mu*X/Y]
7 def agotamiento(t, y, mu_max, Ks, Y):
8     return y[1] - 1.0
9 agotamiento.direction = -1
10 sol = solve_ivp(fermentador, (0.0, 30.0), [0.50, 150.0],
11                 args=(0.30, 1.2, 0.08), method="RK45",
12                 rtol=1e-9, atol=1e-11, dense_output=True,
13                 events=agotamiento)
14 t_agot = sol.t_events[0][0]

```

El comportamiento bajo tres condiciones de operación aparece en la [Figura 3.5](#). Las tres trayectorias comparten el estado final de biomasa, consecuencia de una relación de conservación que se deduce del propio modelo y sirve de verificación exacta, y difieren solo en la escala de tiempo. El [Ejemplo 3.2](#) desarrolla el caso.

Ejemplo 3.2. Fermentación en lote de jugo de caña a tres temperaturas.

Un fermentador en lote parte de 150 g/L de azúcares reductores y 0.50 g/L de biomasa. La cinética es de tipo Monod con $K_s = 1.2$ g/L y rendimiento $Y = 0.08$ gramos de biomasa por gramo de sustrato. La velocidad específi-

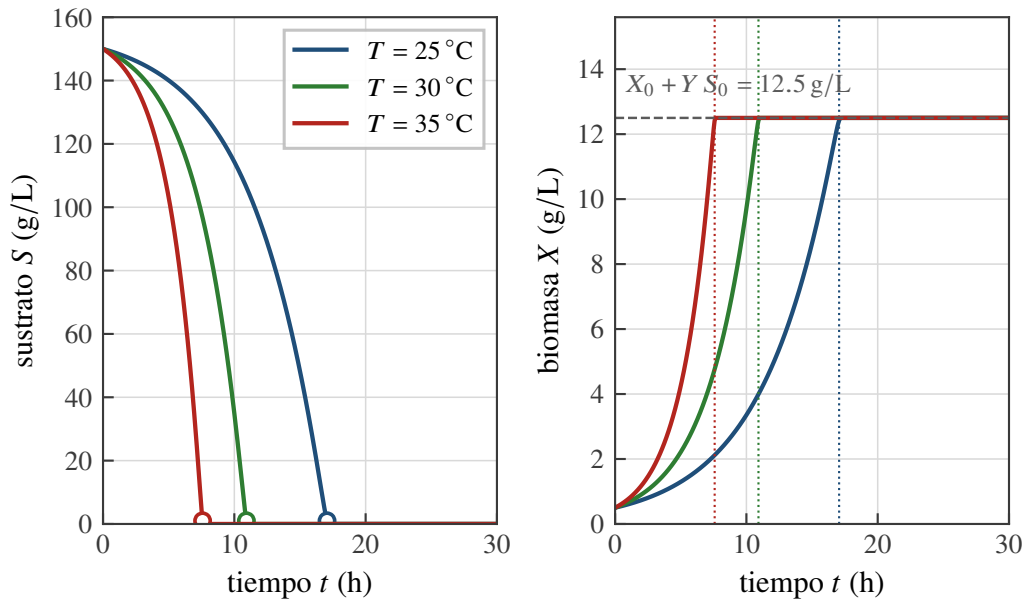


Figura 3.5. Trayectorias del sustrato y de la biomasa en un fermentador en lote para tres temperaturas de operación, con el instante de agotamiento marcado sobre cada curva.

ca máxima depende de la temperatura según el modelo de la raíz cuadrada, $\mu_{\text{máx}}(T) = \mu_{\text{ref}} [(T - T_{\text{mín}})/(T_{\text{ref}} - T_{\text{mín}})]^2$, con $\mu_{\text{ref}} = 0.30/\text{h}$ a $T_{\text{ref}} = 30^\circ\text{C}$ y $T_{\text{mín}} = 5^\circ\text{C}$. Se pide el tiempo necesario para que el sustrato descienda a 1 g/L a 25°C , 30°C y 35°C .

Consideraciones. 1. Mezcla completa, temperatura uniforme y volumen constante durante todo el lote; 2. sin inhibición por producto ni por sustrato; 3. sin muerte celular ni mantenimiento.

Formulación. Los dos balances constituyen el problema de valor inicial de la Ecuación (3.7),

$$\frac{dX}{dt} = \frac{\mu_{\text{máx}} S}{K_s + S} X, \quad \frac{dS}{dt} = -\frac{1}{Y} \frac{\mu_{\text{máx}} S}{K_s + S} X, \quad (3.7)$$

donde X es la concentración de biomasa (g/L); S es la concentración de sustrato (g/L); $\mu_{\text{máx}}$ es la velocidad específica máxima de crecimiento (1/h); K_s es la constante de semisaturación (g/L); y Y es el rendimiento de biomasa sobre sustrato (adimensional).

Solución. Las velocidades específicas máximas resultan 0.192/h, 0.300/h y 0.432/h. Con $\text{rtol} = 1 \times 10^{-9}$ y $\text{atol} = 1 \times 10^{-11}$, RK45 localiza el agotamiento en 17.0605 h, 10.9187 h y 7.5824 h, con 584 evaluaciones del campo en cada caso.

Verificación. Sumando el primer balance con Y veces el segundo se obtiene $d(X + YS)/dt = 0$, invariante exacto que vale 12.5 g/L y que la solución numérica conserva con desviación máxima de 3.6×10^{-15} . El invariante reduce además el sistema a una cuadratura en S , cuya evaluación de alta precisión entrega 17.0604614 h, 10.9186953 h y 7.5824273 h, en coincidencia con el integrador.

Comentario. Un aumento de 10 °C reduce el tiempo de proceso a menos de la mitad, pero el modelo no contempla la pérdida de viabilidad celular por encima de 35 °C. Extrapolarlo a 40 °C produciría un número plausible y falso.

El estudio de tolerancias sobre ese caso ilustra el argumento anterior con cifras. Con $\text{rtol} = 1 \times 10^{-3}$ el integrador necesita 44 evaluaciones y devuelve un instante de agotamiento de 10.9028 h, con error relativo de 1.46×10^{-3} , superior a la tolerancia pedida. Con $\text{rtol} = 1 \times 10^{-6}$ el error cae a 1.85×10^{-7} con 194 evaluaciones y con $\text{rtol} = 1 \times 10^{-9}$ a 2.45×10^{-10} con 584. El error global puede exceder la tolerancia solicitada y el número de pasos crece como $\text{rtol}^{-1/5}$, tal como corresponde a un método de orden cinco.

3.2.2. Estabilidad absoluta y rigidez

Hasta aquí el paso lo ha fijado la exactitud. Existe una segunda restricción que puede volverse dominante y que no se manifiesta como pérdida de precisión sino como una explosión numérica. Para analizarla se recurre a la ecuación de prueba $y' = \lambda y$, con λ igual a un autovalor del jacobiano del campo. Un método de un paso aplicado a esa ecuación produce la recurrencia $y_{n+1} = R(z) y_n$ con $z = h\lambda$, donde R es la *función de estabilidad* del método, y el conjunto $\mathcal{S} = \{z \in \mathbb{C}: |R(z)| \leq 1\}$ es su *región de estabilidad absoluta*. Un método se llama A-estable cuando $\mathcal{S}$ contiene todo el semiplano complejo izquierdo.

La región decide si la solución numérica de un sistema disipativo permanece acotada, pues con $\text{Re}(\lambda) < 0$ la sucesión $\{y_n\}$ tiende a cero si y solo si $|R(h\lambda)| < 1$ y permanece acotada si y solo si $|R(h\lambda)| \leq 1$. Para Euler explícito se tiene $R(z) = 1 + z$, de modo que la región es el disco unitario centrado en -1 y el paso queda limitado por $h \leq 2/|\lambda|$. Para Euler implícito se tiene $R(z) = (1 - z)^{-1}$, cuya región es el exterior del disco unitario centrado en $+1$ y contiene todo el semiplano izquierdo, de manera que ningún paso lo desestabiliza.

La consecuencia es directa. Un método explícito exige que $h\lambda$ caiga dentro de una región acotada, lo cual impone un paso máximo proporcional al inverso del autovalor de mayor magnitud, con independencia de la exactitud requerida. Un método implícito A-estable no impone tal restricción, a cambio de resolver un sistema no lineal en cada paso. La [Figura 3.6](#) compara las tres regiones.

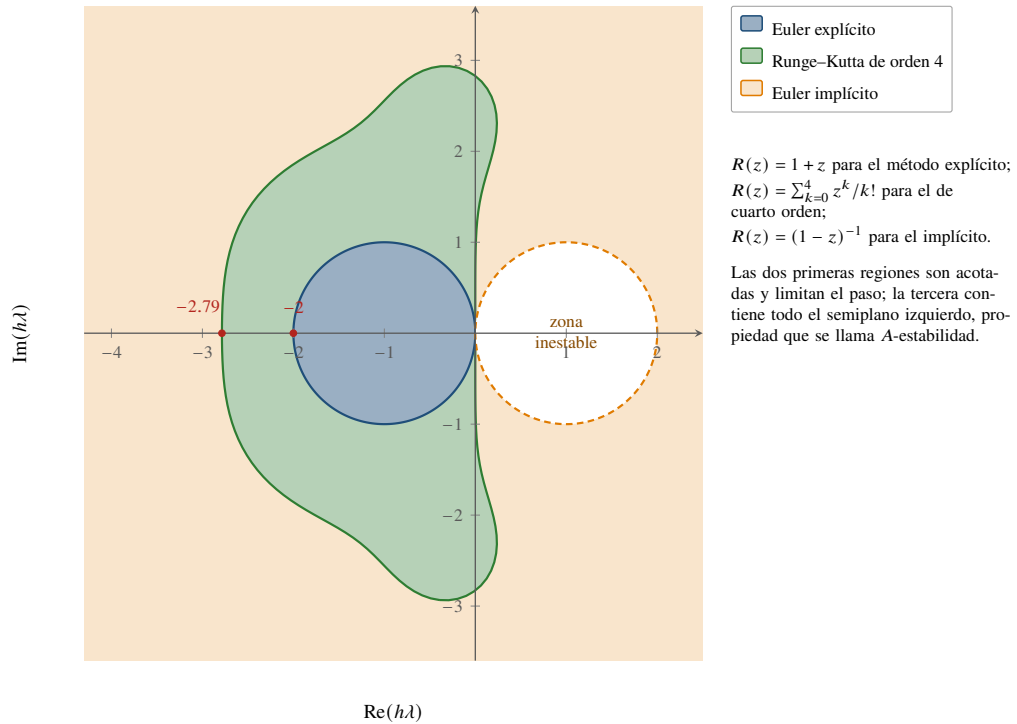


Figura 3.6. Regiones de estabilidad absoluta del método de Euler explícito, del método de Runge–Kutta clásico de orden cuatro y del método de Euler implícito en el plano complejo del producto $h\lambda$.

El problema aparece cuando un mismo modelo contiene fenómenos con escalas de tiempo muy separadas. Una reacción rápida acoplada a un transporte lento o una capacitancia parásita junto a una constante térmica de horas producen jacobianos con autovalores que difieren en varios órdenes de magnitud. El transitorio rápido se extingue en los primeros instantes, pero su autovalor sigue restringiendo el paso durante todo el horizonte.

Definición 3.2. Rigidez de un problema de valor inicial. Un problema de valor inicial es *rígido* sobre un horizonte cuando la solución varía con una escala de tiempo mucho más lenta que la más rápida admitida por la dinámica local, de modo que un método explícito se ve obligado a usar pasos mucho menores que los requeridos por la exactitud. Una medida usual es la razón de rigidez $\max_i |\lambda_i| / \min_i |\lambda_i|$ evaluada sobre los autovalores del jacobiano.

La razón de rigidez es un indicador y no una definición cerrada, porque la rigidez depende también del horizonte y de la condición inicial. El criterio operativo compara el paso que la exactitud requeriría con el que la estabilidad permite, y si el segundo es mucho menor el problema es rígido en la práctica. En un modelo no lineal la razón puede cambiar en varios órdenes a lo largo de la trayectoria.

El diagnóstico y el remedio se aplican en el mismo gesto. El [Listado 3.4](#) integra el problema con un método explícito y con uno implícito e imprime pasos y evaluaciones, de modo que la comparación queda documentada en la corrida. Un cociente de dos o tres órdenes de magnitud entre ambos conteos es la firma inequívoca de la rigidez.

Listado 3.4. Diagnóstico de rigidez por comparación del conteo de evaluaciones entre un método explícito y uno implícito.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 R1, C1, R2, C2, Vs = 1.0e3, 1.0e-6, 2.0e5, 1.0e-4, 5.0
4 A = np.array([[-(1/R1 + 1/R2)/C1, (1/R2)/C1],
5               [(1/R2)/C2, -(1/R2)/C2]])
6 b = np.array([Vs/(R1*C1), 0.0])
7 campo = lambda t, v: A @ v + b
8 for metodo in ("RK45", "BDF"):
9     extra = {"jac": lambda t, v: A} if metodo == "BDF" else {}
10    s = solve_ivp(campo, (0.0, 100.0), np.zeros(2), method=metodo,
11                  rtol=1e-6, atol=1e-9, **extra)
12    print(f"{metodo:5s} pasos={s.t.size-1:6d}
           evaluaciones={s.nfev:7d}")

```

El resultado aparece en la [Figura 3.7](#), con la evolución de las dos tensiones sobre un eje temporal logarítmico a la izquierda y el paso aceptado por cada método a la derecha. El explícito mantiene un paso constante que coincide con el límite de estabilidad, mientras que el implícito lo aumenta en cuatro órdenes de magnitud una vez extinguido el transitorio rápido. El [Ejemplo 3.3](#) recoge el caso.

Ejemplo 3.3. Circuito con dos constantes de tiempo separadas. Una fuente de 5 V alimenta, a través de una resistencia de 1 kΩ, un nodo con una capacitancia a tierra de 1 μF. Ese nodo se conecta mediante una resistencia de 200 kΩ a un segundo nodo con una capacitancia de 100 μF. Se pide la evolución de ambas tensiones durante 100 s partiendo del reposo, y el costo de calcularla con un método explícito y con uno implícito.

Consideraciones. 1. Componentes lineales e invariantes en el tiempo; 2. fuente ideal y corrientes de fuga despreciables; 3. condición inicial de descarga completa.

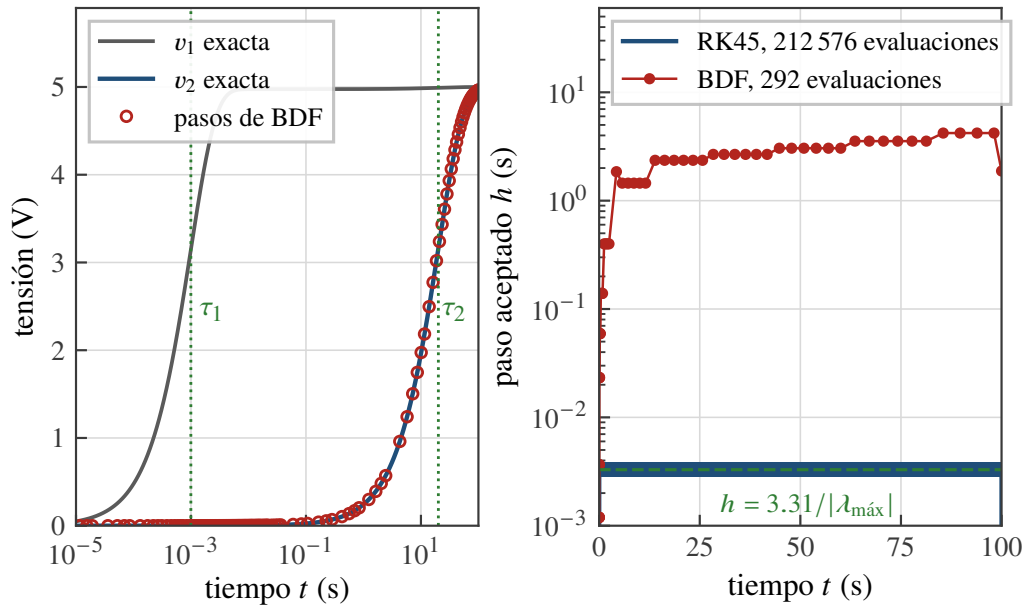


Figura 3.7. Respuesta de un circuito con dos constantes de tiempo separadas y tamaño del paso aceptado por un método explícito y por uno implícito a lo largo del horizonte de simulación.

Análisis. Las leyes de Kirchhoff en los dos nodos conducen al sistema lineal $\dot{\mathbf{v}} = \mathbf{A}\mathbf{v} + \mathbf{b}$, cuyos autovalores valen $\lambda_1 = -1005.0002/\text{s}$ y $\lambda_2 = -0.0497512/\text{s}$. Las constantes de tiempo asociadas son $\tau_1 = 0.995 \text{ ms}$ y $\tau_2 = 20.10 \text{ s}$, de modo que la razón de rigidez de la [Definición 3.2](#) vale 20 200.

Solución. Con $\text{rtol} = 1 \times 10^{-6}$ y $\text{atol} = 1 \times 10^{-9}$, el método RK45 necesita 30 376 pasos y 212 576 evaluaciones del campo. El método BDF, con el jacobiano suministrado, necesita 144 pasos y 292 evaluaciones. El error relativo del estado final es de 9.1×10^{-7} para el primero y de 1.1×10^{-6} para el segundo, de manera que ambos quedan en el orden de la tolerancia solicitada. La relación de costo es de 728 a uno.

Verificación. El sistema es lineal y admite la solución exacta $\mathbf{v}(t) = \mathbf{A}^{-1}(\mathbf{e}^{\mathbf{A}t} - \mathbf{I})\mathbf{b}$ partiendo del reposo, que sirve de referencia. El paso medio aceptado por el método explícito vale 3.292 ms, de manera que $h|\lambda_1| = 3.31$, valor que coincide con el límite de estabilidad real del par encajado de orden cinco. El paso lo fija la estabilidad y no la exactitud.

Comentario. Nada en la respuesta observada delata la rigidez, porque la tensión del segundo nodo evoluciona con suavidad. La rigidez vive en el jacobiano y no en la trayectoria, y de ahí que convenga inspeccionar los autovalores antes de elegir el método.

La familia de integradores de `solve_ivp` cubre las situaciones habituales y la [Tabla 3.2](#) resume el orden, el carácter explícito o implícito y el criterio de selección de cada uno. La implementación de LSODA conmuta de manera automática entre una familia explícita y una implícita según detecte rigidez, comportamiento descrito con detalle por Shampine y Reichelt [60].

Tabla 3.2. Integradores disponibles en `scipy.integrate.solve_ivp` y criterio de selección.

Nombre	Familia	Orden	Cuándo conviene
RK45	Runge–Kutta explícito	5(4)	no rígido, tolerancia moderada
DOP853	Runge–Kutta explícito	8(5,3)	no rígido con tolerancia muy estricta
Radau	Runge–Kutta implícito	5	rígido, se requiere alta exactitud
BDF	paso múltiple implícito	1–5	rígido, campo costoso de evaluar
LSODA	conmutación automática	1–12	rigidez desconocida o variable

^a Los métodos implícitos aceptan el jacobiano como argumento. Suministrarlo reduce de forma apreciable el número de evaluaciones del campo y mejora la robustez.

La [Figura 3.8](#) convierte la tabla en un criterio de decisión que parte del diagnóstico previo sobre los autovalores del jacobiano y desemboca en una de tres ramas según la razón de rigidez sea baja, alta o desconocida. Las tres ramas comparten la misma verificación, que consiste en repetir la corrida con tolerancias más estrictas hasta que la respuesta deje de cambiar, y el modo de falla que la figura recuerda es el del método explícito aplicado a un problema rígido, que no se equivoca sino que se encarece hasta volverse inviable. Los textos que tratan la integración numérica desde Python coinciden en ese criterio y en la conveniencia de suministrar el jacobiano a los métodos implícitos [58, 31].

Queda un asunto que ningún integrador resuelve solo, el de las discontinuidades. Un controlador de dos posiciones, una válvula que abre al alcanzar un nivel o una lluvia que comienza de golpe introducen saltos en el campo o en sus derivadas, y atravesarlos obliga al controlador de paso a reducirlo drásticamente. El tratamiento correcto declara la discontinuidad como evento terminal, detiene la integración en el instante en que ocurre, actualiza el estado o los parámetros y reanuda con una nueva llamada [61].

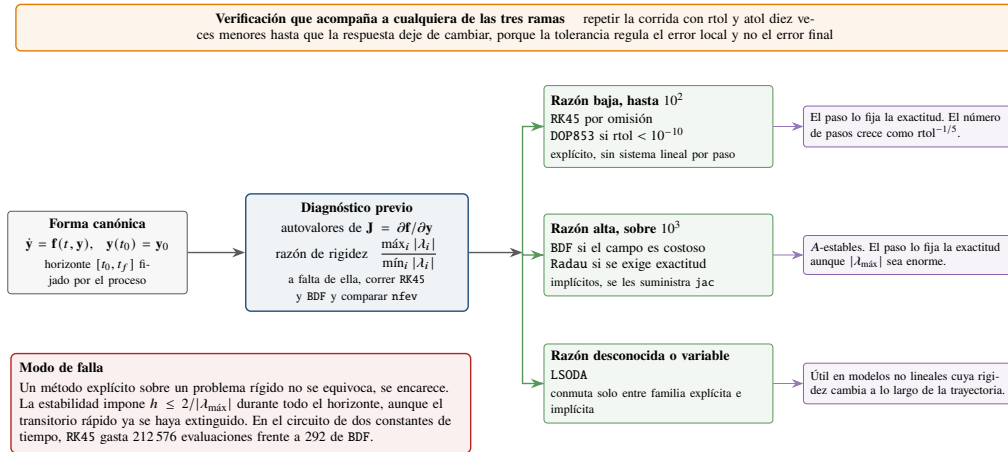


Figura 3.8. Criterio de selección del integrador de `solve_ivp` a partir de la razón de rigidez, con el modo de falla del método explícito y la verificación común a las tres ramas.

3.3. Sistemas distribuidos y método de líneas

La hipótesis de parámetros concentrados supone que el estado es uniforme dentro del sistema, es decir, que el mezclado o la conducción interna son rápidos frente al intercambio con el exterior. El [Capítulo 1](#) mostró que esa suposición se juzga con un número adimensional que compara ambas resistencias y deja de valer cuando el gradiente interno es del orden de la diferencia con el entorno. El estado depende entonces de la posición y el balance conduce a una ecuación en derivadas parciales.

La familia que cubre la mayoría de los casos tiene la forma de transporte con advección, difusión y reacción de la Ecuación (3.8),

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} = D \frac{\partial^2 c}{\partial x^2} - k c + s(x, t), \quad (3.8)$$

donde c es la concentración o el estado transportado (kg/m^3); u es la velocidad advectiva (m/s); D es el coeficiente de dispersión (m^2/s); k es la constante de decaimiento de primer orden ($1/\text{s}$); y s es un término fuente por unidad de volumen. La ecuación de conducción de calor y la de flujo en un medio poroso se obtienen de esta como casos particulares.

3.3.1. Discretización espacial y condiciones de frontera

La idea que hace tratable el problema renuncia a conocer c en todo punto y se conforma con un conjunto finito de nodos, sobre los cuales las derivadas espaciales se reemplazan por cocientes de diferencias deducidos del desarrollo de Taylor. La aproximación centrada de la segunda derivada, recogida en la Ecuación (3.9),

$$\left. \frac{\partial^2 c}{\partial x^2} \right|_i = \frac{c_{i-1} - 2c_i + c_{i+1}}{\Delta x^2} - \frac{\Delta x^2}{12} \left. \frac{\partial^4 c}{\partial x^4} \right|_i + O(\Delta x^4), \quad (3.9)$$

donde c_i es el valor nodal en la posición x_i ; y Δx es el paso de la malla (m), tiene error de truncamiento de segundo orden y solo requiere los dos vecinos inmediatos, orden que comparte la aproximación centrada de la primera derivada. El precio de la simetría es que la advección centrada produce oscilaciones espurias cuando el número de Péclet de celda $Pe = u\Delta x/D$ supera el valor dos, límite que conviene verificar antes de aceptar un perfil.

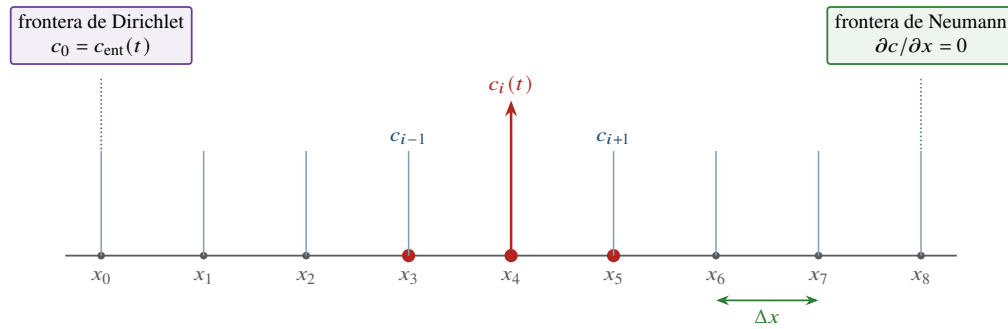
Las condiciones de frontera se imponen sobre los nodos extremos en tres formas. Una de Dirichlet fija el valor nodal y traslada su contribución al término independiente, una de Neumann fija el flujo difusivo mediante un nodo ficticio fuera del dominio y una de Robin combina ambas y modifica el coeficiente diagonal. Un error frecuente impone Dirichlet en la salida de un dominio advectivo, lo cual introduce una capa límite artificial que contamina el interior.

Aplicando la discretización a todos los nodos interiores y dejando el tiempo continuo se obtiene un sistema de ecuaciones diferenciales ordinarias acopladas. Ese paso tiene nombre propio y es la técnica que permite reutilizar toda la maquinaria de la sección anterior.

Definición 3.3. Método de líneas. El *método de líneas* consiste en discretizar únicamente las derivadas espaciales de una ecuación en derivadas parciales sobre una malla fija, con lo cual el problema se transforma en un sistema de ecuaciones diferenciales ordinarias cuyas incógnitas son los valores nodales en función del tiempo. La integración temporal de ese sistema se realiza después con cualquier método para problemas de valor inicial.

La Figura 3.9 representa el procedimiento, desde el stencil de tres puntos hasta la matriz tridiagonal del ensamblaje. Las condiciones de frontera aparecen en dos lugares, el término independiente cuando son de Dirichlet y los coeficientes de las filas extremas cuando son de Neumann o de Robin, y confundirlos genera errores silenciosos.

El ensamblaje se escribe una sola vez y se reutiliza en cada evaluación del campo, ventaja que existe solo si los coeficientes no dependen del estado. El Listado 3.5 construye la matriz en formato disperso, lo cual importa porque una malla de mil nodos generaría un millón de elementos en formato denso frente a tres mil no nulos.



Al reemplazar las derivadas espaciales por diferencias finitas en cada nodo interior se obtiene

$$\frac{dc_i}{dt} = -u \frac{c_{i+1} - c_{i-1}}{2\Delta x} + D \frac{c_{i-1} - 2c_i + c_{i+1}}{\Delta x^2} - k c_i, \quad i = 1, \dots, n$$

↓ n ecuaciones diferenciales ordinarias acopladas

$$\dot{\mathbf{c}} = \mathbf{A} \mathbf{c} + \mathbf{b}(t), \text{ con } \mathbf{A} = \frac{1}{\Delta x^2} \begin{bmatrix} \beta & \gamma & & & \\ \alpha & \beta & \gamma & & \\ & \ddots & \ddots & \ddots & \\ & & \alpha & \beta & \gamma \\ & & & \alpha & \beta \end{bmatrix} \begin{matrix} \alpha = D + \frac{1}{2}u\Delta x \\ \beta = -2D - k\Delta x^2 \\ \gamma = D - \frac{1}{2}u\Delta x \end{matrix}$$

Figura 3.9. Método de líneas aplicado a la ecuación de transporte, desde el stencil de tres puntos sobre la malla espacial hasta el sistema de ecuaciones diferenciales ordinarias con matriz tridiagonal.

Listado 3.5. Ensamblaje disperso de la matriz de diferencias finitas para advección, difusión y decaimiento.

```
1 import numpy as np
2 from scipy.sparse import diags
3 def matriz_transporte(n, dx, u, D, k):
4     """Diferencias centradas para advección, difusión y
5         decaimiento."""
6     sub = np.full(n - 1, D/dx**2 + u/(2*dx))
7     dia = np.full(n, -2*D/dx**2 - k)
8     sup = np.full(n - 1, D/dx**2 - u/(2*dx))
9     return diags([sub, dia, sup], [-1, 0, 1], format="csc")
```

Con la matriz disponible, la integración temporal se reduce a una llamada al mismo `solve_ivp`, con el campo definido como un producto matriz por vector y el jacobiano igual a la propia matriz. El [Listado 3.6](#) completa el procedimiento y calcula el error frente a la solución analítica, verificación mínima exigible a toda implementación de este tipo.

Listado 3.6. Método de líneas para el transporte de un vertimiento en un tramo de río.

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3 def perfil_exacto(x, t, u, D, k, M, area, x0):
4     s2 = 4*D*t
5     return (M/(area*np.sqrt(np.pi*s2))
6             * np.exp(-(x - x0 - u*t)**2/s2)*np.exp(-k*t))
7 L, n = 5000.0, 249
8 x = np.linspace(0, L, n + 2)[1:-1]
9 dx = x[1] - x[0]
10 par = dict(u=0.35, D=12.0, k=0.25/86400, M=25.0, area=18.0,
11            x0=1200.0)
12 A = matriz_transporte(n, dx, par["u"], par["D"], par["k"])
13 c0 = perfil_exacto(x, 600.0, **par)
14 sol = solve_ivp(lambda t, c: A @ c, (600.0, 3600.0), c0,
15                 method="BDF",
16                 jac=lambda t, c: A, rtol=1e-8, atol=1e-14,
17                 t_eval=[3600.0])
18 error = np.max(np.abs(sol.y[:, 0] - perfil_exacto(x, 3600.0,
19 **par)))
```

El procedimiento completo, con la verificación por refinamiento, aparece en el [Algoritmo 3.2](#). Los tres controles que allí se enumeran separan una implementación defendible de una que solo produce gráficas, y solo el último exige la solución exacta, que se sustituye por una de malla muy fina cuando no existe.

3.3.2. Estabilidad del esquema explícito y condición de Courant

El sistema que produce el método de líneas suele ser rígido por una razón que ahora se enuncia con precisión. Los autovalores de la matriz de difusión escalan como $-4D/\Delta x^2$ en el extremo del espectro, de modo que refinar la malla por un factor dos multiplica por cuatro el autovalor de mayor magnitud, y con un método explícito la restricción de estabilidad se vuelve prohibitiva.

Dos números adimensionales resumen esa competencia. El *número de Courant* $C = u \Delta t / \Delta x$ mide cuántas celdas recorre la advección en un paso de tiempo y el *número de difusión* $d = D \Delta t / \Delta x^2$ mide cuántas alcanza la difusión. El primero gobierna una restricción más profunda que la estabilidad lineal, pues expresa una condición geométrica sobre los dominios de dependencia. Si la solución exacta en un nodo depende de información que el estencil no alcanza, ningún refinamiento puede reproducirla. Ese argumento, de Courant, Friedrichs y Lewy, es anterior al análisis de von Neumann y no depende de la linealidad del esquema [62].

Algoritmo 3.2 Simulación de un sistema distribuido por el método de líneas con verificación por refinamiento.

Entradas: Ecuación de transporte, dominio $[0, L]$, horizonte $[t_0, t_f]$, condiciones de frontera, sucesión de mallas $n_1 < n_2 < \dots < n_m$

Salidas: Perfil $\mathbf{c}(t_f)$ y orden observado $\hat{p}$

- 1: **para** $j = 1$ **hasta** m **hacer**
 - 2: $\Delta x \leftarrow L/n_j$; verificar que el número de Péclet de celda $u \Delta x/D$ no supere el valor 2
 - 3: ensamblar $\mathbf{A}$ y $\mathbf{b}$ con las condiciones de frontera e imponer el perfil inicial
 - 4: integrar $\dot{\mathbf{c}} = \mathbf{A}\mathbf{c} + \mathbf{b}$ hasta t_f con tolerancia muy estricta
 - 5: verificar que la masa total cierre el balance con el término de decaimiento
 - 6: $e_j \leftarrow \|\mathbf{c}(t_f) - \mathbf{c}^*(t_f)\|_\infty$
 - 7: **fin para**
 - 8: $\hat{p} \leftarrow$ pendiente de $\log e_j$ frente a $\log \Delta x_j$
 - 9: **si** $\hat{p}$ difiere de forma apreciable del orden teórico **entonces**
 - 10: **devolver** aviso de error en el ensamblaje o en las condiciones de frontera
 - 11: **fin si**
 - 12: **devolver** $\mathbf{c}(t_f)$ y $\hat{p}$
-

Teorema 3.2. Condición de Courant–Friedrichs–Lewy. *Considérese la ecuación de advección lineal $\partial_t c + u \partial_x c = 0$ con $u > 0$, discretizada sobre una malla uniforme mediante un esquema explícito de un paso cuyo stencil espacial se extiende p nodos hacia la izquierda y q hacia la derecha. Una condición necesaria para que el esquema converja cuando Δt y Δx tienden a cero con razón $\Delta t/\Delta x$ constante es que el número de Courant satisfaga $-q \leq C \leq p$. En particular, para el esquema de diferencias regresivas, con $p = 1$ y $q = 0$, la condición es $0 \leq C \leq 1$.*

La condición es necesaria y no suficiente, pues un esquema puede respetarla y ser inestable por otras razones. Para la ecuación de transporte completa, el análisis de von Neumann sobre el esquema explícito centrado añade $d \leq 1/2$, casi siempre más severa que la de Courant en problemas dominados por la dispersión. Con $\Delta x = 20$ m, $u = 0.35$ m/s y $D = 12$ m²/s, la difusión limita el paso a 16.67 s y la advección a 57.14 s.

Una violación moderada de la condición no siempre se manifiesta de inmediato, porque el modo inestable crece desde el nivel del error de redondeo y puede requerir centenares de pasos para hacerse visible. Un horizonte corto la oculta y el resultado parece correcto, razón por la cual la condición se verifica antes de correr. El remedio

definitivo es un integrador implícito, que deja el paso en manos de la exactitud a costa de resolver un sistema lineal disperso por paso [63]. Cuando la geometría deja de ser unidimensional, el mismo razonamiento se traslada a los elementos finitos, cuya implementación en Python está documentada con detalle [64].

El caso que sigue reúne los tres controles del [Algoritmo 3.2](#) sobre un vertimiento accidental. La solución analítica de una inyección instantánea en un canal uniforme e infinito verifica la implementación de manera concluyente.

Ejemplo 3.4. Transporte de un vertimiento en un tramo de río. Un vertimiento accidental descarga 25 kg de un compuesto biodegradable en un punto de un río de sección 18 m², velocidad media 0.35 m/s y coeficiente de dispersión longitudinal 12 m²/s. El compuesto se degrada con una constante de primer orden de 0.25/d. Se pide el perfil de concentración una hora después de la descarga y la verificación del esquema numérico.

Consideraciones. 1. Canal prismático con velocidad, sección y dispersión longitudinal uniformes; 2. mezcla completa en la sección transversal desde el instante inicial; 3. sin aportes laterales ni intercambio con el lecho.

Formulación. El modelo es la [Ecuación 3.8](#) con $s = 0$. Para una inyección instantánea de masa M en $x = x_0$ sobre un dominio infinito, la solución analítica es la de la [Ecuación \(3.10\)](#),

$$c(x, t) = \frac{M}{A\sqrt{4\pi Dt}} \exp\left[-\frac{(x - x_0 - ut)^2}{4Dt}\right] \exp(-kt), \quad (3.10)$$

donde M es la masa vertida (kg); A es el área de la sección transversal (m²); x_0 es la abscisa del vertimiento (m); y las demás variables conservan el significado de la [Ecuación 3.8](#). El dominio numérico abarca 5000 m y el perfil analítico en $t_0 = 600$ s sirve de condición inicial.

Solución. Con 250 intervalos, es decir $\Delta x = 20$ m y número de Péclet de celda igual a 0.583, la integración con BDF desde 600 s hasta 3600 s entrega un pico de 1.8655 mg/L en la abscisa 2460 m, con error máximo de 7.18×10^{-6} kg/m³ frente a la solución analítica, el 0.38 % del pico.

Verificación. La masa remanente calculada por integración numérica del perfil vale 24.740935 kg y coincide en las nueve cifras reportadas con el valor teórico Me^{-kt} . El refinamiento sucesivo de la malla hasta 2.5 m entrega un orden observado de 2.00, coherente con el error de truncamiento de la [Ecuación 3.9](#). Un esquema explícito con $\Delta t = 15$ s, es decir $d = 0.45$, reproduce el perfil con error de 6.3×10^{-5} kg/m³, mientras que con $\Delta t = 20$ s, es decir $d = 0.60$, la solución diverge y alcanza valores del orden de 5×10^2 kg/m³.

Comentario. La condición de Dirichlet nula en ambos extremos es admisible porque el dominio es lo bastante extenso para que la solución analítica sea despreciable en los bordes. Con un dominio de 4000 m y el vertimiento a 300 m del extremo aguas arriba, el error se estanca en 2.7×10^{-7} y deja de disminuir con el refinamiento, señal de que la frontera artificial contamina el interior.

El estancamiento del comentario anterior es el modo de falla más difícil de diagnosticar en un sistema distribuido. Un estudio de convergencia con orden correcto en las mallas gruesas que se aplanan en las finas indica que algún error independiente de Δx domina el residuo, y las tres causas habituales son una frontera artificial demasiado cercana, una tolerancia temporal insuficiente y un error en la condición inicial. El panel derecho de la [Figura 3.10](#) muestra el comportamiento sano, en el cual el error se divide por cuatro cada vez que la malla se refina por dos.

La comparación entre el perfil calculado y la solución analítica, junto con la recta de convergencia en escala doblemente logarítmica, aparece en la [Figura 3.10](#). La coincidencia visual de los puntos con la curva exacta es el criterio débil y la pendiente igual a dos es el fuerte, porque una gráfica superpuesta oculta con facilidad errores del uno por ciento.

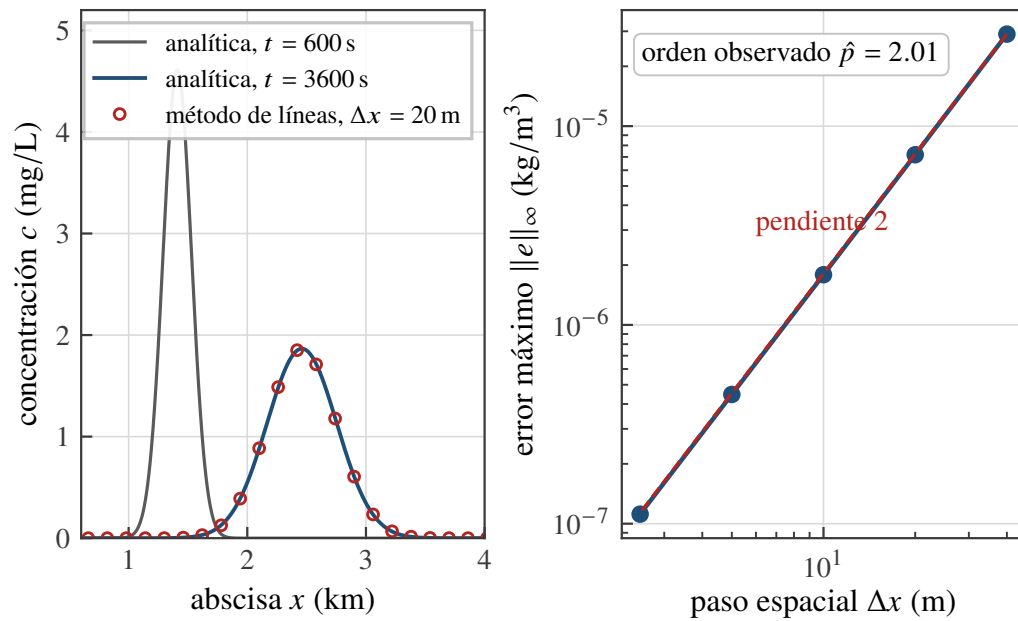


Figura 3.10. Perfil de concentración calculado por el método de líneas frente a la solución analítica y estudio de convergencia del error máximo bajo refinamiento de la malla.

3.4. Análisis de escenarios y comparación de alternativas

Las tres secciones anteriores enseñaron a producir una corrida, y ninguna decisión de ingeniería se toma con una corrida. La pregunta que llega al escritorio rara vez es cuál es el caudal de salida con estos datos, sino cuál de las tres alternativas conviene, qué pasa si el año resulta seco o hasta dónde se puede reducir la inversión sin comprometer el servicio. Responderlas exige ejecutar el modelo muchas veces bajo condiciones deliberadamente elegidas.

Conviene separar dos nociones que el lenguaje corriente confunde. Una corrida es una ejecución del modelo con un juego completo de valores de entrada. Un escenario es de nivel superior, pues describe de manera coherente una situación posible y se traduce en uno o varios juegos de valores. La distinción importa porque las entradas de un escenario deben ser mutuamente compatibles y porque un mismo escenario puede requerir varias corridas.

Definición 3.4. Escenario. Un *escenario* es una combinación coherente y completa de valores asignados a los factores de un modelo, que representa una situación posible del sistema y de su entorno. Los factores se clasifican en factores de decisión, que el ingeniero controla, y factores de contexto, que sufre y no elige, y esa distinción determina cómo se interpreta el resultado de la comparación.

Esa distinción tiene una consecuencia metodológica que se pasa por alto con frecuencia. Las alternativas se comparan dentro de un mismo contexto y nunca a través de contextos distintos, porque una que parece mejor en un año húmedo puede ser peor en uno seco y el promedio de ambas situaciones no representa ninguna. El procedimiento correcto construye la comparación para cada contexto y examina después la robustez de la recomendación.

El primer paso operativo construye la matriz de escenarios, fijando los factores relevantes, eligiendo unos pocos niveles para cada uno y combinándolos. Los niveles no se eligen al azar, pues un factor continuo se representa con un valor central y dos que acoten el rango plausible, tomados de la variabilidad histórica o de la norma aplicable, y uno discreto con las alternativas realmente disponibles. La Figura 3.11 muestra a la izquierda una matriz con tres factores y tres niveles.

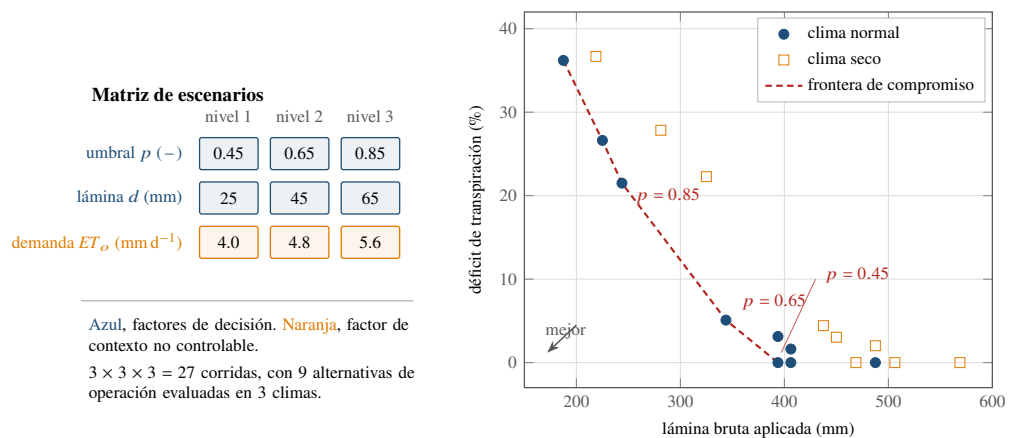


Figura 3.11. Matriz de escenarios con factores de decisión y de contexto, y frontera de compromiso entre el agua aplicada y el déficit de transpiración para las nueve alternativas de operación bajo dos climas.

El costo del barrido completo crece como el producto de los niveles de cada factor. Con tres factores de tres niveles resultan veintisiete corridas, con siete factores resultan 2187 y con diez el barrido se vuelve inviable si cada corrida tarda un minuto. El diseño factorial fraccionado estima entonces los efectos principales y las interacciones de bajo orden con una fracción de las corridas, a costa de confundir efectos de orden alto que suelen ser despreciables [65].

3.4.1. Indicadores de desempeño y comparación multicriterio

Una matriz de escenarios produce una tabla de resultados, y una tabla no es una recomendación. El paso intermedio reduce cada corrida a unas pocas magnitudes escalares que sintetizan lo que importa de la decisión, los indicadores de desempeño, cada uno especificado por su definición matemática, sus unidades, su sentido de preferencia y el umbral que separa lo aceptable de lo inaceptable. Esas magnitudes deben fijarse antes de correr el modelo, para evitar el sesgo de elegir después el indicador que favorece la conclusión preferida.

Los indicadores de un problema de ingeniería casi nunca son compatibles entre sí. Reducir el consumo de agua aumenta el déficit del cultivo y ampliar la capacidad instalada eleva la confiabilidad y el costo. Ante indicadores en conflicto, la tentación es combinarlos en un índice único mediante pesos, procedimiento que traslada la decisión a la elección de los pesos y la oculta tras una cifra aparentemente objetiva. Existe una alternativa que no requiere pesos y no compromete al analista con ninguna preferencia.

Se dice que una alternativa *domina* a otra cuando no es peor en ningún indicador y es estrictamente mejor en al menos uno, y el conjunto de las alternativas no dominadas constituye la *frontera de compromiso*, sobre la cual toda mejora de un indicador exige empeorar otro. Esa frontera hace un trabajo doble, pues elimina las alternativas dominadas, sobre las cuales no cabe discusión razonable, y expone el precio de cada mejora sobre las que quedan. En la [Figura 3.11](#) reducir el déficit del 21.5 % al 5.1 % cuesta 100 mm adicionales de agua y eliminarlo por completo cuesta otros 50 mm. Esa es la información que un tomador de decisiones necesita, y no un índice ponderado que ya decidió por él.

El procedimiento se organiza en tres pasos, la enumeración de las combinaciones de niveles, la ejecución con registro del diagnóstico de convergencia y la extracción de la frontera dentro de cada contexto. Los dos controles que ese orden impone evitan las fallas típicas del análisis de escenarios, que son extraer conclusiones de corridas que no convergieron y comparar alternativas a través de contextos distintos.

La implementación se apoya en el producto cartesiano de los niveles y en una tabla de pandas [26]. El [Listado 3.7](#) construye la tabla y extrae la frontera restringida a un contexto fijo, tal como exige la advertencia anterior.

Listado 3.7. Barrido factorial de escenarios y extracción de la frontera de compromiso.

```
1 import itertools
2 import pandas as pd
3 niveles = {"p": [0.45, 0.65, 0.85],
4           "d": [25.0, 45.0, 65.0],
5           "ETo": [4.0, 4.8, 5.6]}
6 corridas = []
7 for combo in itertools.product(*niveles.values()):
8     esc = dict(zip(niveles, combo))
```

```

9     corridas.append(**esc, **balance_hidrico(**esc))
10    tabla = pd.DataFrame(corridas)
11    normal = tabla.query("ETo == 4.8").reset_index(drop=True)
12    dominado = normal.apply(
13        lambda f: ((normal.bruto <= f.bruto) & (normal.deficit <=
14                    f.deficit))
15                    & ((normal.bruto < f.bruto) | (normal.deficit <
16                    f.deficit))
17                    ).any(), axis=1)
18    frente = normal[~dominado].sort_values("bruto")

```

El caso que sigue aplica el procedimiento a la programación del riego de un cultivo de maíz, con el balance hídrico del suelo formulado según la publicación 56 de la Organización de las Naciones Unidas para la Alimentación y la Agricultura [12].

Ejemplo 3.5. Programación del riego de un cultivo de maíz. Un lote de maíz de 90 d de ciclo se cultiva en un suelo franco con humedad a capacidad de campo de 0.29 y punto de marchitez de 0.15, sobre una profundidad radicular efectiva de 0.90 m. El sistema de riego tiene eficiencia de aplicación de 0.80. Se comparan tres umbrales de agotamiento para disparar el riego, 0.45, 0.65 y 0.85, y tres láminas netas por evento, 25 mm, 45 mm y 65 mm, bajo tres niveles de demanda climática, 4.0 mm/d, 4.8 mm/d y 5.6 mm/d de evapotranspiración de referencia.

Consideraciones. 1. Precipitación nula durante el ciclo, propia de la temporada seca del Caribe colombiano; 2. profundidad radicular constante, escorrentía nula y percolación instantánea del exceso; 3. fracción de agotamiento sin estrés del cultivo igual a 0.55.

Formulación. El agua aprovechable total vale $TAW = 1000 (\theta_{cc} - \theta_{pm}) Z_r = 126$ mm. El agotamiento D_r se actualiza cada día con la Ecuación (3.11),

$$\begin{aligned}
 D_{r,j} &= \min(TAW, D_{r,j-1} - I_j + K_{s,j} K_{c,j} ET_o), \\
 K_{s,j} &= \min\left(1, \frac{TAW - D_{r,j-1}}{TAW - RAW}\right),
 \end{aligned} \tag{3.11}$$

donde $D_{r,j}$ es el agotamiento al final del día j (mm); I_j es la lámina neta aplicada ese día (mm); $K_{s,j}$ es el coeficiente de estrés hídrico (adimensional); $K_{c,j}$ es el coeficiente de cultivo (adimensional); y RAW es el agua fácilmente aprovechable (mm). Los indicadores son la lámina bruta total aplicada y el déficit relativo de transpiración $1 - \sum ET_a / \sum ET_c$.

Solución. El barrido de las 27 combinaciones produce la [Tabla 3.3](#) y, en el clima normal, cinco de las nueve alternativas resultan no dominadas y forman la frontera de la [Figura 3.11](#). La de umbral 0.45 y lámina 45 mm elimina el déficit con 393.8 mm brutos y siete riegos, y la de umbral 0.85 y lámina 65 mm aplica 243.8 mm en tres riegos a cambio de un déficit del 21.5 %.

Verificación. La alternativa de umbral 0.45 y lámina 65 mm aplica 487.5 mm brutos con déficit nulo y percola 35.8 mm, de modo que resulta dominada por la de lámina 45 mm, y ese resultado confirma que la regla de dominancia detecta el desperdicio sin intervención del analista. La suma de la lámina neta, la percolación y el agua residual reproduce además la transpiración acumulada con error inferior a 0.1 mm.

Comentario. Ninguna de las cinco alternativas de la frontera es la respuesta, que depende del precio del agua y del valor del rendimiento perdido, información ausente del modelo hidrológico. El análisis entrega el conjunto de opciones defendibles y el precio de pasar de una a otra, que es todo lo que el modelo puede aportar.

Tabla 3.3. Resultados del barrido de escenarios de riego bajo tres niveles de demanda climática.

Decisión		Lámina bruta (mm)			Déficit (%)		
<i>p</i>	<i>d</i> (mm)	4.0	4.8	5.6	4.0	4.8	5.6
0.45	25	312.5	406.2	468.8	0.00	0.00	0.00
0.45	45	337.5	393.8	506.2	0.00	0.00	0.00
0.45	65	406.2	487.5	568.8	0.00	0.00	0.00
0.65	25	281.2	343.8	437.5	4.82	5.09	4.43
0.65	45	281.2	393.8	450.0	2.27	3.13	3.04
0.65	65	325.0	406.2	487.5	2.03	1.63	2.02
0.85	25	125.0	187.5	218.8	33.48	36.20	36.68
0.85	45	168.8	225.0	281.2	25.16	26.62	27.84
0.85	65	243.8	243.8	325.0	21.71	21.50	22.28

^a Los tres niveles de cada bloque corresponden a evapotranspiración de referencia de 4.0 mm/d, 4.8 mm/d y 5.6 mm/d.

^b Eficiencia de aplicación de 0.80 en todas las corridas.

El paso final es el que más se descuida y consiste en discutir los resultados en función del problema de estudio y no del modelo. Una tabla de corridas se convierte en recomendación cuando se declara qué alternativas quedaron descartadas y por qué, cuál es el precio de cada mejora sobre la frontera, qué tan sensible es el orden de preferencia al cambio de contexto y qué información faltante impide decidir. Aquí la recomendación defendible es que el umbral 0.65 con lámina de 65 mm entrega un déficit inferior al 2.1 % en los tres climas con cinco riegos o menos.

3.5. Simulación estocástica y propagación de la variabilidad

El análisis de escenarios recorre unas pocas combinaciones elegidas, enfoque que responde bien cuando los factores son decisiones discretas y mal cuando las entradas varían de manera continua con una frecuencia conocida. El caudal de un río, la composición de un residuo o el potencial metanogénico de un lote de estiércol no toman tres valores, sino que se distribuyen. Preguntar por el resultado en el escenario medio no basta, porque la respuesta del modelo en el promedio de las entradas rara vez coincide con el promedio de las respuestas.

Conviene distinguir dos orígenes de la dispersión, distinción que el [Capítulo 4](#) retoma. La *variabilidad natural* es la fluctuación real de una magnitud entre repeticiones y no se reduce midiendo mejor, mientras que la *incertidumbre de parámetro* refleja el desconocimiento del valor verdadero de una constante y sí se reduce con más datos. Ambas se propagan con la misma herramienta y conducen a lecturas distintas.

Definición 3.5. Método de Monte Carlo. El *método de Monte Carlo* aplicado a la propagación consiste en generar N realizaciones independientes del vector de entradas conforme a su distribución conjunta, evaluar el modelo en cada una y describir la muestra de respuestas obtenida mediante sus momentos, sus percentiles y sus probabilidades de excedencia. El estimador de la media es el promedio muestral de las respuestas.

Las realizaciones parten de un generador de números pseudoaleatorios uniformes en el intervalo unitario y las distribuciones se obtienen con la función cuantílica correspondiente. Ese generador es determinista y su estado inicial se llama semilla, de modo que fijarla convierte una corrida aleatoria en una reproducible. Declararla en el informe es un requisito, no una cortesía, y el curso emplea el valor 20 262.

La propiedad que justifica el método y que permite dimensionar el esfuerzo de cómputo es la siguiente, y se sigue de la ley de los grandes números junto con el teorema central del límite.

Teorema 3.3. Convergencia del estimador de Monte Carlo. Sea $Y = g(\mathbf{X})$ con $\mathbb{E}[Y] = \mu$ y $\text{Var}[Y] = \sigma^2 < \infty$, y sea $\bar{Y}_N$ el promedio de N realizaciones independientes. Entonces $\bar{Y}_N \rightarrow \mu$ casi seguramente, $\text{Var}[\bar{Y}_N] = \sigma^2/N$ y $\sqrt{N}(\bar{Y}_N - \mu)$ converge en distribución a una normal de media nula y varianza σ^2 . En consecuencia, el error del estimador decrece como $\sigma/\sqrt{N}$ con independencia de la dimensión de $\mathbf{X}$.

Las dos lecturas del teorema apuntan en direcciones opuestas. La tasa no se degrada al aumentar el número de entradas inciertas, cosa que sí ocurre con las cuadraturas de malla, cuyo costo crece de manera exponencial con la dimensión. En cambio la tasa es lenta, pues reducir el error a la décima parte exige multiplicar por cien el número de corridas. Con σ estimada de una corrida piloto basta despejar $N = (1.96 \sigma/\epsilon)^2$ para una semiamplitud ϵ al 95 % de confianza [66].

El Algoritmo 3.3 organiza el procedimiento y explicita dos puntos que las implementaciones apresuradas omiten, la separación entre el muestreo en el hipercubo unitario y la transformación a las variables físicas, que permite cambiar de esquema sin tocar el modelo, y el registro de las corridas fallidas, cuyo descarte silencioso sesga el resultado hacia la región en la cual el modelo converge.

Algoritmo 3.3 Propagación de la variabilidad por muestreo de Monte Carlo.

Entradas: Modelo g , distribuciones marginales $F_1, \dots, F_k$, tamaño de muestra N , semilla s

Salidas: Muestra $\{y_n\}$ y sus estadísticos con su error estándar

- 1: fijar la semilla s y generar $\mathbf{U}$ de tamaño $N \times k$ con entradas uniformes en $(0, 1)$
 - 2: **para** $n = 1$ **hasta** N **hacer**
 - 3: $x_{n,j} \leftarrow F_j^{-1}(u_{n,j})$ para $j = 1, \dots, k$
 - 4: $y_n \leftarrow g(\mathbf{x}_n)$, registrando la falla junto con $\mathbf{x}_n$ si la evaluación no converge
 - 5: **fin para**
 - 6: $\bar{y} \leftarrow$ promedio de $\{y_n\}$; $s_y \leftarrow$ desviación muestral; $ee \leftarrow s_y/\sqrt{N}$
 - 7: **si** ee supera la precisión exigida **entonces**
 - 8: aumentar N y repetir el muestreo
 - 9: **fin si**
 - 10: **devolver** $\bar{y}$, s_y , ee , percentiles y proporción de fallas
-

El muestreo aleatorio simple desperdicia esfuerzo, porque varias realizaciones pueden caer muy cerca unas de otras y dejar regiones enteras sin visitar. El muestreo estratificado corrige el defecto imponiendo una cobertura uniforme de cada margen, y su versión más difundida se debe a McKay, Beckman y Conover [67].

Definición 3.6. Muestreo por hipercubo latino. El *muestreo por hipercubo latino* divide el recorrido de cada una de las k entradas en N estratos de igual probabilidad, extrae exactamente una realización de cada estrato y combina al azar las extracciones de las distintas entradas. La muestra resultante cubre de manera uniforme cada margen por construcción, sin dejar de ser aleatoria en la estructura conjunta.

La ganancia es apreciable cuando la respuesta depende sobre todo de los efectos individuales de cada entrada, situación frecuente en ingeniería, porque el hipercubo latino elimina casi por completo la componente aditiva de la varianza del estimador. El [Listado 3.8](#) muestra las dos formas de muestreo con la misma interfaz, de modo que reemplazar una por la otra no exige tocar el resto del programa.

Listado 3.8. Muestreo aleatorio simple y por hipercubo latino sobre el hipercubo unitario.

```
1 import numpy as np
2 from scipy.stats import qmc
3 def muestra_lhs(n, d, semilla):
4     disenio = qmc.LatinHypercube(d=d, seed=semilla)
5     return disenio.random(n)
6 def muestra_mc(n, d, semilla):
7     return np.random.default_rng(semilla).random((n, d))
8 u_lhs = muestra_lhs(500, 6, 20262)
9 u_mc = muestra_mc(500, 6, 20262)
```

La transformación hasta las variables físicas se hace con las funciones cuantílicas de SciPy, y el [Listado 3.9](#) la reúne con el modelo y el resumen estadístico. La función de transformación es la única pieza que cambia cuando se revisa una distribución, separación que facilita el análisis de sensibilidad de la sección siguiente.

Listado 3.9. Propagación de la variabilidad en un biodigestor por muestreo de Monte Carlo.

```
1 import numpy as np
2 from scipy.stats import lognorm, norm, triang, uniform
3 def transformar(u):
4     """Del hipercubo unitario a las variables físicas del
5         biodigestor."""
6     s = np.sqrt(np.log(1 + 0.18**2))
7     return dict(m=norm.ppf(u[:, 0], 1200.0, 90.0),
8                 SV=norm.ppf(u[:, 1], 0.115, 0.012),
9                 B0=lognorm.ppf(u[:, 2], s, scale=0.21),
10                 k=uniform.ppf(u[:, 3], 0.10, 0.15),
```

```

10         TRH=triang.ppf(u[:, 4], 6/13, loc=22.0,
11                     scale=13.0),
12         eta=triang.ppf(u[:, 5], 0.5, loc=0.28,
13                     scale=0.08))
14 def energia(m, SV, B0, k, TRH, eta):
15     """Energía eléctrica diaria en kW h por día."""
16     return m*SV*B0*(1 - np.exp(-k*TRH))*35.8*eta/3.6
17 rng = np.random.default_rng(20262)
18 E = energia(**transformar(rng.random((20000, 6))))
19 resumen = dict(media=E.mean(), desv=E.std(ddof=1),
20               p05=np.percentile(E, 5), p50=np.percentile(E, 50),
21               p95=np.percentile(E, 95), exced=(E >=
22               110.0).mean())

```

La lectura del resultado exige más que reportar la media. La [Figura 3.12](#) muestra la convergencia del estimador con su banda de confianza y el histograma con los percentiles marcados. La banda se estrecha con la raíz del número de corridas, tal como anuncia el [Teorema 3.3](#), y el histograma revela una asimetría que la media oculta.

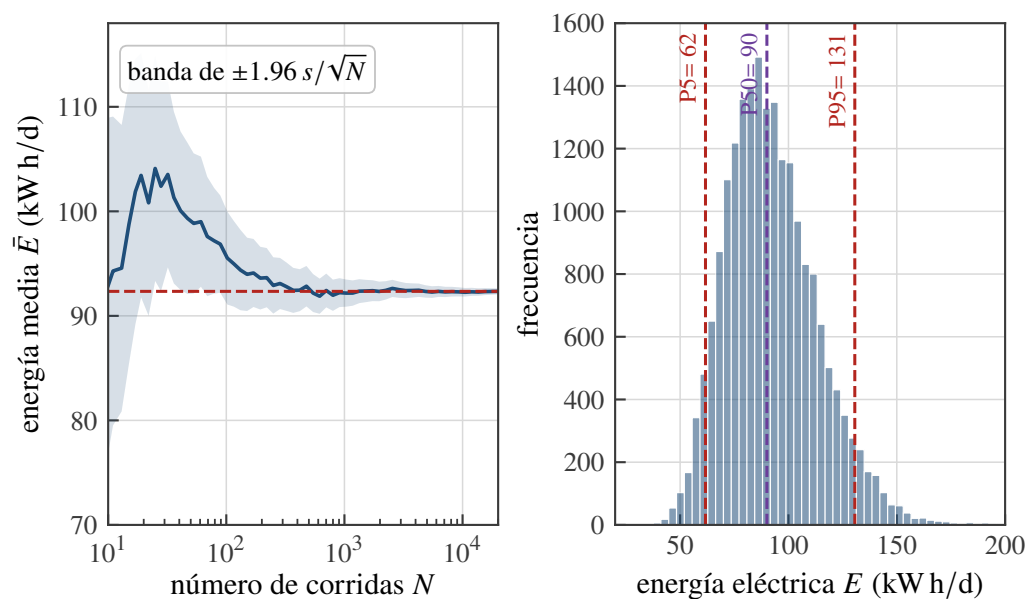


Figura 3.12. Convergencia del estimador de la energía media con su banda de confianza al 95 % e histograma de la respuesta con los percentiles cinco, cincuenta y noventa y cinco.

Ejemplo 3.6. Producción eléctrica de un biodigestor de estiércol bovino. Un biodigestor de mezcla completa recibe estiércol bovino con una disponibilidad diaria incierta de media 1200 kg/d y desviación 90 kg/d. El contenido de sólidos volátiles se distribuye de forma normal con media 0.115 y desviación 0.012 en base másica. El potencial metanogénico sigue una distribución logarítmico normal de mediana 0.21 m³/kg de sólidos volátiles y coeficiente de variación 0.18. La constante de hidrólisis es uniforme entre 0.10/d y 0.25/d, el tiempo de retención es triangular entre 22 y 35 d con moda 28, y la eficiencia eléctrica del motor es triangular entre 0.28 y 0.36 con moda 0.32. Se pide caracterizar la energía eléctrica diaria y la probabilidad de cubrir una demanda de 110 kW h/d.

Consideraciones. 1. Régimen permanente con alimentación continua e hidrólisis de primer orden como etapa limitante; 2. poder calorífico inferior del metano de 35.8 MJ/m³ en condiciones normales; 3. entradas mutuamente independientes.

Formulación. El caudal de metano y la energía eléctrica se obtienen de la Ecuación (3.12),

$$V_{CH_4} = m x_{SV} B_0 [1 - \exp(-k TRH)] , \quad E = \frac{V_{CH_4} PCI \eta}{3.6}, \quad (3.12)$$

donde m es la masa diaria de estiércol (kg/d); x_{SV} es la fracción másica de sólidos volátiles (adimensional); B_0 es el potencial metanogénico (m³/kg); k es la constante de hidrólisis (1/d); TRH es el tiempo de retención hidráulica (d); PCI es el poder calorífico inferior del metano (MJ/m³); y η es la eficiencia eléctrica del grupo motogenerador (adimensional).

Solución. Con $N = 20\,000$ y semilla 20 262, la media resulta 92.34 kW h/d con desviación 21.22 kW h/d, es decir un coeficiente de variación del 23.0 %. Los percentiles cinco, cincuenta y noventa y cinco valen 61.71 kW h/d, 90.05 kW h/d y 130.62 kW h/d. La probabilidad de igualar o superar la demanda de 110 kW h/d es de 0.1945. El error estándar de la media vale 0.150 kW h/d.

Verificación. El modelo evaluado en los valores centrales entrega 91.53 kW h/d, un 0.9 % por debajo de la media muestral, diferencia atribuible a la asimetría de la distribución logarítmico normal y a la curvatura del término exponencial. Con 200 réplicas de 500 corridas, la desviación del estimador de la media vale 0.933 kW h/d con muestreo aleatorio simple y 0.112 kW h/d con hipercubo latino, una reducción de varianza de 69 veces.

Comentario. La conclusión operativa no es que el biodigestor produzca 92 kWh/d, sino que cubre la demanda supuesta en menos de una quinta parte de los días. Si el diseño debe garantizar el suministro, la magnitud pertinente es el percentil cinco y con ella el dimensionamiento cambia por completo.

El contraste entre la media, la mediana y el percentil bajo es la lección central de esta sección. Un informe que reporta solo el valor esperado transmite una certeza que la simulación no respalda, y uno que reporta el intervalo sin declarar el tamaño de muestra ni la semilla no es reproducible. El mínimo defendible incluye el estimador puntual, su error estándar, los percentiles relevantes, el número de corridas, el esquema de muestreo y la semilla. Sobre esa base, el [Capítulo 4](#) construye el tratamiento formal de la incertidumbre.

3.6. Análisis de sensibilidad

La sección anterior entrega una distribución de la respuesta y queda por responder de dónde viene esa dispersión. Si un solo parámetro explica el ochenta por ciento de la varianza, medirlo mejor reduce la incertidumbre del pronóstico y afinar los demás no sirve de nada. Si diez parámetros contribuyen por igual, ninguna campaña de medición razonable mejorará el resultado y la conclusión honesta es que el modelo resulta demasiado ambicioso para los datos disponibles.

El análisis persigue tres objetivos separados, jerarquizar las entradas por su influencia para orientar la medición y la calibración, identificar las irrelevantes, que pueden fijarse en su valor nominal, y detectar comportamientos inesperados del modelo, como una influencia que cambia de signo dentro del rango plausible, síntoma habitual de un error de formulación [68].

3.6.1. Sensibilidad local y elasticidad

El enfoque más simple mide cuánto cambia la respuesta ante una perturbación infinitesimal de una entrada alrededor del punto nominal. Esa *sensibilidad local* es la derivada parcial $\partial g / \partial x_j$ evaluada en $\mathbf{x}^0$, su cálculo por diferencias centradas cuesta dos evaluaciones por entrada y no exige ninguna hipótesis sobre distribuciones. Sus unidades son las de la respuesta divididas por las de la entrada.

Ese detalle de unidades impide usar la derivada como criterio de jerarquía. Una sensibilidad de 0.4 kW h/d por cada grado Celsius y otra de 60 kW h/d por cada unidad de una fracción adimensional no admiten comparación directa, y el orden que sugieren depende de las unidades elegidas. La normalización que resuelve el problema es la misma que usa la economía desde hace un siglo.

Definición 3.7. Elasticidad. La *elasticidad* de la respuesta respecto de la entrada x_j es la sensibilidad local adimensionalizada, $E_j = (\partial g / \partial x_j) (x_j^0 / y^0)$, e indica el cambio porcentual de la respuesta que produce un cambio del uno por ciento en la entrada, con las demás entradas fijas en su valor nominal.

La elasticidad ofrece una verificación gratuita. Si una entrada aparece como factor multiplicativo su elasticidad debe valer exactamente la unidad, y si aparece en el denominador debe valer menos uno, de modo que comprobar esas identidades detecta de inmediato un error de signo o un paso de diferenciación mal elegido. El [Listado 3.10](#) calcula todas las elasticidades de un modelo cuyo argumento es un diccionario de parámetros.

Listado 3.10. Elasticidades de un modelo por diferencias centradas de paso relativo.

```
1 import numpy as np
2 def elasticidades(modelo, nominal, paso=1e-4):
3     """Elasticidad de la respuesta frente a cada factor, por
4         diferencias
5         centradas de paso relativo."""
6     y0 = modelo(**nominal)
7     resultado = {}
8     for nombre, x0 in nominal.items():
9         h = paso*abs(x0)
10        mas, menos = dict(nominal), dict(nominal)
11        mas[nombre], menos[nombre] = x0 + h, x0 - h
12        resultado[nombre] = (modelo(**mas) -
13                             modelo(**menos)) / (2*h) * x0 / y0
14    return resultado
```

La elasticidad describe solo el entorno inmediato del punto nominal. El método de un factor a la vez extiende la idea recorriendo el rango plausible completo de cada entrada mientras las demás permanecen fijas, y su representación es el diagrama de tornado, en el cual cada barra abarca el intervalo de respuestas de una entrada y las barras se ordenan por amplitud decreciente. La [Figura 3.13](#) muestra uno para el costo nivelado de la energía de un sistema fotovoltaico.

Ambos responden preguntas distintas y por eso ordenan los factores de manera distinta. La elasticidad mide el efecto de un cambio porcentual unitario y es una propiedad de la estructura del modelo, mientras que el tornado mide el efecto del rango realmente alcanzable e incorpora el conocimiento sobre la variabilidad de cada entrada. Un factor con elasticidad grande y rango estrecho puede resultar irrelevante, y uno con elasticidad moderada y rango amplio puede dominar. El [Ejemplo 3.7](#) exhibe ese contraste.

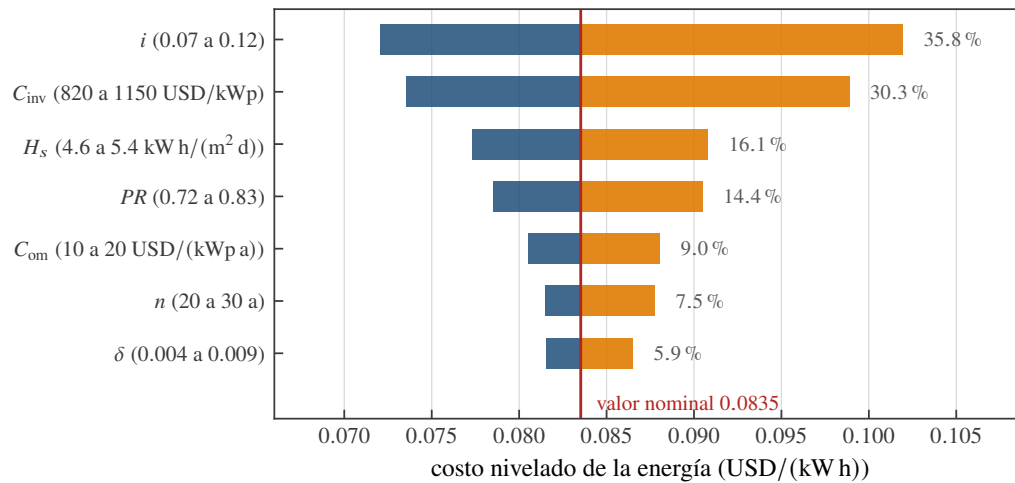


Figura 3.13. Diagrama de tornado del costo nivelado de la energía de un sistema fotovoltaico, con el rango plausible de cada factor y la amplitud relativa que induce sobre la respuesta.

Ejemplo 3.7. Costo nivelado de la energía de un sistema fotovoltaico. Un sistema fotovoltaico de 250 kW nominales se instala en el Caribe colombiano, con una irradiación diaria de referencia de 5.0 kW h/(m² d), razón de desempeño de 0.78, inversión de 950 USD/kW, operación y mantenimiento de 14 USD/(kW a), tasa de descuento de 0.09, vida útil de 25 a y degradación anual de 0.006. Se pide el costo nivelado de la energía, la jerarquía de los factores por elasticidad y por amplitud, y la discusión de la diferencia entre ambas.

Consideraciones. 1. Producción anual constante salvo por la degradación y costos de operación uniformes; 2. sin valor residual ni costos de desmantelamiento; 3. sin restricciones de despacho ni recortes de generación.

Formulación. El costo nivelado se obtiene de la Ecuación (3.13),

$$LCOE = \frac{C_{inv} CRF + C_{om}}{H_s \cdot 365 \cdot PR \cdot f_d}, \quad CRF = \frac{i(1+i)^n}{(1+i)^n - 1}, \quad f_d = \frac{1 - (1-\delta)^n}{n\delta}, \quad (3.13)$$

donde C_{inv} es la inversión específica (USD/kW); C_{om} es el costo anual de operación por unidad de potencia (USD/(kW a)); CRF es el factor de recuperación del capital (1/a); H_s es la irradiación diaria de referencia (kW h/(m² d)); PR es la razón de desempeño (adimensional); f_d es el factor medio de degradación (adimensional); i es la tasa de descuento (adimensional); n es la vida útil (a); y δ es la tasa anual de degradación (adimensional).

Solución. El factor de recuperación vale 0.101806/a y el de degradación medio 0.931205, de modo que la producción específica alcanza 1325.6 kW h/(kW a) y el costo nivelado 0.083523 USD/kWh. Las elasticidades resultan -1.0000 para la irradiación y la razón de desempeño, 0.8736 para la inversión, 0.6370 para la tasa, -0.1735 para la vida útil, 0.1264 para la operación y 0.0705 para la degradación.

Verificación. La irradiación y la razón de desempeño aparecen como factores del denominador, de modo que sus elasticidades deben valer exactamente menos uno, y así resulta. Las elasticidades de la inversión y de la operación deben sumar la unidad porque ambos costos entran de manera aditiva en el numerador, y su suma numérica vale 1.000000. El valor analítico de la elasticidad de la inversión, $C_{\text{inv}}\text{CRF}/(C_{\text{inv}}\text{CRF} + C_{\text{om}})$, es 0.873550 y coincide con el estimado por diferencias centradas.

Comentario. El tornado de la [Figura 3.13](#) invierte el orden que sugieren las elasticidades. La tasa de descuento, con elasticidad de 0.637, encabeza la lista con una amplitud del 35.8 % porque su rango abarca de 0.07 a 0.12, mientras que la irradiación, con elasticidad de -1 , solo produce un 16.1 %. La decisión financiera pesa más que la técnica, conclusión que ninguna elasticidad por sí sola revela.

3.6.2. Tamizado y sensibilidad global

Los métodos anteriores comparten una limitación de fondo, pues al mover un factor a la vez recorren solo los ejes que pasan por el punto nominal y no detectan interacciones ni efectos que cambian con la posición. Cuando el modelo es no lineal y los factores son muchos, el remedio explora el espacio completo, y la primera opción económica es el tamizado propuesto por Morris [69].

El procedimiento construye trayectorias sobre una malla del hipercubo unitario, en las cuales cada factor se perturba una sola vez, y promedia los valores absolutos de los cocientes incrementales. El promedio μ^* mide la influencia global del factor y la desviación σ mide cuánto varía esa influencia según la región, de modo que un σ grande delata no linealidad o interacción. El costo es de $r(k + 1)$ evaluaciones con r trayectorias y k factores, modesto frente al análisis basado en varianza. El [Algoritmo 3.4](#) lo recoge.

El tamizado jerarquiza pero no cuantifica. Para atribuir a cada factor una fracción de la varianza de la respuesta se recurre a la descomposición funcional de la varianza, cuya versión operativa se debe a Sobol' [70].

Algoritmo 3.4 Tamizado de Morris por efectos elementales sobre trayectorias.

Entradas: Modelo g definido sobre el hipercubo unitario, número de factores k , número de trayectorias r , número de niveles p par

Salidas: Medidas μ_j^* y σ_j para cada factor

- 1: $\Delta \leftarrow p/[2(p-1)]$
 - 2: **para** $t = 1$ **hasta** r **hacer**
 - 3: elegir al azar el punto de partida $\mathbf{x}$ sobre la malla de p niveles y el orden de recorrido de los k factores
 - 4: $y_0 \leftarrow g(\mathbf{x})$
 - 5: **para** cada factor j en el orden elegido **hacer**
 - 6: perturbar x_j en $+\Delta$ o en $-\Delta$ sin salir del hipercubo; $y \leftarrow g(\mathbf{x})$
 - 7: $d_{t,j} \leftarrow (y - y_0)/(\pm\Delta)$; $y_0 \leftarrow y$
 - 8: **fin para**
 - 9: **fin para**
 - 10: $\mu_j^* \leftarrow$ promedio de $|d_{t,j}|$ sobre t ; $\sigma_j \leftarrow$ desviación de $d_{t,j}$ sobre t
 - 11: **devolver** $\{\mu_j^*\}$ y $\{\sigma_j\}$
-

Teorema 3.4. Descomposición de la varianza. Sea $Y = g(X_1, \dots, X_k)$ con X_j independientes y $\text{Var}[Y] < \infty$. Entonces la varianza admite la descomposición única

$$\text{Var}[Y] = \sum_j V_j + \sum_{j < l} V_{jl} + \dots + V_{1\dots k}, \quad V_j = \text{Var}_{X_j}(\mathbb{E}[Y | X_j]),$$

en la cual cada término mide la contribución del subconjunto de factores correspondiente y todos los términos son no negativos.

De la descomposición se derivan dos índices que resumen la información en dos números por factor y que constituyen hoy el patrón de referencia del análisis de sensibilidad global.

Definición 3.8. Índices de sensibilidad de Sobol'. El índice de primer orden $S_j = V_j/\text{Var}[Y]$ es la fracción de la varianza de la respuesta que se eliminaría si el factor X_j quedara fijo en su valor verdadero. El índice total S_{T_j} es la fracción que permanecería si todos los demás factores quedaran fijos, de modo que recoge el efecto individual del factor más todas sus interacciones. Siempre se cumple $S_j \leq S_{T_j}$ y $\sum_j S_j \leq 1 \leq \sum_j S_{T_j}$.

La diferencia $S_{T_j} - S_j$ cuantifica cuánto del efecto del factor pasa por su interacción con los demás, y $1 - \sum_j S_j$ mide el peso total de las interacciones. Un modelo con esa suma próxima a uno es aditivo en la práctica. Los índices se estiman con los estimadores de Saltelli para el primer orden y de Jansen para el total, que reutilizan dos matrices de muestreo y sus versiones cruzadas, con un costo de $(k + 2)N$ evaluaciones [71]. El [Listado 3.11](#) lo implementa.

Listado 3.11. Estimación de los índices de Sobol' de primer orden y totales por muestreo cruzado.

```

1 import numpy as np
2 from scipy.stats import qmc
3 def indices_sobol(modelo, d, m, semilla):
4     """Índices de primer orden y totales con los estimadores de
5     Saltelli y de Jansen sobre una secuencia de Sobol'."""
6     Z = qmc.Sobol(d=2*d, scramble=True, seed=semilla).random(2*m)
7     A, B = Z[:, :d], Z[:, d:]
8     yA, yB = modelo(A), modelo(B)
9     V = np.var(np.concatenate([yA, yB]), ddof=1)
10    S, ST = np.empty(d), np.empty(d)
11    for j in range(d):
12        AB = A.copy()
13        AB[:, j] = B[:, j]
14        yAB = modelo(AB)
15        S[j] = np.mean(yB*(yAB - yA))/V
16        ST[j] = np.mean((yA - yAB)**2)/(2*V)
17    return S, ST

```

Una implementación así debe verificarse antes de aplicarla, porque un error de índice o de signo produce resultados plausibles. La verificación estándar emplea la función de prueba de Ishigami, cuyos índices se conocen en forma cerrada. Con $N = 2^{14}$ y la semilla del curso se obtienen $S_1 = 0.3138$, $S_2 = 0.4423$ y $S_3 = -0.0001$, frente a 0.3139, 0.4424 y 0, y $S_{T_3} = 0.2436$ frente a 0.2437.

Aplicada al biodigestor del [Ejemplo 3.6](#) con $N = 2^{14}$, es decir 131 072 evaluaciones, la estimación entrega índices de primer orden de 0.107, 0.207, 0.616, 0.005, 0.001 y 0.050 para la masa de estiércol, la fracción de sólidos volátiles, el potencial metanogénico, la constante de hidrólisis, el tiempo de retención y la eficiencia eléctrica, con totales de 0.112, 0.216, 0.629, 0.005, 0.001 y 0.052. La suma de los primeros vale 0.985, de modo que las interacciones explican el 1.5 % de la varianza.

El tamizado de Morris sobre el mismo modelo, con treinta trayectorias de ocho niveles y solo 210 evaluaciones, entrega la jerarquía B_0 , x_{SV} , m , η , k y TRH , con valores de μ^* de 58.24 kW h/d, 42.55 kW h/d, 33.65 kW h/d, 22.40 kW h/d, 3.93 kW h/d y 2.56 kW h/d. Es el mismo orden que produce el análisis de Sobol' a un costo seiscientas veces menor, contraste que ilustra el papel de cada método, pues el tamizado descarta y jerarquiza mientras que el basado en varianza cuantifica.

La elección entre los tres enfoques se resuelve con dos preguntas, cuántos factores hay y cuánto cuesta una evaluación. La [Tabla 3.4](#) organiza la respuesta con el costo, el supuesto y la limitación de cada familia. La estrategia sensata los encadena, tamizando primero para descartar y cuantificando después sobre los que sobrevivieron.

Tabla 3.4. Comparación de los tres enfoques de análisis de sensibilidad.

Enfoque	Costo ^a	Qué supone	Qué no detecta
Elasticidad local	$2k$	modelo suave en el punto	no linealidad e interacción
Un factor a la vez	$2k$ a $10k$	efectos separables	interacción entre factores
Tamizado de Morris	$r(k + 1)$	nada sobre la forma	el reparto de la varianza
Índices de Sobol'	$(k + 2)N$	factores independientes	nada relevante

^a Evaluaciones del modelo, con k el número de factores, r el número de trayectorias de Morris, con r entre 10 y 50, y N el tamaño de la muestra base de Sobol', con N del orden de 1000 a 10 000.

La [Figura 3.14](#) sitúa esa comparación en el plano definido por el número de factores inciertos y el costo de una evaluación del modelo, sobre el cual cada familia ocupa una región. La elasticidad y el tornado sirven cuando los factores son pocos y el modelo es barato, el tamizado de Morris se extiende a decenas de factores con un costo lineal, y los índices de Sobol' exigen un modelo que se evalúe en fracciones de segundo o un presupuesto de cómputo que rara vez existe en un proyecto de ingeniería.

Queda por decidir qué se hace con el resultado, que es la parte con consecuencias reales. Los factores dominantes exigen inversión en medición, en calibración o en control. Los irrelevantes se fijan en su valor nominal y se declara esa decisión, lo cual simplifica el modelo y reduce la recolección de datos. Los intermedios se reportan tal como están. En el biodigestor la recomendación se resume en medir el potencial metanogénico con un ensayo específico, pues explica el 62 % de la varianza, y fijar el tiempo de retención y la constante de hidrólisis.

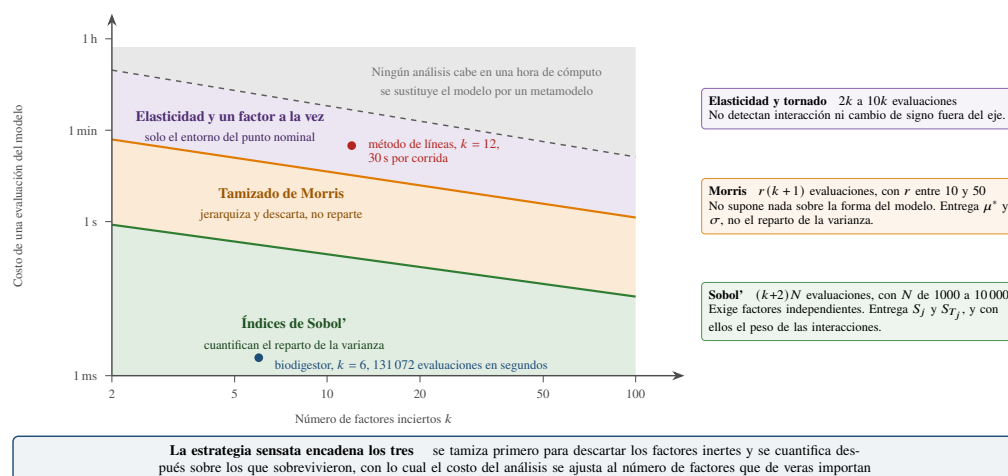


Figura 3.14. Elección del enfoque de sensibilidad según el número de factores inciertos y el costo de una evaluación del modelo, con el costo, el supuesto y el punto ciego de cada familia.

3.7. Discusión de resultados en función del problema de estudio

Todo lo anterior produce cifras, y ninguna cifra es todavía una respuesta. Una tabla de veintisiete corridas, una distribución con sus percentiles y una jerarquía de factores son resultados del modelo, mientras que la pregunta de ingeniería que abrió el capítulo se refería a una decisión sobre el sistema, y el paso que separa lo uno de lo otro es la discusión de resultados. Esa etapa no consiste en describir lo que las figuras muestran, tarea que el lector puede hacer solo, sino en establecer qué significa cada resultado para la decisión en curso, con qué confianza se afirma y qué queda fuera del alcance de lo calculado.

La discusión se organiza alrededor de cinco preguntas que conviene responder en orden. La primera indaga qué alternativas quedaron descartadas y por qué, distinguiendo las que fueron dominadas en todos los contextos de las que solo lo fueron en alguno. La segunda pregunta cuál es el precio de cada mejora sobre la frontera de compromiso, expresado en las unidades de los indicadores y no en un índice ponderado. La tercera examina la robustez, es decir, si el orden de preferencia se conserva al cambiar el contexto o al mover los factores dentro de su rango plausible. La cuarta declara qué información faltante impide cerrar la decisión, y la quinta enumera lo que el modelo no representa y que podría alterar la conclusión. La [Tabla 3.5](#) recoge esas preguntas con la evidencia que las sostiene.

Tabla 3.5. Elementos de la discusión de resultados de una simulación, pregunta que responde cada uno y evidencia que lo sostiene.

Elemento	Pregunta que responde	Evidencia que lo sostiene
Descarte	Qué alternativas no merecen discusión	Regla de dominancia aplicada dentro de cada contexto
Precio de la mejora	Cuánto cuesta pasar de una alternativa a la vecina	Diferencias de indicadores sobre la frontera de compromiso
Robustez	Si la recomendación sobrevive al cambio de contexto	Comparación de fronteras entre contextos y rango de los factores
Confianza	Con qué margen se afirma cada cifra	Percentiles, error estándar, tamaño de muestra y semilla
Información faltante	Qué habría que medir o conocer para decidir	Jerarquía del análisis de sensibilidad
Límites del modelo	Qué proceso omitido podría cambiar la conclusión	Registro de supuestos y dominio de validez

^a Los seis elementos se redactan en función del problema de estudio y no del modelo, de modo que el lector encuentre la decisión y no la descripción de las corridas.

El caso del riego del [Ejemplo 3.5](#) ilustra la diferencia entre describir y discutir. Describir es informar que cinco de nueve alternativas resultaron no dominadas en el clima normal. Discutir es afirmar que reducir el déficit del 21.5 % al 5.1 % cuesta 100 mm brutos adicionales y eliminarlo cuesta otros 50 mm, que el umbral 0.65 con lámina de 65 mm mantiene el déficit por debajo del 2.1 % en los tres climas con cinco riegos o menos, pero que en el clima normal esa misma alternativa resulta dominada por el umbral 0.45 con lámina de 45 mm, que aplica 12 mm menos sin déficit alguno. La recomendación robusta y la recomendación óptima en un contexto no coinciden, y decir cuál se prefiere exige el precio del agua y el valor del rendimiento perdido, que el modelo hidrológico no contiene. Solo esa segunda versión permite que el propietario del lote decida.

El caso del biodigestor del [Ejemplo 3.6](#) muestra la segunda cara de la discusión, que es la lectura de la confianza. La conclusión operativa no es que el sistema produzca 92 kW h/d, sino que cubre la demanda supuesta en menos de una quinta parte de los días, y que la magnitud de diseño es el percentil cinco y no la media. El análisis de sensibilidad completa la lectura al señalar que el potencial metanogénico explica el 62 % de la varianza, de modo que la información faltante tiene nombre y un ensayo de laboratorio concreto puede suministrarla. Una discusión que omitiera ese vínculo entre la dispersión y su causa dejaría al lector con una distribución y sin una acción.

Queda la obligación que da sentido a la etapa, que es declarar lo que el modelo no representa. El modelo de riego supone lluvia nula y percolación instantánea, el del biodigestor supone régimen permanente y entradas independientes, y cada uno de esos supuestos, tomado del registro de la [Sección 3.4](#), define una condición bajo la cual la conclusión podría cambiar. Los mandamientos negativos de la modelación recogidos en el [Capítulo 1](#) se aplican aquí con toda su fuerza, pues distorsionar la lectura de las corridas para que confirmen la alternativa preferida o conservar una recomendación que el cambio de contexto ya desacreditó son las dos maneras más frecuentes de convertir una simulación correcta en una decisión equivocada [1].

La [Figura 3.15](#) cierra la unidad con la cadena de verificaciones que toda cifra reportada debe haber superado antes de entrar en la discusión, con la comprobación obligatoria de cada bloque, la cifra del caso trabajado y la condición bajo la cual el método falla. Ningún resultado de esta unidad se entrega solo, y esa exigencia es la que distingue una simulación de una conjetura con gráficas.

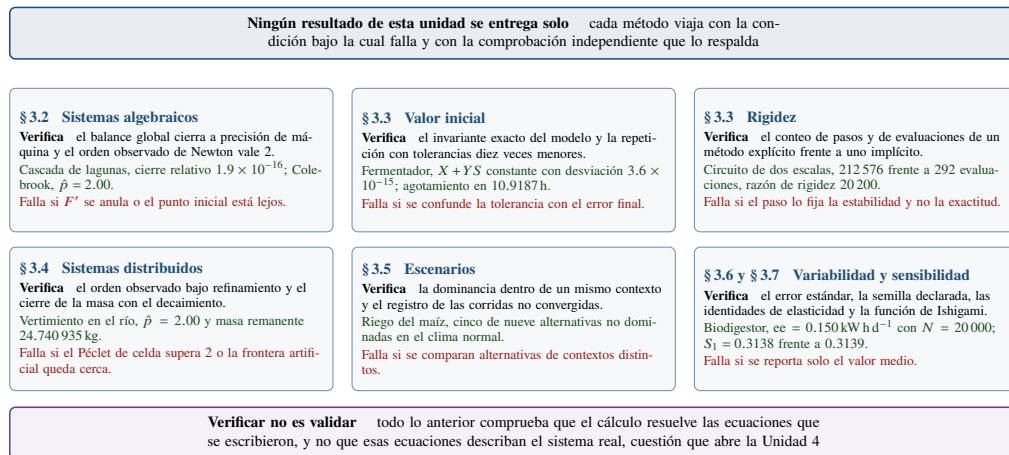


Figura 3.15. Cadena de verificación de la unidad, con la comprobación obligatoria de cada bloque, la cifra del caso trabajado y la condición bajo la cual el método falla.

3.8. Problemas

Los problemas se ordenan en dificultad creciente y cubren los cinco campos de la Facultad. Los tres primeros grupos se resuelven con lápiz, los grupos cuarto y quinto exigen un cuaderno de trabajo y los dos últimos piden implementar y argumentar. Salvo indicación contraria, la semilla es 20 262.

3.8.1. Apropriación conceptual

- 3-1 Explique por qué un modelo estacionario conduce a un sistema algebraico y uno dinámico a un problema de valor inicial, y qué término del balance desaparece en el primero.
- 3-2 Un colega afirma que su simulación es correcta porque el residuo final vale 1×10^{-8} . Discuta qué falta para aceptarlo y el papel del escalamiento de los residuos.
- 3-3 Enuncie la diferencia entre error local de truncamiento y error global, e indique cuál controla la tolerancia de `solve_ivp`.
- 3-4 Justifique por qué un método A-estable no restringe el paso en un problema rígido y qué costo se paga a cambio.
- 3-5 Distinga escenario y corrida con un ejemplo de su área, e identifique dos factores de decisión y dos de contexto.

3.8.2. Simulación estacionaria

- 3-6 Una tubería de PVC de 0.20 m de diámetro y rugosidad 0.0015 mm conduce 45 L/s de agua a 25 °C, con viscosidad cinemática $0.893 \times 10^{-6} \text{ m}^2/\text{s}$. Calcule el número de Reynolds, resuelva Colebrook–White con Newton desde $f_0 = 0.02$ y reporte las iteraciones hasta una tolerancia de 1×10^{-10} .
- 3-7 Un tanque de mezcla recibe salmueras de 12 g/L y 35 g/L con caudales de 4 m³/h y 7 m³/h. Plantee el residuo del balance de sal, resuélvalo y verifique el cierre global de masa.
- 3-8 Para la cascada de la [Ecuación 3.4](#), dibuje el patrón de dispersión del jacobiano al añadir una recirculación de la quinta laguna a la primera y cuente los elementos no nulos.
- 3-9 Demuestre que con una raíz doble Newton converge de manera lineal con razón 0.5, y proponga una modificación que restablezca el orden dos.

3.8.3. Simulación dinámica y rigidez

- 3-10 Un lote de 500 kg de leche a 35 °C se enfría contra un refrigerante a 1 °C, con $UA = 399 \text{ W/K}$ y calor específico de 3.93 kJ/(kg K). Plantee el problema de valor inicial, resuélvalo de forma analítica y halle el tiempo para llegar a 4 °C.
- 3-11 Resuelva el problema anterior con `solve_ivp` y tolerancias 1×10^{-3} , 1×10^{-6} y 1×10^{-9} , y compare el error relativo del tiempo frente al valor analítico. Discuta si el error respeta la tolerancia pedida.

- 3–12** El jacobiano de un modelo de dos estados tiene autovalores $-4500/\text{s}$ y $-0.02/\text{s}$. Calcule la razón de rigidez, el paso máximo admisible para Euler explícito y cuántos pasos exigiría simular 600 s.
- 3–13** Un fermentador con la cinética del [Ejemplo 3.2](#) incluye muerte celular de constante $0.02/\text{h}$. Verifique que el invariante $X + YS$ deja de conservarse y proponga otra relación de verificación.
- 3–14** Un controlador de dos posiciones enciende un compresor sobre 6°C y lo apaga en 3°C . Explique por qué integrarlo de forma directa degrada el desempeño del integrador y describa el tratamiento con eventos terminales.

3.8.4. Sistemas distribuidos

- 3–15** Deduzca la aproximación centrada de la segunda derivada con los desarrollos de Taylor en los nodos vecinos y obtenga el término principal del error de truncamiento.
- 3–16** Una pared de 0.25 m de espesor y difusividad $6.9 \times 10^{-7} \text{ m}^2/\text{s}$ se discretiza con 40 intervalos. Calcule el paso temporal máximo de un esquema explícito y los pasos para simular 12 h.
- 3–17** Para el río del [Ejemplo 3.4](#), halle el paso espacial máximo que mantiene el número de Péclet de celda bajo dos y qué artefacto aparece si se excede.
- 3–18** Un estudio de convergencia entrega errores de 4.0×10^{-4} , 1.0×10^{-4} , 2.6×10^{-5} y 2.1×10^{-5} para pasos de 40 m, 20 m, 10 m y 5 m. Estime el orden observado en cada refinamiento y diagnostique qué ocurre en el último.
- 3–19** Explique por qué el método de líneas produce un sistema más rígido al refinar la malla, y relaciónelo con el número de difusión.

3.8.5. Escenarios, Monte Carlo y sensibilidad

- 3–20** Una matriz de escenarios tiene cinco factores con tres niveles. Calcule las corridas del barrido completo y el tiempo total si cada una tarda 45 s, y proponga una reducción justificada.
- 3–21** Con la [Tabla 3.3](#), construya la frontera del clima seco e indique qué alternativas cambian de estatus frente al clima normal.
- 3–22** Una simulación con 2000 corridas entrega media de 318 kg/h y desviación de 47 kg/h. Calcule el error estándar de la media y las corridas necesarias para llevar la semiamplitud al 95 % hasta 1 kg/h.

- 3–23** Un factor tiene elasticidad -1.8 con rango del 3 % y otro elasticidad 0.4 con rango del 40 %. Determine cuál domina el tornado y por qué la elasticidad sola induce a error.

3.8.6. Problemas de programación

- 3–24** Implemente el Newton amortiguado del [Algoritmo 3.1](#) como función que reciba el residuo, el jacobiano y un predicado de dominio físico, y pruébela con la cascada de lagunas.
- 3–25** Implemente el método de líneas para la conducción transitoria en una pared con una cara a temperatura fija y la otra con convección, y verifique el orden contra la solución en serie de Fourier.
- 3–26** Programe un barrido factorial que registre las corridas no convergidas y produzca la frontera de alternativas no dominadas dentro de cada contexto, sobre un problema de su área con al menos tres factores.
- 3–27** Implemente el tamizado de Morris del [Algoritmo 3.4](#) y verifíquelo con la función de Ishigami, comprobando que el orden de μ^* coincide con el de los índices totales.
- 3–28** Escriba una función que compare RK45, BDF y LSODA sobre un mismo problema, devolviendo pasos, evaluaciones y error frente a una referencia, y úsela para diagnosticar rigidez.

3.8.7. Problemas de diseño y ensayo

- 3–29** Diseñe el estudio que permita recomendar el número de lagunas en serie de una planta para 18 000 habitantes, con volumen total fijo. Declare factores, niveles, indicadores y criterio de comparación, y sustente la recomendación.
- 3–30** Un fabricante de panela debe decidir la temperatura de evaporación. Formule el modelo dinámico mínimo, proponga escenarios, elija dos indicadores en conflicto y explique cómo presentaría la frontera al propietario, que no es ingeniero.
- 3–31** Escoja un modelo de su área con al menos seis parámetros inciertos, asígneles distribuciones sustentadas, ejecute la propagación y el análisis de sensibilidad global, y redacte la recomendación de medición que se deriva.
- 3–32** Argumente, con cifras plausibles, por qué reportar solo el valor medio de una simulación estocástica conduce a un dimensionamiento inseguro, y proponga el conjunto mínimo de cifras que debe acompañarlo.

Capítulo 4

Validación, Interpretación y Comunicación de Resultados

Un modelo que corre no es un modelo que sirve. Al terminar el [Capítulo 3](#) el lector dispone de un código que integra ecuaciones diferenciales, resuelve balances acoplados y recorre escenarios. Ese código produce números con una apariencia de autoridad que no depende de que sean correctos. La pregunta que abre este capítulo consiste en establecer cuánto de lo que el modelo predice merece crédito y en qué condiciones deja de merecerlo.

Responderla exige separar tres preguntas que la conversación cotidiana confunde. La primera indaga si el programa resuelve las ecuaciones que se escribieron. La segunda indaga si los parámetros se estimaron de manera defendible y con qué precisión quedaron determinados. La tercera, la más incómoda, indaga si el modelo reproduce el sistema real frente a mediciones que nadie usó para ajustarlo.

A ese trío se añade una cuarta obligación que domina la práctica profesional. El resultado de una simulación nunca es un número, es un intervalo con un origen declarado, y quien lo entrega debe decir de dónde viene su amplitud y qué parte se puede reducir midiendo mejor. La etapa final es la comunicación, pues un modelo verificado, calibrado y validado carece de valor si el informe que lo transporta no permite que otro ingeniero lo revise y lo reproduzca.

4.1. Verificación del código y de la solución numérica

La práctica profesional exige un orden estricto entre las dos actividades que sostienen la credibilidad de un modelo. Verificar es comprobar que el programa resuelve correctamente las ecuaciones que el modelador escribió; validar es comprobar que esas ecuaciones describen el sistema real con la exactitud que la decisión requiere [2]. Invertir el orden es la fuente más común de conclusiones falsas, pues un error de programación se compensa con un parámetro ajustado y el resultado parece excelente hasta que cambia la condición de operación.

Definición 4.1. Verificación y validación. La *verificación* es el proceso de determinar si una implementación computacional representa con fidelidad el modelo matemático que se pretendía resolver, y si el error de la solución numérica está acotado. La *validación* es el proceso de determinar hasta qué punto ese modelo matemático representa el sistema real desde la perspectiva de los usos previstos.

La definición encierra una particularidad decisiva. La verificación es un problema de matemáticas y de programación que se resuelve dentro del computador, mientras que la validación es un problema de ingeniería que sin datos independientes no se puede siquiera plantear [72]. Un modelo puede estar perfectamente verificado y ser completamente inútil. La Figura 4.1 ordena las cuatro preguntas que este capítulo responde, con los datos que alimenta cada etapa y la evidencia que produce, y recuerda que cuando la validación falla se revisa la estructura del modelo y no se retocan los parámetros, que es la lección que el texto guía extrae de su propio caso de un modelo que supera la calibración y fracasa en la validación [1].

Dentro de la verificación conviene distinguir dos niveles. La del código busca defectos de programación y se apoya en casos con solución conocida; la de la solución cuantifica el error de discretización de una corrida concreta sobre una malla concreta [22]. La primera se hace una vez y se automatiza, la segunda se repite en cada aplicación. El Figura 4.2 sitúa ambas dentro del ciclo completo.

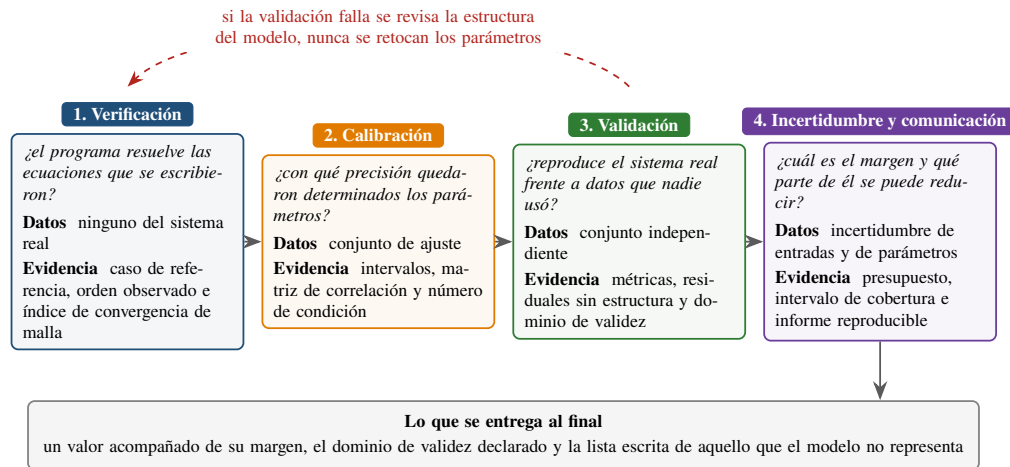


Figura 4.1. Las cuatro preguntas que separan un modelo que corre de un modelo que sirve, con los datos que alimenta cada etapa y la evidencia que produce.

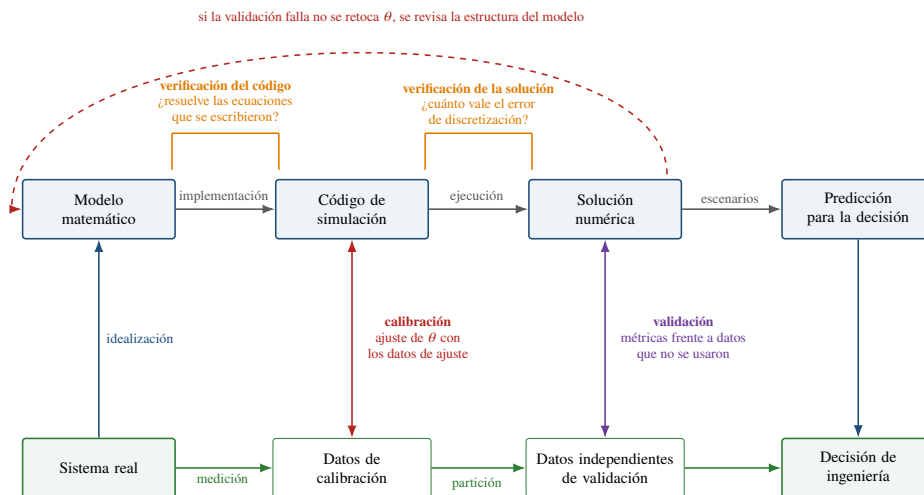


Figura 4.2. Marco de verificación, calibración y validación, con los datos que alimenta cada actividad y el lazo de revisión que se activa cuando la validación falla.

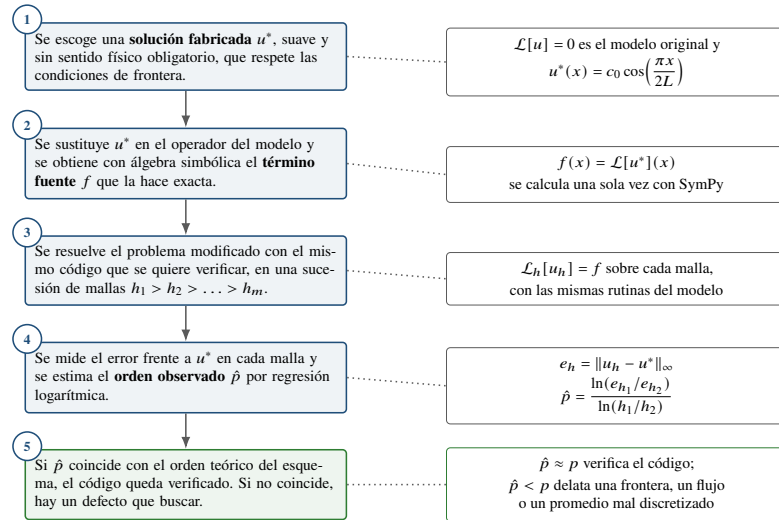
Dos instrumentos cubren la verificación del código. El primero es la solución analítica de referencia, disponible bajo hipótesis restrictivas en la aleta de sección constante o en la respuesta de primer orden a un escalón [19], y siempre con norma de error explícita y tolerancia declarada. El segundo son las comprobaciones de conservación y los casos límite, que no exigen solución cerrada y detectan el signo equivocado de un término fuente o el índice desplazado en el ensamblaje de una matriz.

4.1.1. Método de las soluciones fabricadas

La cobertura de esos dos instrumentos es limitada, pues solo ejercitan las rutas que el caso simplificado recorre. Cuando el modelo tiene coeficientes variables, términos fuente y fronteras mixtas, ninguna solución analítica lo cubre por completo. La técnica que resuelve la dificultad invierte el planteamiento, pues escoge primero la solución y después construye la ecuación que la tiene por respuesta exacta.

Definición 4.2. Solución fabricada. Una *solución fabricada* es una función suave escogida por conveniencia, sin exigencia de significado físico, que se sustituye en el operador del modelo para generar el término fuente que la convierte en solución exacta del problema modificado.

El procedimiento produce un caso de referencia exacto para cualquier ecuación, incluso para aquellas cuya versión física no admite solución cerrada. Basta que la función escogida sea suficientemente derivable, que active todos los términos del operador y que el esquema no la integre de manera exacta, como ocurriría con un polinomio de grado bajo. El Figura 4.3 resume los cinco pasos.



La ventaja del procedimiento es que produce un caso de referencia exacto para cualquier ecuación, incluso cuando el problema físico de interés no tiene solución analítica conocida.

Figura 4.3. Método de las soluciones fabricadas, con la construcción del término fuente por álgebra simbólica y la estimación del orden observado como criterio de aceptación.

El paso que suele ahuyentar a quien lo intenta por primera vez es el cálculo del término fuente, que para un operador con coeficientes variables produce expresiones largas y propensas al error. El álgebra simbólica elimina esa dificultad y genera además la función evaluable [18]. El [Listado 4.1](#) lo construye para una ecuación de difusión con reacción y coeficiente variable.

Listado 4.1. Construcción del término fuente de una solución fabricada con SymPy.

```

1 import sympy as sp
2
3 x, L, D0, k, c0 = sp.symbols("x L D_0 k c_0", positive=True)
4 D = D0 * (1 + x / L)
5 c_fab = c0 * sp.cos(sp.pi * x / (2 * L))
6 flujo = D * sp.diff(c_fab, x)
7 fuente = sp.simplify(-sp.diff(flujo, x) + k * c_fab)
8 f_num = sp.lambdify((x, L, D0, k, c0), fuente, "numpy")
9
10 valores = (0.40, 5.0e-10, 1.0e-6, 50.0)
11 f_centro = f_num(0.20, *valores)

```

4.1.2. Estudio de convergencia y orden observado

Contar con un caso de referencia exacto permite plantear la pregunta central de la verificación de la solución, que consiste en establecer cómo se comporta el error cuando la malla se refina. Para un esquema consistente y estable el error admite un desarrollo asintótico dominado por una potencia del paso, según la Ecuación (4.1).

$$e(h) = \|u_h - u\| = C h^p + O(h^{p+1}), \quad (4.1)$$

donde h es el paso característico de la malla o del tiempo (m o s); u_h es la solución numérica con ese paso; u es la solución exacta de referencia; C es una constante independiente de h ; y p es el orden formal del esquema.

Definición 4.3. Orden observado. El *orden observado* de un esquema numérico es la pendiente de la recta que ajusta el logaritmo del error frente al logaritmo del paso, calculada sobre una sucesión de mallas suficientemente finas para que el término dominante del desarrollo asintótico gobierne el error.

La utilidad del orden observado no está en confirmar lo que ya se sabía del esquema, sino en delatar lo que se hizo mal. Un esquema de segundo orden cuyo estudio de convergencia devuelve un orden cercano a uno tiene casi con seguridad una frontera o un promedio discretizado con precisión inferior a la del interior, defecto que ninguna otra prueba manifiesta.

Teorema 4.1. Consistencia del orden observado. Sea una sucesión de pasos $h_1 > h_2 > \dots > h_m$ con razón de refinamiento constante $r = h_i/h_{i+1}$, y sea el error de la forma $e(h) = Ch^p + O(h^{p+1})$ con $C \neq 0$. Entonces la estimación por pares consecutivos $\hat{p}_i = \ln(e_i/e_{i+1})/\ln r$ satisface $\hat{p}_i \rightarrow p$ cuando $h_i \rightarrow 0$. Si además se desconoce la solución exacta pero se dispone de tres mallas, la estimación $\hat{p} = \ln|(u_1 - u_2)/(u_2 - u_3)|/\ln r$ converge al mismo límite.

La lectura práctica tiene dos partes. La primera indica que basta refinar hasta que dos estimaciones consecutivas coincidan en dos cifras para declarar alcanzado el régimen asintótico. La segunda permite estimar el orden sin conocer la solución exacta, con tres corridas de la aplicación real, y en ese caso conviene usar una magnitud integral en lugar de un valor puntual. El [Algoritmo 4.1](#) organiza el procedimiento.

Algoritmo 4.1 Estudio de convergencia y estimación del orden observado.

Entradas: Modelo M , referencia u^* o magnitud integral F , pasos $h_1 > \dots > h_m$ con razón r

Salidas: Orden observado $\hat{p}$ y diagnóstico de régimen asintótico

- 1: **para** $i = 1$ **hasta** m **hacer**
 - 2: $u_i \leftarrow$ resolver M con paso h_i y las rutinas de la aplicación
 - 3: $e_i \leftarrow \|u_i - u^*\|_\infty$ si se dispone de u^*
 - 4: **fin para**
 - 5: $\hat{p}_i \leftarrow \ln(e_i/e_{i+1})/\ln r$ para $i = 1, \dots, m-1$
 - 6: $\hat{p} \leftarrow$ pendiente de la regresión de $\ln e_i$ sobre $\ln h_i$
 - 7: **si** $|\hat{p}_{m-1} - \hat{p}_{m-2}| > 0.05$ **entonces**
 - 8: informar que no se alcanzó el régimen asintótico y refinar más
 - 9: **fin si**
 - 10: **devolver** $\hat{p}$, la sucesión $\hat{p}_i$ y el diagnóstico
-

El cálculo cabe en pocas líneas y conviene tenerlo como función reutilizable. El [Listado 4.2](#) devuelve las estimaciones por pares y la pendiente global, informaciones complementarias porque la primera revela si el régimen asintótico se alcanzó y la segunda promedia el ruido.

Listado 4.2. Orden observado por pares consecutivos y por regresión logarítmica.

```

1 import numpy as np
2
3 def orden_observado(h, e):
4     """Orden por pares y por regresión logarítmica."""
5     h, e = np.asarray(h, float), np.asarray(e, float)
6     p_par = np.log(e[:-1] / e[1:]) / np.log(h[:-1] / h[1:])
7     p_reg = np.polyfit(np.log(h), np.log(e), 1)[0]

```

```

8     return p_par, p_reg
9
10    h = np.array([6.0, 3.0, 1.5, 0.75, 0.375]) * 1e-3
11    e = np.array([2.1964e-2, 5.5043e-3, 1.3769e-3,
12                  3.4427e-4, 8.6072e-5])
13    p_par, p_reg = orden_observado(h, e)

```

El siguiente ejemplo aplica el procedimiento a un caso con solución analítica conocida y muestra, por contraste, el efecto de un único descuido en el tratamiento de la frontera.

Ejemplo 4.1. Orden observado en una aleta de disipador electrónico. Una aleta rectangular de aluminio de un disipador tiene espesor 1.0 mm, longitud 30 mm y conductividad 205 W/(m K). El coeficiente de convección forzada vale 80 W/(m² K) y la base se mantiene 45 K por encima del aire. Se pide verificar un código de diferencias finitas que resuelve el perfil de temperatura, con extremo adiabático.

Consideraciones. 1. Régimen estacionario y unidimensional a lo largo de la aleta; 2. propiedades constantes y coeficiente de convección uniforme; 3. extremo libre adiabático, de modo que el flujo por la punta se desprecia.

Análisis. El balance de energía sobre un elemento diferencial conduce a la ecuación de la aleta en términos del exceso de temperatura $\theta = T - T_\infty$, según la Ecuación (4.2).

$$\frac{d^2\theta}{dx^2} - m^2\theta = 0, \quad m^2 = \frac{2h_c}{k t}, \quad (4.2)$$

donde θ es el exceso de temperatura sobre el aire (K); x es la coordenada desde la base (m); h_c es el coeficiente de convección (W/(m² K)); k es la conductividad (W/(m K)); y t es el espesor (m). Con los datos $m = 27.94/\text{m}$ y $mL = 0.838$, y la solución exacta es $\theta(x) = \theta_b \cosh[m(L - x)]/\cosh(mL)$.

Solución. Se resuelve el sistema tridiagonal con diferencias centradas en el interior y se trata el extremo adiabático de dos maneras, con nodo fantasma, que conserva el segundo orden, y con una diferencia hacia atrás de primer orden. Los errores aparecen en la [Tabla 4.1](#). Con el nodo fantasma el orden observado converge a 2.000 y la regresión global entrega 1.999; con la diferencia hacia atrás cae a 1.003, y el error con 160 intervalos resulta 2740 veces mayor.

Verificación. Como magnitud integral se toma el calor disipado por unidad de ancho, de valor exacto 176.4898 W/m. Las mallas de 10, 20 y 40 intervalos entregan 176.6111 W/m, 176.5201 W/m y 176.4974 W/m, con lo cual la estimación de tres mallas devuelve $\hat{p} = 2.003$.

Comentario. Ambos códigos convergen, ambos producen perfiles de aspecto razonable y ninguno levanta una excepción. Sin embargo, para igualar el error que el nodo fantasma alcanza con 20 intervalos, la diferencia hacia atrás necesitaría alrededor de 6400, es decir unas 320 veces más nodos. Sin el estudio de convergencia el defecto pasa inadvertido y se paga en tiempo de cómputo durante toda la vida del modelo.

Tabla 4.1. Estudio de convergencia de la aleta con dos tratamientos del extremo adiabático.

N	h (mm)	Nodo fantasma		Diferencia hacia atrás	
		e_{∞} (K)	$\hat{p}$	e_{∞} (K)	$\hat{p}$
5	6.000	2.196×10^{-2}	—	2.011	—
10	3.000	5.504×10^{-3}	1.997	9.735×10^{-1}	1.047
20	1.500	1.377×10^{-3}	1.999	4.787×10^{-1}	1.024
40	0.750	3.443×10^{-4}	2.000	2.373×10^{-1}	1.012
80	0.375	8.607×10^{-5}	2.000	1.181×10^{-1}	1.006
160	0.188	2.152×10^{-5}	2.000	5.894×10^{-2}	1.003

^a N es el número de intervalos y e_{∞} el error en norma del máximo frente a la solución analítica.

^b El orden observado se calcula por pares consecutivos con razón de refinamiento $r = 2$.

El comportamiento de ambas familias se aprecia mejor en escala logarítmica, donde la relación de potencia se convierte en una recta cuya pendiente es el orden. La [Figura 4.4](#) presenta el error frente al paso y la sucesión de órdenes observados.

4.1.3. Extrapolación de Richardson e índice de convergencia de malla

Establecido el régimen asintótico, la misma estructura del error permite estimar el error de la malla más fina eliminando el término dominante con dos soluciones de pasos distintos, según la Ecuación (4.3).

$$u_{\text{ext}} = u_{h_2} + \frac{u_{h_2} - u_{h_1}}{r^{\hat{p}} - 1}, \quad (4.3)$$

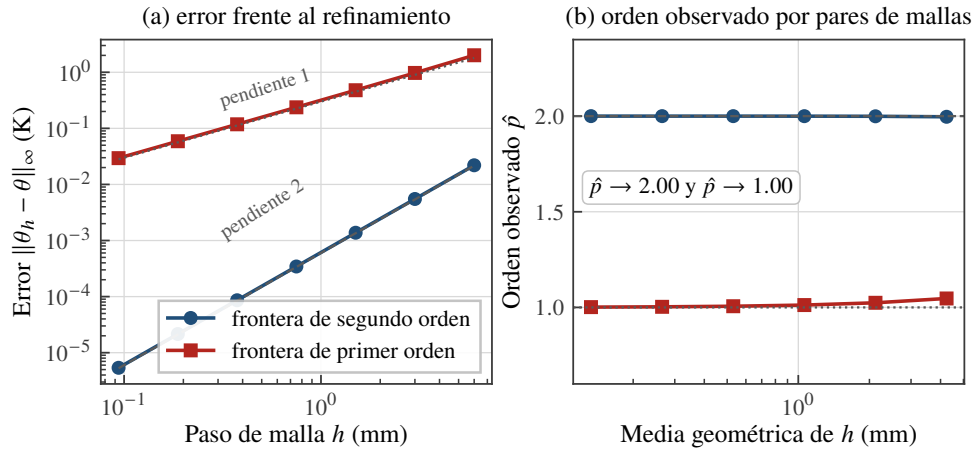


Figura 4.4. Estudio de convergencia de la aleta, con el error en escala logarítmica y la sucesión de órdenes observados para los dos tratamientos del extremo adiabático.

donde u_{ext} es el valor extrapolado, que aproxima la solución con paso nulo; u_{h_1} y u_{h_2} son las soluciones en la malla gruesa y en la fina; r es la razón de refinamiento; y $\hat{p}$ es el orden observado. Su diferencia con la solución fina estima el error de discretización de esta última.

Usar esa diferencia como estimación del error resulta optimista, porque el desarrollo se trunca y porque el orden observado se estima con ruido. La convención más difundida introduce un factor de seguridad y reporta el *índice de convergencia de malla*, estimación conservadora del error de discretización relativo de la solución fina que se interpreta como banda de incertidumbre numérica [22].

El índice se reporta como porcentaje y acompaña a todo resultado en el que la discretización sea fuente relevante de error. La Ecuación (4.4) recoge su definición operativa, con el factor de seguridad de 1.25 que corresponde al caso de tres mallas con orden observado.

$$\text{GCI} = \frac{F_s}{r^{\hat{p}} - 1} \left| \frac{u_{h_2} - u_{h_1}}{u_{h_2}} \right|, \quad (4.4)$$

donde GCI es el índice de convergencia de malla; F_s es el factor de seguridad; y las demás variables conservan el significado de la Ecuación (4.3). El Listado 4.3 las aplica a los tres valores del calor disipado del Ejemplo 4.1.

Listado 4.3. Extrapolación de Richardson e índice de convergencia de malla.

```
1 import numpy as np
2
3 def richardson(f_1, f_2, f_3, r=2.0, fs=1.25):
4     """f_1 en la malla gruesa y f_3 en la fina, razón r."""
```

```

5     p = np.log(abs((f_1 - f_2) / (f_2 - f_3))) / np.log(r)
6     f_ext = f_3 + (f_3 - f_2) / (r**p - 1.0)
7     gci = fs * abs((f_3 - f_2) / f_3) / (r**p - 1.0)
8     return p, f_ext, gci
9
10    p, q_ext, gci = richardson(176.6111, 176.5201, 176.4974)

```

El listado devuelve $\hat{p} = 2.003$, un valor extrapolado de 176.4899 W/m y un índice de 0.0053 %, frente a un error verdadero de la malla fina de 0.0043 %. El índice cumple así su función de cota conservadora, y mide únicamente el error de discretización.

El siguiente ejemplo lleva el procedimiento a un caso sin solución analítica, que es donde la técnica de la solución fabricada muestra su verdadero valor.

Ejemplo 4.2. Verificación de una columna de suelo por solución fabricada.

Un modelo de transporte reactivo en una columna de suelo de 0.40 m resuelve la ecuación de difusión con reacción de primer orden, con coeficiente de difusión que crece linealmente con la profundidad según $D(x) = D_0(1 + x/L)$. Se adoptan $D_0 = 5.0 \times 10^{-10} \text{ m}^2/\text{s}$, constante de reacción $k = 1.0 \times 10^{-6}/\text{s}$ y concentración de referencia $c_0 = 50 \text{ mg/L}$. Se pide verificar el código sin disponer de solución analítica.

Consideraciones. 1. Régimen estacionario y flujo advectivo despreciable frente al difusivo; 2. coeficiente de difusión conocido y variable con la profundidad; 3. reacción de primer orden con constante uniforme.

Análisis. El modelo por verificar es la Ecuación (4.5), cuyo operador incluye un coeficiente variable y por eso no admite solución cerrada elemental.

$$-\frac{d}{dx} \left(D(x) \frac{dc}{dx} \right) + k c = f(x), \quad (4.5)$$

donde c es la concentración (mg/L); x es la profundidad (m); D es el coeficiente de difusión efectivo (m^2/s); k es la constante de reacción ($1/\text{s}$); y f es el término fuente (mg/(L s)). Se escoge $c^*(x) = c_0 \cos[\pi x/(2L)]$, que satisface Dirichlet en ambos extremos y activa los tres términos del operador.

Formulación. Al sustituir la solución fabricada se obtiene la Ecuación (4.6), que el álgebra simbólica genera sin riesgo de error de derivación.

$$f(x) = \frac{\pi c_0 D_0}{2L^2} \left[\sin \frac{\pi x}{2L} + \frac{\pi}{2} \left(1 + \frac{x}{L} \right) \cos \frac{\pi x}{2L} \right] + k c_0 \cos \frac{\pi x}{2L}. \quad (4.6)$$

Solución. Con el término fuente evaluado en los nodos se resuelve el problema en mallas de 8 a 256 intervalos, con el coeficiente de difusión promediado en las caras. El error en norma del máximo pasa de 2.382×10^{-3} mg/L a 2.343×10^{-6} mg/L y el orden observado recorre 1.991, 1.999, 1.999, 2.000 y 2.000.

Comentario. El término fuente en el centro vale 3.594×10^{-5} mg/(L s) y carece de sentido físico, lo cual es irrelevante. La solución fabricada no describe ninguna columna de suelo, solo ejercita todas las rutas del código con una respuesta conocida, incluida la que promedia el coeficiente variable en las caras, que es donde se esconden los defectos.

4.2. Calibración y estimación de parámetros

Verificado el código, el modelo sigue sin poder predecir nada porque sus parámetros son desconocidos. La conductividad hidráulica de un suelo, la constante de semisaturación de un cultivo microbiano y la difusividad efectiva de un grano se estiman a partir de mediciones del sistema concreto. Esa estimación es un problema de optimización, y tratarla como tal permite acompañar cada parámetro de una medida de su precisión.

Definición 4.4. Calibración. La *calibración* de un modelo es el procedimiento de estimar el vector de parámetros que minimiza una función objetivo construida sobre las discrepancias entre las respuestas del modelo y un conjunto de mediciones reservado exclusivamente para ese fin.

La función objetivo más difundida es la suma de cuadrados de los residuales, que la Ecuación (4.7) presenta en su forma ponderada. La ponderación no es un adorno, pues cuando las mediciones tienen precisiones distintas, ignorarla deja que los datos más ruidosos gobiernen el ajuste.

$$S(\theta) = \sum_{i=1}^n w_i [y_i - M(x_i, \theta)]^2, \quad w_i = \frac{1}{\sigma_i^2}, \quad (4.7)$$

donde S es la función objetivo; θ es el vector de parámetros por estimar; y_i es la medición i ; $M(x_i, \theta)$ es la respuesta del modelo en la condición x_i ; w_i es su peso; σ_i es su desviación estándar; y n es el número de mediciones. Con precisiones iguales los pesos coinciden y la expresión se reduce a mínimos cuadrados ordinarios.

Definición 4.5. Residual del ajuste. El *residual del ajuste* en una condición experimental es la diferencia entre la medición y la respuesta del modelo evaluada en el vector de parámetros estimado. Su estructura, y no solo su magnitud, es la fuente principal de información sobre lo que al modelo le falta.

La minimización de la Ecuación (4.7) para un modelo no lineal en los parámetros no tiene solución cerrada y se aborda por iteración. Las dos rutinas de uso corriente en Python resuelven el mismo problema con interfaces distintas [4], y el texto guía desarrolla el mismo problema de estimación sobre modelos de ecuaciones diferenciales, con la advertencia de que la sensibilidad de los parámetros estimados a los datos forma parte del resultado [1]. La primera devuelve la matriz de covarianza aproximada; la segunda admite límites, residuales vectoriales y pérdidas robustas.

4.2.1. Precisión de los parámetros estimados

El valor que devuelve el optimizador es una estimación, y como toda estimación tiene dispersión. Cuantificarla exige linealizar el modelo alrededor del óptimo, lo cual conduce al resultado que sostiene toda la práctica de reportar parámetros con su intervalo.

Teorema 4.2. Covarianza aproximada de los parámetros estimados. Sea $\hat{\theta}$ el minimizador de la suma de cuadrados de n residuales con m parámetros, y sea J la matriz de sensibilidad de elementos $J_{ij} = \partial M(x_i, \theta) / \partial \theta_j$ evaluada en $\hat{\theta}$. Si los errores de medición son independientes, de media nula y varianza común σ^2 , y si el modelo es aproximadamente lineal en un entorno de $\hat{\theta}$, entonces la matriz de covarianza de la estimación se aproxima por $\text{cov}(\hat{\theta}) \approx s^2 (J^T J)^{-1}$, con $s^2 = S(\hat{\theta}) / (n - m)$. El intervalo de confianza al nivel $1 - \alpha$ para el parámetro j es $\hat{\theta}_j \pm t_{1-\alpha/2, n-m} \sqrt{[\text{cov}(\hat{\theta})]_{jj}}$.

El teorema descansa sobre dos hipótesis. La linealidad local falla cuando la función objetivo tiene un valle curvo y alargado, situación frecuente en modelos cinéticos, y entonces el intervalo simétrico deforma la incertidumbre. La varianza común falla cuando las mediciones cubren varios órdenes de magnitud, y se corrige con la ponderación. Cuando alguna resulta insostenible, el remuestreo por bootstrap ofrece intervalos que no dependen de la linealización [73].

El elemento fuera de la diagonal contiene una información que los informes desperdician. Una correlación alta significa que los datos no permiten separar los dos parámetros, pues un aumento de uno se compensa con un aumento del otro sin degradar el ajuste. Reportar los intervalos por separado sugiere una precisión inexistente, y la representación honesta es la región conjunta. El Algoritmo 4.2 organiza el procedimiento.

La implementación con la rutina de ajuste directa aparece en el Listado 4.4, que aplica el Teorema 4.2 a una cinética de crecimiento microbiano medida a diez concentraciones de sustrato.

Listado 4.4. Ajuste no lineal con matriz de covarianza e intervalos de confianza.

```
1 import numpy as np
```

Algoritmo 4.2 Calibración con reporte de precisión de los parámetros.

Entradas: Modelo M , datos de calibración $\{(x_i, y_i)\}$, incertidumbres σ_i , punto inicial $\theta^{(0)}$, límites físicos

Salidas: Estimación $\hat{\theta}$, errores estándar, intervalos y diagnóstico

- 1: escalar los residuales con $w_i = 1/\sigma_i^2$ y los parámetros a orden unidad
 - 2: $\hat{\theta} \leftarrow \arg \min S(\theta)$ sujeto a los límites físicos
 - 3: repetir la minimización desde otros puntos iniciales y comparar el óptimo
 - 4: $J \leftarrow$ matriz de sensibilidad en $\hat{\theta}$; $s^2 \leftarrow S(\hat{\theta})/(n - m)$; $\text{cov} \leftarrow s^2(J^T J)^{-1}$
 - 5: $\kappa \leftarrow$ número de condición de la matriz de sensibilidad escalada
 - 6: **si** κ es grande o alguna correlación excede 0.95 **entonces**
 - 7: informar identificabilidad práctica deficiente y proponer reparametrización
 - 8: **fin si**
 - 9: **devolver** $\hat{\theta}$, errores estándar, intervalos, matriz de correlación y κ
-

```
2 from scipy.optimize import curve_fit
3 from scipy.stats import t as t_student
4
5 S = np.array([0.5, 1.0, 2.0, 3.0, 5.0,
6              8.0, 12.0, 18.0, 25.0, 35.0])
7 mu = np.array([0.0702, 0.1248, 0.1919, 0.2496, 0.2761,
8               0.3205, 0.3689, 0.3709, 0.3770, 0.3773])
9
10 def monod(S, mu_max, K_s):
11     return mu_max * S / (K_s + S)
12
13 theta, cov = curve_fit(monod, S, mu, p0=[0.3, 1.0])
14 n, m = S.size, theta.size
15 ee = np.sqrt(np.diag(cov))
16 t_c = t_student.ppf(0.975, n - m)
17 ic = np.column_stack([theta - t_c * ee, theta + t_c * ee])
18 corr = cov[0, 1] / np.sqrt(cov[0, 0] * cov[1, 1])
```

Cuando el problema exige límites físicos, pesos distintos o una pérdida robusta, la rutina de mínimos cuadrados general resulta preferible. El [Listado 4.5](#) resuelve el mismo ajuste con residuales ponderados por la desviación del instrumento y con los parámetros restringidos al primer cuadrante.

Listado 4.5. Ajuste con residuales ponderados y límites en los parámetros.

```
1 import numpy as np
2 from scipy.optimize import least_squares
3
4 sigma = np.full(S.size, 0.012)
5
```

```

6 def residuo(theta):
7     return (monod(S, *theta) - mu) / sigma
8
9 limites = ([0.0, 0.0], [2.0, 50.0])
10 sol = least_squares(residuo, x0=[0.3, 1.0], bounds=limites)
11 cov_p = np.linalg.inv(sol.jac.T @ sol.jac)
12 ee_p = np.sqrt(np.diag(cov_p))
13 chi2_red = 2 * sol.cost / (S.size - sol.x.size)

```

El siguiente ejemplo desarrolla el ajuste completo y compara los dos caminos para construir el intervalo.

Ejemplo 4.3. Calibración de una cinética de Monod en fermentación. Un laboratorio de agroindustria mide la velocidad específica de crecimiento de una levadura sobre melaza diluida, a diez concentraciones de sustrato entre 0.5 g/L y 35 g/L. La repetibilidad del método analítico corresponde a una desviación estándar de 0.012/h. Se pide estimar los parámetros de la cinética y reportar su precisión.

Consideraciones. 1. Cultivo en lote con sustrato limitante único y temperatura controlada; 2. la velocidad específica depende solo de la concentración de sustrato; 3. los errores de medición son independientes y de varianza común.

Análisis. El modelo adoptado es la expresión de saturación de la Ecuación (4.8), que describe el crecimiento limitado por un sustrato.

$$\mu(S) = \frac{\mu_{\max} S}{K_s + S}, \quad (4.8)$$

donde μ es la velocidad específica de crecimiento (1/h); S es la concentración de sustrato (g/L); $\mu_{\max}$ es la velocidad máxima (1/h); y K_s es la concentración de semisaturación (g/L).

Solución. El ajuste entrega $\hat{\mu}_{\max} = 0.4137(74)/h$ y $\hat{K}_s = 2.231(163) g/L$, con $t_{0.975,8} = 2.306$ y por tanto intervalos de 0.3966 a 0.4308 y 1.855 a 2.606. La desviación residual vale 0.0104/h, algo menor que la repetibilidad declarada de 0.012/h, y el coeficiente de determinación alcanza 0.9923. La correlación entre los dos parámetros es 0.778.

Verificación. El remuestreo por bootstrap de los residuales con 4000 réplicas y semilla 20 262 devuelve intervalos de 0.4020 a 0.4277 y de 1.956 a 2.524, algo más estrechos que los de la aproximación lineal. La coincidencia razonable indica que la linealización es aceptable.

Comentario. La correlación de 0.778 advierte que los dos parámetros no se determinan de manera independiente, pues un valor mayor de la velocidad máxima se compensa con una semisaturación mayor. Las mediciones a concentración alta fijan bien la asíntota, mientras que la curvatura inicial queda definida por unos pocos puntos. Añadir ensayos entre 1 g/L y 4 g/L reduciría la elipse mucho más que duplicar los de concentración alta.

La geometría de la función objetivo explica de un vistazo lo que las cifras anuncian. La Figura 4.5 muestra las curvas de nivel de la suma de cuadrados con la región de confianza conjunta y la nube del bootstrap, y al lado el ajuste con su banda de confianza.

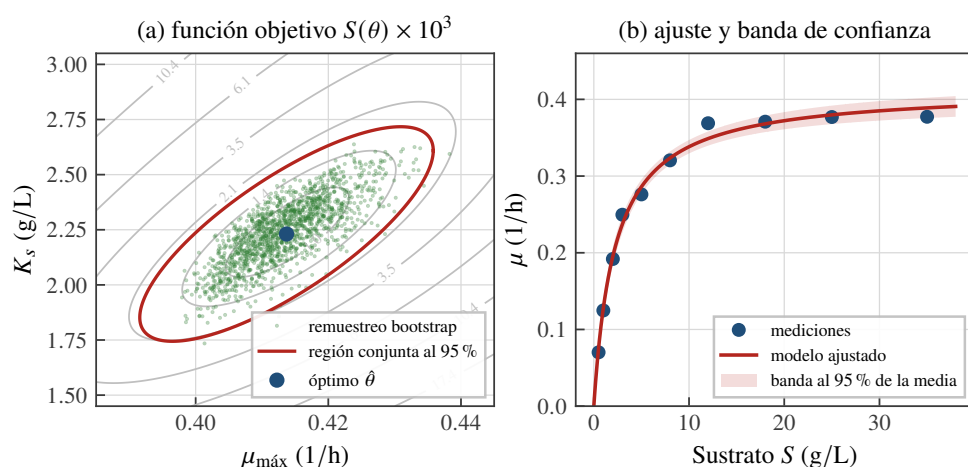


Figura 4.5. Función objetivo de la calibración de Monod, con la región de confianza conjunta al 95 %, la nube del remuestreo y la banda de confianza del ajuste.

4.2.2. Identificabilidad, equifinalidad y parsimonia

El ejemplo anterior terminó bien porque el modelo tenía dos parámetros y los datos cubrían la región donde ambos se manifiestan. Al aumentar el número de parámetros aparece un obstáculo que ninguna rutina de optimización denuncia por sí sola, y que consiste en que distintos vectores producen respuestas indistinguibles.

Definición 4.6. Identificabilidad. Un modelo es *estructuralmente identificable* cuando dos vectores de parámetros distintos producen necesariamente respuestas distintas, para datos perfectos y sin ruido. Es *prácticamente identificable* cuando, además, los datos disponibles y su nivel de ruido permiten distinguir esas respuestas.

La falta de identificabilidad estructural se detecta por análisis del propio modelo y es irreparable con más datos, pues los parámetros solo aparecen en ciertas combinaciones. La práctica depende del diseño experimental y del ruido, y se diagnostica sobre la matriz de sensibilidad escalada [74]. Dos indicadores la resumen, el número de condición de la matriz y el índice de colinealidad de la Ecuación (4.9).

$$\gamma = \frac{1}{\sqrt{\lambda_{\min}(\tilde{J}^T \tilde{J})}}, \quad (4.9)$$

donde γ es el índice de colinealidad (adimensional); $\tilde{J}$ es la matriz de sensibilidad escalada con columnas normalizadas; y $\lambda_{\min}$ denota el menor valor propio. De manera orientativa, valores por debajo de 5 corresponden a parámetros bien separables y valores por encima de 20 señalan combinaciones que los datos no distinguen. El Listado 4.6 calcula ambos indicadores.

Listado 4.6. Matriz de sensibilidad escalada, número de condición e índice de colinealidad.

```

1 import numpy as np
2
3 def sensibilidad_escalada(modelo, theta, x, rel=1e-6):
4     """Columna j igual a theta_j por la derivada."""
5     J = np.zeros((np.size(x), np.size(theta)))
6     for j in range(np.size(theta)):
7         d = rel * max(abs(theta[j]), 1e-12)
8         mas = np.array(theta, float)
9         menos = np.array(theta, float)
10        mas[j], menos[j] = mas[j] + d, menos[j] - d
11        dif = modelo(x, *mas) - modelo(x, *menos)
12        J[:, j] = dif / (2 * d) * theta[j]
13    return J
14
15 def colinealidad(J):
16     Jn = J / np.linalg.norm(J, axis=0)
17     return 1.0 / np.sqrt(np.linalg.eigvalsh(Jn.T @ Jn).min())
18
19 J = sensibilidad_escalada(monod, theta, S)
20 kappa, gamma = np.linalg.cond(J), colinealidad(J)

```

El fenómeno que acompaña a la falta de identificabilidad práctica se llama equifinalidad y consiste en que muchos conjuntos de parámetros reproducen los datos de calibración con calidad equivalente [25]. El conjunto que el optimizador devuelve depende entonces del punto inicial, y las predicciones extrapoladas con uno u otro divergen. La respuesta correcta consiste en reparametrizar, fijar los parámetros que los datos no informan o reportar el conjunto completo de soluciones aceptables.

El criterio que gobierna cuántos parámetros vale la pena estimar se llama parsimonia y se cuantifica con criterios de información, que penalizan la complejidad al comparar modelos ajustados a los mismos datos [24]. La Ecuación (4.10) recoge la versión corregida para muestras pequeñas, que es la pertinente en ingeniería.

$$AIC_c = n \ln \left(\frac{S(\hat{\theta})}{n} \right) + 2K + \frac{2K(K+1)}{n-K-1}, \quad (4.10)$$

donde AIC_c es el criterio corregido; n es el número de mediciones; $S(\hat{\theta})$ es la suma de cuadrados en el óptimo; y K es el número de parámetros estimados incluida la escala del error. Solo las diferencias entre modelos tienen significado, y una superior a diez descarta al modelo peor situado. El ejemplo que sigue reúne los tres conceptos sobre una curva de secado de grano, donde la tentación de añadir parámetros es particularmente fuerte.

Ejemplo 4.4. Identificabilidad en el secado de maíz. Una campaña de secado de maíz a 60 °C registra la razón de humedad en trece instantes entre 0.25 h y 8 h. Se comparan cuatro modelos de la literatura de poscosecha y se examina si sus parámetros quedan determinados por los datos.

Consideraciones. 1. Espesor de capa delgada y condiciones de aire constantes; 2. razón de humedad normalizada entre la inicial y la de equilibrio; 3. ruido de medición con desviación estándar cercana a 0.008.

Análisis. Los cuatro modelos son el exponencial simple con un parámetro, el de Henderson y Pabis con dos, el de Page con dos y el de dos términos con tres. Se ajustan todos por mínimos cuadrados y se comparan con la Ecuación (4.10), además de examinar el número de condición de la matriz de sensibilidad escalada.

Solución. El modelo de Page entrega $\hat{k} = 0.3430$ y $\hat{n} = 1.1627$, con suma de cuadrados de 5.81×10^{-4} , número de condición 3.18 e índice de colinealidad 2.31. El de dos términos alcanza 9.97×10^{-3} , diecisiete veces peor, y su número de condición se dispara a 1.3×10^{10} porque el optimizador converge a un punto en el que las dos constantes coinciden. Dos puntos iniciales distintos entregan conjuntos de parámetros distintos con idéntica suma de cuadrados.

Verificación. Un quinto modelo, el de difusión en lámina plana truncado al primer término, ilustra la falta de identificabilidad estructural. Sus dos parámetros aparecen solo en el cociente D_{ef}/L^2 , de modo que la pareja $3.07 \times 10^{-10} \text{ m}^2/\text{s}$ con 3.0 mm y la pareja $1.36 \times 10^{-10} \text{ m}^2/\text{s}$ con 2.0 mm producen predicciones idénticas. Al reparametrizar con el cociente único se obtiene 0.1226(149) 1/h y número de condición 1.00.

Comentario. El resultado desmiente la creencia de que más parámetros siempre ajustan mejor, pues el modelo de dos términos tiene uno más y ajusta peor. La lección práctica consiste en revisar el número de condición antes de creer en cualquier ajuste con más de dos parámetros.

La comparación completa aparece en la [Tabla 4.2](#), que incluye el error de predicción fuera de la muestra obtenido por validación cruzada. La coincidencia entre el ordenamiento del criterio de información y el de la validación cruzada refuerza la conclusión.

Tabla 4.2. Selección entre cuatro modelos de secado de capa delgada ajustados a los mismos trece datos.

Modelo	m	$S(\hat{\theta})$	s	AIC_c	$RMSE_{vc}$
Exponencial simple	1	9.971×10^{-3}	0.0288	-88.05	0.0292
Henderson y Pabis	2	3.173×10^{-3}	0.0170	-99.47	0.0196
Page	2	5.815×10^{-4}	0.0073	-121.53	0.0075
Dos términos	3	9.971×10^{-3}	0.0316	-80.25	0.0292

^a m es el número de parámetros del modelo y s la desviación residual de la razón de humedad.

^b $RMSE_{vc}$ es el error de predicción fuera de la muestra con validación cruzada de cinco bloques y semilla 20 262.

4.3. Validación frente a datos independientes

Un modelo calibrado reproduce por construcción los datos con los que se calibró, y ese acuerdo no constituye evidencia de nada, pues los parámetros se escogieron para minimizar la discrepancia. Preguntar después si esa discrepancia es pequeña equivale a examinar un examen que uno mismo respondió con el solucionario a la vista.

Definición 4.7. Conjunto de validación. Un *conjunto de validación* es un grupo de mediciones del sistema real que no intervino en ninguna etapa de la estimación de parámetros, ni siquiera en la selección de la estructura del modelo o en el descarte de valores atípicos.

La cláusula final es la que más se incumple. Cuando el modelador prueba cinco estructuras, calcula las métricas de las cinco sobre el mismo conjunto reservado y se queda con la mejor, ese conjunto dejó de ser independiente. La partición honesta exige tres bloques, uno para ajustar, otro para escoger y un tercero para el reporte final [75].

La forma de la partición depende del sistema. En una serie temporal la separación cronológica es obligatoria, porque partir al azar deja puntos vecinos en ambos bloques y la correlación temporal simula una capacidad predictiva inexistente. En ensayos conviene separar por lote o por campaña, y en un modelo espacial, por sitio.

4.3.1. Métricas de desempeño y lo que cada una esconde

Comparar dos series de números exige resumirlos, y cada resumen sacrifica algo. Las métricas de uso corriente miden el sesgo, la magnitud del error o la fracción de variabilidad explicada. Las Ecuaciones (4.11) y (4.12) recogen las más extendidas en ingeniería.

$$\text{ME} = \frac{1}{n} \sum_{i=1}^n (s_i - o_i), \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |s_i - o_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (s_i - o_i)^2}, \quad (4.11)$$

$$\text{NSE} = 1 - \frac{\sum_{i=1}^n (s_i - o_i)^2}{\sum_{i=1}^n (o_i - \bar{o})^2}, \quad \text{RSR} = \frac{\text{RMSE}}{\sigma_o}, \quad \text{PBIAS} = 100 \frac{\sum_{i=1}^n (s_i - o_i)}{\sum_{i=1}^n o_i}, \quad (4.12)$$

donde o_i es la medición i ; s_i es la simulación correspondiente; $\bar{o}$ y σ_o son la media y la desviación estándar de las mediciones; n es el número de parejas; y ME, MAE, RMSE, NSE, RSR y PBIAS designan el error medio, el error absoluto medio, la raíz del error cuadrático medio, el coeficiente de eficiencia, la razón de error y el sesgo porcentual.

El coeficiente de eficiencia se introdujo para el pronóstico de caudales y compara el modelo contra el predictor trivial que devuelve la media observada [76]. Un valor nulo indica que el modelo no aporta nada sobre esa media y uno unitario corresponde al acuerdo perfecto. Su punto débil está en el denominador, pues un periodo muy variable agranda la referencia y regala un valor alto a un modelo mediocre, razón por la cual nunca debe compararse entre cuencas, periodos o estudios distintos.

Teorema 4.3. Relación entre el coeficiente de eficiencia y la razón de error.

Si la desviación estándar de las observaciones se calcula con divisor n , es decir $\sigma_o^2 = n^{-1} \sum_i (o_i - \bar{o})^2$, entonces las dos métricas satisfacen $\text{NSE} = 1 - \text{RSR}^2$ de manera exacta, para cualquier pareja de series.

La consecuencia práctica es inmediata. Reportar ambas métricas no aporta información adicional, porque una es función determinista de la otra. Un coeficiente de 0.50 corresponde a una razón de 0.707 y uno de 0.75 a una razón de 0.50. Lo que sí aporta información nueva es acompañarlas de una métrica de sesgo y de una de magnitud en unidades físicas.

El índice de concordancia se propuso para separar el acuerdo del simple ajuste de la media [77]. Es menos sensible a los valores extremos que el coeficiente de eficiencia, pero rara vez baja de 0.65 incluso para modelos deficientes, lo cual lo vuelve poco discriminante. La [Tabla 4.3](#) resume qué mide cada métrica y cuándo engaña.

Tabla 4.3. Métricas de desempeño de uso corriente, con lo que miden y la situación en la que engañan.

Métrica	Intervalo	Qué mide	Cuándo engaña
ME	$(-\infty, \infty)$	Sesgo medio en unidades físicas	Errores de signo opuesto se cancelan
MAE	$[0, \infty)$	Magnitud típica del error	No aísla un error grande solitario
RMSE	$[0, \infty)$	Error con peso en los extremos	Un dato mal medido domina el valor
PBIAS	$(-\infty, \infty)$	Sesgo relativo al total observado	Inestable si la media es casi nula
NSE	$(-\infty, 1]$	Mejora sobre el predictor de la media	Periodos variables lo inflan
RSR	$[0, \infty)$	Error en unidades de la dispersión	Función exacta del anterior
d	$[0, 1]$	Concordancia normalizada	Rara vez desciende de 0.65

^a Ninguna métrica sustituye la inspección de los residuales; todas resumen y por tanto ocultan.

La implementación conviene tenerla en una sola función que devuelva todas las métricas a la vez, para que el informe nunca reporte una sin las demás. El [Listado 4.7](#) las recoge y sirve de bloque de construcción del resto de la sección.

Listado 4.7. Cálculo conjunto de las métricas de desempeño de un modelo.

```

1 import numpy as np
2
3 def metricas(obs, sim):
4     """Métricas de uso corriente en validación."""
5     obs, sim = np.asarray(obs, float), np.asarray(sim, float)
6     e = sim - obs
7     rmse = np.sqrt(np.mean(e**2))
8     dif = np.abs(sim - obs.mean()) + np.abs(obs - obs.mean())
9     den_d = np.sum(dif**2)
10    return {"ME": e.mean(), "MAE": np.abs(e).mean(), "RMSE": rmse,
11           "PBIAS": 100 * e.sum() / obs.sum(),

```

```

12     "NSE": 1 - np.sum(e**2) / np.sum((obs -
13     obs.mean())**2),
    "RSR": rmse / obs.std(), "d": 1 - np.sum(e**2) /
    den_d}

```

Queda por saber a partir de qué valor se declara aceptable un modelo. La literatura de modelación de cuencas propuso umbrales orientativos muy difundidos, que se reproducen en la [Tabla 4.4 \[78\]](#) y que conviene leer con tres advertencias.

Tabla 4.4. Umbrales orientativos de desempeño publicados para caudal simulado en paso mensual.

Calificación	NSE	RSR	PBIAS (%)
Muy bueno	$0.75 < \text{NSE} \leq 1.00$	$0.00 \leq \text{RSR} \leq 0.50$	< 10
Bueno	$0.65 < \text{NSE} \leq 0.75$	$0.50 < \text{RSR} \leq 0.60$	10 a 15
Satisfactorio	$0.50 < \text{NSE} \leq 0.65$	$0.60 < \text{RSR} \leq 0.70$	15 a 25
Insatisfactorio	$\text{NSE} \leq 0.50$	$\text{RSR} > 0.70$	≥ 25

^a Los umbrales se propusieron para caudal simulado en paso mensual con modelos de cuenca, y los autores advierten que deben ajustarse al paso de tiempo, a la variable y a la calidad de los datos disponibles.

^b Para sedimentos y nutrientes los mismos autores proponen límites de sesgo más holgados.

La primera advertencia es que los umbrales se establecieron para una variable, un paso de tiempo y una familia de modelos concretos, de modo que trasladarlos a la temperatura de un secador carece de fundamento. La segunda es que la agregación mensual eleva el coeficiente respecto del paso diario sin que el modelo haya mejorado. La tercera es que ningún umbral sustituye el criterio del ingeniero.

4.3.2. Diagrama de paridad y análisis de residuales

Las métricas resumen y por tanto ocultan, de manera que un modelo puede alcanzar valores excelentes y arrastrar un defecto estructural evidente. La herramienta que lo revela es el examen de los residuales, y tres diagnósticos concentran casi todo el valor, según muestra el [Listado 4.8](#).

Listado 4.8. Diagnóstico de paridad, tendencia, autocorrelación y heterocedasticidad.

```

1 import numpy as np
2
3 def diagnostico_residuales(obs, sim):
4     """Paridad, tendencia y estructura del residual."""
5     r = sim - obs
6     pend_par = np.polyfit(obs, sim, 1)

```

```

7 pend_res = np.polyfit(obs, r, 1)
8 lag1 = np.corrcoef(r[:-1], r[1:])[0, 1]
9 hetero = np.corrcoef(np.abs(r), obs)[0, 1]
10 return dict(paridad=pend_par[0], tendencia=pend_res[0],
11             autocorrelacion=lag1, heterocedasticidad=hetero)

```

El diagrama de paridad enfrenta lo simulado contra lo observado y le añade la recta de pendiente unitaria. Una nube centrada sobre esa recta indica ausencia de sesgo sistemático, mientras que una pendiente ajustada inferior a la unidad delata que el modelo comprime el rango, firma de un proceso que solo se activa en un extremo. La tendencia del residual frente a lo observado cuantifica esa compresión con un solo número.

La autocorrelación de retardo unitario detecta si los errores llegan en rachas. Una racha del mismo signo indica que al modelo le falta memoria, típicamente un almacenamiento no representado, e invalida cualquier prueba que suponga residuales independientes. La correlación entre la magnitud del residual y el valor observado detecta heterocedasticidad. La [Figura 4.6](#) muestra las cuatro firmas que un residual puede exhibir y el defecto que cada una denuncia, y solo la primera autoriza a leer las métricas como un resumen fiel del desempeño. El [Algoritmo 4.3](#) reúne los tres diagnósticos dentro del protocolo completo.

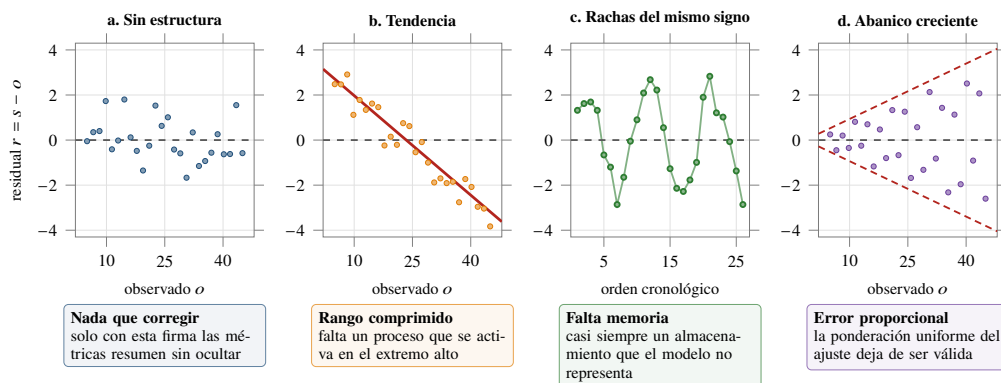


Figura 4.6. Cuatro firmas del residual y el defecto que denuncia cada una, con la única que autoriza a leer las métricas como resumen fiel.

El paso primero merece comentario aparte porque es el que más se omite. Declarar el criterio de aceptación antes de mirar los datos de validación impide escoger después, ya vistos los resultados, la métrica que mejor le queda al modelo. En un trabajo de posgrado ese criterio se escribe en el protocolo y se firma, con la misma seriedad con que se declara una hipótesis.

Algoritmo 4.3 Protocolo de validación con criterio de aceptación declarado de antemano.

Entradas: Modelo calibrado, datos independientes, uso previsto y exactitud requerida

Salidas: Veredicto de aceptación y dominio de validez declarado

- 1: escribir el criterio de aceptación antes de mirar los datos de validación
 - 2: separar los datos por criterio físico, cronológico o por lote, nunca al azar
 - 3: ejecutar el modelo con los parámetros calibrados, sin retocar ninguno
 - 4: calcular las métricas de sesgo, de magnitud y de eficiencia
 - 5: examinar paridad, tendencia, autocorrelación y heterocedasticidad
 - 6: **si** alguna métrica no alcanza el criterio o los residuales muestran estructura **entonces**
 - 7: declarar la falla, diagnosticar el proceso ausente y volver a la formulación
 - 8: **en otro caso**
 - 9: declarar el dominio de validez como el rango cubierto por los datos
 - 10: **fin si**
 - 11: **devolver** veredicto, métricas, diagnóstico y dominio de validez
-

Ejemplo 4.5. Validación de un modelo de caudal medio mensual. Un modelo lluvia–escorrentía de una cuenca del Caribe colombiano se calibra con veinticuatro meses de registro y se valida con los veinticuatro meses siguientes, que incluyen un periodo notablemente más húmedo. El criterio de aceptación se declaró antes del ejercicio y exige coeficiente de eficiencia superior a 0.65 y sesgo porcentual inferior al 15 % en valor absoluto.

Consideraciones. 1. Paso mensual y partición cronológica, sin mezcla entre periodos; 2. parámetros congelados tras la calibración, sin retoque alguno; 3. las mediciones de caudal tienen incertidumbre relativa cercana al 8 %.

Análisis. Se calculan las siete métricas de la [Tabla 4.3](#) sobre los dos periodos por separado, y se aplica después el diagnóstico de residuales. La comparación entre periodos es tan informativa como los valores absolutos, pues la degradación mide cuánto del acuerdo original provenía del ajuste.

Solución. En calibración se obtiene un coeficiente de eficiencia de 0.951, una razón de error de 0.222, un sesgo porcentual de 0.04 % y una raíz del error cuadrático medio de 1.58 m³/s. En validación las mismas métricas valen 0.747, 0.504, –11.5 % y 4.42 m³/s. El error medio pasa de 0.01 m³/s a –1.89 m³/s.

Verificación. Los residuales de validación revelan lo que las métricas esconden. La pendiente del diagrama de paridad cae a 0.663, la tendencia del residual frente a lo observado vale –0.337, la autocorrelación de retardo unitario alcanza 0.456 y la correlación entre la magnitud del residual y el caudal observado llega a 0.726.

Comentario. El modelo cumple el criterio declarado y aun así no debe usarse para dimensionar una obra de excedencias. La firma de los residuales indica que satura la respuesta en crecidas, porque no representa la escorrentía superficial que se activa con el suelo saturado, proceso ausente en el periodo seco de calibración. Aceptarlo con dominio de validez restringido a caudales inferiores a $20 \text{ m}^3/\text{s}$ es la conclusión defendible.

La Figura 4.7 reúne las cuatro vistas que sostienen ese diagnóstico. La serie revela el desfase, el diagrama de paridad revela la compresión del rango, la gráfica de residuales revela la tendencia y el histograma revela el sesgo global.

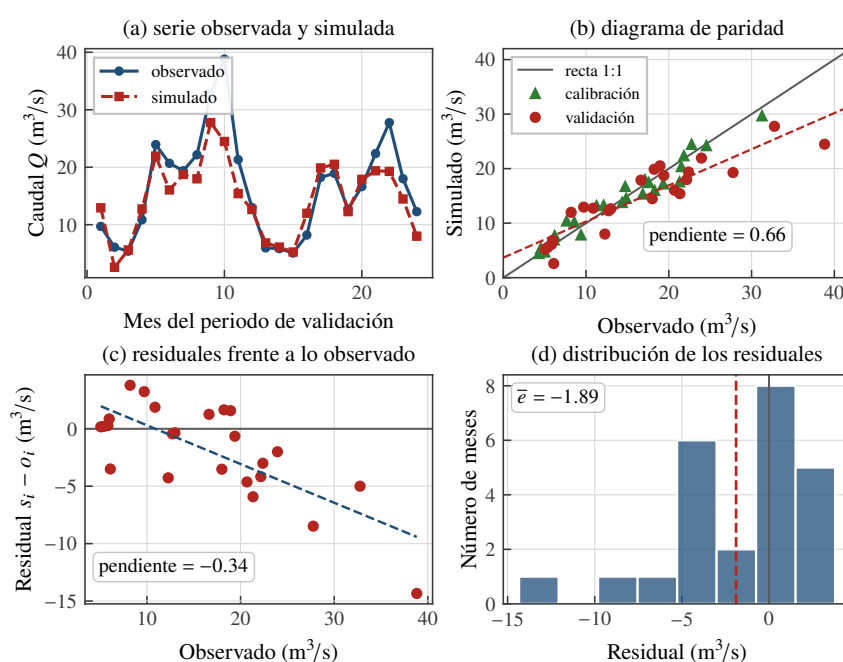


Figura 4.7. Validación del modelo de caudal medio mensual, con la serie del periodo reservado, el diagrama de paridad, la tendencia de los residuales y su distribución.

Cuando los datos escasean y reservar un tercio del registro resulta inaceptable, la validación cruzada aprovecha todas las mediciones. El procedimiento divide los datos en bloques, ajusta con todos menos uno y predice el que quedó fuera, repitiendo el ciclo hasta agotarlos, con lo cual estima el error de predicción fuera de la muestra sin sacrificar datos. Esa es la vía que produjo la última columna de la Tabla 4.2. La validación cruzada es también el instrumento central con que la literatura de aprendizaje estadístico compara modelos fenomenológicos de complejidad creciente, y sus variantes por bloques y por dejar uno fuera se describen allí con detalle [6, 79].

Conviene cerrar la sección con una distinción que la validación cruzada no separa. Una cosa es preguntar cuánto se equivoca el modelo al predecir un dato nuevo del mismo tipo de los que ya vio, y otra muy distinta es preguntarlo en condiciones que nunca observó. La primera se responde con validación cruzada, la segunda solo con una campaña de medición en el nuevo régimen. Extrapolar sin esa campaña convierte el modelo en una conjetura elegante.

4.4. Incertidumbre, limitaciones y consistencia

El resultado de una simulación validada sigue siendo una afirmación incompleta mientras no declare su margen. Ese margen no proviene de una sola fuente, y confundir sus orígenes conduce a decisiones equivocadas sobre dónde invertir el siguiente peso. Reducir la incertidumbre de una predicción exige saber primero qué parte de ella se puede reducir.

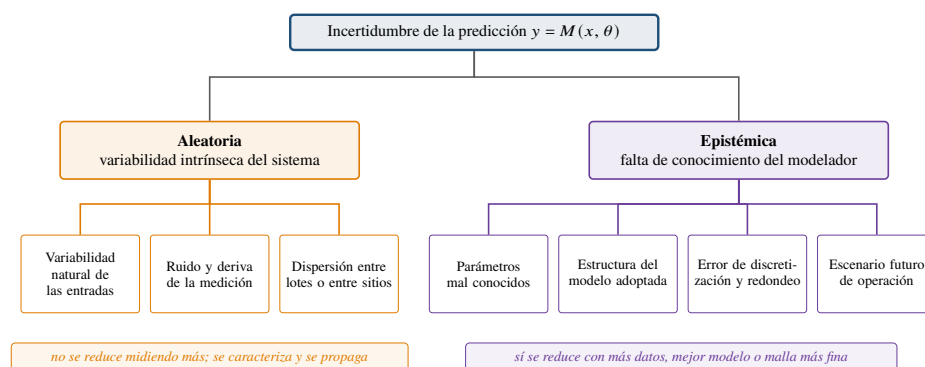
Definición 4.8. Incertidumbre aleatoria. La *incertidumbre aleatoria* es la que proviene de la variabilidad intrínseca del sistema o de sus entradas, y no disminuye al acumular más mediciones porque describe una dispersión real y no un desconocimiento.

Su tratamiento consiste en caracterizarla con una distribución y propagarla a través del modelo. La variabilidad de la radiación solar entre días y la dispersión de la humedad inicial entre lotes de grano pertenecen a esta categoría.

Promediar esa variabilidad antes de entrar al modelo es un error frecuente y caro, porque un modelo no lineal evaluado en la media de las entradas no devuelve la media de las respuestas. La segunda categoría se comporta de manera opuesta.

Definición 4.9. Incertidumbre epistémica. La *incertidumbre epistémica* es la que proviene del desconocimiento del modelador acerca del valor de un parámetro, de la estructura del proceso o del error numérico cometido, y sí disminuye con más información, mejor modelo o malla más fina.

La distinción gobierna el presupuesto del proyecto. Si la incertidumbre dominante de una predicción de producción de biogás es epistémica y reside en la constante cinética, una campaña de ensayos la reduce; si es aleatoria y reside en la composición del sustrato que llega a planta, lo que corresponde es dimensionar con margen. La [Figura 4.8](#) organiza ambas naturalezas frente a los cuatro orígenes recorridos.



La clasificación no es una etiqueta permanente. Una incertidumbre epistémica que nadie está dispuesto a financiar se trata en la práctica como aleatoria, y esa decisión debe quedar escrita en el informe.

Figura 4.8. Taxonomía de la incertidumbre de una predicción según su naturaleza y según su origen, con la vía de reducción que corresponde a cada rama.

4.4.1. Propagación de la incertidumbre

La guía internacional para la expresión de la incertidumbre en la medida estableció el procedimiento que hoy constituye la referencia obligada en metrología y en ingeniería [23]. Su núcleo es la misma linealización alrededor del punto nominal que el [Capítulo 1](#) introdujo en términos de varianzas, escrita ahora en la notación y con las convenciones de la metrología.

Teorema 4.4. Ley de propagación de primer orden. Sea $y = f(x_1, \dots, x_k)$ con entradas independientes de incertidumbres típicas $u(x_i)$. Si f es continuamente derivable y las incertidumbres son pequeñas frente a la escala de curvatura de f , la incertidumbre típica combinada de la salida se aproxima por

$$u_c^2(y) = \sum_{i=1}^k c_i^2 u^2(x_i), \quad c_i = \left. \frac{\partial f}{\partial x_i} \right|_{x=\bar{x}},$$

y cuando las entradas están correlacionadas se añade el término $2 \sum_{i < j} c_i c_j u(x_i, x_j)$. Si además f es un producto de potencias $y = A \prod x_i^{a_i}$, la expresión se reduce a la forma relativa $[u_c(y)/y]^2 = \sum a_i^2 [u(x_i)/x_i]^2$.

La hipótesis que sostiene el resultado es la linealidad local, y su violación se paga con una incertidumbre mal estimada y con un intervalo asimétrico que la fórmula no representa. La forma relativa es útil en ingeniería porque el exponente de cada variable actúa como amplificador de su incertidumbre relativa. Una carga elevada a la potencia 1.5 contribuye una vez y media la suya, y una velocidad al cubo la contribuye tres veces.

Definición 4.10. Presupuesto de incertidumbre. Un *presupuesto de incertidumbre* es la tabla que reúne, para cada magnitud de entrada, su valor nominal, su incertidumbre típica, su coeficiente de sensibilidad, su contribución a la varianza de la salida y la fracción que esa contribución representa del total.

La utilidad del presupuesto está en la columna de fracciones, que ordena las fuentes y señala cuál conviene atacar. Reducir a la mitad una fuente que aporta el 2% de la varianza no mejora nada, mientras que reducir un veinte por ciento la que aporta el 70% sí se nota. El [Listado 4.9](#) lo calcula con derivadas numéricas.

Listado 4.9. Propagación de primer orden con derivadas por diferencias centradas.

```
1 import numpy as np
2
3 def propagacion_primer_orden(f, x, u, rel=1e-6):
4     """Ley del GUM con derivadas por diferencias."""
5     x, u = np.asarray(x, float), np.asarray(u, float)
6     c = np.zeros_like(x)
7     for i in range(x.size):
8         d = rel * max(abs(x[i]), 1e-12)
9         mas, menos = x.copy(), x.copy()
10        mas[i], menos[i] = mas[i] + d, menos[i] - d
11        c[i] = (f(mas) - f(menos)) / (2 * d)
12    contrib = (c * u) ** 2
13    return f(x), np.sqrt(contrib.sum()), contrib / contrib.sum()
14
15 def caudal(p):
16     return (2 / 3) * p[0] * p[1] * np.sqrt(2 * 9.81) * p[2]**1.5
17 Q, u_c, aporte = propagacion_primer_orden(
18     caudal, [0.620, 0.500, 0.150], [0.015, 0.0010, 0.0015])
```

Cuando la linealidad local no se sostiene, el suplemento de la misma guía propone propagar distribuciones en lugar de varianzas [80]. El procedimiento es el método de Monte Carlo aplicado a las incertidumbres de entrada, y devuelve la distribución completa de la salida, con su asimetría y sus colas, sin suponer normalidad. El [Algoritmo 4.4](#) fija el criterio de decisión entre ambos métodos.

El [Listado 4.10](#) implementa el muestreo con entradas normales independientes y devuelve el intervalo de cobertura por percentiles, que es el reporte que corresponde cuando la distribución de salida no es simétrica.

Listado 4.10. Propagación de distribuciones por el método de Monte Carlo.

```
1 import numpy as np
2
3 def propagacion_monte_carlo(f, x, u, n=2000000, semilla=20262):
```

Algoritmo 4.4 Propagación de incertidumbre por Monte Carlo y contraste con la ley de primer orden.

Entradas: Modelo f , valores nominales $\bar{x}$, incertidumbres $u(x_i)$, distribuciones y correlaciones, semilla

Salidas: Estimación de $u_c(y)$, intervalo de cobertura y veredicto sobre la linealización

- 1: calcular u_c y el presupuesto por la ley de primer orden
 - 2: generar N muestras de las entradas con la semilla declarada y evaluar $y_j = f(x_j)$
 - 3: estimar media, desviación y percentiles 2.5 y 97.5 de la salida
 - 4: **si** la desviación de Monte Carlo difiere de u_c en más del cinco por ciento **entonces**
 - 5: declarar insuficiente la linealización y reportar el intervalo de Monte Carlo
 - 6: **fin si**
 - 7: repetir con otra semilla y con $2N$ muestras para verificar la estabilidad
 - 8: **devolver** u_c , presupuesto, intervalo de cobertura y veredicto
-

```

4      """Propagación con entradas normales independientes."""
5      rng = np.random.default_rng(semilla)
6      muestras = rng.normal(np.asarray(x, float),
7                             np.asarray(u, float), (n, np.size(x)))
8      y = np.array([f(fila) for fila in muestras])
9      inf, sup = np.percentile(y, [2.5, 97.5])
10     return y.mean(), y.std(ddof=1), (inf, sup)
11
12 media, desv, intervalo = propagacion_monte_carlo(
13     caudal, [0.620, 0.500, 0.150], [0.015, 0.0010, 0.0015])

```

Ejemplo 4.6. Presupuesto de incertidumbre del caudal en un vertedero rectangular. Una estación de aforo mide el caudal de un canal de riego con un vertedero rectangular de pared delgada sin contracciones. El ancho de cresta vale 0.5000(10) m, la carga medida sobre la cresta vale 0.1500(15) m y el coeficiente de descarga adoptado vale 0.620(15). Se pide el caudal con su incertidumbre expandida.

Consideraciones. 1. Flujo permanente y carga medida lejos de la zona de contracción; 2. las tres magnitudes de entrada son independientes; 3. las incertidumbres declaradas son típicas, es decir con factor de cobertura unitario.

Análisis. La ecuación de descarga del vertedero rectangular es la Ecuación (4.13), un producto de potencias al que se aplica directamente la forma relativa del Teorema 4.4.

$$Q = \frac{2}{3} C_d b \sqrt{2g} h^{3/2}, \quad (4.13)$$

donde Q es el caudal (m^3/s); C_d es el coeficiente de descarga; b es el ancho de la cresta (m); h es la carga (m); y g es la gravedad, tomada como 9.81 m/s^2 sin incertidumbre.

Solución. El caudal nominal vale 53.18 L/s . Las incertidumbres relativas contribuyen con 2.42% el coeficiente, 0.20% el ancho y 1.50% la carga, esta última ya multiplicada por el exponente 1.5 . La combinación cuadrática entrega 2.854% , es decir 1.52 L/s , y con factor de cobertura 2 el resultado se reporta como $53.2(30) \text{ L/s}$.

Verificación. El muestreo de Monte Carlo con $200\,000$ réplicas y semilla $20\,262$ devuelve una desviación de 1.52 L/s y un intervalo de cobertura de 50.22 a 56.20 L/s , frente a 50.21 a 56.16 L/s de la fórmula lineal. La discrepancia es del 0.2% , de modo que la linealización es adecuada.

Comentario. El presupuesto ordena las prioridades sin ambigüedad, pues el coeficiente de descarga aporta el 71.9% de la varianza, la carga el 27.6% y el ancho apenas el 0.5% . Medir la cresta con calibrador de precisión no mejora nada, mientras que calibrar el vertedero en sitio contra un aforo volumétrico reduciría la incertidumbre casi a la mitad.

El presupuesto completo se recoge en la [Tabla 4.5](#), con el formato que conviene reproducir en todo informe que entregue una magnitud derivada de varias mediciones.

Tabla 4.5. Presupuesto de incertidumbre del caudal medido con vertedero rectangular.

Magnitud	Valor	$u(x_i)$	c_i	$c_i u(x_i)$ (L/s)	Aporte (%)
C_d (—)	0.620	0.015	0.0858	1.287	71.9
b (m)	0.500	0.0010	0.1064	0.106	0.5
h (m)	0.1500	0.0015	0.5318	0.798	27.6
$u_c(Q)$				1.518	100.0

^a Caudal nominal 53.18 L/s ; incertidumbre expandida 3.04 L/s con factor de cobertura $k = 2$.

^b Los coeficientes de sensibilidad están expresados en m^3/s por unidad de la magnitud de entrada.

La adecuación de la linealización no es propiedad del método sino del modelo. La [Figura 4.9](#) compara ambos procedimientos sobre el vertedero y explora el límite de validez para tres formas funcionales. Una potencia moderada tolera incertidumbres relativas grandes, un cociente empieza a fallar cerca del 15% y una exponencial de argumento inverso, como la de toda cinética de Arrhenius, falla antes de lo que la intuición sugiere.

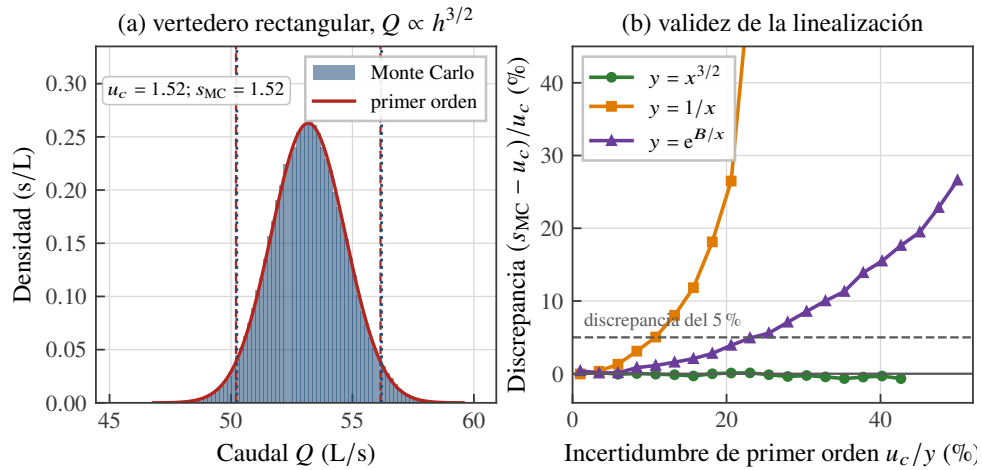


Figura 4.9. Propagación de primer orden frente a Monte Carlo en el vertedero y límite de validez de la linealización para tres formas funcionales.

4.4.2. Consistencia, cifras significativas y limitaciones

Antes de propagar cualquier incertidumbre conviene comprobar que el resultado es consistente. Cuatro comprobaciones baratas atrapan la mayoría de los errores gruesos, y son la homogeneidad dimensional de toda ecuación implementada, el cierre de los balances, la comparación del orden de magnitud con un cálculo independiente y el contraste con valores publicados.

El reporte de la cifra final obedece a una regla sencilla que casi nadie aplica. La incertidumbre se redondea a una o dos cifras significativas y el valor a la misma posición decimal, de modo que escribir 53.1810 L/s con incertidumbre de 3 L/s es una contradicción interna. Automatizar ese redondeo en el código que produce las tablas evita que un solo número del informe la incumpla.

Queda por declarar aquello que el modelo no puede hacer, y esa declaración es parte del entregable profesional. Cinco limitaciones deben aparecer siempre por escrito, que son el dominio de validez cubierto por los datos de validación, los procesos omitidos, las fuentes de incertidumbre no cuantificadas, la dependencia respecto de escenarios futuros que nadie puede verificar y el error numérico residual. Omitirlas transfiere al lector un riesgo que no le corresponde.

4.5. Tablas, gráficas e informes técnicos

Una tabla y una figura no ilustran un informe, lo argumentan. Cada una ocupa un espacio costoso y debe ganarlo aportando evidencia que el texto no transmite con la misma economía. La tabla sirve cuando el lector necesita leer valores concretos para compararlos o reutilizarlos, y la figura cuando lo que importa es una tendencia o la presencia de estructura [50].

La anatomía de una y otra obedece a reglas de función. En la tabla el título va encima y es autosuficiente, la unidad se declara una sola vez en el encabezado, las cifras significativas las gobierna la incertidumbre, las reglas son solo horizontales y una nota al pie fija el alcance. En la figura el pie va debajo e interpreta en lugar de describir, los ejes llevan símbolo, nombre y unidad, la incertidumbre se dibuja y la leyenda no repite el pie [49]. La Figura 4.10 señala cada elemento.

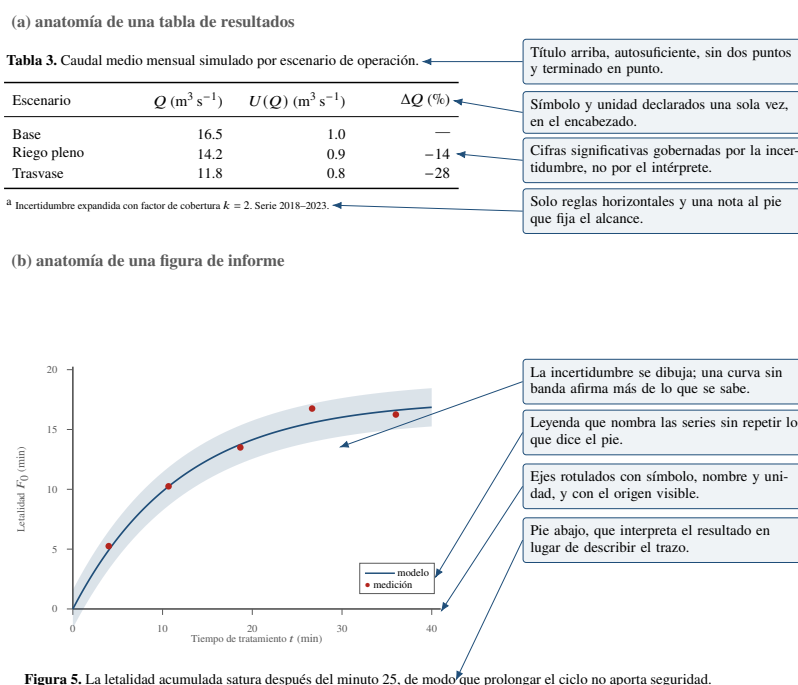


Figura 5. La letalidad acumulada satura después del minuto 25, de modo que prolongar el ciclo no aporta seguridad.

Figura 4.10. Anatomía de una tabla y de una figura de informe técnico, con los elementos que deciden si el lector puede verificar el resultado.

La consecuencia operativa es que ni la tabla ni la figura deben construirse a mano. Copiar números de una consola a un procesador de texto introduce errores de transcripción que nadie detecta y obliga a repetir el trabajo cada vez que cambia un dato. El Listado 4.11 genera la tabla directamente desde los datos.

Listado 4.11. Generación automática de la tabla de resultados desde los datos del proyecto.

```
1 import numpy as np
2 import pandas as pd
3
4 P, u_P, PR, u_PR, u_rel_H = 5.00, 0.10, 0.78, 0.03, 0.04
5 tabla = pd.DataFrame({"H": [1980.0, 2035.0, 1910.0]},
6                       index=["Base", "Óptima", "Sombreado"])
7 u_rel = np.sqrt((u_P / P) ** 2 + u_rel_H**2 + (u_PR / PR) ** 2)
8 tabla["E"] = P * tabla["H"] * PR
9 tabla["u"] = u_rel * tabla["E"]
10 tabla["Y"] = tabla["E"] / P
11 salida = tabla.to_latex(column_format="lrrrr",
                          float_format="%.0f")
```

El ejemplo que sigue muestra por qué el reporte de una comparación exige algo más que reportar dos valores con sus márgenes.

Ejemplo 4.7. Tabla de resultados de tres configuraciones fotovoltaicas. Un estudio compara tres configuraciones de un arreglo fotovoltaico de 5.00(10) kW en Sincelejo. La irradiación anual en el plano del arreglo vale 1980 kWh/m², 2035 kWh/m² y 1910 kWh/m² para las tres configuraciones, con incertidumbre relativa del 4 %, y la razón de desempeño vale 0.78(3). Se pide construir la tabla de resultados del informe.

Consideraciones. 1. Energía anual estimada como el producto de la potencia nominal, la irradiación en el plano y la razón de desempeño; 2. las tres incertidumbres son independientes dentro de una misma configuración; 3. las tres irradiaciones provienen del mismo modelo de radiación y están fuertemente correlacionadas entre configuraciones.

Solución. La incertidumbre relativa combinada vale 5.90 % y es la misma para las tres configuraciones. Las energías anuales resultan de 7.72(46) MWh, 7.94(47) MWh y 7.45(44) MWh, con rendimientos específicos de 1544, 1587 y 1490 kWh/kW.

Verificación. Comparadas por sus intervalos las tres parecen indistinguibles, pues la diferencia entre la mejor y la de referencia vale 215 kWh frente a márgenes de 460 kWh. Ese razonamiento es incorrecto, porque la diferencia comparte la potencia y la razón de desempeño, y las dos irradiaciones están correlacionadas con coeficiente cercano a 0.95. La incertidumbre de la diferencia vale apenas 100 kWh, de modo que la ventaja alcanza poco más de dos incertidumbres típicas.

Comentario. La lección se generaliza a toda comparación de escenarios. Cuando dos alternativas se evalúan con el mismo modelo y las mismas entradas, buena parte de la incertidumbre es común y se cancela en la diferencia. Reportar solo los intervalos absolutos oculta esa cancelación y lleva a declarar que no hay diferencia donde sí la hay.

4.5.1. El informe reproducible

La estructura del informe técnico no es negociable, porque cada parte responde a una pregunta que el revisor formulará de todos modos. Diez secciones cubren el ciclo, y son el problema con su alcance, el modelo con sus supuestos, la verificación, la calibración, la validación, los escenarios, la incertidumbre, las conclusiones, las limitaciones y el anexo reproducible.

Definición 4.11. Informe reproducible. Un *informe reproducible* es aquel cuyas figuras, tablas y cifras se regeneran íntegramente a partir de los datos crudos ejecutando el código que lo acompaña, en un entorno declarado y con las semillas fijadas, sin que ninguna cifra se transcriba manualmente.

La condición es apenas la aplicación de lo practicado en las unidades anteriores [37]. Los datos crudos se tratan como de solo lectura, cada figura y cada tabla tiene un guion que la produce, el entorno se declara en un archivo y un único comando reconstruye todo desde cero [11]. La [Figura 4.11](#) muestra la correspondencia con los artefactos del repositorio.

El beneficio inmediato de esa disciplina no es la virtud sino la comodidad, pues cuando el cliente entrega un dato corregido a tres días de la entrega el informe se rehace en minutos. El beneficio de largo plazo es que el trabajo sobrevive a quien lo hizo. La [Tabla 4.6](#) recoge la lista de comprobación previa a la entrega.

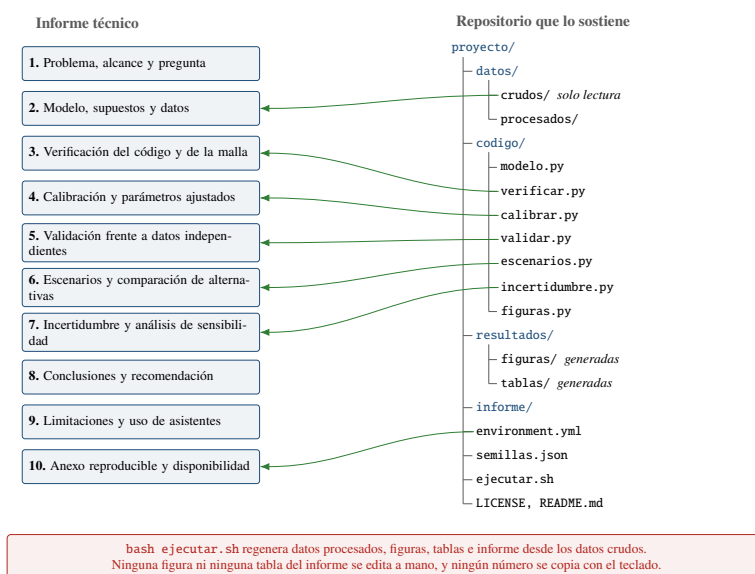


Figura 4.11. Estructura del informe técnico y del repositorio que lo sostiene, con el guion de reconstrucción que regenera figuras, tablas e informe desde los datos crudos.

Tabla 4.6. Lista de verificación del informe de simulación antes de su entrega.

#	Comprobación	Evidencia que debe existir
1	La pregunta de ingeniería está enunciada y el alcance delimitado	Párrafo con la decisión que apoya
2	Los supuestos están listados y justificados	Registro con criterio de validez
3	El código está verificado	Orden observado y caso de referencia
4	El error de discretización está acotado	Estudio de convergencia con índice
5	Los parámetros llevan intervalo y correlación	Matriz de covarianza o remuestreo
6	La validación usó datos independientes	Partición y criterio previo de aceptación
7	Los residuales fueron examinados	Paridad y diagnóstico de estructura
8	La incertidumbre está presupuestada	Tabla con contribuciones ordenadas
9	Las cifras significativas son coherentes	Redondeo gobernado por la incertidumbre
10	Las limitaciones y el dominio están escritos	Sección propia, no una frase de cortesía
11	El informe se regenera con un solo comando	Guion probado desde cero
12	El uso de asistentes está declarado	Nota con alcance y verificación aplicada

^a La lista se recorre con el informe terminado y ninguna casilla se marca sin señalar la página donde está la evidencia.

4.6. Comunicación científica y aplicaciones en investigación

El paso del informe técnico al artículo científico no es un cambio de formato sino de destinatario. El informe responde a un cliente que necesita decidir algo concreto, mientras que el artículo se dirige a una comunidad que necesita saber algo general, y por eso exige una afirmación defendible que nadie haya establecido antes. La [Figura 4.12](#) muestra la correspondencia entre ambas estructuras y el hilo argumental que las atraviesa.

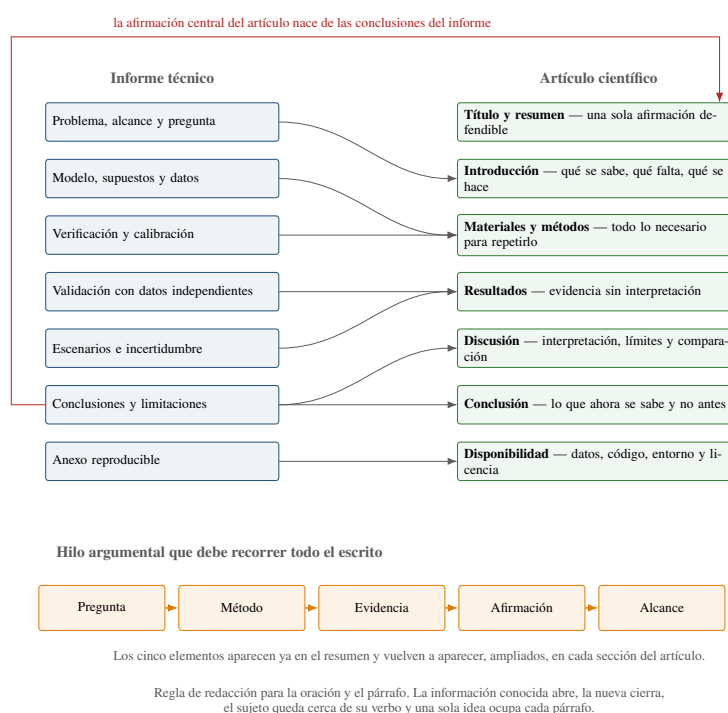


Figura 4.12. Correspondencia entre las secciones del informe técnico y las del artículo científico, con el hilo argumental que atraviesa ambos documentos.

La regla de organización que mejor resultado da consiste en tratar cada sección como respuesta a una pregunta única y en subordinar el escrito a una sola afirmación central [81]. La introducción establece qué se sabe, qué falta y qué aporta el trabajo; los métodos entregan lo necesario para repetirlo; los resultados presentan evidencia sin interpretarla; y la discusión interpreta y declara los límites. A escala de la oración la información conocida abre, la nueva cierra y el sujeto queda cerca de su verbo [82].

4.6.1. Métodos reproducibles y publicación de datos y de código

La sección de métodos de un trabajo que usa simulación debe permitir que otro grupo reconstruya un objeto computacional completo. Omitir la versión de una biblioteca o la semilla de un generador equivale a omitir la temperatura de un ensayo [83]. La [Tabla 4.7](#) enumera el contenido mínimo, que en un artículo se distribuye entre el cuerpo y el material suplementario.

Tabla 4.7. Contenido mínimo de la sección de métodos de un trabajo basado en simulación.

Elemento	Qué debe declararse
Modelo	Ecuaciones gobernantes, hipótesis, condiciones iniciales y de frontera
Datos	Origen, periodo, resolución, tratamiento de faltantes y de atípicos
Método numérico	Esquema, malla, paso de tiempo, tolerancias y criterio de parada
Verificación	Caso de referencia, orden observado e índice de convergencia
Calibración	Función objetivo, algoritmo, límites, punto inicial y datos empleados
Validación	Partición, métricas, criterio de aceptación declarado de antemano
Incertidumbre	Fuentes consideradas, distribuciones, método de propagación y semilla
Entorno	Lenguaje, bibliotecas con versión, sistema operativo y semillas
Disponibilidad	Repositorio con identificador persistente, licencia y datos

^a Cuando alguno de estos elementos no aplica, se declara explícitamente en lugar de omitirse.

La publicación del código y de los datos dejó de ser una cortesía y se convirtió en requisito de un número creciente de revistas y de agencias [84]. Los principios que la orientan exigen objetos localizables por un identificador persistente, accesibles por un protocolo abierto, interoperables por el uso de formatos estándar y reutilizables gracias a una licencia clara [40]. Un cuaderno con celdas ejecutadas fuera de orden no es reproducible ni por su autor [39].

La exposición oral obedece a una economía distinta de la del documento. El auditorio no puede releer, de modo que la charla se organiza alrededor de una sola figura que sostiene el argumento. Conviene declarar de entrada la pregunta y la afirmación, mostrar después la evidencia y cerrar con las limitaciones, sin reservar la conclusión para el final como si fuera un desenlace. La [Figura 4.13](#) fija ese guion para la sustentación del proyecto integrador de la asignatura, con el reparto del tiempo entre los tramos, la evidencia que sostiene cada uno y lo que descalifica una defensa.

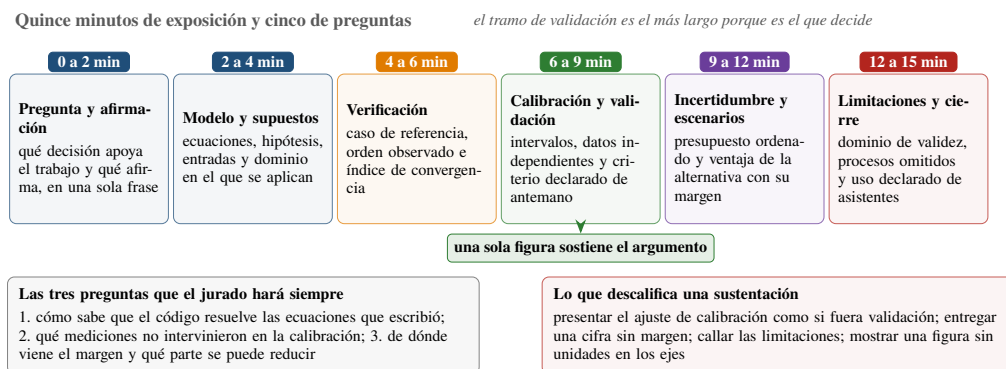


Figura 4.13. Guion de la sustentación del proyecto integrador, con el reparto del tiempo, la evidencia que sostiene cada tramo y lo que descalifica una defensa.

La declaración del uso de asistentes de programación forma parte de esa misma honestidad instrumental. Lo que se declara es el alcance del uso y sobre todo la verificación aplicada a lo generado. Un fragmento producido por un modelo de lenguaje que pasó el caso de referencia analítico, el estudio de convergencia y la prueba de conservación tiene el mismo estatus que uno escrito a mano. La responsabilidad no se transfiere.

4.6.2. La simulación en el diseño de una investigación

En un trabajo de investigación el modelo rara vez es el resultado, casi siempre es el instrumento. Su función más frecuente consiste en convertir una hipótesis cualitativa en una predicción cuantitativa que el experimento pueda refutar, lo cual obliga a formularla antes de medir. Su segunda función consiste en decidir dónde medir, pues el análisis de sensibilidad orienta el diseño experimental hacia las variables que gobiernan la respuesta [10].

Los cinco dominios de la Facultad ofrecen ejemplos inmediatos. En ingeniería civil e hidráulica un modelo de tránsito de crecientes convierte una hipótesis sobre la respuesta de la cuenca en un hidrograma que la estación de aforo puede contrastar. En ingeniería agrícola un balance hídrico con evapotranspiración de referencia permite diseñar un ensayo de campo que compare láminas de riego [12]. En agroindustria un modelo cinético de degradación define a qué temperaturas conviene ensayar para estimar la energía de activación.

En ingeniería ambiental y sanitaria un modelo de digestión anaerobia predice la producción de metano ante un cambio de sustrato y separa la variabilidad de la composición de la del arranque del reactor [85]. En ingeniería eléctrica y energética un modelo de generación fotovoltaica separa el efecto de la irradiación del de la temperatura de celda [86]. En los cinco casos el modelo no reemplaza al experimento, lo hace más barato y más informativo.

Dos desarrollos recientes amplían el repertorio de instrumentos de investigación sin alterar el criterio anterior. El primero es la disponibilidad de datos de sensores remotos, de drones y de plataformas de procesamiento en la nube, que suministran entradas espacialmente distribuidas a los modelos de balance hídrico, de estado de los cultivos y de calidad ambiental a escala de lote y de región [33, 36, 35]. El segundo es la madurez de los modelos de aprendizaje estadístico y de las redes neuronales profundas, que construyen modelos fenomenológicos de gran flexibilidad a partir de esos datos [79, 8, 34]. Ambos son bienvenidos en un trabajo de investigación siempre que se sometan a la misma disciplina que este capítulo exige a cualquier modelo, esto es, una partición honesta de los datos, una validación frente a observaciones independientes y una declaración explícita del dominio dentro del cual la predicción tiene respaldo [7].

El capítulo cierra con la obligación que da sentido a todo lo anterior. Quien entrega el resultado de una simulación a quien decide asume una responsabilidad que no se agota en la corrección técnica, pues debe comunicar el margen, el dominio de validez y aquello que el modelo no representa, con claridad suficiente para que la decisión se tome con conocimiento de causa. Un modelo bien verificado y mal comunicado produce el mismo daño que uno equivocado. La Figura 4.14 recoge el arco completo del libro, con lo que cada unidad deja en el proyecto integrador y el peso que la evaluación de la asignatura asigna a cada instrumento, de modo que el lector pueda reconocer en qué punto de esa construcción se encuentra su propio trabajo.

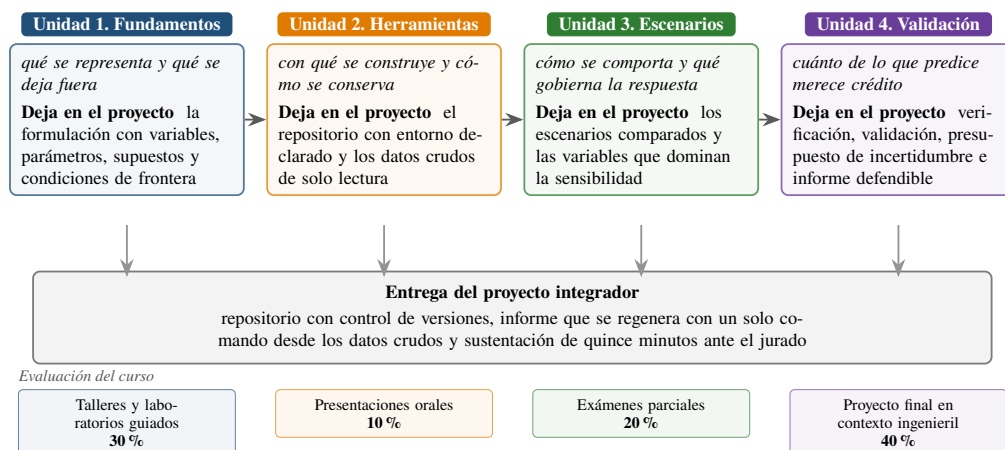


Figura 4.14. El arco del curso, con lo que cada unidad deja en el proyecto integrador y el peso que la evaluación asigna a cada instrumento.

4.7. Problemas

Los problemas siguientes recorren el capítulo en complejidad creciente. Donde se indique semilla aleatoria se emplea la del curso, que es 20 262.

4.7.1. Apropiación conceptual

- 4-1** Explique por qué la verificación debe ejecutarse antes de la calibración, y describa un caso de su disciplina en el que invertir el orden produciría una conclusión falsa.
- 4-2** Un colega afirma que su modelo está validado porque reproduce con error inferior al 3 % los datos con los que estimó sus parámetros. Redacte la objeción técnica y proponga el ejercicio que sí constituiría una validación.
- 4-3** Distinga incertidumbre aleatoria de epistémica sobre tres magnitudes de su área, e indique para cada una si más mediciones la reducirían.
- 4-4** Explique por qué una solución fabricada no necesita tener sentido físico, y por qué sí necesita activar todos los términos del operador.
- 4-5** Un estudio reporta eficiencias de 0.82 y 0.61 en dos cuencas y concluye que el modelo funciona mejor en la primera. Señale por qué la comparación es inválida.

4.7.2. Verificación y convergencia

- 4-6 Un esquema de segundo orden entrega errores de 4.10×10^{-3} , 1.05×10^{-3} y 2.64×10^{-4} para pasos de 0.04, 0.02 y 0.01. Calcule el orden observado por pares y diga si se alcanzó el régimen asintótico.
- 4-7 Con los valores 18.42, 18.61 y 18.68 de una magnitud integral en mallas de razón 2, estime el orden observado, el valor extrapolado y el índice de convergencia de malla.
- 4-8 Un código de conducción transitoria entrega orden observado 0.98 pese a usar diferencias centradas en el espacio y un integrador de cuarto orden. Proponga tres causas ordenadas por probabilidad y el ensayo que las distingue.
- 4-9 Construya una solución fabricada para la ecuación de advección con difusión estacionaria, con velocidad constante y coeficiente variable, y escriba su término fuente.

4.7.3. Calibración e identificabilidad

- 4-10 Un ajuste de dos parámetros entrega errores estándar de 0.021 y 1.45 con correlación de 0.987. Interprete el resultado y proponga una reparametrización.
- 4-11 Explique por qué la ponderación por el inverso de la varianza es obligatoria cuando las mediciones cubren tres órdenes de magnitud, y qué ocurre si se omite.
- 4-12 Se ajusta una curva de retención de humedad de un suelo con cuatro parámetros a partir de seis puntos. Discuta la identificabilidad práctica y proponga el diseño experimental que la mejoraría.
- 4-13 Un modelo de secado ajustado desde dos puntos iniciales devuelve dos conjuntos de parámetros con sumas de cuadrados que difieren en el cuarto decimal. Nombre el fenómeno y describa cómo debe reportarse.
- 4-14 Dos modelos ajustan los mismos quince datos con sumas de cuadrados de 2.4×10^{-3} y 1.9×10^{-3} , con dos y cuatro parámetros. Calcule el criterio de información corregido y decida cuál se prefiere.

4.7.4. Validación y métricas de desempeño

- 4-15 Demuestre la relación exacta entre el coeficiente de eficiencia y la razón entre el error y la desviación observada, y diga qué consecuencia tiene para el reporte.
- 4-16 Una validación entrega error medio de -0.02°C y raíz del error cuadrático medio de 2.8°C . Explique cómo coexisten y qué gráfica revelaría lo que ocultan.

- 4-17 Un modelo de calidad de agua calibrado en estiaje se valida en invierno con eficiencia de 0.31. Diagnostique la causa y proponga la modificación del modelo antes de tocar los parámetros.
- 4-18 Explique por qué partir al azar una serie temporal produce métricas optimistas, y proponga la partición correcta para un registro de cinco años de caudal diario.

4.7.5. Incertidumbre y consistencia

- 4-19 La potencia hidráulica de una bomba es el producto de densidad, gravedad, caudal y altura dividido por la eficiencia. Con incertidumbres relativas del 0.1 %, 3 %, 2 % y 4 %, construya su presupuesto.
- 4-20 La pérdida de carga de una tubería depende del cuadrado del caudal. Con incertidumbre relativa del 6 % en el caudal, calcule la de la pérdida por la ley de primer orden y verifíquela por Monte Carlo con 100 000 réplicas.
- 4-21 La vida útil por Arrhenius depende de la temperatura a través de una exponencial de argumento inverso. Explique por qué la ley de primer orden falla antes que en un producto de potencias.
- 4-22 Un resultado se reporta como 1247.836 kWh con incertidumbre de 63 kWh. Corrija el reporte y justifique la regla aplicada.
- 4-23 Explique por qué la incertidumbre de la diferencia entre dos escenarios evaluados con el mismo modelo es menor que la de cada uno por separado, y bajo qué condición deja de serlo.

4.7.6. Problemas de programación

- 4-24 Implemente una función que reciba una lista de mallas, estime el orden observado y avise cuando el régimen asintótico no se haya alcanzado.
- 4-25 Implemente el método de las soluciones fabricadas para la difusión con reacción transitoria, y verifique el orden en espacio y en tiempo por separado.
- 4-26 Escriba una función que calibre con límites físicos y devuelva parámetros, errores estándar, correlaciones, número de condición e índice de colinealidad.
- 4-27 Implemente el bootstrap de los residuales de un ajuste no lineal y compare sus intervalos con los de la aproximación lineal.
- 4-28 Escriba un guion que regenere todas las figuras y tablas del informe desde los datos crudos, con semilla declarada y sin rutas absolutas.

4.7.7. Problemas de diseño y ensayo

- 4–29** Diseñe el protocolo de verificación, calibración y validación de un modelo de su área, con los datos de cada etapa y el criterio de aceptación declarado de antemano.
- 4–30** Evalúe la sección de métodos de un artículo publicado de su disciplina frente a la [Tabla 4.7](#), y señale qué permitiría reproducirlo y qué lo impediría.
- 4–31** Un cliente exige una predicción puntual y rechaza el intervalo. Redacte la respuesta profesional que lo defiende y la forma de presentarlo que él sí aceptaría.
- 4–32** Tome un modelo de las unidades anteriores y prepare su paquete de reproducibilidad, con repositorio, entorno declarado, semillas, guion de reconstrucción y licencia.

Referencias

- [1] K. Velten, D. M. Schmidt y K. Kahlen. *Mathematical Modeling and Simulation: Introduction for Scientists and Engineers*. 2.^a ed. Weinheim: Wiley-VCH, 2024. ISBN: 978-3-527-41414-7. URL: <https://www.hs-geisenheim.de/mms/>.
- [2] W. L. Oberkampf y C. J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010. DOI: [10.1017/CB09780511760396](https://doi.org/10.1017/CB09780511760396).
- [3] G. E. P. Box. «Science and Statistics». En: *Journal of the American Statistical Association* 71.356 (1976), págs. 791-799. DOI: [10.1080/01621459.1976.10480949](https://doi.org/10.1080/01621459.1976.10480949).
- [4] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau et al. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17.3 (2020), págs. 261-272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [5] A. B. Downey. *Modeling and Simulation in Python: An Introduction for Scientists and Engineers*. No Starch Press, 2023. URL: <https://allendowney.github.io/ModSimPy/>.
- [6] G. James, D. Witten, T. Hastie y R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2.^a ed. New York: Springer, 2021. DOI: [10.1007/978-1-0716-1418-1](https://doi.org/10.1007/978-1-0716-1418-1). URL: <https://www.statlearning.com/>.
- [7] K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. Cambridge, MA: MIT Press, 2022. ISBN: 978-0-262-04682-4. URL: <https://probml.github.io/pml-book/book1.html>.
- [8] S. J. D. Prince. *Understanding Deep Learning*. Cambridge, MA: MIT Press, 2023. URL: <https://udlbook.github.io/udlbook/>.
- [9] P. V. Danckwerts. «Continuous Flow Systems: Distribution of Residence Times». En: *Chemical Engineering Science* 2.1 (1953), págs. 1-13. DOI: [10.1016/0009-2509\(53\)80001-1](https://doi.org/10.1016/0009-2509(53)80001-1).
- [10] A. J. Jakeman, R. A. Letcher y J. P. Norton. «Ten Iterative Steps in Development and Evaluation of Environmental Models». En: *Environmental Modelling & Software* 21.5 (2006), págs. 602-614. DOI: [10.1016/j.envsoft.2006.01.004](https://doi.org/10.1016/j.envsoft.2006.01.004).

- [11] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt y T. K. Teal. «Good Enough Practices in Scientific Computing». En: *PLOS Computational Biology* 13.6 (2017), e1005510. DOI: [10.1371/journal.pcbi.1005510](https://doi.org/10.1371/journal.pcbi.1005510).
- [12] R. G. Allen, L. S. Pereira, D. Raes y M. Smith. *Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements*. FAO Irrigation and Drainage Paper 56. Food y Agriculture Organization of the United Nations, 1998. URL: <https://www.fao.org/4/x0490e/x0490e00.htm>.
- [13] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau et al. «Array Programming with NumPy». En: *Nature* 585.7825 (2020), págs. 357-362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [14] V. T. Chow. *Open-Channel Hydraulics*. McGraw-Hill, 1959.
- [15] E. Hairer y G. Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. 2.^a ed. Springer, 1996. DOI: [10.1007/978-3-642-05221-7](https://doi.org/10.1007/978-3-642-05221-7).
- [16] A. K. Datta. *Heat and Mass Transfer: A Biological Context*. 2.^a ed. CRC Press, 2017. URL: <https://www.routledge.com/Heat-and-Mass-Transfer-A-Biological-Context-Second-Edition/Datta/p/book/9781138033603>.
- [17] E. Buckingham. «On Physically Similar Systems; Illustrations of the Use of Dimensional Equations». En: *Physical Review* 4.4 (1914), págs. 345-376. DOI: [10.1103/PhysRev.4.345](https://doi.org/10.1103/PhysRev.4.345).
- [18] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin et al. «SymPy: Symbolic Computing in Python». En: *PeerJ Computer Science* 3 (2017), e103. DOI: [10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103).
- [19] F. P. Incropera, D. P. DeWitt, T. L. Bergman y A. S. Lavine. *Fundamentals of Heat and Mass Transfer*. 7.^a ed. Wiley, 2011.
- [20] C. V. Theis. «The Relation between the Lowering of the Piezometric Surface and the Rate and Duration of Discharge of a Well Using Ground-Water Storage». En: *Eos, Transactions American Geophysical Union* 16.2 (1935), págs. 519-524. DOI: [10.1029/TR016i002p00519](https://doi.org/10.1029/TR016i002p00519).
- [21] H. H. Cooper y C. E. Jacob. «A Generalized Graphical Method for Evaluating Formation Constants and Summarizing Well-Field History». En: *Eos, Transactions American Geophysical Union* 27.4 (1946), págs. 526-534. DOI: [10.1029/TR027i004p00526](https://doi.org/10.1029/TR027i004p00526).
- [22] C. J. Roy y W. L. Oberkampf. «A Comprehensive Framework for Verification, Validation, and Uncertainty Quantification in Scientific Computing». En: *Computer Methods in Applied Mechanics and Engineering* 200.25–28 (2011), págs. 2131-2144. DOI: [10.1016/j.cma.2011.03.016](https://doi.org/10.1016/j.cma.2011.03.016).

- [23] Joint Committee for Guides in Metrology. *Evaluation of Measurement Data – Guide to the Expression of Uncertainty in Measurement*. JCGM 100:2008. Bureau International des Poids et Mesures, 2008. URL: <https://www.bipm.org/en/committees/jc/jcgm/publications>.
- [24] K. P. Burnham y D. R. Anderson. *Model Selection and Multimodel Inference: A Practical Information-Theoretic Approach*. 2.^a ed. Springer, 2002. DOI: [10.1007/b97636](https://doi.org/10.1007/b97636).
- [25] K. Beven. «A Manifesto for the Equifinality Thesis». En: *Journal of Hydrology* 320.1–2 (2006), págs. 18-36. DOI: [10.1016/j.jhydrol.2005.07.007](https://doi.org/10.1016/j.jhydrol.2005.07.007).
- [26] W. McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3.^a ed. O'Reilly Media, 2022. URL: <https://wesmckinney.com/book/>.
- [27] J. D. Hunter. «Matplotlib: A 2D Graphics Environment». En: *Computing in Science & Engineering* 9.3 (2007), págs. 90-95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [28] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic et al. «Jupyter Notebooks – a Publishing Format for Reproducible Computational Workflows». En: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. Ed. por F. Loizides y B. Schmidt. IOS Press, 2016, págs. 87-90. DOI: [10.3233/978-1-61499-649-1-87](https://doi.org/10.3233/978-1-61499-649-1-87).
- [29] J. VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. 2.^a ed. O'Reilly Media, 2023. URL: <https://jakevdp.github.io/PythonDataScienceHandbook/>.
- [30] R. Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. 3.^a ed. Apress, 2024. DOI: [10.1007/979-8-8688-0413-7](https://doi.org/10.1007/979-8-8688-0413-7). URL: <https://doi.org/10.1007/979-8-8688-0413-7>.
- [31] S. Linge y H. P. Langtangen. *Programming for Computations – Python: A Gentle Introduction to Numerical Simulations with Python 3.6*. 2.^a ed. Springer, 2019. DOI: [10.1007/978-3-030-16877-3](https://doi.org/10.1007/978-3-030-16877-3). URL: <https://doi.org/10.1007/978-3-030-16877-3>.
- [32] J. Sundnes. *Introduction to Scientific Programming with Python*. Springer, 2020. DOI: [10.1007/978-3-030-50356-7](https://doi.org/10.1007/978-3-030-50356-7). URL: <https://doi.org/10.1007/978-3-030-50356-7>.
- [33] J. A. Cardille, M. A. Crowley, D. Saah y N. E. Clinton, eds. *Cloud-Based Remote Sensing with Google Earth Engine: Fundamentals and Applications*. Cham: Springer, 2024. DOI: [10.1007/978-3-031-26588-4](https://doi.org/10.1007/978-3-031-26588-4). URL: <https://doi.org/10.1007/978-3-031-26588-4>.

- [34] S. S. Chouhan, A. Saxena, U. P. Singh y S. Jain, eds. *Artificial Intelligence Techniques in Smart Agriculture*. Singapore: Springer, 2024. DOI: [10.1007/978-981-97-5878-4](https://doi.org/10.1007/978-981-97-5878-4). URL: <https://doi.org/10.1007/978-981-97-5878-4>.
- [35] S. S. Chouhan, U. P. Singh y S. Jain, eds. *Applications of Computer Vision and Drone Technology in Agriculture 4.0*. Singapore: Springer, 2024. DOI: [10.1007/978-981-99-8684-2](https://doi.org/10.1007/978-981-99-8684-2). URL: <https://doi.org/10.1007/978-981-99-8684-2>.
- [36] J. A. Domínguez-Gómez y J. Kumhálová, eds. *Advanced in Remote Sensing Approaches for Agricultural Monitoring at Field and Regional Scale*. Special Issue Reprint of *Remote Sensing*. Basel: MDPI, 2025. DOI: [10.3390/books978-3-7258-5844-6](https://doi.org/10.3390/books978-3-7258-5844-6). URL: <https://doi.org/10.3390/books978-3-7258-5844-6>.
- [37] G. K. Sandve, A. Nekrutenko, J. Taylor y E. Hovig. «Ten Simple Rules for Reproducible Computational Research». En: *PLOS Computational Biology* 9.10 (2013), e1003285. DOI: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- [38] S. Chacon y B. Straub. *Pro Git*. 2.^a ed. Apress, 2014. URL: <https://git-scm.com/book/es/v2>.
- [39] A. Rule, A. Birmingham, C. Zuniga, I. Altintas, S.-C. Huang, R. Knight et al. «Ten Simple Rules for Writing and Sharing Computational Analyses in Jupyter Notebooks». En: *PLOS Computational Biology* 15.7 (2019), e1007007. DOI: [10.1371/journal.pcbi.1007007](https://doi.org/10.1371/journal.pcbi.1007007).
- [40] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak et al. «The FAIR Guiding Principles for Scientific Data Management and Stewardship». En: *Scientific Data* 3 (2016), pág. 160018. DOI: [10.1038/sdata.2016.18](https://doi.org/10.1038/sdata.2016.18).
- [41] H. Wickham. «Tidy Data». En: *Journal of Statistical Software* 59.10 (2014), págs. 1-23. DOI: [10.18637/jss.v059.i10](https://doi.org/10.18637/jss.v059.i10).
- [42] R. J. A. Little y D. B. Rubin. *Statistical Analysis with Missing Data*. 3.^a ed. Hoboken, NJ: John Wiley & Sons, 2019.
- [43] F. R. Hampel. «The Influence Curve and Its Role in Robust Estimation». En: *Journal of the American Statistical Association* 69.346 (1974), págs. 383-393. DOI: [10.1080/01621459.1974.10482962](https://doi.org/10.1080/01621459.1974.10482962).
- [44] P. J. Rousseeuw y C. Croux. «Alternatives to the Median Absolute Deviation». En: *Journal of the American Statistical Association* 88.424 (1993), págs. 1273-1283. DOI: [10.1080/01621459.1993.10476408](https://doi.org/10.1080/01621459.1993.10476408).

- [45] D. Goldberg. «What Every Computer Scientist Should Know about Floating-Point Arithmetic». En: *ACM Computing Surveys* 23.1 (1991), págs. 5-48. doi: [10.1145/103162.103163](https://doi.org/10.1145/103162.103163).
- [46] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Inf. téc. IEEE Std 754-2019. New York: Institute of Electrical y Electronics Engineers, 2019. doi: [10.1109/IEEESTD.2019.8766229](https://doi.org/10.1109/IEEESTD.2019.8766229).
- [47] W. Kahan. «Pracniques: Further Remarks on Reducing Truncation Errors». En: *Communications of the ACM* 8.1 (1965), pág. 40. doi: [10.1145/363707.363723](https://doi.org/10.1145/363707.363723).
- [48] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2.^a ed. Society for Industrial y Applied Mathematics, 2002. doi: [10.1137/1.9780898718027](https://doi.org/10.1137/1.9780898718027).
- [49] N. P. Rougier, M. Droettboom y P. E. Bourne. «Ten Simple Rules for Better Figures». En: *PLOS Computational Biology* 10.9 (2014), e1003833. doi: [10.1371/journal.pcbi.1003833](https://doi.org/10.1371/journal.pcbi.1003833).
- [50] E. R. Tufte. *The Visual Display of Quantitative Information*. 2.^a ed. Graphics Press, 2001.
- [51] C. F. Colebrook. «Turbulent Flow in Pipes, with Particular Reference to the Transition Region between the Smooth and Rough Pipe Laws». En: *Journal of the Institution of Civil Engineers* 11.4 (1939), págs. 133-156. doi: [10.1680/ijoti.1939.13150](https://doi.org/10.1680/ijoti.1939.13150).
- [52] F. M. White. *Fluid Mechanics*. 8.^a ed. McGraw-Hill Education, 2016.
- [53] J. Monod. «The Growth of Bacterial Cultures». En: *Annual Review of Microbiology* 3 (1949), págs. 371-394. doi: [10.1146/annurev.mi.03.100149.002103](https://doi.org/10.1146/annurev.mi.03.100149.002103).
- [54] I. Metcalf & Eddy, G. Tchobanoglous, H. D. Stensel, R. Tsuchihashi y F. L. Burton. *Wastewater Engineering: Treatment and Resource Recovery*. 5.^a ed. McGraw-Hill Education, 2014.
- [55] S. C. Chapra y D. E. Clough. *Applied Numerical Methods with Python for Engineers and Scientists*. McGraw Hill, 2022. URL: <https://www.mheducation.com/highered/product/applied-numerical-methods-with-python-for-engineers-and-scientists-chapra.html>.
- [56] S. C. Chapra. *Applied Numerical Methods with MATLAB for Engineers and Scientists*. 4.^a ed. McGraw-Hill Education, 2018.
- [57] Q. Kong, T. Siauw y A. Bayen. *Python Programming and Numerical Methods: A Guide for Engineers and Scientists*. Academic Press, 2020. URL: <https://pythonnumericalmethods.studentorg.berkeley.edu/>.

- [58] J. Sundnes. *Solving Ordinary Differential Equations in Python*. Springer, 2023. DOI: [10.1007/978-3-031-46768-4](https://doi.org/10.1007/978-3-031-46768-4). URL: <https://doi.org/10.1007/978-3-031-46768-4>.
- [59] J. R. Dormand y P. J. Prince. «A Family of Embedded Runge–Kutta Formulae». En: *Journal of Computational and Applied Mathematics* 6.1 (1980), págs. 19-26. DOI: [10.1016/0771-050X\(80\)90013-3](https://doi.org/10.1016/0771-050X(80)90013-3).
- [60] L. F. Shampine y M. W. Reichelt. «The MATLAB ODE Suite». En: *SIAM Journal on Scientific Computing* 18.1 (1997), págs. 1-22. DOI: [10.1137/S1064827594276424](https://doi.org/10.1137/S1064827594276424).
- [61] D. E. Seborg, T. F. Edgar, D. A. Mellichamp y F. J. Doyle. *Process Dynamics and Control*. 4.^a ed. Wiley, 2017. URL: <https://www.wiley.com/en-us/Process+Dynamics+and+Control,+4th+Edition-p-9781119285915>.
- [62] R. Courant, K. Friedrichs y H. Lewy. «Über die partiellen Differenzengleichungen der mathematischen Physik». En: *Mathematische Annalen* 100.1 (1928), págs. 32-74. DOI: [10.1007/BF01448839](https://doi.org/10.1007/BF01448839).
- [63] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial y Applied Mathematics, 2007. DOI: [10.1137/1.9780898717839](https://doi.org/10.1137/1.9780898717839).
- [64] H. P. Langtangen y A. Logg. *Solving PDEs in Python: The FEniCS Tutorial I*. Springer, 2016. DOI: [10.1007/978-3-319-52462-7](https://doi.org/10.1007/978-3-319-52462-7). URL: <https://doi.org/10.1007/978-3-319-52462-7>.
- [65] D. C. Montgomery. *Design and Analysis of Experiments*. 10.^a ed. Wiley, 2020. URL: <https://www.wiley.com/en-us/Design+and+Analysis+of+Experiments,+10th+Edition-p-9781119492443>.
- [66] J. C. Helton y F. J. Davis. «Latin Hypercube Sampling and the Propagation of Uncertainty in Analyses of Complex Systems». En: *Reliability Engineering & System Safety* 81.1 (2003), págs. 23-69. DOI: [10.1016/S0951-8320\(03\)00058-9](https://doi.org/10.1016/S0951-8320(03)00058-9).
- [67] M. D. McKay, R. J. Beckman y W. J. Conover. «A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code». En: *Technometrics* 21.2 (1979), págs. 239-245. DOI: [10.1080/00401706.1979.10489755](https://doi.org/10.1080/00401706.1979.10489755).
- [68] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli et al. *Global Sensitivity Analysis: The Primer*. Wiley, 2008. DOI: [10.1002/9780470725184](https://doi.org/10.1002/9780470725184).

- [69] M. D. Morris. «Factorial Sampling Plans for Preliminary Computational Experiments». En: *Technometrics* 33.2 (1991), págs. 161-174. DOI: [10.1080/00401706.1991.10484804](https://doi.org/10.1080/00401706.1991.10484804).
- [70] I. M. Sobol'. «Global Sensitivity Indices for Nonlinear Mathematical Models and Their Monte Carlo Estimates». En: *Mathematics and Computers in Simulation* 55.1-3 (2001), págs. 271-280. DOI: [10.1016/S0378-4754\(00\)00270-6](https://doi.org/10.1016/S0378-4754(00)00270-6).
- [71] A. Saltelli, P. Annoni, I. Azzini, F. Campolongo, M. Ratto y S. Tarantola. «Variance Based Sensitivity Analysis of Model Output. Design and Estimator for the Total Sensitivity Index». En: *Computer Physics Communications* 181.2 (2010), págs. 259-270. DOI: [10.1016/j.cpc.2009.09.018](https://doi.org/10.1016/j.cpc.2009.09.018).
- [72] R. G. Sargent. «Verification and Validation of Simulation Models». En: *Journal of Simulation* 7.1 (2013), págs. 12-24. DOI: [10.1057/jos.2012.20](https://doi.org/10.1057/jos.2012.20).
- [73] B. Efron. «Bootstrap Methods: Another Look at the Jackknife». En: *The Annals of Statistics* 7.1 (1979), págs. 1-26. DOI: [10.1214/aos/1176344552](https://doi.org/10.1214/aos/1176344552).
- [74] R. Brun, P. Reichert y H. R. Künsch. «Practical Identifiability Analysis of Large Environmental Simulation Models». En: *Water Resources Research* 37.4 (2001), págs. 1015-1030. DOI: [10.1029/2000WR900350](https://doi.org/10.1029/2000WR900350).
- [75] N. D. Bennett, B. F. W. Croke, G. Guariso, J. H. A. Guillaume, S. H. Hamilton, A. J. Jakeman et al. «Characterising Performance of Environmental Models». En: *Environmental Modelling & Software* 40 (2013), págs. 1-20. DOI: [10.1016/j.envsoft.2012.09.011](https://doi.org/10.1016/j.envsoft.2012.09.011).
- [76] J. E. Nash y J. V. Sutcliffe. «River Flow Forecasting through Conceptual Models Part I – A Discussion of Principles». En: *Journal of Hydrology* 10.3 (1970), págs. 282-290. DOI: [10.1016/0022-1694\(70\)90255-6](https://doi.org/10.1016/0022-1694(70)90255-6).
- [77] C. J. Willmott. «On the Validation of Models». En: *Physical Geography* 2.2 (1981), págs. 184-194. DOI: [10.1080/02723646.1981.10642213](https://doi.org/10.1080/02723646.1981.10642213).
- [78] D. N. Moriasi, J. G. Arnold, M. W. Van Liew, R. L. Bingner, R. D. Harmel y T. L. Veith. «Model Evaluation Guidelines for Systematic Quantification of Accuracy in Watershed Simulations». En: *Transactions of the ASABE* 50.3 (2007), págs. 885-900. DOI: [10.13031/2013.23153](https://doi.org/10.13031/2013.23153).
- [79] G. James, D. Witten, T. Hastie, R. Tibshirani y J. Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Cham: Springer, 2023. DOI: [10.1007/978-3-031-38747-0](https://doi.org/10.1007/978-3-031-38747-0). URL: <https://www.statlearning.com/>.
- [80] Joint Committee for Guides in Metrology. *Evaluation of Measurement Data – Supplement 1 to the GUM: Propagation of Distributions Using a Monte Carlo Method*. JCGM 101:2008. Bureau International des Poids et Mesures, 2008. URL: <https://www.bipm.org/en/committees/jc/jcgm/publications>.

- [81] B. Mensh y K. Kording. «Ten Simple Rules for Structuring Papers». En: *PLOS Computational Biology* 13.9 (2017), e1005619. doi: [10.1371/journal.pcbi.1005619](https://doi.org/10.1371/journal.pcbi.1005619).
- [82] G. D. Gopen y J. A. Swan. «The Science of Scientific Writing». En: *American Scientist* 78.6 (1990), págs. 550-558.
- [83] R. D. Peng. «Reproducible Research in Computational Science». En: *Science* 334.6060 (2011), págs. 1226-1227. doi: [10.1126/science.1213847](https://doi.org/10.1126/science.1213847).
- [84] V. Stodden, M. McNutt, D. H. Bailey, E. Deelman, Y. Gil, B. Hanson et al. «Enhancing Reproducibility for Computational Methods». En: *Science* 354.6317 (2016), págs. 1240-1241. doi: [10.1126/science.aah6168](https://doi.org/10.1126/science.aah6168).
- [85] D. J. Batstone, J. Keller, I. Angelidaki, S. V. Kalyuzhnyi, S. G. Pavlostathis, A. Rozzi et al. «The IWA Anaerobic Digestion Model No 1 (ADM1)». En: *Water Science and Technology* 45.10 (2002), págs. 65-73. doi: [10.2166/wst.2002.0292](https://doi.org/10.2166/wst.2002.0292).
- [86] J. A. Duffie y W. A. Beckman. *Solar Engineering of Thermal Processes*. 4.^a ed. Wiley, 2013. doi: [10.1002/9781118671603](https://doi.org/10.1002/9781118671603).